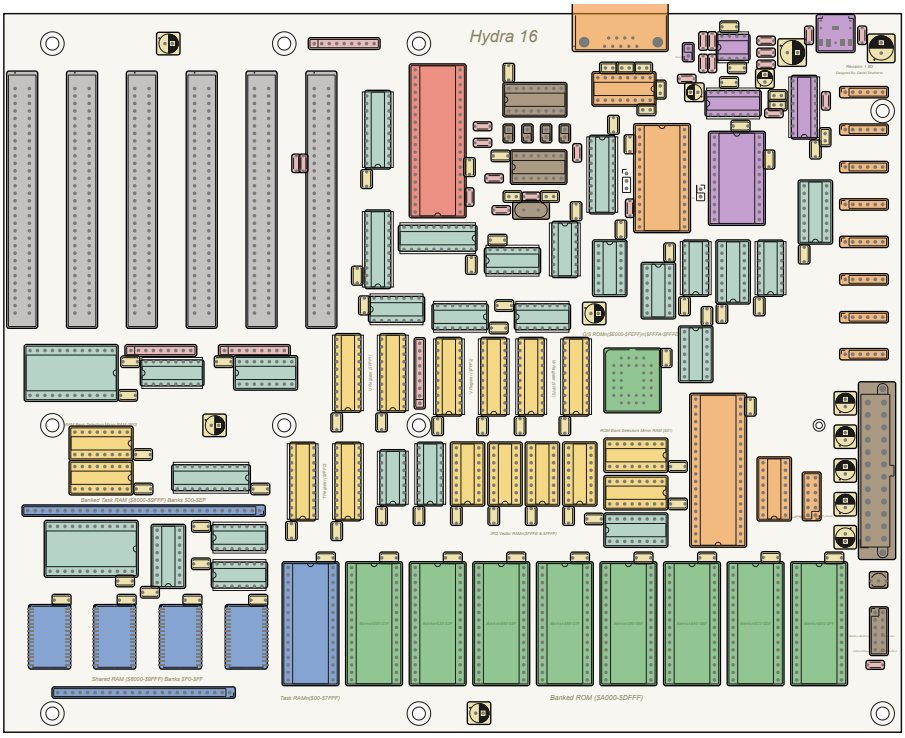
THE HYDRA PROJECT

HYDRA-16

User's and Builder's Manual



For builders, programmers and their teachers
Ages 12 and up

HydraOS 1.0

About This Book

Welcome! You are holding the manual for the **Hydra-16**, a computer you build yourself, one part at a time, and then learn to program from the inside out. This book is your guide for the whole trip: from an empty circuit board and a bag of parts, to a working computer, to the programs you will write for it.

You don't need to know anything about electronics or computers to start. If you can follow a recipe, use a screwdriver carefully, and type on a keyboard, you can build and use a Hydra. The first chapters explain everything from the beginning. Later chapters go deeper, and by the end of the book you will be writing programs in four different languages, one of them the language the computer's chip speaks itself.

Who this book is for

This book was written for **students aged 12 and up**, for their **teachers**, and for anyone else who wants to understand how a computer really works. Some readers will build a Hydra from a kit; others will use the Hydra *emulator*, a program that makes a PC behave exactly like a Hydra. Everything in this book except the building chapters works the same way on both.

> **FOR TEACHERS:** Boxes like this one have suggestions for classrooms: what to try together, what to watch for, and how much time an activity takes. A whole class can use the emulator (Chapter 2) at the same time, with one or two real Hydras built as a group project.

Two more books go with this one. The **Student Companion** is a workbook: a build log with a box to tick for every part, lab sheets and exercises for each chapter, projects, and quick-reference cards. The **Educator's Course Guide** is for whoever is running a class or a club: course plans, lesson by lesson, a safety policy, how to equip a room, how to mark the work, and the answers. You don't need either to use this book. It's complete on its own.

How this book is organized

The book has five parts, plus appendices at the back for looking things up.

Part One, Meet the Hydra, introduces the computer, shows you how to run it on a PC, and takes you through building and switching on your own.

Part Two, Inside HydraOS, starts with your first hour at the keyboard. Then it follows the computer from the moment it is switched on, and teaches you to use it: its shells, its files, its windows, and every command it knows.

Part Three, Devices, explains how the Hydra talks to the world: the screen and keyboard, disks, sound, video and the connectors on the board, and the *control files* that let you command each one.

Part Four, Programming Languages, teaches you to program in HyForth, hylang and HyBASIC, starting with the way the Hydra handles numbers.

Part Five, Machine Language, goes all the way down to the 65C02 chip at the heart of the Hydra, with the assembler, disassembler and debugger that let you program it directly.

The appendices hold the complete language references, the 65C02's instructions, the character codes, the memory and I/O map, error messages, the emulator's options, the parts list and board diagrams, a glossary of computer words, and every system call. The index, at the very end, finds the page where a word is explained.

Which way through?

You don't have to read the parts in order. After Chapter 6, every reader knows enough to go where they like:

IF YOU WANT TO	READ
Try a Hydra today, on a PC	Chapters 1 and 2, then Chapter 6
Build one	Chapters 1 to 5, in order and carefully, then Chapter 6
Start programming as soon as you can	Chapters 2 and 6, then Part Four: Chapter 18, and the language you like
Understand how the machine and its system work	Parts Two and Three, in order
Make sounds and pictures	Chapter 6, then Chapters 15 and 16, and the second chapter of your language in Part Four
Program the processor itself	Chapter 6, a language from Part Four, then Part Five

Each chapter opens with a box, **IN THIS CHAPTER**, that says what you'll be able to do by the end of it, and what it expects you to know already.

When you want every detail, the Hydra's **reference book** has it: the folder `docs` in the Hydra's files (start at `docs/README.md`) describes the architecture, every driver's files and commands, the expansion cards and the quirks. This book explains; that one specifies.

How to read the examples

Most chapters have examples you can try. In the examples, what **you type** is printed in **bold**, and what the Hydra prints is in plain type:

```
/> 2 3 + .
5
/> echo Hello, Hydra!
Hello, Hydra!
```

The text before your typing, like `/>` or `%`, is the **prompt**: the computer's way of saying "I'm ready, type something." You type only the bold part, then press the RETURN key (it may be called ENTER on your keyboard).

Keys you press are shown like this: RETURN, ESC, TAB. When two keys are joined by a dash, hold down the first and tap the second: CTRL-C means hold the CTRL key and tap C. Keys like CTRL-] work the same way.

Computer people often write numbers in *hexadecimal* (base 16). In this book a number written with a dollar sign, like **`$FF20`**, is hexadecimal. Section 1.6 explains what that means.

This book's Hydra

Every example in this book was typed into a Hydra running HydraOS 1.0, and the answers shown are the ones it gave. It was always the same Hydra:

	THIS BOOK'S HYDRA HAS
Memory	Three RAM modules, on one or two memory cards (so `free` shows 384 KB for each task)
SD card	HydraOS's sample card, in the first reader: `/sd/0`
Clock chip	None. Its clock starts at midnight on January 1, 2000 each time it's switched on
Vera X video card	None, except in Chapter 16 and where an example says it has one
Processor speed	3.58 MHz

Your Hydra may not be the same, and then a few of its answers won't be either. That's not a mistake. Here's what to expect:

YOUR HYDRA	WHAT'S DIFFERENT FROM THE BOOK
The web emulator, as it comes	It has a Vera X and a clock chip. The start-up messages have a few more lines, `date` shows today's date, and `ps` lists one more task, `vid`.
The terminal emulator, as it comes	It has two RAM modules, so `free` shows 256 KB for each task.
A real Hydra with one RAM module	`free` shows 128 KB for each task.
A real Hydra with no SD card reader	There's no `/sd/0`. Read section 6.7 before you save anything.
Any Hydra	The numbers that count time (the `CPU` column of `ps`, the seconds of `date`) are never quite the same twice.

To make an emulator into this book's Hydra exactly: in the web emulator, press **Setup...**, clear the boxes for the Vera X card and the clock chip, and press **Power cycle with these**; in the terminal emulator, add `--modules 3` when you start it (section 2.4). You don't have to. Nothing in this book depends on it.

If your Hydra's answer is different in some other way, check that you typed exactly what the book shows. Spaces matter!

Boxes

IN THIS CHAPTER: Each chapter starts with one of these: what you'll be able to do by its end, and what you need to know, or to have, before you begin.

NOTE: A note gives you extra information that is useful but not essential.

TIP: A tip is a shortcut, or a way to stay out of trouble.

TRY THIS: A "try this" box suggests something to do at the keyboard right now.

CAUTION: A caution warns you about something that could hurt you, damage a part, or lose your work. Always read these, especially in the building chapters.

HOW IT WORKS: These boxes explain what is going on inside, for readers who want to know more. You can skip them the first time through.

About the Hydra-16

The Hydra-16 was designed by Daniel Struthers; his name is on the circuit board, next to the board's revision number, **1.8D**. This book describes revision 1.8D of the board and version 1.0 of its operating system, **HydraOS**.

Contents

About This Book 2

PART ONE: MEET THE HYDRA

1 Welcome to the Hydra-16 15
- 1.1 What is a computer? 15
- 1.2 The parts of a computer 15
- 1.3 Meet the Hydra-16 16
- 1.4 A tour of the board 17
- 1.5 HydraOS, the Hydra's software 19
- 1.6 Decimal, binary and hexadecimal 19
- 1.7 What you need 23
- 1.8 Where to go next 23

2 The Hydra on Your PC 24
- 2.1 The emulator in a web browser 24
- 2.2 What the emulator can and can't do 27
- 2.3 Getting the Hydra's files 28
- 2.4 The emulator in a terminal window 28
- 2.5 Sharing files with your PC 30
- 2.6 SD card images 30
- 2.7 Building HydraOS yourself 31

3 Tools, Parts and Soldering 32
- 3.1 Safety first 33
- 3.2 The tools you need 33
- 3.3 Static electricity 34
- 3.4 Knowing your parts 34
- 3.5 How to solder 37
- 3.6 Good joints and bad joints 38
- 3.7 Soldering surface-mount chips 39
- 3.8 Cleaning up 39
- 3.9 Getting a board and the parts 40

4 Building the Hydra, Step by Step 41
- 4.1 Getting ready 41
- 4.2 Soldering the main board 41
- 4.3 The first power test 49
- 4.4 Programming the ROM chips 49
- 4.5 Putting in the chips 50
- 4.6 Setting the jumpers 51
- 4.7 The memory card 52
- 4.8 Power, switch, reset and light 55
- 4.9 Ready to go 56

5 Switching On 57
- 5.1 Talking to the Hydra 57
- 5.2 A terminal program 58
- 5.3 Switching on 59
- 5.4 Is it all there? 60
- 5.5 A faster line 61
- 5.6 Switching off 61
- 5.7 When something goes wrong 62

PART TWO: INSIDE HYDRAOS

6 First Steps 65
6.1 The prompt 65
6.2 Your first commands 65
6.3 Typing, and fixing what you typed 66
6.4 When the Hydra doesn't understand 67
6.5 Looking around 68
6.6 Making a file 69
6.7 Keeping your work 71
6.8 Running programs 72
6.9 A taste of three languages 73
6.10 Finding help 74
6.11 Stopping for the day 75
6.12 What you can do now 75

7 How the Hydra Starts 76
7.1 Waking up 76
7.2 The memory map 76
7.3 Tasks: sixteen computers in one 78
7.4 Step by step 79
7.5 What's running now 82

8 The Shells 84
8.1 Two shells in one 84
8.2 Commands 84
8.3 rc: words and quotes 85
8.4 Variables 85
8.5 Joining words 86
8.6 Input and output 86
8.7 Pipes 87
8.8 A command's output as words 88
8.9 Success and failure 88
8.10 Patterns 88
8.11 Decisions and loops 88
8.12 Functions 89
8.13 Scripts 89
8.14 rc's built-in commands 90
8.15 HyForth's shell words 91
8.16 Choosing your shell 92

9 Files and Namespaces 93
9.1 Files and directories 93
9.2 Looking at files 93
9.3 The disks 94
9.4 Devices as files 95
9.5 Namespaces 96
9.6 The Hydra's tree 96
9.7 Unions 98
9.8 Your own namespace 99
9.9 /pc: a folder on your PC 100

10 The Console and Its Windows 101
10.1 Windows on a terminal 101
10.2 A new window 102
10.3 Groups 103
10.4 Side by side 104
10.5 Scrolling back 105
10.6 Holding the output 105
10.7 Bars, headings and footers 105
10.8 A window's files 107

11 Commands from A to Z 108
as: the assembler 108
basic: HyBASIC 108
calc: the calculator 109
cat: show files 109
cmp: compare two files 110
cp: copy files 110
date: the date and time 110
db: the debugger 111
df: free disk space 111
dis: the disassembler 111
du: disk usage 111
echo: print words 112
ed: the line editor 112
edit: the screen editor 113
forth: HyForth 115
free: memory 115
fsck: check a disk 116
grep: find lines 116
head: the first lines 117
hello: say hello 117
hwtest: the hardware test 117
hylang 117
kill: stop a task 118
label: a disk's name 118
ls: list files 118
mkdir: make directories 118
mkfs: make a file system 119
mods: the ROM's modules 119
more: a screen at a time 120
mv: move or rename 120
new-window: open a window 120
ns: show the namespace 121
play: play a song 121
ps: the tasks 121
pwd: where you are 122
rc: the shell 122
rm: remove files 122
rmdir: remove directories 123
scom: a song 123
slay: stop a program by its name 123
sleep: wait 123
sort: sort lines 123
tail: the last lines 124
tee: copy to a file too 124
top: the busiest tasks 124
touch: make a file, or change its time 125
uniq: remove repeats 125
wc: count lines, words and bytes 126
xd: hex dump 126
xmodem: send and receive files 126
The system's own programs 126
The shell's own commands 127

PART THREE: DEVICES

12 How Drivers Work 130
12.1 Devices are files 130

12.2 Drivers are servers 130
12.3 Device names 131
12.4 Control files 132
12.5 When it goes wrong 132
12.6 Claims 133

13 The Console and the Terminal 134
13.1 The console's files 134
13.2 Reading lines 134
13.3 Raw mode and keys 135
13.4 consctl: the window's settings 135
13.5 wctl: the windows 136
13.6 The serial port 137
13.7 The terminal 138
13.8 /pc 142

14 Disks and HydraFS 143
14.1 Three devices 143
14.2 SD cards 143
14.3 The RAM disks and the ROM disk 144
14.4 The disks' control files 145
14.5 HydraFS 146
14.6 Checking a disk 146
14.7 Moving files to and from a PC 147

15 Sound 148
15.1 The sound hardware 148
15.2 Playing notes from the shell 148
15.3 /dev/sndctl: the sound driver's commands 149
15.4 Patches and drums 150
15.5 The other sound files 152
15.6 MML: writing music 153
15.7 play 154
15.8 Recorded sound 155
15.9 Audio In 155

16 The Vera X: Screen, Keyboard and Mouse 156
16.1 The Vera X 156
16.2 The console on the screen 157
16.3 /dev/vid/ctl: the screen's settings 157
16.4 Drawing 158
16.5 The screen's other files 159
16.6 Taking over the chip 160
16.7 Recorded sound: pcm and pcmctl 160
16.8 The mouse 161
16.9 The keyboard 161

17 The Other Devices 163
17.1 The I/O header, J27 163
17.2 /dev/gpio: the pins 163
17.3 /dev/i2c: the I²C bus 164
17.4 /dev/spi: the SPI devices 165
17.5 The clock 165
17.6 null, zero and kmesg 166
17.7 /env: the environment 166
17.8 /proc: the tasks 166
17.9 /dev/seg: shared memory segments 167
17.10 /dev/mod: the ROM's modules 168
17.11 Pipes, and raw memory 168

PART FOUR: PROGRAMMING LANGUAGES

18 Numbers and Bases 170
18.1 Four kinds of number 170
18.2 Functions and precision 171
18.3 Bases 171
18.4 The named bases 172
18.5 Radixes and digits of your own 173
18.6 The modifiers 174
18.7 Writing a number in a base 174
18.8 Fractions in other bases 175
18.9 The base setting 175
18.10 Bits 176
18.11 When numbers go wrong 176

19 Learning HyForth 177
19.1 Talking to HyForth 177
19.2 The stack 177
19.3 Arithmetic 177
19.4 Moving things on the stack 179
19.5 Making new words 179
19.6 Printing 180
19.7 True and false 180
19.8 Making decisions 181
19.9 Loops 181
19.10 Variables and constants 182
19.11 Characters and strings 182
19.12 Memory 183
19.13 Numbers in other bases 183
19.14 Words that call themselves 184
19.15 When things go wrong 184
19.16 Keeping your words 185
19.17 Whole programs 185

20 Programming the Hydra in HyForth 188
20.1 Libraries 188
20.2 Running commands 189
20.3 Files 189
20.4 The terminal 190
20.5 The devices 192
20.6 Sound 193
20.7 The screen 194
20.8 Big numbers 195
20.9 The Hydra library 196
20.10 Blocks 196

21 Learning hylang 197
21.1 Starting hylang 197
21.2 Expressions 197
21.3 Values 198
21.4 Names 198
21.5 Functions 199
21.6 Lists, and code as lists 200
21.7 Doing something to every item 201
21.8 Decisions 201
21.9 Loops 201
21.10 Names inside functions 202
21.11 Strings and characters 202
21.12 Hashes 202
21.13 Errors 203
21.14 Files of hylang 204
21.15 Prefixes: writing less 204

21.16 Whole programs 204

22 Programming the Hydra in hylang 207
22.1 Libraries 207
22.2 Files and directories 207
22.3 Programs 208
22.4 Time, memory and the system 209
22.5 The devices 210
22.6 Sound 210
22.7 Drawing 211
22.8 Numbers 211
22.9 hylang as your shell 212

23 Learning HyBASIC 213
23.1 Starting HyBASIC 213
23.2 PRINT 213
23.3 Variables 214
23.4 Programs 214
23.5 INPUT 216
23.6 Arithmetic 216
23.7 Decisions 216
23.8 Loops 217
23.9 Strings 217
23.10 Arrays 218
23.11 DATA and READ 218
23.12 SUBs and FUNCTIONs 218
23.13 Errors 219
23.14 Whole programs 220

24 Programming the Hydra in HyBASIC 222
24.1 Files 222
24.2 The terminal 223
24.3 Other programs 223
24.4 Records 224
24.5 Sound 224
24.6 Graphics 225
24.7 Numbers 226
24.8 Memory and machine code 227
24.9 HyBASIC as your shell 227

25 One Program, Three Languages 228
25.1 The game 228
25.2 In HyBASIC 228
25.3 In hylang 229
25.4 In HyForth 230
25.5 Side by side 231
25.6 How fast? 231
25.7 Choosing 232

PART FIVE: MACHINE LANGUAGE

26 The 65C02 234
26.1 Machine code 234
26.2 Bytes, hex, and negative numbers 234
26.3 The registers 235
26.4 Try it: watching the registers 236
26.5 Memory 237
26.6 Addressing modes 237
26.7 Moving bytes 238
26.8 Arithmetic 239
26.9 Logic, bits and shifts 239

26.10 Comparing, branching and the stack 240
26.11 Interrupts 241
26.12 How fast? 241
26.13 Learning more 242

27 The Assembler 243
27.1 Your first program 243
27.2 A bigger program: hi.s 245
27.3 Inside hi.s 245
27.4 What's on a line 247
27.5 Labels 248
27.6 Numbers and expressions 248
27.7 Constants 249
27.8 Segments: where everything goes 249
27.9 Data 250
27.10 Including files 250
27.11 Macros 250
27.12 Conditions and checks 251
27.13 System calls 252
27.14 Reading a file 253
27.15 More memory: RAM banks 255
27.16 The SDK's libraries 255
27.17 as's options and the labels file 256
27.18 When as finds a mistake 257
27.19 How as works 258
27.20 Assembly language in HyBASIC 259
27.21 Machine code from HyForth 259

28 The Disassembler 261
28.1 Taking a program apart 261
28.2 Names from a labels file 262
28.3 And back again 263
28.4 Code and data 263
28.5 The other options 264
28.6 Inside the Hydra's tools 265
28.7 Disassembling in HyForth 266
28.8 In the debugger 267

29 The Debugger 268
29.1 The commands 268
29.2 Starting 269
29.3 Looking at the program 269
29.4 Stepping 270
29.5 Breakpoints 270
29.6 Until, and next 271
29.7 Changing the program 271
29.8 Finding a bug 272
29.9 Stopping a running program 273
29.10 Debugging a running task 273
29.11 How db works 273
29.12 Making show1.s 273

APPENDICES

A HyForth Reference 276
A.1 How to read the entries 276
A.2 The core 276
A.3 Core Extension (`coreext`) 281
A.4 Exception (`exception`) 282
A.5 File Access (`file`) 282

A.6 Programming-Tools (tools) 283
A.7 Facility (facility) 284
A.8 String (string) 285
A.9 Search-Order (search) 285
A.10 Double-Number (double) 286
A.11 Memory-Allocation, Locals and Block 286
A.12 The shell (shell) 287
A.13 The Hydra (hydra) 288
A.14 Numbers (numbers) 289
A.15 Sound (sound) 290
A.16 Bits, random numbers and the disassembler 290
A.17 The device libraries 291
A.18 Errors 291

B hylang Reference 292
B.1 Syntax 292
B.2 Special forms 293
B.3 Lists 294
B.4 Numbers 295
B.5 Strings and characters 297
B.6 Types 298
B.7 Hashes 298
B.8 Errors, output and input 299
B.9 Files and streams 300
B.10 Buffers 300
B.11 Programs, time and the system 301
B.12 The library 302

C HyBASIC Reference 304
C.1 The rules 304
C.2 The keywords 304
C.3 Examples 309
C.4 Errors 312

D The 65C02's Instructions 313
D.1 The addressing modes 313
D.2 The flags and the cycles 313
D.3 The instructions 314
D.4 The opcodes in order 316

E Characters 318
E.1 The control characters 318
E.2 ASCII 319
E.3 ISO-8859-15 320
E.4 The DEC Special Graphics 321
E.5 The screen's fonts 321

F The Memory and I/O Map 324
F.1 The 64K the 65C02 sees 324
F.2 A task's 32K 324
F.3 The RAM banks 324
F.4 The paged ROM 325
F.5 The I/O ports 325
F.6 The interrupt lines 326
F.7 HydraOS's addresses 327

G Error Codes 328
G.1 HydraOS's errors 328
G.2 In each language 329
G.3 The numbers library's errors 329

H The Emulator and the PC Tools 331
H.1 The terminal emulator's options 331
H.2 The emulator's keys 333
H.3 The monitor 333
H.4 The web emulator 333
H.5 The PC tools 334

I Parts and Board Diagrams 336
I.1 The main board's parts 336
I.2 What each chip does 338
I.3 The memory cards 339
I.4 The board 340

J Glossary 341

K System Calls 347
K.1 The system 347
K.2 Tasks 347
K.3 Memory 349
K.4 Files 350
K.5 Namespaces 351
K.6 The console 351
K.7 The clock 351
K.8 Drivers 351
K.9 Debugging HydraOS 352

Index 353

PART ONE

Meet the Hydra

What the Hydra-16 is, how to run it on your PC, and how to build your own, from a bag of parts to a working computer.

- Chapter 1. Welcome to the Hydra-16
- Chapter 2. The Hydra on Your PC
- Chapter 3. Tools, Parts and Soldering
- Chapter 4. Building the Hydra, Step by Step
- Chapter 5. Switching On

CHAPTER 1

Welcome to the Hydra-16

Before you pick up a soldering iron or type your first command, let's look at what a computer *is*, and at the particular computer you are about to meet. By the end of this chapter you will know the names of the Hydra's main parts, what each one does, and how to read the strange numbers, like `$FF20`, that computer people love.

IN THIS CHAPTER: You'll learn what a computer is made of, meet the Hydra-16's parts and its software, and learn to read binary and hexadecimal numbers. **You need:** nothing. This is where everybody starts.

1.1 What is a computer?

A computer is a machine that **follows instructions**. That's all. A list of instructions is called a **program**, and the computer works through a program one instruction at a time, very quickly, without getting bored or making mistakes. Each instruction is tiny: "add these two numbers," "copy this number over there," "if this number is zero, skip ahead." But a computer can follow millions of them every second, and from millions of tiny steps come games, music, drawings, and everything else computers do.

Every computer, from a pocket calculator to a supercomputer, does the same four jobs:

JOB	WHAT IT MEANS	ON THE HYDRA
Input	Getting information in	The keyboard, files on a card, a mouse
Processing	Working on it	The 65C02 processor chip
Memory	Remembering it	RAM and ROM chips
Output	Showing the results	Text on a screen, sound, files

Think of a computer as a cook in a kitchen. The **program** is the recipe. The **processor** is the cook, who reads one step at a time and does it. The **memory** is the kitchen counter and the recipe book. And **input** and **output** are the doors the groceries come in through and the plates go out through.

1.2 The parts of a computer

Open up almost any computer and you will find the same kinds of parts. The Hydra has all of them, and on the Hydra they are big enough to see and touch.

The processor

The **processor**, or **CPU** (central processing unit), is the chip that follows the instructions. The Hydra's processor is a **W65C02S**, made by the Western Design Center. It is a modern version of the 6502, the processor that powered many famous home computers and game machines of the late 1970s and 1980s. The 65C02 is small and simple enough that a person can learn every single instruction it knows (there are about seventy), which is one of the reasons the Hydra uses it. Part Five of this book teaches you to program it directly.

Memory

A computer needs somewhere to keep its programs and the information they work on. That's **memory**, and it comes in two kinds.

RAM (random-access memory) is like a whiteboard: the computer can write on it, read it back, and wipe it clean, as often as it likes. RAM forgets everything when the power goes off.

ROM (read-only memory) is like a printed book: the computer can read it, but not change it, and it remembers everything even with the power off. The Hydra keeps its operating system, its languages and its tools in ROM, so they are ready the moment you switch on. (The Hydra's ROM chips are actually *flash* memory, a kind of ROM that a special programmer can rewrite. That's how you put the software in them. See Chapter 4.)

Memory is divided into **bytes**. A byte holds one small number, from 0 to 255: enough for one letter of text, one colour, or one piece of a bigger number. Each byte has an **address**, a number that says where it is, just as each house on a street has a number. When the processor wants a byte, it sends out the address, and the memory answers with the byte that lives there.

Input and output

Input and output, often written **I/O**, are how the computer connects to the world. Special chips handle each kind of connection: one sends and receives text over a cable, another makes sound, another reads the keyboard. To the processor, each I/O chip looks like a few bytes of memory at special addresses. Writing a byte to the sound chip's address plays a note; reading the serial chip's address gives you the key someone just typed.

The clock

Inside every computer is a **clock**, not one that tells the time of day, but a steady beat, like a drummer keeping time for a band. On each beat the processor does one small step of its work. The Hydra's clock beats 3,579,545 times every second, written **3.58 MHz** (megahertz: millions of beats per second). You can make it go twice as fast, or slower, by moving a little jumper on the board.

The bus

All these parts are connected by the **bus**: dozens of copper paths, called *traces*, that run between the chips. The address bus carries addresses from the processor, the data bus carries bytes back and forth, and the control lines say what's happening ("I'm reading," "I'm writing," "now!"). On the Hydra's circuit board you can see the bus as the many thin lines running in parallel between the chips.

1.3 Meet the Hydra-16

The Hydra-16 is a computer you build yourself. It is a single large circuit board, about the size of a sheet of paper, covered in chips that you put in their places one by one. Its name comes from the Hydra of Greek legend, a monster with many heads. The Hydra-16 has sixteen "heads": it can run up to **sixteen programs at the same time**, each in its own private part of the memory, without any of them getting in the others' way. (Four of the sixteen are HydraOS's own, so twelve are yours.)

That ability, called **multitasking**, is something the home computers of the 1980s could not do. A Commodore 64 or an Apple II ran one program at a time. The Hydra is built from the same kind of chips as those machines, but it adds a few clever extra chips that give every program its own memory. That lets its operating system, **HydraOS**, juggle many programs at once, the way modern computers do.

Here is what's inside:

PART	THE HYDRA-16 HAS
Processor	One W65C02S, at 3.58 MHz (or 1.79 or 0.89 MHz)

PART	THE HYDRA-16 HAS
Programs at once	Up to 16 *tasks*, each with its own 32K of memory
RAM	512K for the tasks, 2 MB shared between them, and 2 MB more for each RAM module on the memory cards (up to 15 modules)
ROM	128K for the kernel (the heart of HydraOS), and up to 4 MB for the rest of the system
Serial port	A DE-9 connector for a PC or terminal: the Hydra's keyboard and screen
Sound	A Yamaha YM2151 FM sound chip, with a stereo headphone/line output
Storage	Up to eight SD card readers, and RAM disks
Connectors	Six expansion slots, eight SPI device headers, a 12-pin I/O header (six I/O pins, two handshake pins and the I²C bus)
Video (optional)	The Vera X card: a colour screen, sprites, a keyboard and mouse
Power	A standard ATX computer power supply

NOTE: "K" means 1,024 bytes, and "MB" (megabyte) means 1,024 K, or 1,048,576 bytes. Computers count in powers of two, so 1,024 (that is, 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2) is a "round number" to a computer. The Hydra's 32K for each task is 32,768 bytes.

1.4 A tour of the board

Figure 1-1 shows the Hydra-16's circuit board from above, with its main parts labelled. Every part on the board has a name printed next to it in white letters: a letter for the kind of part and a number. **U** is a chip (integrated circuit), **R** a resistor, **C** a capacitor, **J** a connector, **Y** the crystal. You will use these names all through the building chapters.

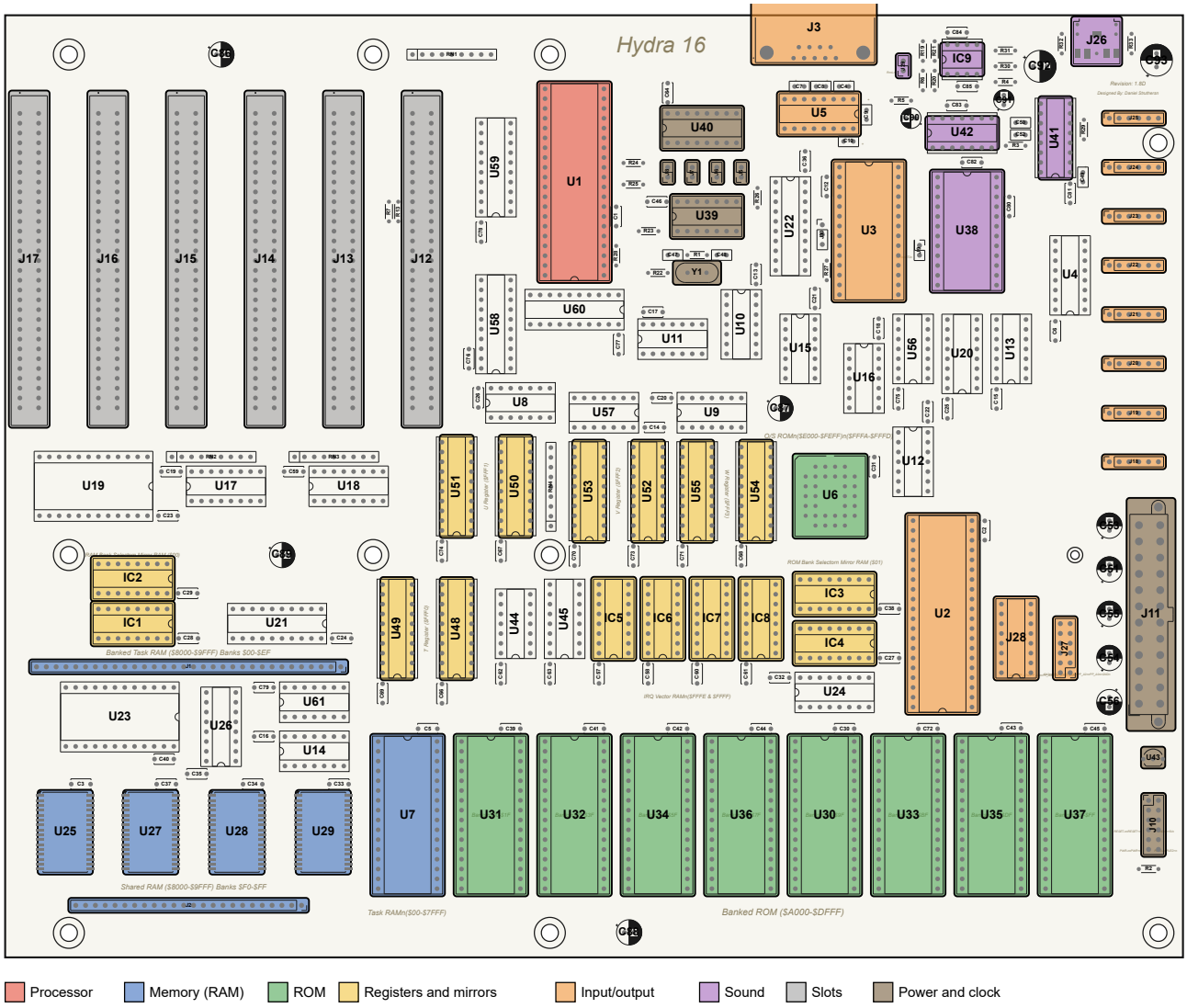


Figure 1-1. The Hydra-16's circuit board and its main parts.

Let's take a walk around the board.

The processor (U1). The big 40-pin chip near the top of the board, next to the expansion slots, is the W65C02S. Everything else on the board is there to help it.

The clock (Y1, U39, U40). The little silver can marked *14.31818MHz* is the **crystal**, which vibrates 14,318,180 times a second. Two chips divide that beat down into the clocks the computer uses. Four jumpers, J5 to J8, choose the processor's speed.

The task RAM (U7). One 512K RAM chip, near the bottom left, holds sixteen separate 32K blocks, one for each task. When the Hydra switches from one program to another, it simply switches which block the processor sees.

The shared RAM (U25, U27, U28, U29). Four small surface-mounted RAM chips along the bottom edge hold 2 MB that any program can use, a piece at a time.

The memory cards (J1, J2). Two long sockets on the left take the **memory cards**. A card holds one or two **RAM modules** of 2 MB each, and every module gives every task 128K more memory of its own.

The kernel ROM (U6). The square chip in the middle of the board, in a socket with pins on all four sides, holds the **kernel**: the heart of HydraOS, the part that runs the other programs.

The paged ROM (U30 to U37). Eight sockets along the bottom right hold up to eight more ROM chips, 512K each. They hold the rest of HydraOS: its drivers, shells, languages and tools, and a whole disk's worth of files.

The registers and mirrors. The rows of smaller chips in the middle of the board are the Hydra's special trick. Four of them, labelled **T**, **U**, **V** and **W Register**, choose which task's memory the processor sees, which shared memory, which interrupt, and which page of the kernel ROM. Others, the **mirror RAMs**, remember which bank of memory each task has chosen. Chapter 7 tells you how they work together.

The VIA (U2). The R65C22 *Versatile Interface Adapter* has two timers and two 8-bit ports. One timer ticks the Hydra's clock for multitasking. One port runs the **SPI bus**, which talks to SD cards and other gadgets through the eight **SPI device headers** (J18 to J25). The other port goes to the **I/O header** (J27), where you can connect switches, lights and I²C sensors.

The serial port (U3, U5, J3). The R65C51 *ACIA* chip sends and receives characters one bit at a time, and the MAX232 chip changes them into the voltages of a standard serial cable. The DE-9 connector at the top, J3, is where your PC plugs in. This cable is how you type to the Hydra and see its answers.

The sound (U38, U41, U42, IC9, J26). The Yamaha **YM2151** is an FM synthesizer, the same kind of chip that made the music in many 1980s arcade games. It has eight voices. The YM3012 turns its numbers into sound waves, and the mixer adds in the sound from slot 0 and the *Audio In* header. Plug headphones or a speaker into the jack, J26.

The expansion slots (J12 to J17). Six long card-edge sockets let you plug in extra boards. **Slot 0** is special: it's the one for the **Vera X**, a video card that gives the Hydra a colour screen, a keyboard and a mouse.

The power (J11, J10, U43). The big 24-pin connector, J11, takes a standard ATX computer power supply. The front-panel header, J10, is for the power switch, the reset button and the power light, and U43 makes sure the computer starts cleanly every time the power comes on or the reset button is pressed.

1.5 HydraOS, the Hydra's software

A computer without software is just a pile of chips. The Hydra's software is called **HydraOS**, and it lives in the ROM chips, so it starts the moment you switch on. HydraOS is an **operating system**: the program that runs the other programs. It decides which program gets the processor next, gives each one its memory, and lets them use the files and devices without fighting over them.

HydraOS gives you:

- **Shells**: programs that read the commands you type and do them. The Hydra greets you with **HyForth**, a shell that understands both commands and the Forth programming language, and also has **rc**, a classic command shell.
- **Files**: a tree of folders and files on SD cards, in ROM, and in RAM disks, all in one tree.
- **Windows**: up to sixteen text windows on one screen, each with its own program, that you switch between with a couple of keys.
- **About fifty commands**: for listing, copying and editing files, watching the programs that are running, calculating, playing music, and much more.
- **Three programming languages**: **HyForth**, **hylang** and **HyBASIC**, plus an **assembler**, a **disassembler** and a **debugger** for programming the 65C02 itself.

Part Two of this book is all about using HydraOS. Part Four teaches the languages.

1.6 Decimal, binary and hexadecimal

Sooner or later every computer book shows you a number like `$FF20` or `01000001`. These are ordinary numbers, written in a different way. This section shows you how to read them. Take it slowly: the rest of the book leans on it, and it is much easier than it looks.

Decimal: the way you already count

Take the number 3,405. You read it without thinking, but look at what each digit is doing:

PLACE	THOUSANDS	HUNDREDS	TENS	ONES
Worth	1,000	100	10	1
Digit	3	4	0	5

Three thousands, four hundreds, no tens and five ones: 3,000 + 400 + 0 + 5. A digit's **place** says what it's worth, and each place is worth *ten times* the one to its right.

That's why there are ten digits, 0 to 9. When you've counted up to 9 in a place you've run out of digits, so you put that place back to 0 and add 1 to the place on its left. That's how 9 is followed by 10, and 99 by 100.

This way of writing numbers is called **decimal**, or **base 10**. There's nothing special about ten, except that people have ten fingers. Any number can be the base, and computers use two others all the time.

Binary: counting with two digits

Everything inside a computer is kept in millions of tiny switches, and a switch can only be *off* or *on*. So a computer has just two digits to count with: **0** for off, and **1** for on. Counting with two digits is called **binary**, or **base 2**.

It works exactly like decimal, except that you run out of digits much sooner. After 0 comes 1, and then you've run out already: so back to 0, and add 1 to the place on the left. That gives `10`, which is two. Then `11`, three; and now both places have run out, so the next number is `100`, four.

DECIMAL	0	1	2	3	4	5	6	7	8
Binary	`0`	`1`	`10`	`11`	`100`	`101`	`110`	`111`	`1000`

In decimal each place is worth ten times the one to its right. In binary each place is worth *twice* the one to its right: 1, 2, 4, 8, 16, 32, 64, 128, and so on.

One binary digit is called a **bit** (short for "binary digit"). Eight bits in a row make a **byte**, and a byte is the size of thing the Hydra's processor works with.

To read a binary number, write the place values over its bits, and add up the ones with a 1 under them:

WORTH	128	64	32	16	8	4	2	1
`01000001`	0	1	0	0	0	0	0	1

64 + 1 = 65. So the byte `01000001` is the number 65. (It is also the code for the letter **A**: a computer keeps letters as numbers too. Appendix E has the whole list.)

To write a number in binary, go the other way. Start at the biggest place value, and at each one ask: does it fit into what's left? If it does, write a 1 and take it away. If it doesn't, write a 0. Here is 200:

WORTH	128	64	32	16	8	4	2	1
Does it fit?	yes	yes	no	no	yes	no	no	no
What's left	72	8	8	8	0	0	0	0
Bit	1	1	0	0	1	0	0	0

128 fits into 200, leaving 72. 64 fits into 72, leaving 8. 32 and 16 don't fit into 8. 8 fits, leaving nothing. So 200 is `11001000`.

The smallest byte is `00000000`, which is 0. The biggest is `11111111`, which is 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255. So a byte can hold any whole number from 0 to 255, which is 256 different values. Each bit you add doubles that:

BITS	HOW MANY VALUES	THE BIGGEST NUMBER
1	2	1
4	16	15
8 (one byte)	256	255
16 (two bytes)	65,536	65,535

Hexadecimal: binary made shorter

Binary is what the computer uses, but it's hard on people. `1111111100100000` is easy to copy wrongly and impossible to say aloud. So programmers write binary numbers in a shorter way, called **hexadecimal**, or **hex**. Hex is **base 16**: each place is worth *sixteen times* the one to its right.

Base 16 needs sixteen digits, and there are only ten. So after 9, hex carries on with letters: **A** is ten, **B** eleven, **C** twelve, **D** thirteen, **E** fourteen and **F** fifteen.

DECIMAL	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Binary	`0000`	`0001`	`0010`	`0011`	`0100`	`0101`	`0110`	`0111`	`1000`	`1001`	`1010`	`1011`	`1100`	`1101`	`1110`	`1111`

Look at the bottom row. Four bits make sixteen different patterns, and there are sixteen hex digits, so **each hex digit stands for exactly four bits**. That's the whole reason hex is used. To turn binary into hex, split it into groups of four bits from the right, and write each group's digit:

BINARY	`1100`	`1000`
Hex	C	8

So `11001000` is **`$C8`**. To turn hex into binary, do the opposite: write the four bits for each digit. A byte is always exactly two hex digits, from **`$00`** to **`$FF`**.

To read a hex number as decimal, use the place values again. In a two-digit number the left digit counts sixteens and the right one counts ones:

HEX	WORKED OUT	DECIMAL
`$C8`	12 × 16 + 8	200 (the same 200 as before)
`$FF`	15 × 16 + 15	255
`$10`	1 × 16 + 0	16 (not ten!)

With four digits, the places are worth 4,096, 256, 16 and 1:

WORTH	4,096	256	16	1
`$FF20`	F (15)	F (15)	2	0

15 × 4,096 + 15 × 256 + 2 × 16 + 0 = 61,440 + 3,840 + 32 = 65,312.

To write a number in hex, divide it by 16. The answer is the left digit and the remainder is the right one: 200 ÷ 16 is 12, remainder 8, and 12 is C, so 200 is `$C8`.

You will rarely need to do these sums. What matters is to recognise hex when you see it, to know that two hex digits are one byte, and to remember a handful of sizes.

How this book writes numbers

- A plain number, like 200, is decimal.
- A number with a **$** in front, like **`$C8`**, is hexadecimal, as it was on many computers of the 1980s. So **`$10`** is sixteen, not ten.
- A binary number is written out as its bits, like `11001000`, and the words around it say that it's binary.

Each of the Hydra's languages has its own way to write a hex or a binary number in a program. Their chapters explain them; here they are side by side, for the number 200:

WHERE	HEX	BINARY
The assembler and HyForth	`$C8`	`%11001000`
HyBASIC	`&HC8`	`&B11001000`
hylang and `calc`	`#xC8`	`#b11001000`

Sizes worth knowing

DECIMAL	HEX	WHAT IT IS
255	`$FF`	The biggest number in a byte
256	`$100`	How many values a byte can have
1,024	`$400`	1K: a kilobyte of memory
8,192	`$2000`	8K: one of the Hydra's RAM banks
65,535	`$FFFF`	The biggest number in 16 bits
65,536	`$10000`	64K: all the memory the processor can see at once

The Hydra numbers each byte of its memory, and that number is the byte's **address**. An address is 16 bits long, so addresses run from **`$0000`** to **`$FFFF`**: always four hex digits.

Let the Hydra do it

Once you have a Hydra running (or the emulator, in the next chapter), the `calc` command converts for you. `-b b` asks for the answer in binary, and `-b x` in hex. A hex number is typed with `#x` in front and a binary one with `#b`, inside single quotes:

```
/> calc -b b 200
11001000
/> calc -b x 200
C8
/> calc '#xFF20'
65312
/> calc '#b01000001'
65
```

> **TRY THIS:** Work these out on paper. What is `00101010` in decimal? What is 100 in binary? What is `$20` in decimal? What is `$7F` in binary? Then check your answers with `calc` when your Hydra is running. (They are 42, `01100100`, 32 and `01111111`.)

Chapter 18 has much more about the Hydra's numbers: other bases, fractions, and numbers too big for any calculator.

1.7 What you need

To use the Hydra on a PC, all you need is a web browser and one file: the emulator's page, `hydra-16.html` (someone may give it to you; or section 2.3 shows how to make it yourself, in a minute). Chapter 2 shows you where it comes from, and how to use it.

To build and use a real Hydra-16, you will need:

- The Hydra-16 kit: the main circuit board, its parts, and at least one memory card. (A kit is planned; until there is one, section 3.9 says how to get a board and the parts.)
- A soldering iron and a few other tools (Chapter 3 lists them).
- An **ATX power supply**, the kind used in desktop PCs, with a 24-pin connector.
- A **power switch** (a toggle switch that stays on), and if you like, a reset button and a power light.
- A PC (Windows, Mac or Linux) and a **USB-to-serial adapter** with a DE-9 plug. It must be an "RS-232" adapter, the kind with a 9-pin connector, not a bare "TTL" cable.
- A programmer for flash ROM chips (such as a TL866 or T48), to put HydraOS into the ROM chips, unless your kit came with them already programmed.
- An **SD card reader module** for the SPI headers, and an SD card. A Hydra works without one, but it's where your files are kept: without it, you can only save your work on your PC, through the serial cable.
- Optional: headphones or speakers, and the Vera X video card.

1.8 Where to go next

- If you want to try the Hydra right away, turn to **Chapter 2** and run it on your PC.
- If you are building a Hydra, read **Chapters 3, 4 and 5** in order, carefully, before you start.
- If your Hydra is already built and working, skip ahead to **Part Two** and learn your way around HydraOS.

CHAPTER 2

The Hydra on Your PC

You don't have to build a Hydra to start using one. The **Hydra emulator** is a program that makes your PC behave exactly like a Hydra-16: it imitates every chip on the board, one clock beat at a time, and runs the very same HydraOS that lives in a real Hydra's ROM chips. Programs you write in the emulator run unchanged on a real Hydra, and the other way round.

IN THIS CHAPTER: You'll run a Hydra on a PC, in a web browser and in a terminal window, choose its hardware, share files with your PC, and make SD card images. **You need:** a PC with a web browser; and for sections 2.3 to 2.7, permission to install Node.js on it.

This chapter shows you two ways to run the emulator: in a **web browser**, the easiest way, with nothing to install; and in a **terminal window** on your PC, which gives you extra tools for exploring. At the end you'll see how to share files between your PC and the Hydra.

FOR TEACHERS: The web emulator is a single file. Make it once, on one computer (section 2.3), then copy it to every computer in the room (or put it on a shared drive), and each student can run a complete Hydra with no installation and no network. Everything the students save stays inside their own browser, on that computer: a student who moves to another computer needs **Save card** and **Load…** to take their work along.

2.1 The emulator in a web browser

The web emulator is one file called **hydra-16.html**. Your teacher may give it to you, or you can make it yourself: section 2.3 shows how, and it takes one command. To start it, open the file in a web browser: Chrome, Edge, Firefox or Safari all work. Double-clicking the file is usually enough.

The Hydra switches on at once. Within a few seconds the **serial console** fills with the Hydra's start-up messages, and then the prompt `/>` appears, which means HydraOS is ready for you to type.

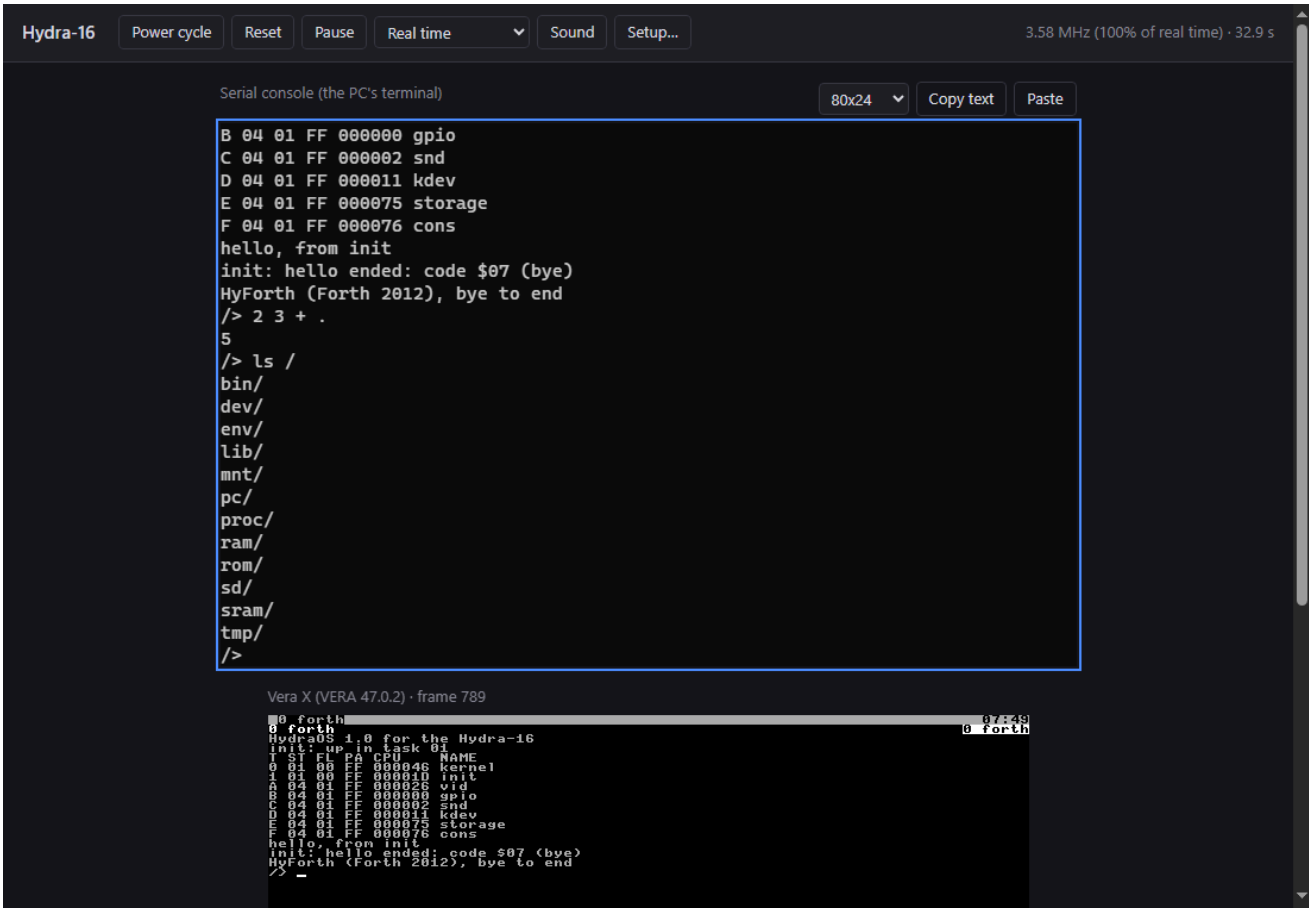


Figure 2-1. The web emulator. The serial console is at the top; the Vera X card's screen is below it.

Click in the console and try your first two commands. Type `2 3 + .` and press RETURN, then type `ls /` and press RETURN:

```
/> 2 3 + .
5
/> ls /
bin/
dev/
env/
lib/
mnt/
pc/
proc/
ram/
rom/
sd/
sram/
tmp/
```

The first command added two and three (Chapter 19 explains why the `+` comes after the numbers). The second listed the folders at the top of the Hydra's file tree. Congratulations: you are using a Hydra.

The buttons

Along the top of the page are the emulator's controls:

BUTTON	WHAT IT DOES
Power cycle	Switches the Hydra off and on again. Everything in RAM is lost, but the SD cards are kept.
Reset	Presses the Hydra's reset button. The computer starts again, but RAM is not cleared.
Pause	Stops the Hydra's clock until you press it again.
Speed	**Real time** runs the Hydra at its true speed, 3.58 MHz. **2×** and **4×** run it faster, and **As fast as it goes** runs it as fast as your PC can.
Sound	Turns on the sound, so you can hear the YM2151 and the Vera X's sound chips.
Setup...	Chooses what is plugged into this Hydra (see below).

At the top right, the page shows the Hydra's clock speed, how close to real time it is running, and how many seconds the Hydra has been switched on.

The serial console

The big black box is the **serial console**. On a real Hydra, it's the terminal program on a PC connected to the Hydra's serial port. Whatever you type here goes to the Hydra, and whatever the Hydra prints appears here.

Above the console:

- The **size** menu sets how many columns and rows the console has. The Hydra notices the change and fits its windows to the new size.
- **Copy text** copies everything in the console, including the lines that have scrolled away, so you can paste it into a document.
- **Paste** types whatever is on your PC's clipboard into the Hydra, as if you typed it. The Hydra asks the page to wait whenever it falls behind (XON/XOFF, section 13.6), so a whole program arrives whole. For programs you'll change often, `/pc` (section 2.5) is handier still. (CTRL-SHIFT-V does the same.)

You can also scroll back through old lines with the mouse wheel, and drag across text to select and copy it.

The Vera X screen

Below the console is the screen of the **Vera X**, the Hydra's optional video card. When the Vera X is plugged in (it is, in the emulator, unless you turn it off), HydraOS shows its windows on this screen as well as on the serial console. Programs can also draw pictures, move sprites and play sounds through the Vera X; Chapter 16 shows you how. If you turn on the Vera X's keyboard and mouse in Setup, click on the screen and you can type and use the mouse there, just as on a real Hydra with a keyboard and mouse plugged in.

Setup

The **Setup...** button opens the setup panel, where you choose the Hydra's hardware:

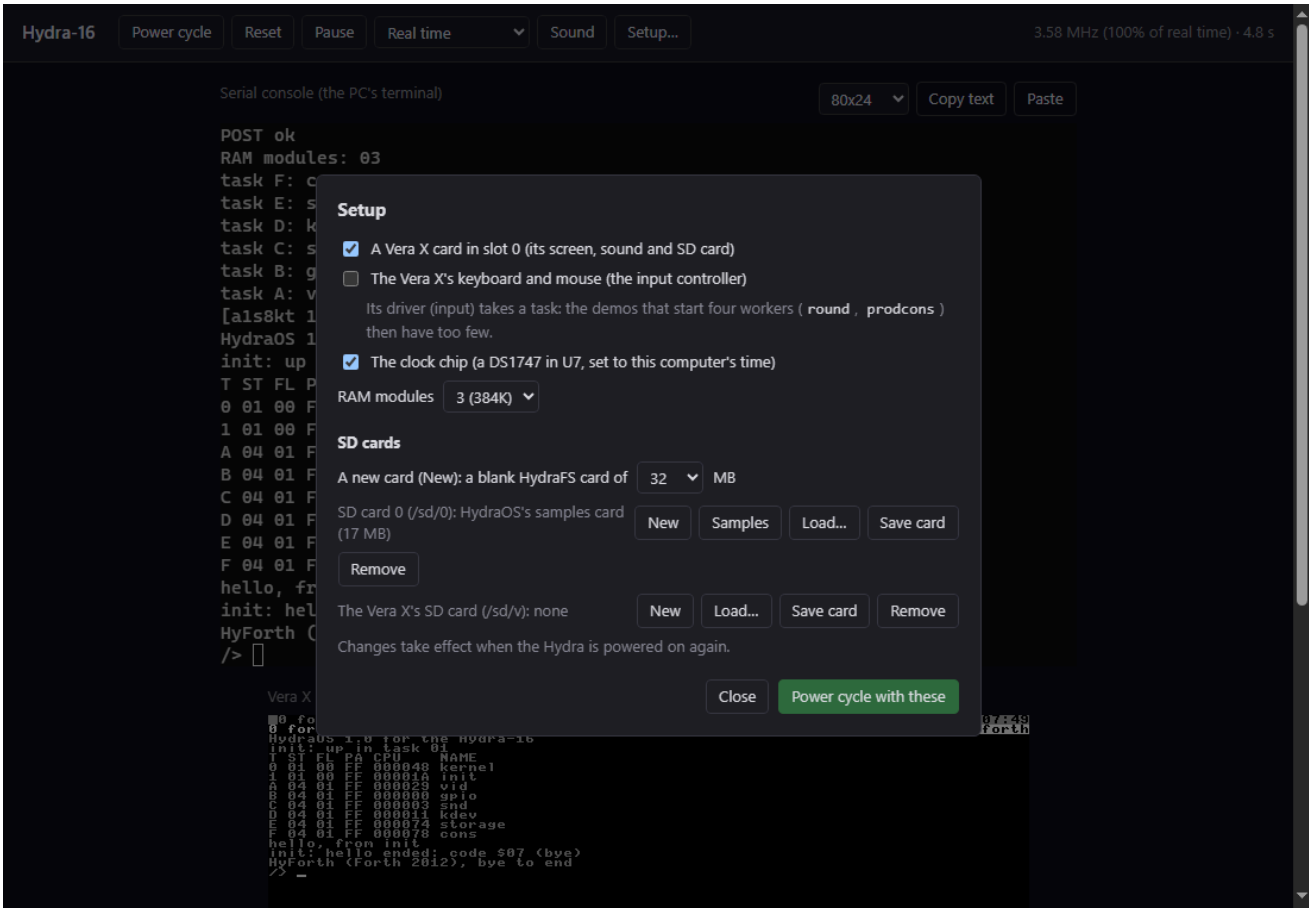


Figure 2-2. The Setup panel.

- **A Vera X card in slot 0** adds the video card: its screen, its sound chips and its own SD card.
- **The Vera X's keyboard and mouse** adds the card's input controller, so the screen takes keys and the mouse. (The `kdev` driver reads them, between its other jobs.)
- **The clock chip** fits a DS1747 clock chip in place of the task RAM chip U7. It keeps the date and time, which the emulator sets from your PC's clock.
- **RAM modules** chooses how many memory cards are plugged in: one, two or three.
- **SD cards**: **SD card 0** appears to the Hydra as `/sd/0`. **New** puts in a new, empty card (of the size you pick), **Samples** puts back HydraOS's sample card (programs, songs and examples), **Load...** loads a card image file from your PC, **Save card** saves the card as an image file, and **Remove** takes the card out. The Vera X's own card slot works the same way, and appears as `/sd/v`.

Changes take effect when the Hydra is switched on again: press **Power cycle with these**.

> **NOTE:** The web emulator keeps your SD cards inside your browser, so your files are still there the next time you open the page on the same computer. To keep a copy somewhere else, or to move your files to another computer or to a real Hydra, use **Save card**.

2.2 What the emulator can and can't do

The emulator imitates the Hydra very closely: the processor, every chip on the board, the serial port, the sound chip, the Vera X, the SD cards, even the exact time each instruction takes. Programs can't tell the difference.

A few things are different from a real board:

- The emulator's "serial cable" never drops a character, and its SD cards never wear out.
- In **As fast as it goes** mode the Hydra's clock runs faster than real time, so music and other timed things play too fast. Use **Real time** for sound.
- The clock chip is only in the web emulator (its Setup). In the terminal emulator, the Hydra has no clock chip, so its clock starts at midnight on January 1, 2000 each time it is switched on.
- The web emulator has no `/pc` folder for sharing files with the PC. The terminal emulator does (section 2.5), and so does a real Hydra with the PC tool (Chapter 5).

2.3 Getting the Hydra's files

To run the emulator in a terminal window, make your own copy of the web emulator, or build HydraOS yourself, you need two things on your PC:

1. **Node.js**, a free program that runs JavaScript programs outside a web browser. Download the "LTS" version from **nodejs.org** and install it. (On a school computer, ask your teacher; you may already have it.)
2. **The Hydra-16's files**: HydraOS, its emulator, the board's design files and this book. They are kept on GitHub at **github.com/danstruthers/hydra-16**. Use the green **Code** button and **Download ZIP**, then unzip the file. Or, if you have **git**, copy the whole project with:

```
git clone https://github.com/danstruthers/hydra-16.git
```

The project's folder holds these folders:

FOLDER	WHAT'S IN IT
`base`	The kernel (the heart of HydraOS) and the core of the emulator
`reborn`	The rest of HydraOS: its drivers, shells, languages and tools; its ROM images; and its emulator
`board`	The Hydra-16's circuit board design, for the KiCad design program
`manual`	This book
`docs`	The reference book: the architecture, every driver's files and commands, the expansion cards, the quirks
`old`	The first HydraOS, kept for the record; nothing in this book needs it

Everything you need to *run* the emulator is already there, including the ROM images in `reborn/bin`. You do not need to build anything.

To make your own web emulator, open a terminal window (on Windows, *Command Prompt* or *PowerShell*; on a Mac or Linux, *Terminal*), go to the project's folder, and type:

```
node reborn/sim/web.js
```

That writes the page to `reborn/obj/web/hydra-16.html`. Copy it anywhere you like. To try it straight away, `node reborn/sim/web.js --serve` serves the page on your PC, at **http://localhost:8017**.

2.4 The emulator in a terminal window

The terminal emulator runs the Hydra with your PC's terminal window as its serial console. It also has extra tools for exploring: a monitor that stops the processor and looks at its memory, and switches for every piece of hardware. In a terminal window, in the project's folder, type:

```
node reborn/sim/run.js -i
```

The `-i` means *interactive*: your terminal becomes the Hydra's serial console, running in real time. After the start-up messages you'll see the `/>` prompt, just as in the browser.

The emulator's keys

While the Hydra is running, CTRL-A is the emulator's own key. Press it, let go, then press one of these:

KEYS	WHAT THEY DO
CTRL-A then X	Quit the emulator
CTRL-A then R	Press the Hydra's reset button
CTRL-A then S	Show the status: each task and what it's doing
CTRL-A then B	Stop the processor and open the **monitor** (below)
CTRL-A then V	Show the Vera X's screen as text
CTRL-A then P	Save the Vera X's screen as a picture (a PNG file)
CTRL-A then H	Show this list
CTRL-A then CTRL-A	Send a CTRL-A to the Hydra

Choosing the hardware

You choose the Hydra's hardware by adding **options** after `-i`. For example, this command starts a Hydra with a Vera X card and its keyboard and mouse, and shows the Vera X's screen in your web browser:

```
node reborn/sim/run.js -i --vera --smc --view
```

Open **http://localhost:8016** in your browser to see the screen, and click on it to use its keyboard and mouse. The most useful options are:

OPTION	WHAT IT DOES
`--vera`	Plugs a Vera X card into slot 0
`--smc`	Adds the Vera X's keyboard and mouse
`--view`	Shows the Vera X's screen in a web browser, at http://localhost:8016
`--sound`	Plays the Hydra's sound through a web browser page
`--modules N`	Plugs in N memory cards (the emulator's normal number is 2)
`--clock 7.15909`	Runs the processor at 7.16 MHz: the emulator can, the board's parts can't (section 4.6)
`--sd FILE`	Puts an SD card image file in the next SD reader
`--pc-dir FOLDER`	Lets the Hydra see a folder on your PC (or an SD card image file) as `/pc` (section 2.5)
`--speed N`	Runs N times faster than real time (0: as fast as possible)

Appendix H lists every option the emulator has.

The monitor

CTRL-A then B stops the Hydra's processor and opens the **monitor**, the emulator's own little command line. The monitor lets you see exactly what the processor is doing, one instruction at a time. Its commands are:

COMMAND	WHAT IT DOES
`c`	Continue: start the processor again
`n` *N*	Run the next *N* instructions (one, if you leave out *N*), showing each one
`r`	Show the processor's registers
`t`	Show the tasks
`m` *ADDR N T*	Show *N* bytes of memory at address *ADDR* (64, if you leave out *N*), as task *T* sees it
`b` *SPEC*	Stop when the processor reaches an address (with no *SPEC*, list the stops)
`d` *N*	Delete stop number *N*
`w` *ADDR*	Stop when anything writes to address *ADDR*
`x`	Quit the emulator

Nothing in this book needs the monitor (Part Five uses the Hydra's own debugger, `db`, which works on a real Hydra too). But it's fun to peek inside: stop the Hydra while it's waiting at the prompt, type `r` to see the registers, and `c` to carry on.

Running without a terminal

The emulator can also run by itself, type a few things, and print what happened. Leave out `-i`, and give it the keys to type with `--input`:

```
node reborn/sim/run.js --input "\w\w2 3 + .\r" --cycles 40000000
```

`\w` waits two million clock beats, and two of them first give HydraOS time to start: keys that arrive before its console is running are lost. `\r` stands for the RETURN key, and `--cycles` says how long to run (40 million clock beats is a little over 11 seconds of the Hydra's time). At the end the emulator prints everything the Hydra printed, followed by a report on its tasks. This is how HydraOS's own tests check that everything works.

2.5 Sharing files with your PC

When you start the terminal emulator with `--pc-dir`, the Hydra can see a folder on your PC as its own folder `/pc`:

```
node reborn/sim/run.js -i --pc-dir C:\Users\you\hydra-files
```

On the Hydra, `ls /pc` lists the files in that folder, and you can read them, run them, copy files into it and change them. It's the easiest way to move programs between the Hydra and your PC: write a program in your favourite PC editor, save it in the folder, and run it on the Hydra. Add `--pc-read-only` if you want the Hydra to be able to read the folder but not change it.

`--pc-dir` takes an SD card image file (section 2.6) in place of a folder, too. Then `/pc` is the card in that image, and what the Hydra writes there goes into the image.

A real Hydra can do the same through its serial cable, with the PC tool. Chapter 5 shows you how.

2.6 SD card images

The Hydra keeps files on SD cards. The emulator's SD cards are **image files** on your PC: a single file that holds everything that would be on a card, block by block.

When you start the terminal emulator without any `--sd` option, it uses HydraOS's own sample card, `reborn/bin/sdcard.img`, and keeps any changes you make only in memory, so the sample card is never changed. To use your own card that keeps its changes, make an image with the `hydrafs.js` tool and give it to the emulator:

```
node reborn/sim/tools/hydrafs.js mkfs mycard.img 32 MYCARD
node reborn/sim/run.js -i --sd mycard.img
```

The first command makes a new, empty 32 MB card called MYCARD. The Hydra sees it as `/sd/0`, and everything you save there stays in `mycard.img`.

`hydrafs.js` can also copy files in and out of an image, list it, and check it:

COMMAND	WHAT IT DOES
`mkfs IMAGE MB [LABEL]`	Makes a new, empty card image of MB megabytes
`info IMAGE`	Shows the card's label, size and free space
`ls IMAGE [-l] [PATH]`	Lists a folder on the card
`put IMAGE FILE PATH`	Copies a file from your PC onto the card
`get IMAGE PATH FILE`	Copies a file from the card to your PC
`mkdir IMAGE PATH`	Makes a folder on the card
`rm IMAGE PATH`	Removes a file, or an empty folder
`import IMAGE FOLDER [PATH]`	Copies a whole folder from your PC onto the card
`check IMAGE`	Checks the card for errors

To put an image onto a real SD card for a real Hydra, use `reborn/sim/tools/sdwrite.js`, or any "disk imager" program. `node reborn/sim/tools/sdwrite.js --list` lists your PC's disks, so you can find the card. Then `node reborn/sim/tools/sdwrite.js IMAGE DISK` writes it. It checks that the disk really is a removable card, and asks you to confirm, before it writes anything.

> **CAUTION:** Writing an image to a disk erases everything that was on the disk. Make very sure you choose the SD card, not your PC's own disk! `sdwrite.js` refuses to write to the disk your PC starts from, but take care anyway.

2.7 Building HydraOS yourself

You never *need* to build HydraOS, because the ROM images come ready-made. But if you want to change HydraOS itself, or to build programs in C, you'll need the **cc65** compiler and assembler (from **cc65.github.io**). Install it, put its `bin` folder on your PATH (or set `CC65_HOME` to the folder you unpacked it in), then in the project's folder type:

```
node reborn/build.js
```

In about twenty seconds, it builds the kernel, every driver, shell, language and tool, and writes new ROM images to `reborn/bin`: `bios.bin` for the kernel ROM, and `prom0.bin`, `prom1.bin` and `prom2.bin` for the paged ROM chips. The emulator uses the new images the next time you start it, and Chapter 4 shows you how to put them into a real Hydra's ROM chips.

Now that you can run a Hydra, you can try everything in Parts Two to Five of this book. If you're building a real Hydra, read on: Chapter 3 gets you ready to solder.

CHAPTER 3

Tools, Parts and Soldering

Building a Hydra means soldering about 2,200 connections on the main board, and 240 more on a memory card, one at a time. That sounds like a lot, and it is: this is a big project, not an afternoon's. But each joint takes only a few seconds, more than half of them are in neat rows along the chip sockets, and after the first fifty or so your hands will know what to do. This chapter gets you ready: the tools you need, how to stay safe, how to recognize every kind of part in the kit, and how to make a good solder joint, and fix a bad one.

IN THIS CHAPTER: You'll learn the tools, the safety rules, how to recognize each kind of part and read its value, and how to make, judge and repair a solder joint. **You need:** nothing yet. Read this chapter before you buy tools or open the kit.

How long will it take? At a careful beginner's pace, the soldering alone is eight to ten hours. Fitting the parts, checking your work and cleaning up take about as long again. So plan on **fifteen to twenty hours**, in sessions of an hour or two, and don't hurry: a joint made in a rush is the one you'll be hunting for later.

STAGE (CHAPTER 4)	PARTS	JOINTS
Step 1: the surface-mount RAM chips (often done for you)	4	128
Step 2: the resistors and resistor networks	27	82
Step 3: the ceramic capacitors	78	156
Step 4: the chip sockets	64	1,282
Step 5: the crystal, the reset chip, the pin headers and sockets	21	148
Step 6: the electrolytic capacitors	13	26
Step 7: the connectors	9	412
The main board	**216**	**2,234**
A memory card (section 4.7)	13	240

FOR TEACHERS: A Hydra is a term's project for a small group, not a lesson's. Before students touch the Hydra's board, have each one practise on a scrap circuit board or an inexpensive "learn to solder" kit until their joints look like the good one in Figure 3-3. The *Educator's Course Guide* has a plan for sharing the build among a group, and a safety policy to adapt; the *Student Companion* has a build log with a box to tick for every part.

3.1 Safety first

> **CAUTION:** A soldering iron's tip is about 350 °C (660 °F), hot enough to burn you badly in an instant. Always put the iron back in its stand when you aren't using it, never touch the metal parts, and never leave it on when you walk away. Younger builders should work with an adult nearby.

Follow these rules every time you solder:

- **Protect your eyes.** Wear safety glasses. When you clip wire leads, they can fly off fast; hold the end of the lead as you cut it, or point it away from people.
- **Don't breathe the smoke.** The smoke comes from the *flux* inside the solder. Work in a room with fresh air, and use a small fan or a fume extractor to move the smoke away from your face.
- **Wash your hands** when you finish, and don't eat or drink while you solder. This matters most if your solder contains lead.
- **Choose your solder with care.** Solder with lead in it ("63/37" or "60/40") flows more easily, but lead is a poison if it gets into your body. **Lead-free** solder is the safer choice, and the right one for a school or for a young builder. It needs a slightly hotter iron (about 370 °C) and a little more patience.
- **Keep your workspace clear.** Use a heat-proof mat, keep cables away from the hot iron, and keep a damp sponge or brass tip cleaner ready. Tie back long hair, and roll up loose sleeves.
- **Never catch a falling iron.** Step back and let it fall. A mark on the floor is better than a burn on your hand.
- **The power supply stays off and unplugged** whenever you are soldering, inserting chips or changing jumpers.

If you burn yourself, put the burn under cool running water straight away, and keep it there for at least ten minutes. Don't use ice, and don't put cream or butter on it. Tell an adult. A burn bigger than a coin, or one that blisters, needs a doctor or a nurse to look at it.

3.2 The tools you need

TOOL	WHY YOU NEED IT
Soldering iron with temperature control (about 40 to 60 watts) and a fine conical or small chisel tip	To heat the joints. A cheap iron with no temperature control makes the job much harder.
Iron stand with a sponge or brass tip cleaner	To hold the hot iron safely, and to keep its tip clean
Solder: thin (0.5 to 0.8 mm) rosin-core solder	The metal that makes the joints. Lead-free solder is safer; "63/37" tin-lead solder is easier to use
Flux pen	Extra flux helps solder flow, and is essential for the surface-mount chips
Desoldering braid (solder wick) and/or a **solder sucker**	To remove solder when you make a mistake
Flush cutters	To trim the wire leads after soldering
Needle-nose pliers	To bend leads and hold parts
Tweezers	To place the small surface-mount chips
Magnifier, or a magnifying lamp	To inspect your joints
Multimeter	To check for short circuits and measure voltages
Anti-static wrist strap	To protect the chips from static electricity
IC insertion and extraction tools (or careful fingers)	To put chips into their sockets, and take them out

TOOL	WHY YOU NEED IT
Isopropyl alcohol and an old toothbrush	To clean flux off the board when you're done
Safety glasses	To protect your eyes

3.3 Static electricity

When you walk across a carpet and get a shock from a doorknob, that's static electricity: a spark of thousands of volts. You can feel a spark of about 3,000 volts. A spark too small to feel can still destroy a chip. The Hydra's processor, its memory chips and its input and output chips are made with **CMOS** technology (the "C" in 65C02), which uses very little power but is easily damaged by static. Most of the small logic chips (the ones whose names start with 74F or 74LS) are an older, tougher kind (the few marked 74ACT are CMOS too), but it's simplest, and safest, to treat every chip the same careful way.

To protect the chips:

- Wear an **anti-static wrist strap** clipped to a grounded metal object, or touch a grounded metal object, such as the metal case of a plugged-in PC, before you handle chips.
- Keep chips in their **anti-static foam or tubes** until you are ready to put them in.
- Hold chips by their plastic bodies, not their pins.
- Don't work on a carpet, and avoid wool sweaters.

3.4 Knowing your parts

Every part in the kit has a **reference** printed on the circuit board where it goes: **R** for resistors, **C** for capacitors, **U** and **IC** for chips (integrated circuits), **J** for connectors, **Y** for the crystal, and **RN** for resistor networks. Appendix I lists every part, its reference, and its value.

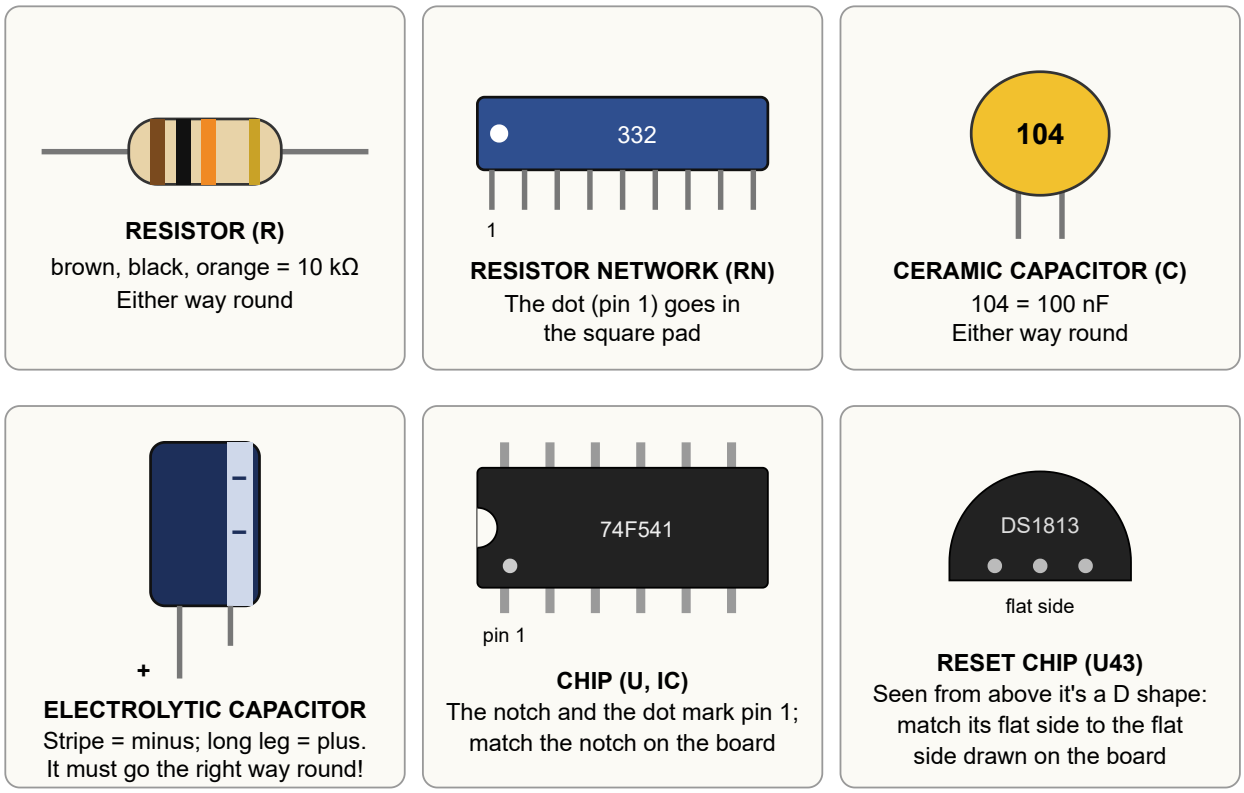


Figure 3-1. The kinds of parts in the Hydra kit, and how to tell which way round they go.

Resistors

A **resistor** limits how much electricity flows through a wire. Resistors are small tan or blue cylinders with a wire at each end, and coloured **bands** that tell you their value in **ohms** (Ω). Resistors can go in either way round.

Read the bands from the end where they are bunched up. The first two bands are digits, and the third says how many zeros to add. The fourth band (gold or silver), if there is one, is the *tolerance*, how accurate the value is. Some resistors have five bands: three digits, then the zeros, then the tolerance.

COLOUR	BLACK	BROWN	RED	ORANGE	YELLOW	GREEN	BLUE	VIOLET	GREY	WHITE
Digit	0	1	2	3	4	5	6	7	8	9

For example, **brown, black, orange** is 1, 0 and three zeros: 10,000 ohms, or **10 kΩ** ("10K").

The Hydra's parts list writes resistor values in a three-digit code that works the same way: two digits, then the number of zeros. Here are all the resistor values in the kit:

CODE	VALUE	BANDS	PARTS
220	22 Ω	red, red, black	R23 to R27
330	33 Ω	orange, orange, black	R30, R31
101	100 Ω	brown, black, brown	R3
331	330 Ω	orange, orange, brown	R22
471	470 Ω	yellow, violet, brown	R2
103	10 kΩ	brown, black, orange	R4 to R7, R13, R19 to R21, R28, R32, R33

CODE	VALUE	BANDS	PARTS
105	1 MΩ	brown, black, green	R1
0	0 Ω	one black band	R29 (a "zero-ohm" resistor: just a wire)

TIP: If you're not sure about a resistor's colours, measure it with your multimeter set to ohms (Ω). Do this *before* you solder it in.

Resistor networks

A **resistor network** (RN1 to RN4) is several resistors in one flat package with a row of nine pins. Eight of the resistors share one pin, the *common* pin, marked with a **dot** or a line at one end. **The common pin goes in the square pad** marked "1" on the board. A resistor network put in backwards won't work.

CODE	VALUE	PARTS
332	3.3 kΩ (8 resistors)	RN1, RN2, RN3
103	10 kΩ (8 resistors)	RN4

Ceramic capacitors

A **capacitor** stores a tiny amount of electricity, like a very small, very fast battery. Ceramic capacitors are small, flat, usually yellow, orange or blue discs, with a code printed on them. They can go in either way round.

The code works like the resistor code, but in **picofarads** (pF): two digits and the number of zeros. So **104** is 10 followed by four zeros: 100,000 pF, which is 100 nF or 0.1 µF.

CODE	VALUE	PARTS
330	33 pF	C47, C48 (beside the crystal)
332	3.3 nF	C50, C52 (in the sound circuit)
104	100 nF (0.1 µF)	C1 to C3, C5, C6, C12 to C46, C57 to C64, C66 to C85: 68 of them
105	1 µF	C4, C7 to C10 (around the serial chip)
475	4.7 µF	C49

Most of the 104 capacitors are **decoupling capacitors**. Each chip has one beside it, between its power pins. When the chip suddenly needs a little extra current, its capacitor supplies it instantly, so the chip's power stays steady.

Electrolytic capacitors

Electrolytic capacitors are bigger cylinders, usually black or blue. They store much more electricity than ceramic ones, and most of them **must go in the right way round**. An electrolytic capacitor has a **minus stripe** down one side of its body, and its **longer leg is the plus (+)** side. The board marks the plus side with a "+" and usually shades the minus side of the circle.

CAUTION: An electrolytic capacitor put in backwards can get hot, bulge, or even burst with a pop when the power comes on. Always check its stripe against the board's markings before you solder it.

CODE	VALUE	PARTS	POLARITY
106	10 µF	C90, C91	Yes: stripe to minus
476	47 µF	C86 to C89	Yes: stripe to minus

CODE	VALUE	PARTS	POLARITY
107	100 µF	C51, C53 to C56	Yes: stripe to minus
227 NP	220 µF, non-polarized	C92, C93	**No**: either way round

C92 and C93 are a special **non-polarized** ("NP" or "bipolar") kind, which has no stripe and can go in either way.

Chips

Most of the Hydra's chips come in **DIP** packages (dual in-line package): a black plastic body with two rows of legs. At one end of the body is a **notch** or a small **dot**. The dot marks **pin 1**, and the pins are numbered counter-clockwise from there, looking down on the chip.

On the circuit board, each chip's outline has a matching notch, and its pin 1 has a **square pad**. **The chip's notch must match the board's notch.** A chip in backwards will not work, and may be damaged when the power comes on.

The Hydra's DIP chips don't get soldered in: their **sockets** do. You solder in an empty socket, and plug the chip into it at the end. That way a chip is never hurt by the soldering iron, and you can change a chip easily. The sockets have notches too; match them to the board, so you'll know which way the chip goes later.

Three other kinds of chip packages are in the kit:

- **U6**, the kernel ROM, is a square **PLCC** chip with legs on all four sides. It goes in a square socket. One corner of the chip, and of the socket, is cut off; the corners must match.
- **U25, U27, U28 and U29**, the shared RAM, are **surface-mount** chips (SOP packages): flat chips with short legs that solder directly to pads on the top of the board. Section 3.7 shows you how. (Your kit may have these already soldered in place.)
- **U43**, the reset chip, is a small black **TO-92** package, like a transistor, with three legs and one flat side. Its flat side matches the flat side of its outline on the board.

The crystal

Y1, the crystal, is a small silver can marked **14.31818MHz**. It can go in either way. Don't let the iron rest on its can for long; a crystal doesn't like too much heat.

Connectors

The kit has many kinds of connectors:

- **Pin headers** are rows of square pins: J5 to J9, J10 (the front panel), J27 (the I/O header), J29 and J30. Small **jumpers** (shunts) push onto pairs of pins to connect them.
- **Pin sockets** are rows of little holes: J1 and J2 (for the memory card) and J18 to J25 (the SPI devices).
- **Card-edge sockets**, J12 to J17, are the six long expansion slots.
- **J11** is the 24-pin power connector, **J3** the 9-pin serial connector, and **J26** the audio jack.

All of these go in only one way, because their pins fit only one pattern of holes. But make sure each one sits flat and straight on the board before you solder all its pins.

3.5 How to solder

Solder is a soft metal that melts at about 190 °C (or about 220 °C for lead-free solder). When melted solder flows over a part's lead and the copper pad around its hole, and then cools, it joins them together, both mechanically and electrically. The **flux** inside the solder cleans the metal as it melts, so the solder can stick.

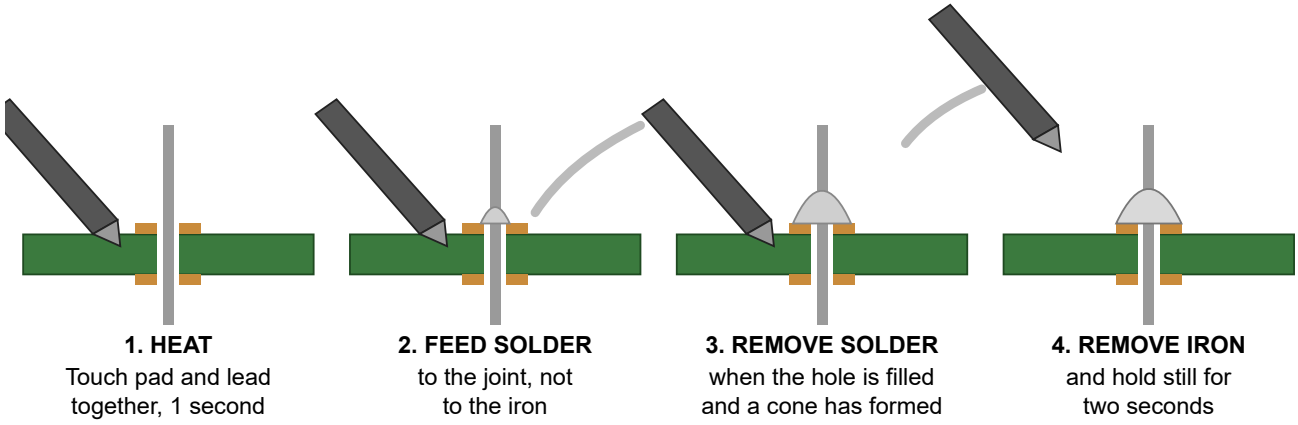


Figure 3-2. Soldering a joint: heat, feed, remove the solder, remove the iron.

To solder a lead:

1. **Prepare.** Set the iron to about 330 °C (370 °C for lead-free solder). Wipe its tip on the sponge, and melt a little solder onto the tip so it's shiny. This is called *tinning* the tip.
2. **Heat.** Touch the tip so it touches **both** the copper pad **and** the lead at the same time. Hold it there for about one second.
3. **Feed.** Touch the end of the solder wire to the joint, on the opposite side from the iron, not to the iron itself. The solder should melt and flow around the lead and across the pad. Feed in just enough to fill the hole and make a little cone; this takes a second or two.
4. **Remove the solder**, then...
5. **Remove the iron.** Lift the tip straight away. Don't move the part until the solder has cooled, which takes about two seconds.

The whole joint takes about three to five seconds. If the solder doesn't flow nicely after five seconds, take the iron away, let everything cool, clean and re-tin the tip, and try again.

3.6 Good joints and bad joints

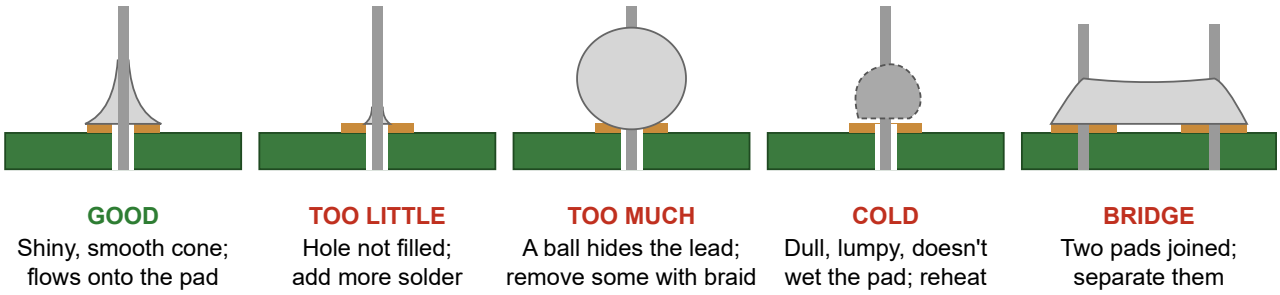


Figure 3-3. A good joint, and the common problems.

A **good joint** is smooth and shiny (lead-free solder is a little duller), shaped like a small cone or volcano, with the lead sticking up out of its middle, and the solder flowing smoothly onto the pad all the way round.

Common problems, and how to fix them:

PROBLEM	WHAT IT LOOKS LIKE	FIX
Too little solder	The hole isn't filled; you can see a gap around the lead	Heat it again and add a little more solder
Too much solder	A big round blob that hides the lead	Remove the extra with desoldering braid
Cold joint	Dull, grey, lumpy or cracked; often a ball that sits on the pad without flowing onto it	Heat it again until the solder flows, adding a touch of flux
Bridge	Solder joining two pads that should be separate	Drag the iron between the pads, or remove the extra with braid
Lifted pad	A copper pad peeling off the board, from too much heat	Ask for help: it can be fixed with a short wire

Desoldering braid is a ribbon of fine copper wire that soaks up melted solder. Lay it on the joint, press the hot iron onto the braid, and as the solder melts it is drawn into the braid. Lift both away together, and cut off the used part of the braid.

After soldering a few joints, **clip the leads**. Use flush cutters to cut each lead just above the top of its solder cone. Hold the end of the lead, or cover it with your hand, so the clipped piece doesn't fly away.

> **TRY THIS:** After every group of parts, inspect every joint with your magnifier, from two or three angles. It takes a few minutes, and finding a bad joint now is much easier than finding it later, when the Hydra doesn't work and you don't know why.

3.7 Soldering surface-mount chips

If your kit's shared RAM chips (U25, U27, U28 and U29) are not already soldered in, you'll need to solder them yourself. Surface-mount chips have legs only 1.27 mm (0.05 inch) apart, which is fiddly, but quite possible with a fine tip, plenty of flux, and patience. Do these **first**, before anything else on the board, while the board is still flat and nothing is in the way.

1. **Find pin 1.** Each chip has a dot or a dimple near pin 1, and the board's outline marks pin 1. Turn the chip so they match.
2. **Tin one pad.** Melt a small blob of solder onto one corner pad of the chip's outline.
3. **Tack the chip.** Hold the chip in place with tweezers, exactly over its pads, with every leg centred on its pad. Touch the iron to the leg over the tinned pad so the solder melts and holds it.
4. **Check the alignment.** Look closely: every leg must sit squarely on its pad, on both sides. If not, melt the tack and nudge the chip until it's right.
5. **Tack the opposite corner** the same way, so the chip can't move.
6. **Solder the rest.** Brush flux along each row of legs. Then either solder each leg separately with a tiny amount of solder, or **drag-solder**: load a little solder on the tip and draw it slowly along the row. The flux makes the solder flow onto each leg and pad, and pull away from the gaps between them.
7. **Remove the bridges.** If two legs are joined, add flux and drag a clean tip, or a piece of braid, across them.
8. **Inspect** every leg with a magnifier. Each one should have a small, smooth fillet of solder joining it to its pad.

3.8 Cleaning up

Flux leaves a sticky brown residue on the board. When you finish a section of the board, scrub it gently with an old toothbrush dipped in isopropyl alcohol, and let it dry completely. A clean board is easier to inspect, and the flux won't collect dust.

Now you know your tools and your parts. In the next chapter you'll put them all together.

3.9 Getting a board and the parts

A Hydra-16 kit is planned. Until there is one, you can build a Hydra from the design itself, and this section says where everything comes from.

The boards. The circuit boards are designed with KiCad (version 9), and the designs are in the `board` folder of the Hydra's files (section 2.3): `hydra-16.kicad_pro` is the main board, `MemoryDaughterCard` holds the two memory cards (through-hole, and surface-mount), and `HydraBusBreakoutCard` the breakout card that a Vera X plugs into. Boards are made to order from those designs: open a project in KiCad, plot its **Gerber** files, and send them to any circuit-board maker.

The parts. Appendix I lists every part by its place, value and package. The reference book's parts list, `docs/02-hardware/parts.md`, has the same list with the part numbers to order, the sockets to buy, and a section on buying: read it before you order. A few parts need care:

PART	WHAT TO KNOW
The processor, U1	Buy WDC's current part, the W65C02S6TPG-14: it's still made
The VIA and the serial chip, U2 and U3	The Rockwell R65C22 and R65C51 are long out of production. WDC's W65C22N is a drop-in replacement for the VIA; its W65C51N works for the serial chip, but HydraOS must be built for it (section 4.5)
The RAM chips	Hitachi's HM628512 is surplus now; Alliance's AS6C4008 is current and fits the same socket
The 74LS219s, IC1 to IC8	Out of production, but easy to find as surplus. The 74LS189 looks the same and **won't** do
The sound chips, U38 and U41	Fake YM2151s are common: buy from a seller with a record, and let the hardware test check them (section 5.7). Or fit an **OPM2151**, a modern pin-compatible replacement, with its own DAC board in U41's socket
The logic chips	Buy the families Appendix I names (74F, 74LS, 74ACT); don't substitute 74HC or 74HCT parts, which are slower
The ROM chips	Three SST39SF040 and one SST39SF010A, all current; and a programmer with a PLCC-32 adapter (section 4.4)

The Hydra works without its sound chips, without an SD card reader and without a Vera X: if a part is hard to find, build without it and add it later.

CHAPTER 4

Building the Hydra, Step by Step

This is the chapter where your Hydra comes to life. You'll fit the parts in a careful order, test the board before any chips go in, program the ROM chips, put the chips in their sockets, build the memory card, and connect the power. Read the whole chapter once before you start, then come back and follow it step by step.

IN THIS CHAPTER: You'll solder the main board in eight steps, test its power, program the ROM chips, fit the chips, set the jumpers, build a memory card, and wire the power. **You need:** all of Chapter 3; the kit and the tools; and fifteen to twenty hours, a little at a time.

CAUTION: Never solder, insert chips, or change jumpers while the power supply is plugged in. Unplug it first, every time.

4.1 Getting ready

Work in order. The steps go from the lowest parts (the ones that sit closest to the board) to the tallest. If you fit a tall part first, it gets in the way of the iron when you solder the small parts around it. When you put a part in, turn the board over, and solder it, the board can rest on its taller parts... so do the short ones first.

Hold parts in place. When you turn the board over to solder, parts can fall out. Bend their leads outward a little on the back of the board, or hold them with a piece of masking tape, or press the board face-down onto a piece of foam so the parts are held against it.

Solder one pin, then check. For anything with many pins (sockets, headers, connectors), solder **one** pin at a corner first. Then look at the part from the side: is it flat on the board, and straight? If not, melt that one joint and push the part into place. Only when it's right, solder the opposite corner, and then all the rest.

Tick them off. Appendix I has a complete parts list with a box beside each part. Tick each box as you fit the part. It's the best way to make sure nothing is forgotten.

Check the kit. Before you start, sort the parts into groups (resistors, ceramic capacitors, electrolytic capacitors, sockets, chips, connectors) and check them against the parts list. Leave the chips in their anti-static foam until section 4.5.

4.2 Soldering the main board

Each step below has a drawing of the board with the step's parts coloured, so you can find their places. Every part's place is also marked on the board itself, with its reference (like **R22**) printed beside it.

Step 1: The shared RAM chips

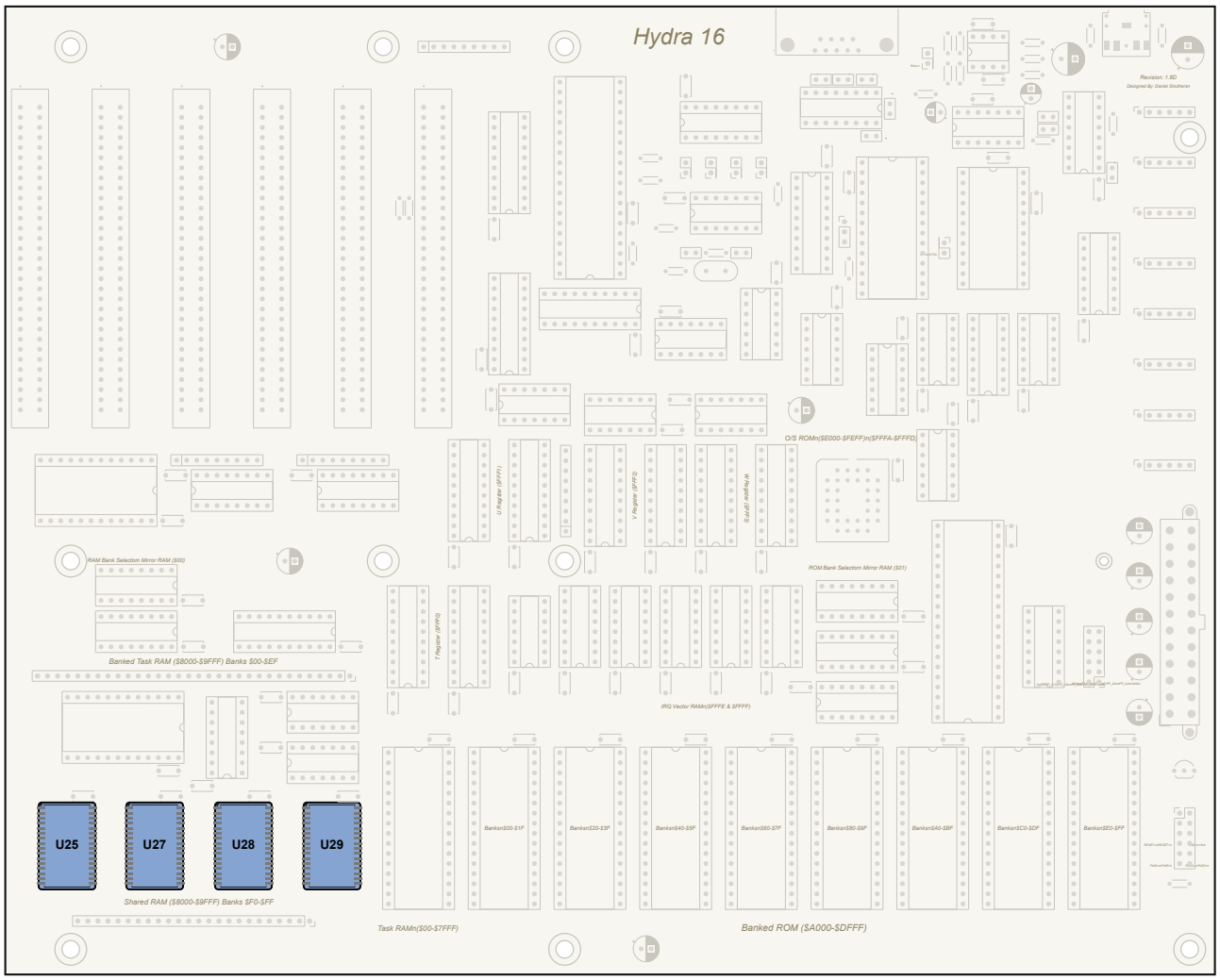


Figure 4-1. Step 1: the four surface-mount RAM chips.

If your kit's shared RAM chips, **U25, U27, U28 and U29** (HM628512 chips in flat SOP packages), are already soldered in, skip this step. Otherwise, solder them now, before anything else, following section 3.7. Each chip's pin 1 matches the dot or bevelled corner on its outline. Inspect every leg with a magnifier before you go on.

Step 2: The resistors and resistor networks

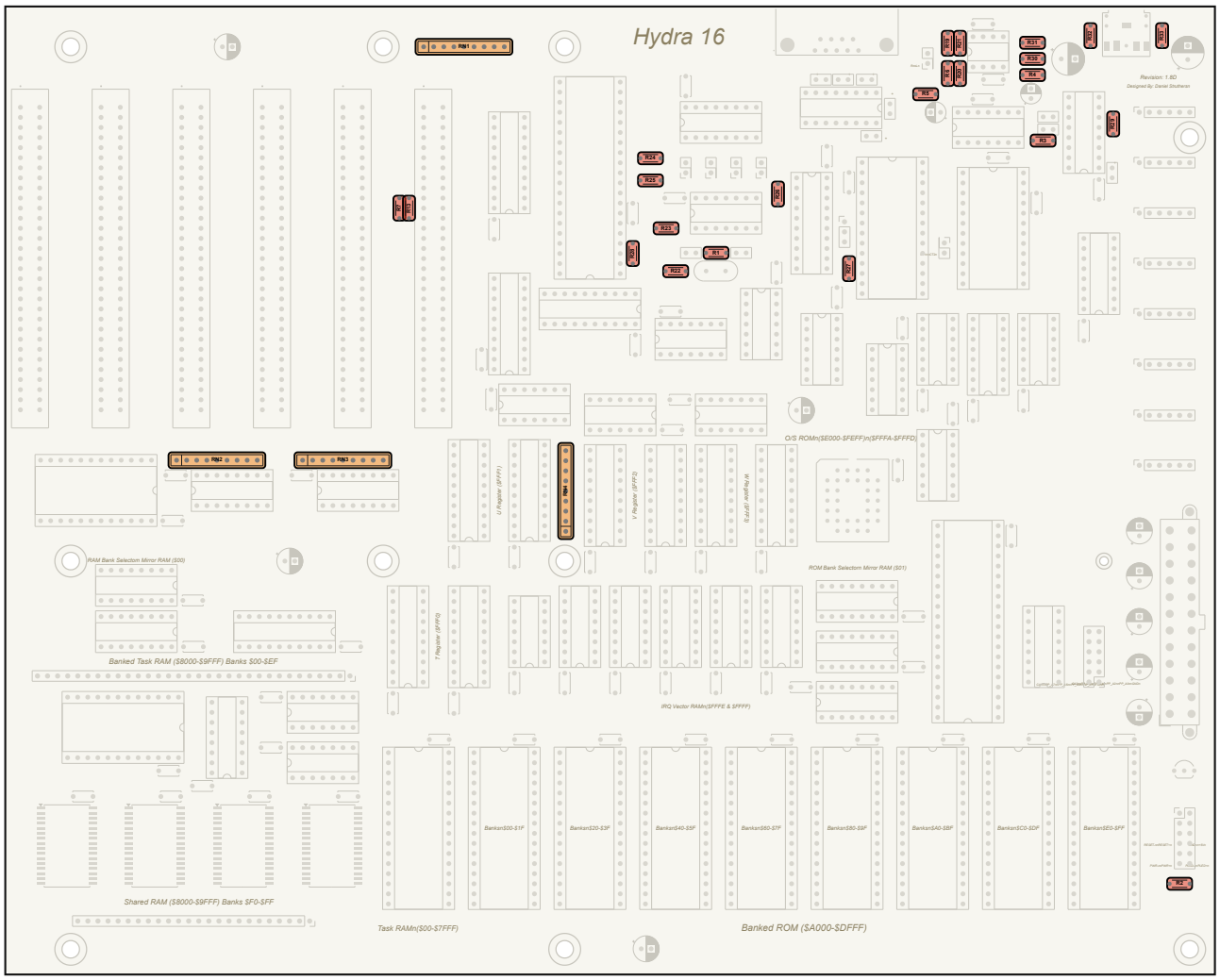


Figure 4-2. Step 2: the resistors (red) and resistor networks (orange).

Bend each resistor's leads at right angles, close to its body, so it fits its two holes, and push it down flat against the board. Resistors can go either way round, but it's neat to point all their colour bands the same way.

VALUE	CODE	PARTS
22 Ω	220	R23, R24, R25, R26, R27
33 Ω	330	R30, R31
100 Ω	101	R3
330 Ω	331	R22
470 Ω	471	R2
10 kΩ	103	R4, R5, R6, R7, R13, R19, R20, R21, R28, R32, R33
1 MΩ	105	R1
0 Ω	0	R29

Then fit the four **resistor networks**. Their dotted end, the *common pin*, goes in the **square pad**:

VALUE	CODE	PARTS
3.3 kΩ	332	RN1, RN2, RN3
10 kΩ	103	RN4

Step 3: The ceramic capacitors

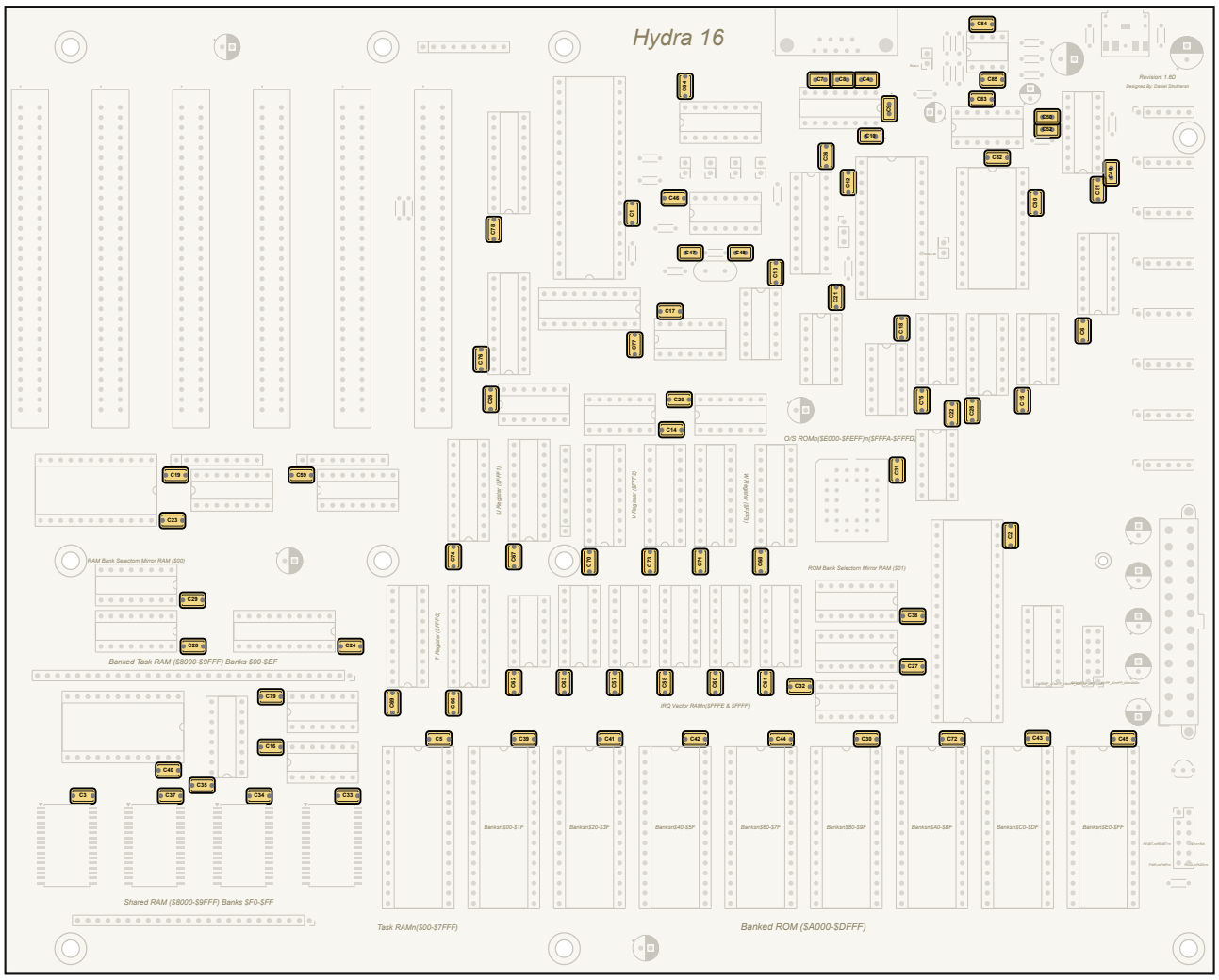


Figure 4-3. Step 3: the ceramic capacitors.

There are 78 small ceramic capacitors, and they can go either way round. Most are 104s (0.1 µF), one beside each chip. Push each one down as far as its leads allow.

VALUE	CODE	PARTS
33 pF	330	C47, C48
3.3 nF	332	C50, C52
100 nF	104	C1, C2, C3, C5, C6, C12 to C46, C57 to C64, C66 to C85 (68 in all)
1 µF	105	C4, C7, C8, C9, C10
4.7 µF	475	C49

> **NOTE:** There is no C11 or C65 on the board, and no R8 to R12 or R14 to R18. Those numbers aren't used.

Step 4: The chip sockets

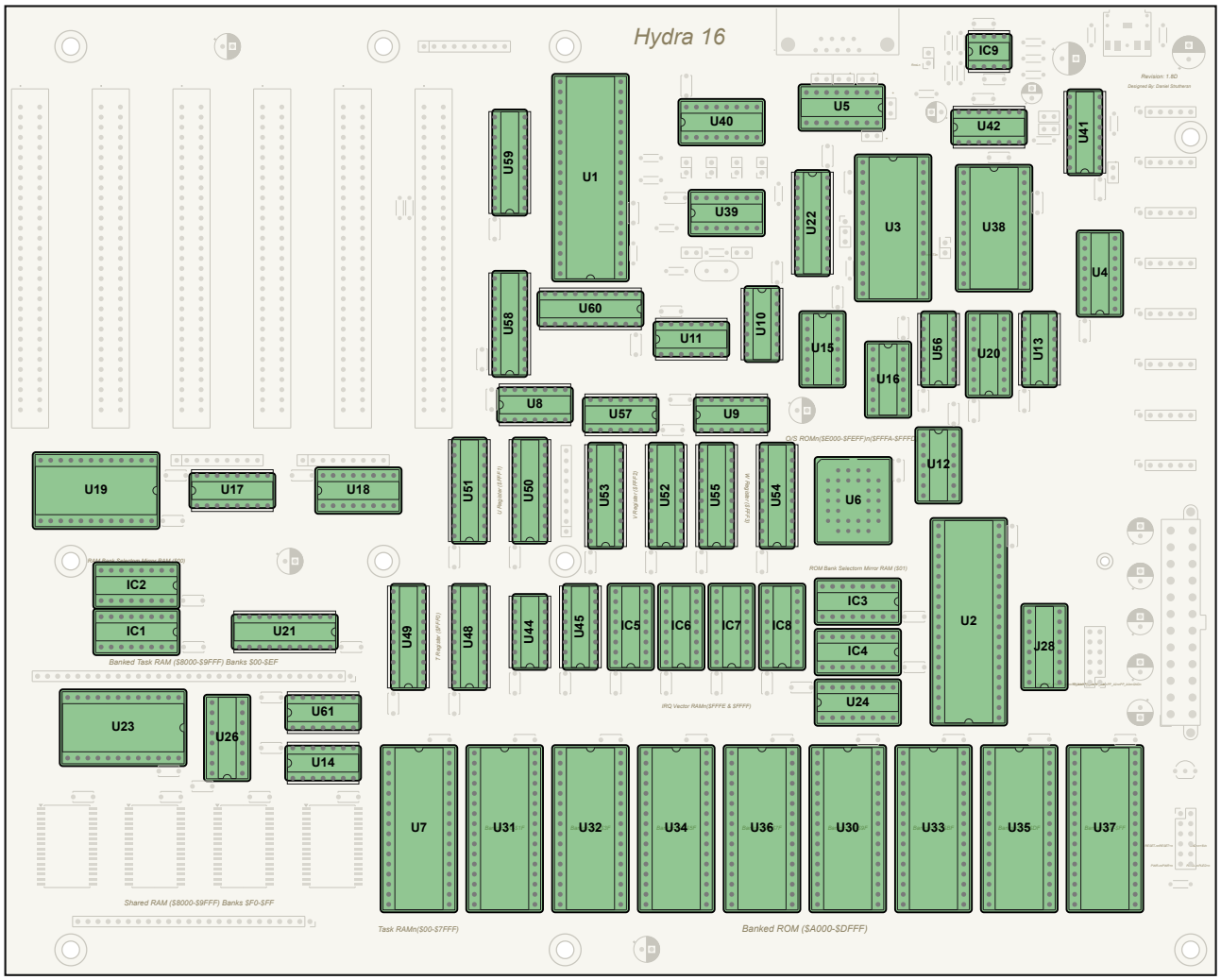


Figure 4-4. Step 4: the chip sockets.

Now the sockets. **Match each socket's notch to the notch on the board's outline.** The socket doesn't care which way it goes in, but the notch tells you, later, which way the chip goes. Solder two opposite corner pins first, check the socket is flat on the board, then solder the rest.

SOCKET	HOW MANY	PLACES
8 pins	1	IC9
14 pins	15	U8, U9, U10, U11, U12, U13, U14, U15, U16, U39, U42, U44, U56, U57, U61
16 pins	19	IC1 to IC8, U4, U5, U17, U18, U20, U24, U26, U40, U41, U45, J28
20 pins	13	U21, U22, U48, U49, U50, U51, U52, U53, U54, U55, U58, U59, U60
24 pins (wide)	3	U19, U23, U38
28 pins (wide)	1	U3

SOCKET	HOW MANY	PLACES
32 pins (wide)	9	U7, U30 to U37
40 pins (wide)	2	U1, U2
PLCC 32 (square)	1	U6

The **wide** sockets have their two rows of pins 15.24 mm (0.6 inch) apart; the others, 7.62 mm (0.3 inch).

The square **PLCC socket** for U6 has a bevelled corner: match it to the bevelled corner of U6's outline.

Step 5: The crystal, the reset chip, and the headers

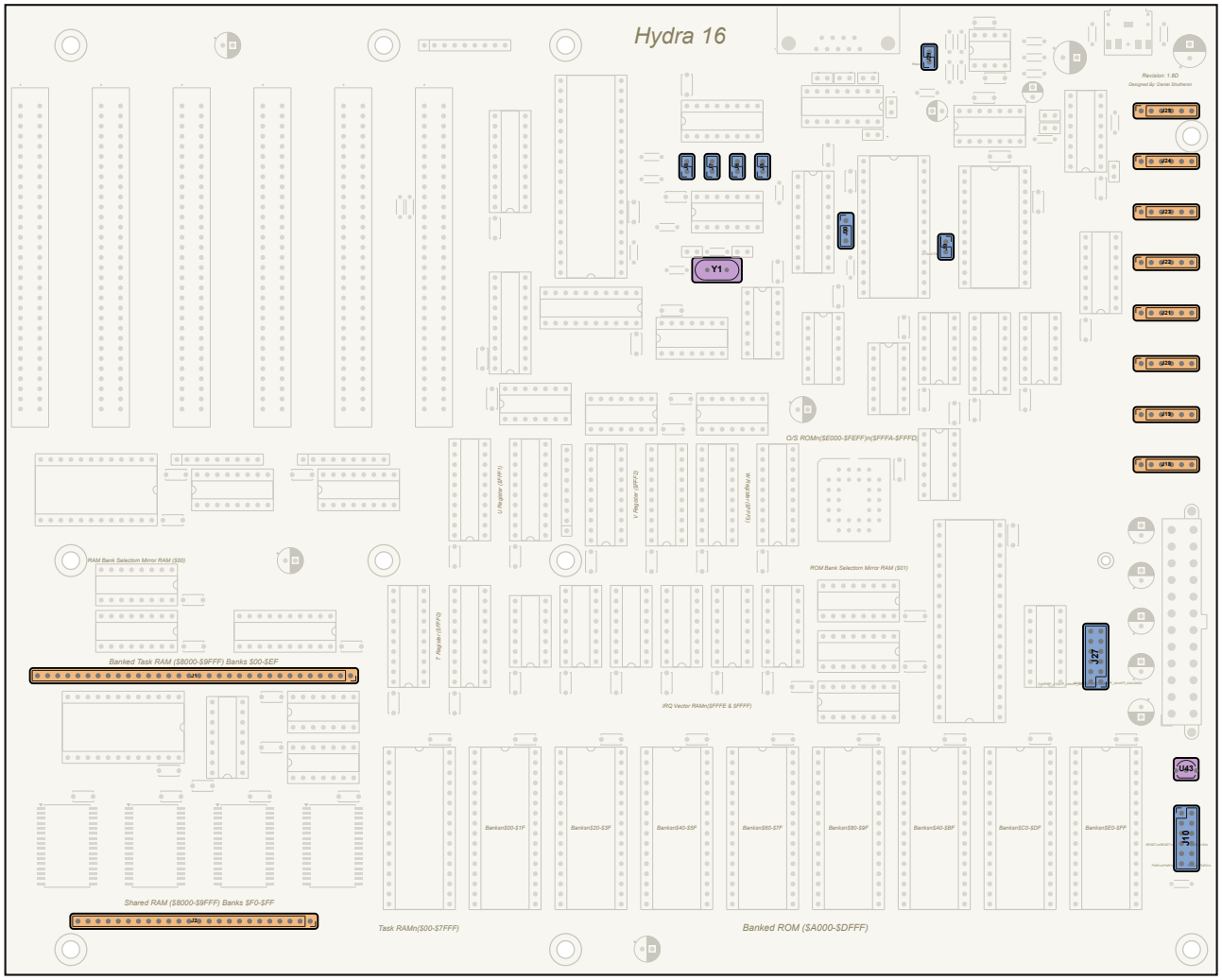


Figure 4-5. Step 5: the crystal and reset chip (purple), the pin headers (blue), and the pin sockets (orange).

- **Y1**, the 14.31818 MHz crystal. Either way round. Solder it quickly, without heating its can.
- **U43**, the DS1813 reset chip. Its **flat side** matches the flat side of its outline.
- **The pin headers**: J5, J6, J7 and J8 (2 pins each, the clock jumpers), J9 (2 pins), J29 (2 pins), J30 (3 pins), J10 (the front panel, 2 rows of 6) and J27 (the I/O header, 2 rows of 6). Solder one pin of each, check that it stands straight up, then solder the rest.
- **The pin sockets**: J18 to J25 (the SPI devices, 6 holes each), and J1 (32 holes) and J2 (24 holes) for the memory card. Make sure J1 and J2 are perfectly straight; the memory card plugs into both at once.

Step 6: The electrolytic capacitors

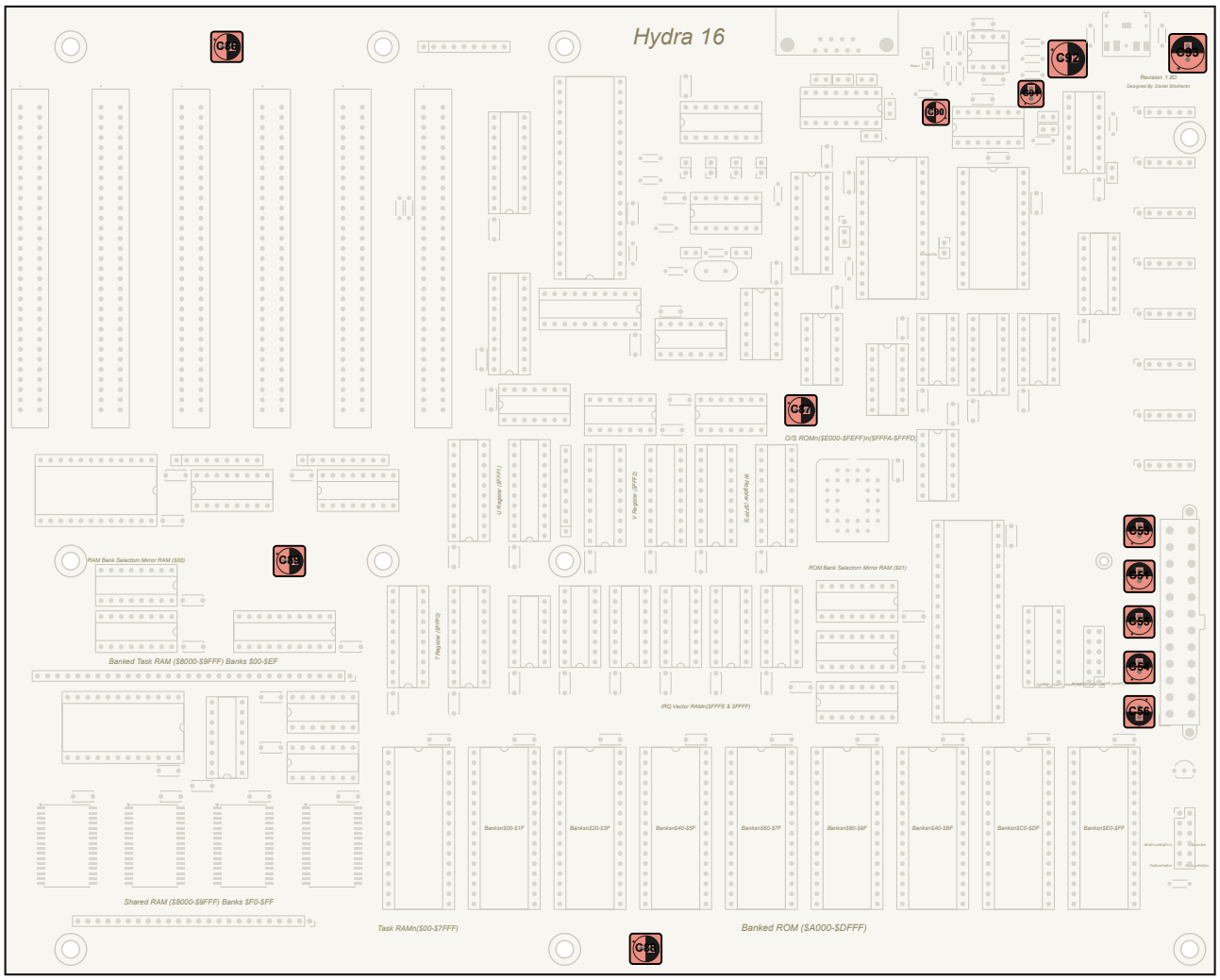


Figure 4-6. Step 6: the electrolytic capacitors.

These must go the right way round. The longer leg is plus (+), and the stripe on the side marks minus (−). Match the plus leg to the + on the board.

VALUE	CODE	PARTS	POLARITY
10 µF	106	C90, C91	Yes
47 µF	476	C86, C87, C88, C89	Yes
100 µF	107	C51, C53, C54, C55, C56	Yes
220 µF non-polarized	227 NP	C92, C93	No: either way

Check every one twice before you solder it.

Step 7: The big connectors

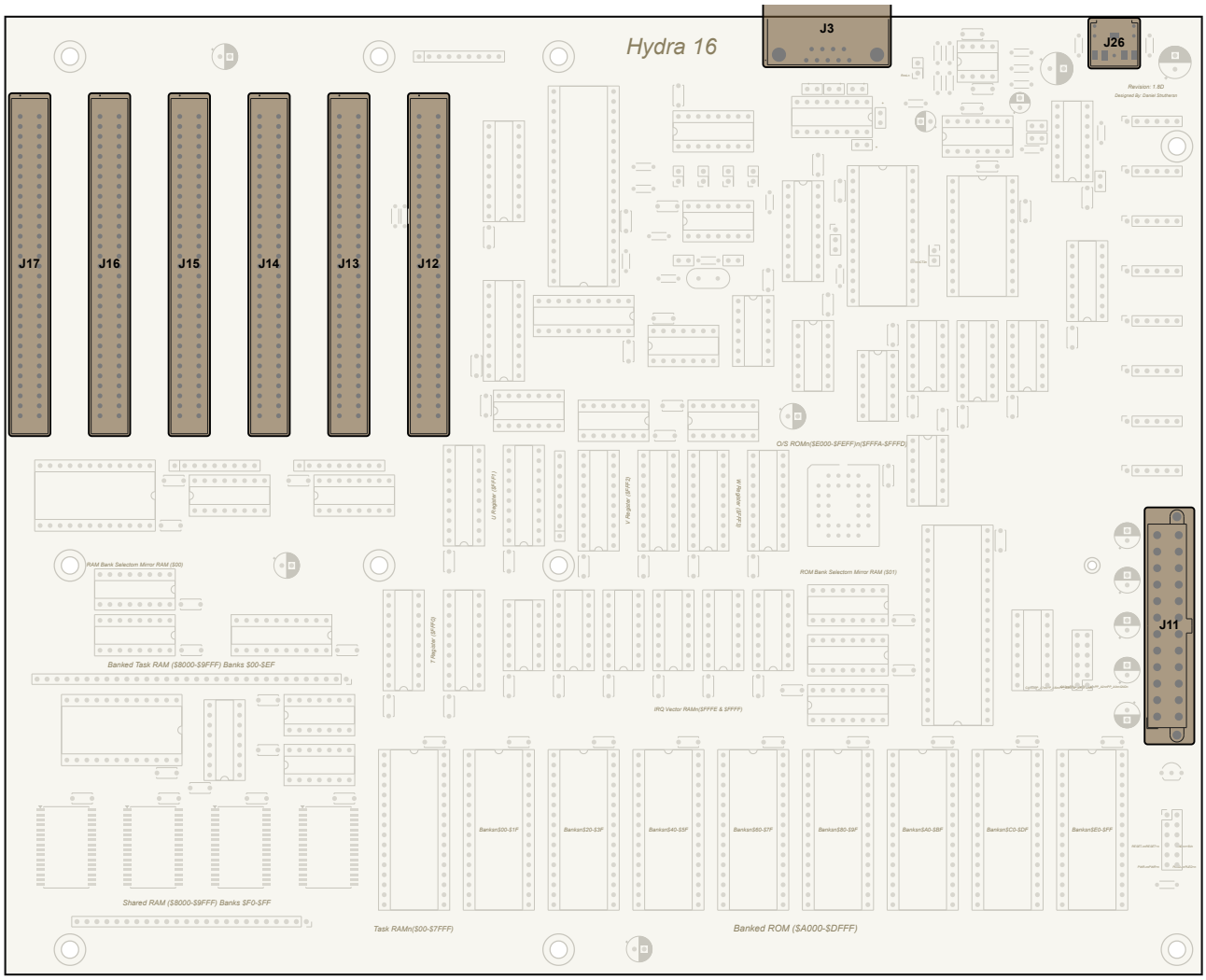


Figure 4-7. Step 7: the connectors.

Last come the tallest parts:

- **J3**, the 9-pin serial connector. Solder its two big mounting lugs as well as its nine pins.
- **J26**, the audio jack.
- **J12 to J17**, the six expansion slots. Each has 62 pins. Solder the two end pins first and check that the slot is flat and straight, then solder the rest. Take your time: that's 372 joints!
- **J11**, the 24-pin ATX power connector. Match its latch to the outline printed on the board. These pins carry the most current on the board, so make sure each joint is full and shiny.

Step 8: Inspect and clean

Now go over the whole back of the board with a magnifier, slowly, row by row. Look for joints you forgot, bridges between neighbouring pins, and dull or lumpy joints. Fix anything you find. Then clean the flux off with isopropyl alcohol and let the board dry.

4.3 The first power test

Before any chips go in, test the power. If something is wrong, a test now protects your chips.

Check for short circuits. Set your multimeter to measure resistance (ohms). Touch one probe to pin 8 of the U1 socket (the processor's +5 V pin) and the other to pin 21 (its ground pin). You should read *more* than about 100 ohms; the reading may slowly climb as the capacitors charge from the meter. A reading close to zero means a short circuit between +5 V and ground: look for a solder bridge, or an electrolytic capacitor in backwards, before going on.

Connect the power supply and a switch. Section 4.8 shows how: turn to it now, plug the ATX power supply into J11, connect a power switch to J10, and come back here.

Measure the voltages. Switch on. Set the multimeter to DC volts, hold the black probe on pin 21 of the U1 socket (ground), and touch the red probe to each of these socket pins in turn:

RED PROBE ON	MEASURES	SHOULD READ
U1 socket, pin 8	+5 V, for the chips	+4.75 to +5.25 V
U42 socket, pin 4	+12 V, for the sound circuit	about +12 V
U42 socket, pin 11	−12 V, for the sound circuit	about −12 V

If the power light is wired up (section 4.8), it lights.

Switch off again. If anything got hot or smelled of burning, or a voltage was missing, don't go on: find the problem first. Appendix I and Chapter 5's troubleshooting section can help.

4.4 Programming the ROM chips

HydraOS lives in four flash ROM chips. If your kit's ROM chips came already programmed (they should have labels on them), skip to section 4.5.

To program them yourself you need a **flash programmer** that connects to your PC by USB and supports SST39SF chips, such as the XGecu TL866II Plus or T48, and its software. For the PLCC chip you'll also need the programmer's **PLCC-32 adapter**.

The files are in the HydraOS folder `reborn/bin` (see Chapter 2):

FILE	CHIP TYPE	GOES IN	WHAT IT HOLDS
`bios.bin`	SST39SF010A (128K, PLCC-32)	**U6**	The kernel, the heart of HydraOS
`prom0.bin`	SST39SF040 (512K, DIP-32)	**U31**	The paged ROM, part 1
`prom1.bin`	SST39SF040 (512K, DIP-32)	**U32**	The paged ROM, part 2
`prom2.bin`	SST39SF040 (512K, DIP-32)	**U34**	The paged ROM, part 3

Together, the three paged ROM chips hold the drivers, the shells, the languages and the ROM disk's files.

For each chip:

1. In the programmer's software, choose the chip type (for example *SST39SF040* or *SST39SF010A*).
2. Load the file.
3. Put the chip in the programmer (the PLCC chip in its adapter) with pin 1 where the programmer's picture shows it.
4. **Program** the chip, with *Verify* turned on. The software erases it, writes it, and reads it back to check.
5. Put a **label** on the chip with its file's name and its place (for example "prom1 → U32").

NOTE: The other five paged ROM sockets, U30, U33, U35, U36 and U37, stay empty. They are there for HydraOS to grow into: the board can hold up to 4 MB of ROM.

TIP: If you can only get a 512K SST39SF040 in a PLCC package for U6, you can use it instead of the 128K SST39SF010A, but you must program `bios.bin` into it **four times**: at addresses `$00000`, `$20000`, `$40000` and `$60000`. (Most programmer software can load a file at a given address.) The board doesn't clear two of the chip's extra address lines when it starts, so the processor might start in any of the four copies.

4.5 Putting in the chips

CAUTION: Put on your anti-static wrist strap. And check twice that the power supply is unplugged.

Chips' legs are made a little wider apart than their sockets, to grip them firmly. To straighten them, hold the chip by its ends, rest one whole row of legs flat on the table, and gently roll the chip until the legs are straight up and down. Do the other side the same way. Then:

1. Turn the chip so **its notch matches the socket's notch**.
2. Line up all its legs with the socket's holes, and check that none are folded under.
3. Press it straight down, firmly, with your thumb on the middle of the chip.
4. Look at all the legs from the side: each one should be in its hole, with none bent outward or under.

Put each chip in the socket with its reference:

CHIP	PLACES
W65C02S (processor)	U1
R65C22 (VIA)	U2
R65C51 (ACIA, serial)	U3
MAX232 (serial levels)	U5
SST39SF010A, programmed with `bios.bin`	U6
HM628512 (task RAM), or a DS1747 for a real-time clock	U7
SST39SF040, programmed with `prom0.bin`, `prom1.bin`, `prom2.bin`	U31, U32, U34
YM2151 (sound)	U38
YM3012 (sound converter)	U41
TL074 (op-amps)	U42
NJM4556A (headphone amplifier)	IC9
SP720 (protection for the I/O header)	J28
74F138	U4, U20, U24
74LS260	U8, U9
74F20	U10
74ACT08	U11
74F86	U12

CHIP	PLACES
74F32	U13, U16, U56
74F30	U14
74F00	U15, U44
74LS148	U17, U18
74LS154	U19, U23
74F541	U21, U49, U51, U53, U55, U58, U60
74ACT240	U22
74F139	U26
74ACT14	U39
74F191	U40
74F157	U45
74F573	U48, U50, U52, U54
74F02	U57
74ACT245	U59
74F125	U61
74LS219	IC1 to IC8

NOTE: U3, the serial chip, should be a Rockwell **R65C51**. A WDC **W65C51N** fits too, but it has a fault in the way it reports that it's ready to send, and HydraOS must be built to work around it: `node reborn/build.js --acia wdc` (Chapter 2 shows how to build HydraOS).

U2, the VIA, is a Rockwell **R65C22** on the schematic. It's out of production too, and WDC's **W65C22N** is a drop-in replacement that needs no change to HydraOS.

CAUTION: Read each chip's markings carefully: some chips look alike. A 74F08 isn't a 74ACT08, and a 74LS139 isn't a 74F139, even though they have the same number of pins. The letters between the "74" and the number tell you the chip's *family*, and the Hydra needs exactly the families in this table.

The **DS1747** is a special RAM chip with a built-in clock and battery, which fits U7's socket instead of the HM628512. With it, the Hydra always knows the date and time, even when switched off. Without it, the Hydra's clock starts at midnight on January 1, 2000 every time you switch on.

4.6 Setting the jumpers

A **jumper** is a tiny plastic cap with a metal clip inside. Pushed onto two pins of a header, it connects them.

The clock jumpers, J5 to J8. Fit **one** jumper, on **J7**, for the normal speed, 3.58 MHz.

JUMPER	CLOCK SPEED	NOTES
J5	0.89 MHz	Very slow: for testing
J6	1.79 MHz	Slow: for testing
J7	**3.58 MHz**	**Normal**

JUMPER	CLOCK SPEED	NOTES
J8	7.16 MHz	Not with these parts: see below

CAUTION: Fit only **one** clock jumper at a time. Two jumpers would connect two different clocks together.

The kernel needs to know how fast the processor runs, to keep time and run the serial port, and the ROM images are built for 3.58 MHz. **With the parts in Appendix I, 3.58 MHz is the fastest speed the Hydra works at.** At J8's 7.16 MHz the memory chips and the bank registers can't answer within the processor's shorter cycle, and the self test fails nearly every RAM test. J8 is there for a future board with faster parts, which would also need HydraOS built for it (`node reborn/build.js --clock 2`). The two slower speeds, J5 and J6, work, but the clock and the serial port then run slow (section 5.7).

J30, the serial port's CTS. Fit a jumper on pins **1 and 2** (the normal setting). If your serial cable has only three wires (send, receive and ground), move the jumper to pins 2 and 3.

J9 and J29: no jumpers. J9 brings out two spare outputs of the sound chip, called CT1 and CT2, for your own experiments. J29, *Audio In*, lets you mix sound from another device into the Hydra's sound (Chapter 15 shows how). Never put a jumper on either of them.

4.7 The memory card

The **memory card** is a small board that plugs into J1 and J2 on the main board, and holds the Hydra's extra RAM. Its RAM chips make up a **RAM module**: four 512K chips, 2 MB in all. A module gives each of the Hydra's sixteen tasks 128K more memory of its own.

There are two kinds of memory card:

- The **through-hole card** has one module, in four chip sockets (Figure 4-8). This is the one to build first.
- The **surface-mount card** has two modules: eight RAM chips soldered to the card (Figure 4-10).

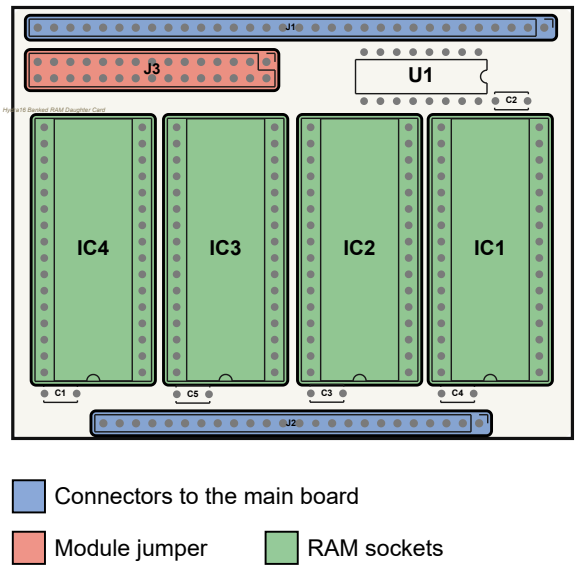


Figure 4-8. The through-hole memory card.

Building the through-hole card

The card's parts are marked on it with their references, just like the main board's:

PART	WHAT IT IS
C1 to C5	100 nF ceramic capacitors (104)
U1	74LS139 chip (soldered straight to the card, or in a 16-pin socket if you have one)
IC1 to IC4	32-pin wide sockets, for the HM628512 RAM chips
J3	Pin header, two rows of 15: the module number
J1	Pin header, one row of 32: the connector to the main board
J2	Pin header, one row of 24: the connector to the main board

1. Fit and solder C1 to C5.
2. Fit U1 (or its socket), matching its notch to the outline.
3. Fit the four 32-pin sockets, matching their notches.
4. Fit J3, with its short pins through the card and its long pins standing up.
5. **J1 and J2 are fitted upside down**: their long pins point *down*, out of the bottom of the card, to plug into the main board. Here is an easy way to get them perfectly lined up. Push the long pins of the two headers into the main board's J1 and J2 sockets, so their short ends point up. Lower the card onto them, chips' side up, so the short ends come up through the card's J1 and J2 holes: the card's J1 over the main board's J1, and its J2 over J2. Now solder the headers from the top of the card. The main board holds them straight while you solder.
6. Lift the card off, inspect and clean it, and put the four RAM chips in their sockets, matching their notches.

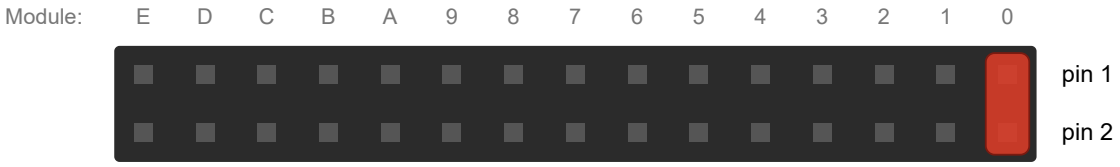
Setting the module number

Every RAM module has a number, from 0 to 14, set by a jumper on the card. When you have more than one module, each one must have a different number. (They don't have to be in order: modules 0, 1 and 5 work just as well as 0, 1 and 2.)

On the through-hole card, the jumper goes on **J3**, across its two rows, in the column for the module's number. The card has the numbers printed beside J3, in hex (section 1.6): 0 to 9 and then A to E, for 10 to 14.

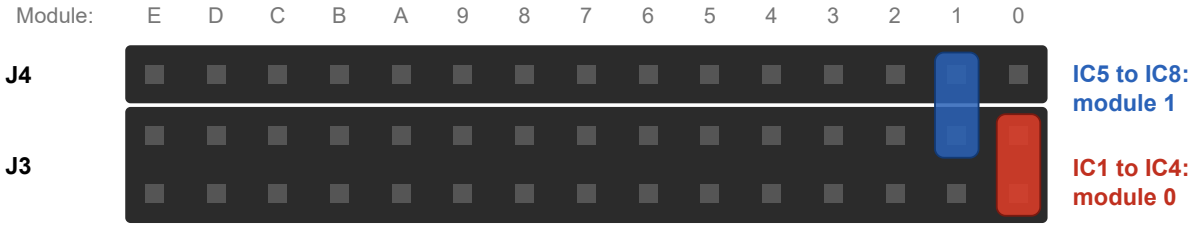
For your first memory card, put the jumper in column 0 (pins 1 and 2): module 0.

THE THROUGH-HOLE CARD: J3



The jumper in column 0 (pins 1 and 2) makes the card module 0. Column 1 (pins 3 and 4) would make it module 1, and so on, up to column E (pins 29 and 30) for module 14.

THE SURFACE-MOUNT CARD: J4 AND J3



The red jumper (J3 to J3) sets IC1 to IC4's module; the blue jumper (J4 to J3) sets IC5 to IC8's.

Figure 4-9. Setting the module numbers.

The surface-mount card

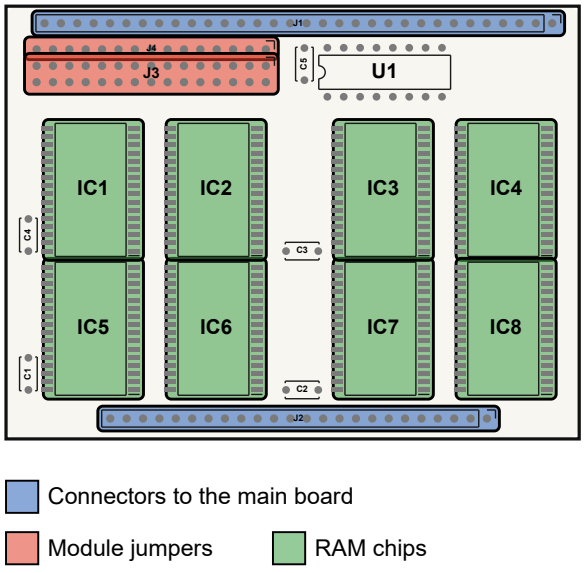


Figure 4-10. The surface-mount memory card.

The surface-mount card's eight RAM chips are soldered the same way as the main board's shared RAM chips in Step 1 (section 3.7 shows how). It has two modules, so it needs two module numbers, and two jumpers:

- **IC1 to IC4**: a jumper across J3's two rows, in the column for the module's number, just like the through-hole card.
- **IC5 to IC8**: a jumper from **J4**, the single row of pins beside J3, to the J3 pin next to it, in the column for the module's number.

Give the two modules different numbers, such as 0 and 1 (Figure 4-9).

More than one card

The memory cards stack, one on top of another: each card's pins plug into the card below it. To build a card that another card will plug into, use **stacking headers** for its J1 and J2 instead of plain pin headers. A stacking header has a socket on top and long pins underneath. Fit them from the top of the card, so the long pins pass right through the card and point down; then solder them underneath. The top card of the stack can use plain headers.

Remember: every module in the stack needs its own number.

Plugging in the card

With the power off, push the memory card's J1 and J2 pins straight down into the main board's J1 and J2 sockets, evenly, until the card is fully seated. When the Hydra starts, it tests every module it finds, and counts them. With one module, it prints:

```
RAM modules: 01
```

The count is in hex, so with three modules it prints `03`, and with eleven, `0B`.

4.8 Power, switch, reset and light

The power supply

The Hydra uses a standard **ATX power supply**, the kind found in desktop PCs. Plug its 24-pin connector into **J11**. (A supply with a "20+4" connector works too; keep the two parts together.) The Hydra uses the supply's +5 V for everything, and its +12 V and −12 V for the sound circuit and the expansion slots.

An ATX supply doesn't turn on just because it's plugged into the wall: its **PS_ON** wire must be connected to ground. On the Hydra, that wire comes to the front-panel header, J10.

The front-panel header, J10

J10 has two rows of six pins. Pin 1 has a square pad; the odd-numbered pins are in one row and the even-numbered pins in the other:

PINS	WHAT TO CONNECT
11 and 9	**Power switch.** A switch that *stays on* (a toggle or latching switch) between pins 11 and 9 turns the supply on and off.
7 and 5	**Reset button.** A push button between pins 7 and 5 resets the Hydra.
12 (+) and 10 (−)	**Power light.** An LED: its longer leg (+) to pin 12, its shorter leg (−) to pin 10. Its resistor is already on the board.
8	+5 V, for your own front-panel gadgets
5, 6, 9, 10	Ground
1, 2, 3, 4	Not used

NOTE: Use a switch that stays in place, not a push button. A PC's push button only "taps" its motherboard, which remembers whether the power should be on; the Hydra needs the switch to *stay* closed for as long as it's on. If you have no switch yet, a jumper across pins 11 and 9 works too; then use the switch on the back of the power supply.

> **CAUTION:** An ATX power supply has dangerous voltages inside. Never open its case.

4.9 Ready to go

Before you switch on for the first time, check:

- Every chip is in the right socket, the right way round, with no legs bent.
- The ROM chips are programmed and in U6, U31, U32 and U34.
- One clock jumper is on J7, and the J30 jumper is on pins 1 and 2.
- The memory card is plugged in, with its module jumper set.
- The power supply is plugged into J11, and the power switch into J10.

The SD card reader. If you have an SD card module, this is the time to connect it: six wires, to the SPI header J18. Section 14.2 has the wiring, and shows how to get a new card ready. To start with HydraOS's sample card (its example programs and songs), write its image, `reborn/bin/sdcard.img`, onto a card with your PC first (section 2.6).

Now turn to Chapter 5, connect the Hydra to your PC, and switch it on!

CHAPTER 5

Switching On

Your Hydra is built. In this chapter you'll connect it to a PC, switch it on, watch it start, and check that it's all there. If something doesn't work, the last part of the chapter will help you find out why.

IN THIS CHAPTER: You'll connect the Hydra to a PC, set up a terminal program, switch on, read the start-up messages, and find the fault if there is one. **You need:** a built Hydra (Chapter 4), a PC, and a USB to RS-232 adapter. If you're using the emulator, skip to Chapter 6.

5.1 Talking to the Hydra

The Hydra doesn't need a screen and keyboard of its own. It talks to you through its **serial port**, J3: a connector that sends and receives letters one *bit* at a time, down a cable, to a PC. On the PC, a **terminal program** shows what the Hydra sends, and sends the Hydra what you type. The PC becomes the Hydra's screen and keyboard.

(With a Vera X card in slot 0, the Hydra can have a screen and a keyboard of its own too. Chapter 16 shows how.)

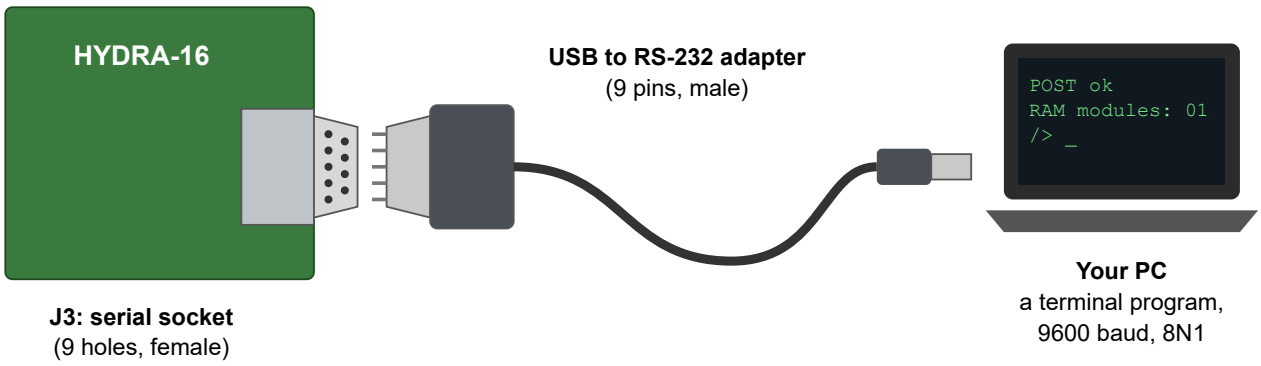


Figure 5-1. Connecting the Hydra to a PC.

The cable. Modern PCs don't have serial ports any more, so you need a **USB to RS-232 adapter**: a cable with a USB plug at one end and a 9-pin serial plug at the other. Its 9-pin plug has pins (it's *male*), and it pushes straight into the Hydra's J3 socket. Tighten its two screws, gently.

CAUTION: Be sure the adapter says **RS-232**. Some adapters, often sold for robot kits and small computer boards, are "TTL" or "3.3 V" serial adapters, with loose wires or a row of pins instead of a 9-pin plug. They use different voltages, and won't work with the Hydra's serial port.

If the adapter's cable doesn't reach, you can put a serial **extension cable** (a "straight-through" cable, with pins at one end and holes at the other) between them. Don't use a **null-modem** cable: it swaps the wires over, and the Hydra and the PC won't hear each other.

Finding the port. When you plug the adapter into the PC, the PC gives it a name:

ON	THE PORT IS CALLED	WHERE TO FIND IT
Windows	`COM3`, `COM4`, ...	*Device Manager*, under *Ports (COM & LPT)*
macOS	`/dev/tty.usbserial-...`	type `ls /dev/tty.*` in the Terminal app
Linux	`/dev/ttyUSB0`	type `ls /dev/ttyUSB*` in a terminal

5.2 A terminal program

You can use the Hydra's own **PC tool**, or almost any terminal program.

The Hydra PC tool

The PC tool comes with HydraOS. It's a terminal, and it does one more thing: it lets the Hydra use a folder on your PC as if it were one of its own disks, called `/pc` (Chapter 9). It needs Node.js, which you installed in Chapter 2. The first time, install the part it uses to talk to serial ports:

```
cd hydra-16/reborn/sim
npm install
```

Then, to see the PC's serial ports:

```
node tools/hydrapc.js --list
```

and to start it, give it the port and a folder on your PC for the Hydra to use:

```
node tools/hydrapc.js COM3 C:\hydra-files
```

In place of the folder you can give it an SD card **image file** (section 2.6):

```
node tools/hydrapc.js COM3 C:\hydra-files\mycard.img
```

Then `/pc` is that card: the Hydra sees the files in the image, and what it writes there goes into the image. It's a way to look at, or fill, a card's image from a real Hydra with no card reader on the PC, and to keep a set of work in one file. The image is complete on your PC whenever the Hydra isn't writing to it, so you can copy it, or write it to a real card (Appendix H). Don't open the same image in an emulator at the same time.

Everything you type now goes to the Hydra. The PC tool's own keys start with CTRL-A:

KEYS	WHAT THEY DO
CTRL-A then X	Quit the PC tool
CTRL-A then L	Turn the log on or off: a list of the Hydra's file requests
CTRL-A then S	Say the counts so far: the Hydra's requests, and any that came damaged or had to be answered again (a bad cable shows here)
CTRL-A then CTRL-A	Send a CTRL-A to the Hydra

It can also take these options, after the folder:

OPTION	WHAT IT DOES
`--read-only`	The Hydra can read the folder's files (or the image's), but not change them
`--baud N`	The serial port's speed (9600 unless you say)
`--log`	Start with the log on (it lists what the Hydra asks for, and any frame that came damaged)
`--no-size`	Don't tell the Hydra the size of the window
`--win32-input`	For Windows Terminal: lets keys like CTRL-TAB reach the Hydra

Other terminal programs

Any terminal program that can use a serial port works too: **PuTTY** or **Tera Term** on Windows, or `screen` or `minicom` on macOS and Linux. Set it up like this:

SETTING	SET IT TO
Speed (baud)	**9600**
Data bits	8
Parity	None
Stop bits	1
Flow control	**XON/XOFF**
Terminal type	VT100 or xterm

The first four are often written **9600 8N1**. In PuTTY, choose *Serial* as the connection type, type the port's name (like `COM3`) and the speed, and set *Connection → Serial → Flow control* to *XON/XOFF*. On macOS or Linux, `screen /dev/ttyUSB0 9600,ixon` starts `screen` on the port (to leave it, press CTRL-A, then K, then Y).

> **HOW IT WORKS:** **Flow control** is how the Hydra says "wait!" when keys come faster than it can take them, as they do when you paste a page of text. It sends one byte, called **XOFF**, to ask the terminal to stop, and another, **XON**, when it's ready for more. A terminal set to *XON/XOFF* obeys them, and nothing you paste is lost. With flow control set to *None* everything else still works, but a long paste may lose its end (section 13.6). The PC tool obeys XON and XOFF without being told.

> **HOW IT WORKS:** The Hydra's serial chip sends only while its *CTS* ("clear to send") line is on, and that line comes from the PC's *RTS* ("request to send"). The PC tool and most terminal programs turn RTS on as they open the port. If yours doesn't, move the jumper on J30 to pins 2 and 3, which turns CTS on for good.

5.3 Switching on

Start the terminal program first, and then switch on the Hydra. The power light comes on, and in about a second the Hydra starts to talk:

```
Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000 20:0/00/0000
POST ok
RAM modules: 03
task F: cons
task E: storage
task D: kdev
task C: snd
task B: vid
task 1: init
HydraOS 1.0 for the Hydra-16
init: up in task 01
T ST FL PA CPU    NAME
0 01 00 FF 000053 kernel
1 01 00 FF 000019 init
C 04 01 FF 000003 snd
D 06 01 FF 000010 kdev
E 04 01 FF 000073 storage
F 08 01 FF 00003F cons
hello, from init
init: hello ended: code $07 (bye)
HyForth (Forth 2012), bye to end
/>
```

Congratulations: you've built a computer! Here's what the messages mean (Chapter 7 tells the whole story):

MESSAGE	WHAT IT MEANS
`Hydra-16: kernel 0.1, ABI 1`	The **kernel**, the heart of HydraOS, has started.
`POST ...` and `RAM ...`	The **power-on self test** has checked the Hydra's memory. A `0` everywhere means all is well...
`POST ok`	...and so does this.
`RAM modules: 03`	It found three RAM modules, on the memory card. (Yours may have one or two.)
`task F: cons` ... `task B: vid`	The **drivers** have started: the programs that look after the Hydra's devices. (`vid` looks for a Vera X card and, finding none, ends at once, which is why task B isn't in the list below: section 7.4.)
`task 1: init`	The first program, **init**, has started. It sets up the system.
`HydraOS 1.0 for the Hydra-16`	HydraOS is running.
The table and `hello, from init`	A little program called **hello** says hello, and shows the tasks running.
`HyForth (Forth 2012), bye to end`	**HyForth**, the shell, has started, ready for your commands.
`/>`	The **prompt**: the Hydra is waiting for you to type.

> **NOTE:** The numbers in the `CPU` column may be a little different on your Hydra. They count how much work each task has done so far.

5.4 Is it all there?

Type `date` and press RETURN. If the Hydra answers, its keyboard side works too:

```
/> date
2000-01-01 00:00:04
```

The Hydra keeps the time, but without a clock chip (section 4.5) it doesn't know the date, so its clock starts at midnight on January 1, 2000, each time you switch on. Chapter 11 shows how to set it.

Now ask it how much memory it found. `free` shows the memory each task can have, and the memory they share. This Hydra has a memory card with three RAM modules:

```
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

Each RAM module gives every task 128 KB, so one module shows `128 KB a task (1 module)`, and two show 256 KB. If the number of modules is less than the number you fitted, see section 5.7.

> **NOTE:** The board has 2 MB of shared RAM, but `free` shows 1024 KB. That's the half HydraOS hands out to programs, in pieces called *segments* (section 17.9). One segment is in use already: the RAM disk `/sram`, of 512 KB. HydraOS 1.0 keeps the other half of the shared RAM in reserve.

Your Hydra works. **Chapter 6, First Steps**, is where you learn to use it: turn there now. The rest of this chapter is about the serial line's speed, switching off, and what to do when something isn't right.

5.5 A faster line

At 9600 baud, the serial port sends about 900 letters a second: a screenful takes two seconds. It can go more than ten times faster. Type:

```
/> echo b115200 >/dev/serctl
```

Then switch your terminal program to 115200 baud too. (In the PC tool, quit with CTRL-A X and start it again with `--baud 115200`.) The Hydra always starts at 9600, so you'll need to do this each time you switch on. Chapter 13 shows the other speeds.

> **HOW IT WORKS:** The serial chip's speeds come from a clock of 1.79 MHz, and its speeds are about 3% slower than their names: "9600" is really 9,322 bits a second. That's close enough for a PC's serial port, which allows for small differences like this.

5.6 Switching off

You can switch the Hydra off at any time, but first:

- **Save your work, and finish with the files you're changing.** If you're in an editor, save and quit. A program writing to an SD card finishes the job as it closes the file.
- **Copy anything you want to keep off the RAM disks**, `/ram` and `/sram` (Chapter 9). They're made of memory, and they're empty again each time the Hydra starts.

The **reset button** starts the Hydra again without switching it off: it's the button to press when a program has gone wrong and won't stop. A reset empties the RAM disks too.

5.7 When something goes wrong

Don't worry if your Hydra doesn't work the first time: most home-built computers don't! Finding out why is part of the fun, and you'll learn a lot about how the Hydra works. **Always switch off before you touch the board.**

Nothing appears in the terminal

1. Is the **power light** on? If not, check the power supply, its plug in J11, and the switch on J10.
2. Is the terminal program using the right **port**, at **9600** baud? Unplug the adapter and plug it in again, and check which port appears.
3. Is the cable a straight-through one, not a null-modem cable?
4. Press the **reset button**. The messages should appear again.
5. Try the jumper on **J30** on pins 2 and 3 (section 5.2).
6. Is there exactly one **clock jumper**, on J7?
7. Check the chips the Hydra needs to start: U1 (the processor), U6 (the kernel ROM), U3 and U5 (the serial port), U39 and U40 (the clock) and U43 (the reset chip). Are they in the right sockets, the right way round, with no legs bent under?
8. Then check the **address decoder**: U8 to U16 and U19 to U24 (Appendix I). These small logic chips work out which chip answers each address, so one of them in the wrong socket, or backwards, can keep the ROM or the serial chip from ever being heard. Many of them look alike: read the number printed on each, and compare it with the number printed on the board beside its socket.
9. Check the joints on the back of the board around those chips, and around J3.

Letters appear, but they're nonsense

The terminal's speed doesn't match the Hydra's. Set it to 9600 baud, 8N1, and press reset. If that doesn't help, check the clock jumper: on J5 or J6 the Hydra runs slower, and its serial port does too.

POST finds faults

When the power-on self test finds a fault, it says so instead of `POST ok`:

```
POST found faults: shared RAM
```

The Hydra goes on starting if it can, leaving any bad memory unused, but you should find the fault. The numbers on the `POST` and `RAM` lines point to it. Each one that isn't `0` is a fault (Chapter 7 explains them all):

PART OF THE LINE	IT TESTS	IF IT ISN'T 0, CHECK
`ZP ST OS HI`	Keeping the tasks' memory apart	U7 (the task RAM), U21, and U48 (the task register)
`SH:X` (not `S`)	Sharing memory between tasks	The shared RAM chips U25, U27, U28, U29
`W`	The kernel ROM's pages	U6
`U`	The U register's lines to the shared RAM	The shared RAM chips, and their joints
`F0: F4: F8: FC:`	Each of the four shared RAM chips	U25, U27, U28, U29
`00: 10:` …	Each RAM module	The memory card and its chips

`RAM modules: 00`

The Hydra didn't find a memory card. Is the card pushed all the way into J1 and J2? Is there a jumper on its J3? Are its RAM chips in their sockets the right way round?

The hardware test

HydraOS has a **hardware test** that checks each part of the Hydra in turn. There are two ways to start it:

- Hold down the T key in the terminal program, and press and let go of the reset button. Keep holding T until the test's menu appears. (This works even if HydraOS can't start.)
- Or, at the prompt, type `hwtest`.

The test shows a menu. Press A to run all its quick tests:

```
/> hwtest
hwtest: the system starts again, into the hardware test

Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000 20:0/00/0000
POST ok


Hydra-16 hardware test
It overwrites all of memory, and ends with a reset (R).
A  all tests (quick)   F  all, full memory   L  all, until a key   R  reset
1  CPU                 2  T U V W registers   3  shared RAM
4  RAM bank registers  5  task RAM           6  RAM modules
7  BIOS ROM            8  paged ROM          I  interrupts
V  VIA                 Y  sound chip (YM2151)  K  CPU clock
S  serial port (ACIA)  P  SPI devices         C  I2C bus
X  slot cards          H  hold a task (probe)
> A
CPU ................ ok
T U V W registers .. ok
shared RAM ......... ok
RAM bank registers . ok
task RAM ........... ok
RAM modules ........ 0 1 2 ok
BIOS ROM ........... ok
paged ROM .......... ok
interrupts .........  ok
VIA ................ ok
sound chip (YM2151) . ok
CPU clock .......... 3.58 MHz (first 16397, next 129) ok
serial port (ACIA) .   9600 baud ok
SPI devices ........ SD cards 0 ok
I2C bus ............ no devices ok
slot cards ......... all empty ok
hwtest: all passed
```

A test that fails says what it found, which tells you where to look. To leave the hardware test, press R: the Hydra starts again. (The test uses all of the Hydra's memory, so anything in memory is lost.)

> **TIP:** If you get stuck, take a break, and then check the board again slowly with a magnifier, one joint at a time. A fresh pair of eyes, a friend's or your teacher's, often spots in a minute what you've looked past for an hour.



PART TWO

Inside HydraOS

Your first hour at the keyboard; then how HydraOS starts, how you give it commands, where it keeps its files, and the console with its windows. The last chapter of this part is a guide to every command, from `as` to `xmodem`.

- Chapter 6. First Steps
- Chapter 7. How the Hydra Starts
- Chapter 8. The Shells
- Chapter 9. Files and Namespaces
- Chapter 10. The Console and Its Windows
- Chapter 11. Commands from A to Z

CHAPTER 6

First Steps

You have a Hydra in front of you: a real one you've just switched on (Chapter 5), or the emulator on a PC (Chapter 2). Either way, it's showing you this, and waiting:

```
/>
```

This chapter is your first hour with it. There's almost no theory here, just things to type. The chapters after this one explain *why* it all works; this one makes sure you can find your way around, fix a mistake, save your work and get out of trouble.

IN THIS CHAPTER: You'll type your first commands, see what a mistake looks like and how to get out of one, make a file and keep it, run the sample programs, and say hello in three programming languages. **You need:** a Hydra at its `/>` prompt. Nothing else.

6.1 The prompt

The `/>` is the **prompt**. It's the Hydra's way of saying "your turn". You type a line, press RETURN, the Hydra does what the line says, prints its answer, and shows the prompt again. That's the whole conversation, and everything in this book is built from it.

The program that prints the prompt and reads your lines is called the **shell**. The `/` in the prompt tells you where you are among the Hydra's files (section 6.5); the `>` is just a pointer to where your typing goes.

> **NOTE:** You can't hurt the Hydra by typing. Its system is kept in ROM chips, which a program can't change. If you get into a muddle that you can't get out of, press the reset button (in the web emulator, the **Reset** button), and the Hydra starts again, as good as new. So try things!

6.2 Your first commands

Type `date` and press RETURN:

```
/> date
2000-01-01 00:00:04
```

`date` is a **command**: the name of a program. The Hydra found the program, ran it, and the program printed the date and the time. (A Hydra without a clock chip starts its clock at midnight on January 1, 2000 each time it's switched on. If yours shows today's date, it has a clock chip, as the web emulator's Hydra does.)

Most commands take more words after their name, which tell them what to work on. They're called **arguments**, and there's a space between each. `echo` prints its arguments back to you:

```
/> echo Hello, Hydra!
Hello, Hydra!
```

`ls` lists files. With no arguments, it lists the files where you are:

```
/> ls
bin/
dev/
env/
lib/
mnt/
pc/
proc/
ram/
rom/
sd/
sram/
tmp/
```

Each name that ends with a `/` is a **directory**: a folder, with more files inside. Give `ls` a directory's name, and it lists what's in there. The SD card is the directory `/sd/0`:

```
/> ls /sd/0
bench/
sample/
songs/
```

And the Hydra can do arithmetic. Its shell is also a programming language called HyForth, where the numbers come first, then what to do with them, and `.` ("dot") prints the answer:

```
/> 2 3 + .
5
```

That looks back to front, but there's a good reason for it, which Chapter 19 explains. For now, notice the spaces: each number and each sign is a separate word.

> **TRY THIS:** Type `free` to see how much memory your Hydra has, `ps` to see the programs that are running right now, and `ls /rom/bin` to see the names of all the programs on the Hydra's ROM disk. Every one of those names is a command you can type. Chapter 11 has a page or so about each.

6.3 Typing, and fixing what you typed

Nothing happens until you press RETURN, so until then you can change the line as much as you like:

KEYS	WHAT THEY DO
BACKSPACE or DELETE	Rub out the letter before the cursor
← and →	Move along the line
HOME or CTRL-A	Go to the start of the line
END or CTRL-E	Go to the end of the line
CTRL-U	Rub out the whole line, and start again
↑ and ↓	Bring back the lines you typed before (the last four)
RETURN	Send the line to the Hydra

The ↑ key is the one you'll use most: when a line was nearly right, bring it back, fix it, and press RETURN again.

Two more keys work at any time, whatever the Hydra is doing:

KEYS	WHAT THEY DO
CTRL-C	Stop the program that's running
CTRL-]	The **window key**: the start of the console's own commands (Chapter 10)

> **TIP:** With the Hydra PC tool and the terminal emulator, CTRL-A is the tool's own key, so press it twice to go to the start of a line, or use HOME.

6.4 When the Hydra doesn't understand

Everybody mistypes. The Hydra can't guess what you meant, so it tells you what it didn't understand, and gives you the prompt back. Nothing is harmed. Here are the mistakes everyone makes in their first hour:

```
/> dat
rc: dat: not found
/> LS
rc: LS: not found
/> 2 3 plus .
plus ?
/> 2 3+ .
3+ ?
```

- **`rc: dat: not found`** means there's no program called `dat`. (`rc` is the part of the shell that looks for programs: Chapter 8.) Press ↑, fix the name, and try again.
- **`rc: LS: not found`**: capital letters count. `ls` and `LS` are different names, and the Hydra's commands are in small letters.
- **`plus ?`** is HyForth asking "what's `plus`?". When a line starts with a number, or with a word that HyForth knows, the line is HyForth's, and every word in it has to be one that HyForth knows.
- **`3+ ?`**: without the space, `3+` is one word, and HyForth doesn't know it. Spaces matter!

Other programs say what's wrong in their own way, usually with their name, the file's name, and the trouble:

```
/> cat /ram/nothing.txt
cat: /ram/nothing.txt: not found
```

Appendix G lists the messages, and what each one means.

When the prompt doesn't come back

If you've pressed RETURN and there's no prompt, a program is still running. Either it's busy, or it's waiting for *you*.

It's busy. `sleep 100` does nothing for a hundred seconds. CTRL-C stops it:

```
/> sleep 100

/>
```

It's waiting for you. Some programs read what you type. `cat` with no arguments copies each line you type back to you, until you tell it there's no more by pressing CTRL-D at the start of a line. (CTRL-D means "that's the end of my typing".)

```
/> cat
Is anybody there?
Is anybody there?
/>
```

When the prompt looks different

The prompt tells you which program is listening. If it isn't `/>`, you've started another program that reads lines, and it's waiting for *its* kind of line:

THE PROMPT	WHO'S LISTENING	TO GET BACK
`/>`, `/ram>`, `0:/work>`	HyForth, the Hydra's shell. This is home.	
Eight spaces, and no `>`	HyForth, in the middle of a definition you began with `:`	Type `;`
`%`	rc, the command shell (Chapter 8)	Type `exit`
`>`	HyBASIC (Chapter 23)	Type `SYSTEM`
`hylang>`	hylang (Chapter 21)	Type `exit` or `(exit)`
`*`	`ed`, the line editor (Chapter 11)	Type `q`
`db>`	`db`, the debugger (Chapter 29)	Type `q`

And three programs take over the whole screen: `edit` (leave it with CTRL-X), `more` and `top` (leave them with Q).

> **TIP:** Lost? Try CTRL-C first, then CTRL-D, then the table above. If nothing brings the prompt back, press the reset button. You'll lose what was in memory, but not what's on the SD card.

6.5 Looking around

The Hydra's files are arranged like a family tree, upside down. At the top is the **root**, written `/`. In it are directories, in them are more directories and files, and so on. You're always *in* one directory, your **current directory**, and the prompt shows which. `cd` ("change directory") moves you, and `pwd` ("print working directory") tells you where you are:

```
/> cd /sd/0/sample
0:/sample> ls
as/
c/
hi
nsum
tick
upper
0:/sample> pwd
/sd/0/sample
0:/sample> cd /
/>
```

On an SD card, the prompt is short: `0:/sample` means "card 0, in `/sample`". A file's full name is its **path**: the directories you go through to reach it from the root, with a `/` between them. `/sd/0/sample/hi` is the file `hi`, in `sample`, on card 0.

These are the places you'll use most:

DIRECTORY	WHAT'S THERE
`/rom`	The ROM disk: HydraOS's own programs and libraries. You can read it, but nothing can change it.
`/ram`	A RAM disk: fast, and empty each time the Hydra starts. A scratch pad.

DIRECTORY	WHAT'S THERE
`/sd/0`	The first SD card. What you put here is kept.
`/bin`	Every program the Hydra can find, gathered from all the disks into one directory
`/pc`	A folder on your PC, if you're using the PC tool or the terminal emulator's `--pc-dir`

Chapter 9 explores the whole tree.

6.6 Making a file

The quickest way to make a file is to send a command's output into one. A `>` and a file's name after a command means "don't print it: put it in this file". `cat` shows what's in a file:

```
/> echo Remember the milk >/ram/note.txt
/> cat /ram/note.txt
Remember the milk
/> echo Feed the cat >>/ram/note.txt
/> cat /ram/note.txt
Remember the milk
Feed the cat
```

`>` starts the file afresh each time. `>>` adds to the end of it.

The screen editor

For anything longer than a line, use `edit`, the Hydra's **screen editor**. It works like a simple word processor. Type `edit` and the name of the file you want to make (or change):

```
/> edit /ram/poem.txt
```

The screen clears. Type your text. The arrow keys move around it, BACKSPACE rubs out, and RETURN starts a new line:

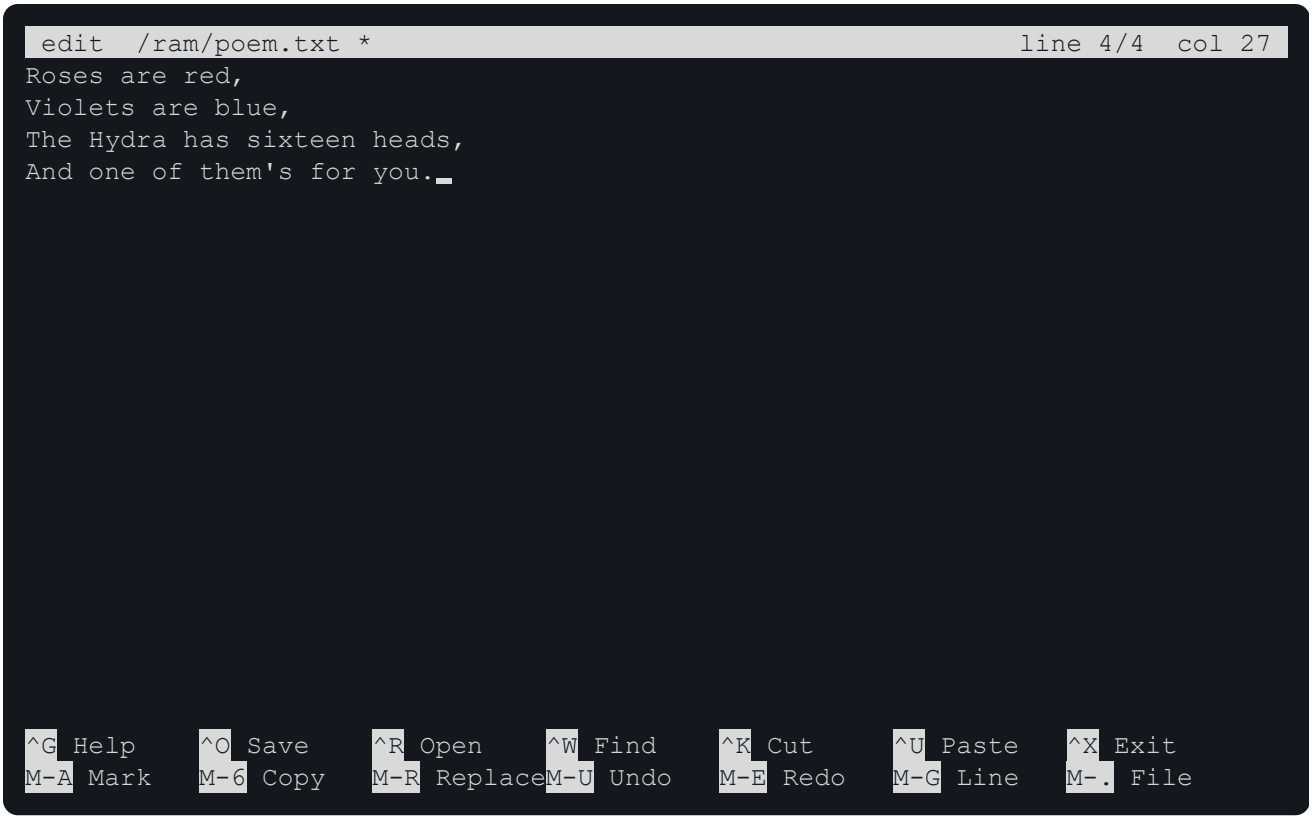


Figure 6-1. A poem in the screen editor. The star after the file's name means there are changes that haven't been saved.

The two lines at the bottom of the screen remind you of the editor's keys. `^O` means CTRL-O. You need only three to begin with:

KEYS	WHAT THEY DO
CTRL-O, then RETURN	**Save**: write the text out to the file. (It asks for the name: RETURN keeps the one it has.)
CTRL-X	**Leave** the editor. If you haven't saved, it asks whether you want to.
CTRL-G	Show **all** the keys: finding, cutting and pasting, undo, and more

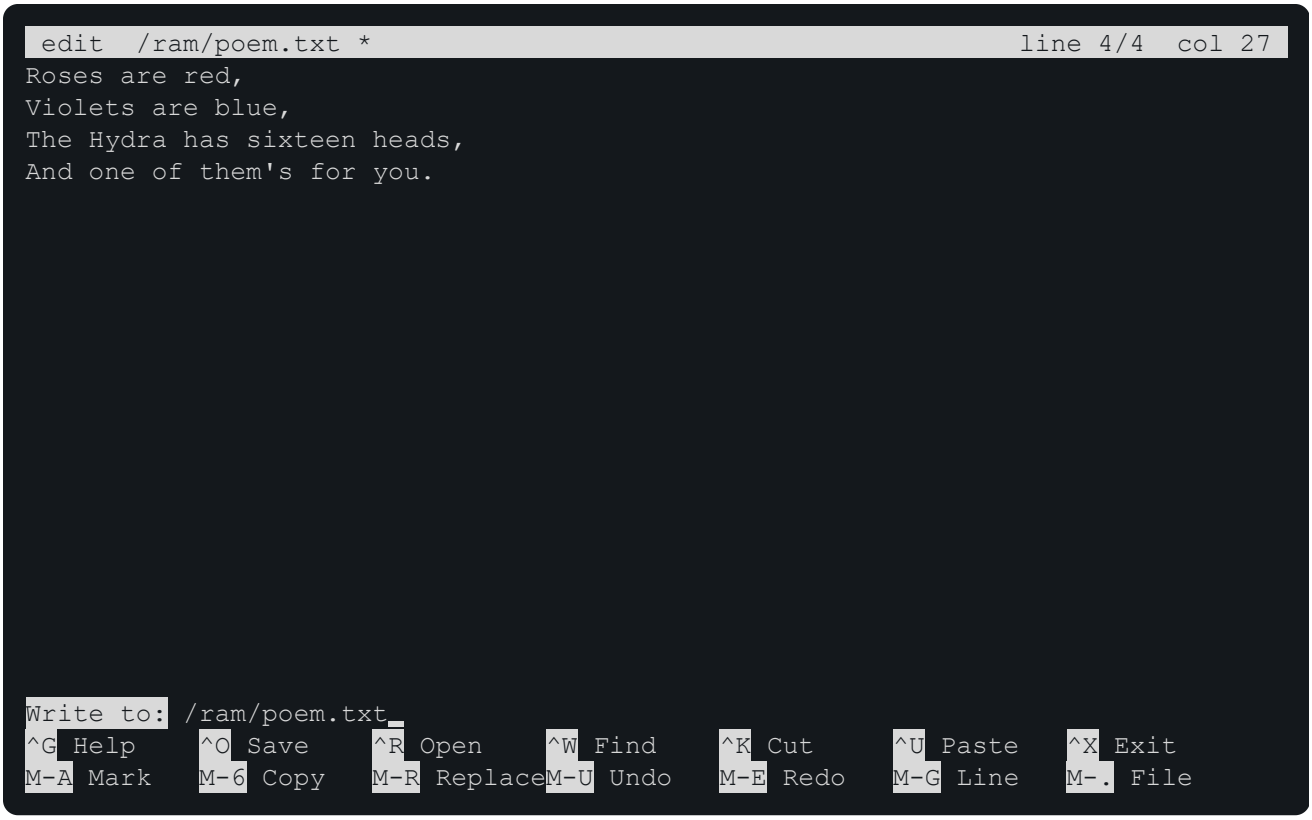


Figure 6-2. Saving: after CTRL-O, the editor asks for the file's name.

Back at the prompt, `cat /ram/poem.txt` shows your poem, and `edit /ram/poem.txt` opens it again to change it. The `edit` entry in Chapter 11 has the whole story.

> **TRY THIS:** Write a four-line poem of your own with `edit`, save it, and leave. Show it with `cat`. Then open it again, change one word, and save it again.

Tidying up

COMMAND	WHAT IT DOES
`ls -l` *directory*	List a directory the long way, with each file's size and date
`cp` *from to*	Copy a file
`mv` *from to*	Move a file, or give it a new name
`rm` *file*	Remove a file. There's no way to get it back!
`mkdir` *directory*	Make a new directory

6.7 Keeping your work

Here's the most important thing in this chapter: **the RAM disk forgets**. `/ram` is made of memory, and it's emptied whenever the Hydra is switched off or reset. It's a fine place to try things. It's a terrible place to keep them.

Work you want to keep goes on the SD card, `/sd/0`. Make a directory of your own there, and copy your files into it:

```
/> mkdir /sd/0/work
/> cp /ram/note.txt /sd/0/work/note.txt
/> ls /sd/0/work
note.txt
```

Better still, make your files there in the first place: `edit /sd/0/work/poem.txt`.

Where the SD card really is depends on your Hydra:

YOUR HYDRA	`/SD/0` IS	KEPT UNTIL
A real Hydra with an SD card reader	The card in the reader	You erase it
A real Hydra with no reader	Not there. Use `/pc`, a folder on your PC (section 5.2)	
The web emulator	A card kept inside your browser, on that one computer	You press **New** or **Samples** in Setup, or clear the browser's stored data. **Save card** makes a copy you can keep anywhere.
The terminal emulator	The sample card, with your changes kept only until you quit	You quit. To keep your work, give it a card image of your own (section 2.6), or use `/pc` (section 2.5).

CAUTION: Before you switch off, reset, or close the emulator, ask yourself: is everything I want in `/sd/0` (or `/pc`)? If it's only in `/ram`, it's about to disappear.

Working from your PC. If you'd rather write your programs in an editor on your PC, let the Hydra see a folder of yours as `/pc`: the PC tool does it for a real Hydra (section 5.2), and `--pc-dir` for the terminal emulator (section 2.5). Then you run and copy its files like any others. Pasting a program into the Hydra works too, as long as your terminal program's flow control is set to XON/XOFF (section 5.2), so that the Hydra can ask it to wait. With no flow control, paste a few lines at a time: keys beyond about 300 that the Hydra hasn't read yet are lost.

6.8 Running programs

A program is a file, and you run it by typing its name. For the programs in `/bin`, the name alone is enough, which is how `date` and `ls` worked. For any other, type its path. The SD card that comes with HydraOS has a directory of sample programs:

```
/> /sd/0/sample/hi
Hello, you!
I'm task 2, in /.
/> /sd/0/sample/hi Ann Bob
Hello, Ann!
Hello, Bob!
I'm task 2, in /.
```

Or go to the program's directory first. The shell looks for a program where you are, and then in `/bin`:

```
/> cd /sd/0/sample/c
0:/sample/c> hello world
hello from C, world
```

Here are the samples that are fun to watch. CTRL-C stops any of them.

IN `/SD/0/SAMPLE`	WHAT IT DOES
`hi` *names*	Says hello, and which of the Hydra's sixteen tasks it's running in

IN `/sd/0/sample`	WHAT IT DOES
`tick`	Prints a dot each second until you press CTRL-C, then says how long that was
`upper`	Prints what you type in capitals, until CTRL-D
`nsum` *numbers*	Adds its arguments exactly (try `nsum 1/3 0.5 2`), and finds the sum's square root
`c/chorus`	Four programs at once print a line each of a song, a letter at a time. First they tangle; then they learn to take turns.
`c/race`	Four programs add to the same counter, and lose count, until they use a lock
`c/philo`	Five philosophers share five forks: a famous puzzle about sharing
`c/prodcons`	Two programs make things, two use them up, and nobody drops anything
`c/keys`	Shows the code of each key you press, and the terminal's colours. Q ends it.
`c/tones`	Plays scales, chords and drums on the sound chip (turn the sound on!)
`c/round`	Four programs sing "Row, Row, Row Your Boat" as a round, a voice each
`c/shapes -w`	Draws shapes on the Vera X's screen (it needs a Vera X)
`c/sketch`	A drawing program for the Vera X's mouse (it needs the Vera X's keyboard and mouse)

The first four were written in assembly language, and their sources are there to read, in `/sd/0/sample/as` (Chapter 27). The others were written in C, on a PC. The ones with four or five programs at once show off what the Hydra is named for: **multitasking** (Chapter 7).

And there's music. `play` plays a song file (in the emulator, press the **Sound** button first):

```
/> play /rom/songs/scom.zsm
```

> **TRY THIS:** Run `/sd/0/sample/c/chorus` and watch what happens to the song when four programs print at once with nothing to keep them apart. Then watch the same four take turns. Chapter 7 explains what a task is, and why sharing takes care.

6.9 A taste of three languages

The Hydra has three programming languages built in. Part Four teaches each of them properly. Here's the same little job in all three, so you can see how they feel: print the squares of some numbers.

HyForth is the language of the shell itself, so you're already in it. A new word is defined with `:` and ends with `;`. This one makes a copy of the number on top (`dup`) and multiplies the two (`*`):

```
/> : square dup * ;
/> 7 square .
49
/> : hi ." Hello, I'm a Hydra!" ;
/> hi
Hello, I'm a Hydra!
```

HyBASIC is the language most home computers of the 1980s had. Type `basic` to start it. Lines that begin with a number are kept as a program, `RUN` runs it, and `SYSTEM` goes back to the shell:

```
/> basic
> 10 FOR I = 1 TO 5
> 20 PRINT I; "squared is"; I * I
> 30 NEXT I
> RUN
 1 squared is 1
 2 squared is 4
 3 squared is 9
 4 squared is 16
 5 squared is 25
> SYSTEM
/>
```

hylang is the Hydra's own language, where everything is written in parentheses, with what to do first. Type `hylang` to start it, and `(exit)` to leave:

```
/> hylang
hylang (danlang on the Hydra-16)
Type 'exit' to Exit

hylang> (print "Hello!")
Hello!
=> NIL
hylang> (* 7 7)
=> 49
hylang> (exit)
=> exit
/>
```

Which should you learn first? If you've never programmed before, start with **HyBASIC** (Chapter 23): it reads almost like English, and it's the gentlest. **HyForth** (Chapter 19) is the one you'll meet all through this book, because it's the shell. **hylang** (Chapter 21) suits people who like mathematics and lists. And when you want to know what the processor itself is doing, Part Five is waiting. Chapter 25 writes one game in all three languages, side by side, if you'd like to compare them before you choose.

6.10 Finding help

The Hydra has no `help` command. This book is its help, and it's arranged for looking things up:

YOU WANT TO KNOW	LOOK IN
What a command does, and its options	Chapter 11, which has every command in alphabetical order
What an error message means	Appendix G
What a computer word means	Appendix J, the glossary
A word of HyForth, hylang or HyBASIC	Appendices A, B and C
Where anything is explained	The index, at the very back

The Hydra can tell you a few things itself. A command with the wrong arguments usually shows its **usage**: the arguments it takes, with the ones you can leave out in square brackets. And `whatis` says where a command's program is:

```
/> cp
usage: cp [-r] from to, or cp [-r] from ... dir
/> whatis date
/bin/date
```

`ls /bin` lists every command there is, and in HyForth, `words` lists every word it knows.

6.11 Stopping for the day

1. Save what you're working on, and leave the editor.
2. Copy anything worth keeping from `/ram` to `/sd/0` (section 6.7).
3. Switch off. On a real Hydra, use the power switch (section 5.6). In the web emulator, close the page: your SD card stays in the browser. In the terminal emulator, press CTRL-A and then X.

6.12 What you can do now

You can now:

- type a command and its arguments, and fix a line before and after you send it;
- tell from a message what the Hydra didn't understand;
- stop a program, end your input, and tell from the prompt which program is listening;
- move around the files with `ls`, `cd` and `pwd`;
- make a file with `echo` or `edit`, look at it with `cat`, and copy, rename and remove it;
- keep your work on the SD card, where it's safe;
- run a program by its name or its path;
- start and leave each of the three languages.

That's enough to follow everything else in this book. **Chapter 7** goes back to the moment of switching on, and explains what the Hydra did in its first second. If you'd rather get on with *using* it, skip to **Chapter 8**, the shells, and come back to Chapter 7 when you're curious. And if you can't wait to program, jump to **Part Four**: you know enough already.

CHAPTER 7

How the Hydra Starts

Every time it's switched on, the Hydra takes about a second to start (section 5.3 shows what it prints as it does). In that second, a lot happens. This chapter follows it step by step, from the moment the power comes on to the prompt. On the way, you'll meet the most important ideas in HydraOS: memory and its banks, tasks, the kernel, drivers and the shell.

IN THIS CHAPTER: You'll learn how the 65C02 reaches megabytes of memory through "windows", what tasks, the kernel, drivers and init are, and what every line of the start-up messages means. **You need:** Chapter 6, and hexadecimal (section 1.6). This is the deepest chapter in Part Two. On a first reading, sections 7.1 to 7.3 and 7.5 are enough: come back to section 7.4 when you want to know what a start-up message means, or when you reach Part Five.

7.1 Waking up

When the power comes on, the Hydra's memory chips are full of random numbers, and the processor knows nothing at all, except one thing: where to find its first instruction.

Every 65C02 processor does the same thing when it starts. It reads two bytes from the very top of its memory, at addresses `$FFFC` and `$FFFD`. Together they make the **address** of the first instruction: the **reset vector**. The processor jumps there and starts work.

But the processor doesn't start straight away. The little three-legged **reset chip**, U43, watches the +5 V supply. It holds the processor's *reset* pin down until the power is steady, and then for 150 milliseconds (thousandths of a second) more. The reset button works through it too: while the button is pressed, the processor waits, and 150 milliseconds after it's let go, the processor starts again from the reset vector.

7.2 The memory map

The 65C02 can only see 65,536 bytes of memory at once (that's 64K, or `$10000` in hex). Each byte has an **address**, from `$0000` to `$FFFF`. But the Hydra has far more memory than that: 512K of task RAM, 2 MB of shared RAM, 2 MB more on each RAM module, and up to 4 MB of ROM. How does the 65C02 reach it all?

The answer is **windows**. Parts of the 64K are windows onto bigger memories behind them. A **register** (a small memory of a few bits, which the processor writes like any other address) chooses which part of the bigger memory shows through the window. It's like reading a long scroll through a hole in a card: you can only see a little of it at once, but you can slide the scroll to see any part.

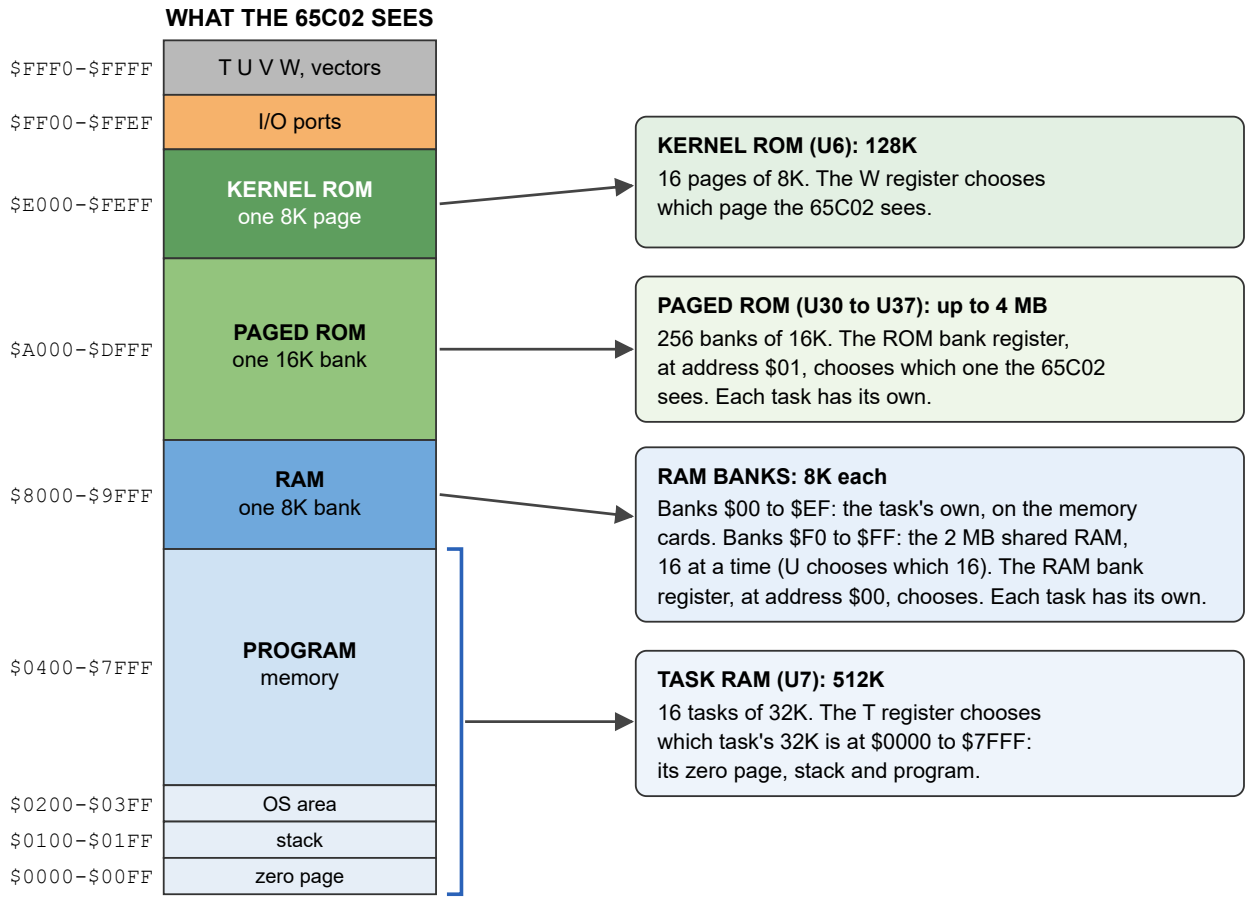


Figure 7-1. The Hydra's memory map: what the 65C02 sees, and the bigger memories behind its windows.

ADDRESSES	WHAT'S THERE	CHOSEN BY
`$0000-$7FFF`	**Task RAM**: 32K of U7, the running task's own	T
`$8000-$9FFF`	A **RAM bank**: 8K of the memory cards or the shared RAM	The RAM bank register (`$00`), and U
`$A000-$DFFF`	A **paged ROM bank**: 16K of the paged ROM chips	The ROM bank register (`$01`)
`$E000-$FEFF`	A **kernel ROM page**: 8K of U6	W
`$FF00-$FFEF`	The **I/O ports**: the chips that talk to the world	(fixed)
`$FFF0-$FFF3`	The registers **T, U, V, W**	(fixed)
`$FFFE-$FFFF`	The **vector RAM**: where interrupts go	V, or the interrupt

The four registers at `$FFF0` to `$FFF3` are the Hydra's own:

REGISTER	ADDRESS	WHAT IT CHOOSES
T	`$FFF0`	The **task**: which 32K of U7 is at `$0000-$7FFF`
U	`$FFF1`	Which 16 of the shared RAM's 256 banks are banks `$F0-$FF`
V	`$FFF2`	Which interrupt address the 65C02 reads when no interrupt is waiting
W	`$FFF3`	The **kernel ROM page** at `$E000-$FEFF`

And two more registers belong to each task: the **RAM bank register**, written at address `$00`, and the **ROM bank register**, written at `$01`. Each task has its own pair, so each task can look at a different bank. (On the board, four 74LS219 chips, IC1 to IC4, are tiny memories that keep every task's bank registers. The other four, IC5 to IC8, are the vector RAM.)

> **HOW IT WORKS:** Addresses `$00` and `$01` are also the first two bytes of each task's zero page. Writing them sets both the bank register and the RAM behind it, so a program can read back the bank it chose from the same address.

The I/O ports are the chips that do the Hydra's input and output:

PORT	ADDRESSES	CHIP	WHAT IT DOES
0	`$FF00-$FF0F`	65C22 VIA (U2)	The tick (a timer), the SPI devices, the I/O header J27
1	`$FF10-$FF1F`	65C51 ACIA (U3)	The serial port
2, 3	`$FF20-$FF3F`	The card in slot 0	The Vera X: screen, sound, SD card, keyboard
4	`$FF40-$FF4F`	YM2151 (U38)	Sound
5-9	`$FF50-$FF9F`	Slots 1 to 5	Expansion cards (select A)
10-14	`$FFA0-$FFEF`	Slots 1 to 5	Expansion cards (select B)

7.3 Tasks: sixteen computers in one

The Hydra can run sixteen programs at once. Each program runs in a **task**, and each task has its own 32K of memory at `$0000-$7FFF`, chosen by the T register. Each task has its own zero page, its own stack, its own bank registers and its own program. A program in one task can't see or spoil the memory of another. It's almost like having sixteen small computers, sharing one processor.

But the processor can only do one thing at a time, so the tasks take turns. Two hundred times a second, a timer in the VIA gives the processor a nudge, called an **interrupt**, and the **scheduler** in the kernel decides whose turn it is next. It saves where the running task was, writes the next task's number into T, and carries on with that task where it left off. The turns are so short and so quick that the tasks seem to run all at once.

The tasks are numbered from 0 to 15, or in hex, 0 to F:

- **Task 0** is the **kernel's**: the heart of HydraOS, which looks after all the others. When nothing else needs to run, it puts the processor to sleep until the next interrupt.
- **Task 1** is **init**, the first program.
- **Programs** take the lowest free task: 2, 3, 4, and so on.
- **Drivers** take the highest: F, E, D, and so on down.

A task's 32K is laid out like this:

ADDRESSES	WHAT'S THERE
`$00-$01`	The bank registers
`$02-$21`	r0 to r15, sixteen 2-byte **call registers**: how a program passes values to HydraOS and gets answers back
`$22-$7F`	The program's own zero page
`$80-$FF`	HydraOS's zero page for this task
`$0100-$01FF`	The stack
`$0200-$03FF`	HydraOS's area for this task: its open files, its name, and more
`$0400-$7FFF`	The program's memory: its code (unless it runs from ROM) and its data

7.4 Step by step

Here is everything the Hydra does, from the reset to the prompt.

Step 1: The reset

The reset vector points to address `$E000`, in the kernel ROM. But which kernel ROM page is there? The W register isn't cleared by a reset, so it could be any of the sixteen. That's why every page has the same instruction at `$E000`: `STZ W_REGISTER`, which stores zero in W. The very next instruction comes from page 0, wherever the processor started, and page 0 goes on with the start.

The kernel turns off interrupts, chooses task 0 (it stores 0 in T), and sets up its stack.

Step 2: Every task made ready

T, U, V, the bank registers and all the RAM start with random values, so the kernel sets every one before it's used. It visits each of the sixteen tasks in turn, writing T, and clears its bank registers and HydraOS's part of its zero page. Then it marks each task "free", and clears its own tables.

Step 3: The self test

Next the kernel starts a simple way of printing on the serial port, the **bring-up console**, and prints its greeting:

```
Hydra-16: kernel 0.1, ABI 1
```

(*ABI 1* is the version of the way programs call HydraOS, so that a program can check it's talking to a kernel it understands.)

Then it runs the **power-on self test**, or **POST**, which checks the memory and the registers that everything else depends on, and prints what it found:

```
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000
POST ok
```

The first line tests the task registers and the kernel ROM pages:

ITEM	WHAT POST DID	WHAT'S PRINTED
`ZP`	Tasks 8, 4, 2, 1 and 0 each wrote their own number into address `$00FF`, in their zero page, then read it back	The T lines (in hex: bit 0 is T0) that didn't keep the tasks apart. 0 is good.
`ST`	The same, at `$0100`, in the stack	The same
`OS`	The same, at `$03FF`, in HydraOS's area	The same
`HI`	The same, at `$7FF0`, near the top of the task's 32K	The same
`SH`	Shared bank `$F0`, written by task 0 and read by task 1	`S` if it's shared, as it should be; `X` if not
`W`	Each kernel ROM page has its own number at `$FDFF`; POST chose each page with W and read it	The W lines that didn't choose their page. 0 is good.

The second line tests the RAM banks. `U:` is the U register's lines. Then comes one item for each RAM chip and module it tested: the first bank of each shared RAM chip (banks `$F0`, `$F4`, `$F8` and `$FC`) and the first bank of each RAM module it found (`$00` for module 0, `$10` for module 1, and so on). Each item looks like `bb:x/dd/aaaa`:

PART	WHAT IT SHOWS
`bb`	The bank tested
`x`	The bank register's lines that don't work

PART	WHAT IT SHOWS
`dd`	The **data lines** (D0 to D7, one bit each) that don't work
`aaaa`	The **address lines** (A0 to A12, one bit each) that don't work

So `F4:0/00/0000` means "bank `$F4`: all good." If POST finds a fault it prints `POST found faults:` and names them (Chapter 5 has a table of where to look), and leaves any bad memory unused.

> **HOW IT WORKS:** Testing the lines one at a time finds the faults a builder is most likely to make: a missing joint or a bridge between two pins. A data line that doesn't work changes the same bit of every byte, so writing `$01`, `$02`, `$04` ... and reading them back finds it. An address line that doesn't work makes two addresses into one, so writing different values to address `$0000` and address `$0001`, `$0002`, `$0004` ... and reading them back finds it.

POST also counts the RAM modules on the memory cards (it tries each of the 15 possible module numbers), and prints how many it found, in hex:

```
RAM modules: 01
```

If a T was typed while POST ran, the Hydra starts the **hardware test** now, instead of HydraOS (section 5.7).

The kernel keeps a copy of everything it prints from here on. Any time later, you can read it again:

```
/> cat /dev/kmesg

Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000
POST ok
RAM modules: 01
task F: cons
task E: storage
task D: kdev
task C: snd
task B: vid
task 1: init
```

Step 4: Interrupts

An **interrupt** is a signal from a chip that says "I need attention now": a key has arrived at the serial port, a timer has run out, the sound chip has finished a note. The Hydra has sixteen interrupt lines, numbered 0 to 15; nothing is wired to line 15, so the kernel uses its entry for `BRK`. When one is active, a chip on the board (U17 and U18 find the most urgent line) makes the 65C02 read the interrupt's own address from the vector RAM, so each line goes straight to its own code. The lower the number, the more urgent:

LINE	COMES FROM
0	The VIA: its timers (the tick) and CA1
1	The ACIA: the serial port
2, 3	Slot 0: the Vera X
4	The YM2151: its timers
5-9	Slots 1 to 5 (select A)
10-14	Slots 1 to 5 (select B)
15	Nothing: used for `BRK`

The kernel sets up the vector RAM, and gives itself line 0, for the tick.

Step 5: The drivers

A **driver** is a program that looks after a device: the serial port, the SD cards, the sound chip. Programs don't touch the devices themselves; they ask the drivers. The kernel starts the **boot drivers**, each in its own task, from the highest down, and prints each one's name:

TASK	DRIVER	WHAT IT LOOKS AFTER
F	`cons`	The console: the serial port, its windows, and `/pc`
E	`storage`	The SPI bus, the SD cards, the RAM disks and the ROM disk, and their files
D	`kdev`	The kernel's own devices: the top of the file tree, the tasks (`/proc`), pipes, and more; and the I/O header J27, the I²C bus on it, and a Vera X's keyboard and mouse
C	`snd`	The sound chips: the YM2151, and the Vera X's
B	`vid`	The Vera X's screen. With no Vera X, it finds nothing to do and ends

There are only sixteen tasks, so HydraOS keeps few of them standing: small jobs share a driver's task (the I/O header's pins are `kdev`'s), and a program that would only wait gets out of the way (as init does, below).

Like most of HydraOS, the drivers live in the paged ROM, in **modules**. A module runs *in place*: its task's ROM bank register shows its bank at `$A000-$DFFF`, so its code never needs copying into RAM. A big module has a bank (or several) of its own; small ones share a bank, each in its own part of it.

Chapter 12 explains how drivers work, and Part Three describes each one.

Step 6: The tick, and init

The kernel starts timer 1 in the VIA, set to interrupt **200 times a second**: the **tick**, which drives the scheduler and the clock. It turns interrupts on, and the drivers run their start-up code. When they're ready, the kernel starts the first program, **init**, in task 1, and then becomes the **idle task**: whenever no other task needs the processor, it lets the processor sleep (the 65C02's `WAI` instruction) until the next interrupt.

Step 7: init sets up the system

init does everything needed to make the Hydra ready for you:

1. It opens the **console**: window 0 of the `cons` driver. Its output now goes there.
2. It starts the two **RAM disks**: `r`, 256K of RAM banks on the memory cards, and `s`, 512K of the shared RAM. (With less memory, it makes them smaller.)
3. It builds its **namespace**, its tree of files, from the file `/rom/lib/namespace` (and from `/lib/namespace` on an SD card, if there is one). Chapter 9 explains namespaces.
4. It sets up the console's **windows** from `/lib/windows` (Chapter 10).
5. It prints `HydraOS 1.0 for the Hydra-16`, and which task it's in, and lists the tasks:

```
init: up in task 01
T ST FL PA CPU    NAME
0 01 00 FF 000052 kernel
1 01 00 FF 000017 init
C 04 01 FF 000003 snd
D 06 01 FF 000011 kdev
E 04 01 FF 000075 storage
F 08 01 FF 00003F cons
```

The columns are all in hex:

COLUMN	WHAT IT SHOWS
`T`	The task's number
`ST`	Its **state**: 01 ready to run, 02 waiting, 03 calling a driver, 04 a driver waiting for work, 06 asleep, 07 waiting for a busy driver, 08 waiting for a driver to have news
`FL`	Its **flags**: 01 means it's a driver
`PA`	Its **parent**, the task that started it (FF: none)
`CPU`	How much of the processor's time it has had, in ticks (200 a second)
`NAME`	The program running in it

(Task B isn't there: without a Vera X, `vid` has already ended. `kdev` is asleep for a moment: it's looking for a Vera X's keyboard, three times in its first three seconds. And `cons` is waiting for its own news: a key from you.)

6. It runs a small program, **hello**, and waits for it to end. hello greets its arguments, `from init`, and ends with the code 7 and the message `bye`. It's a quick check that programs can start, run and end:

```
hello, from init
init: hello ended: code $07 (bye)
```

7. It **becomes the shell** in window 0. Which shell? The one named in `/lib/shell`. On the ROM disk, that file says `/bin/forth -l`: HyForth, as a login shell. init doesn't start the shell in another task and wait for it: it asks the kernel to put the shell *in its own place* (the `EXEC` call), so task 1, with its console and its files, is the shell's from then on, and init is gone.

So no task is used up just waiting. Two jobs that a waiting init would do are the kernel's instead:

- **If the shell in window 0 ends**, the kernel starts init again in a free task, and init becomes a new shell there (this time it skips steps 2, 4, 5 and 6). There's always a shell to type at.
- **When you ask for a new window** (Chapter 10), the console's driver makes the window and asks the kernel to start init for it; init opens that window's console, and becomes its shell.

Step 8: The shell starts

HyForth starts, and loads its start-up file, `/lib/forth/startup.fs`, which adds four sets of words to its **dictionary**: the Core Extension, Exception, File Access and Programming-Tools word sets (Chapter 19). Because it was started with `-l`, as a **login shell**, it then reads `/lib/forth/profile.fs`, which turns HyForth into a shell: it loads `shell.fl`, the words that let it run commands, and sets up the window. Then it says hello, and prints its prompt:

```
HyForth (Forth 2012), bye to end
/>
```

The prompt shows where you are in the file tree: `/` is the top. The Hydra is ready.

7.5 What's running now

The `ps` command lists the tasks, in a friendlier form than init's table, with the task numbers in decimal:

```
/> ps
task  state   parent     cpu group  name
   0  ready   -          1.6     0  kernel
   1  wait    -          0.5     1  forth
   2  ready    1         0.0     1  ps
  12  idle    -          0.0    12  snd
  13  idle    -          0.1    13  kdev
  14  idle    -          1.6    14  storage
  15  event   -          0.4    15  cons
```

You can see the kernel (task 0), the shell, `forth` (task 1: it's where init was), `ps` itself (task 2: the shell started it, so its parent is 1), and the four drivers in tasks 12 to 15 (C to F). That leaves ten tasks free for your programs and for more windows. The `cpu` column is in seconds. Chapter 11 explains the `group` column.

TRY THIS: Run `ps` again. Has anything changed? Which task do you think did most of the work while the Hydra started?

CHAPTER 8

The Shells

A **shell** is the program that reads your commands and carries them out. It's called a shell because it's the outside of the system: the part you touch. This chapter shows how HydraOS's shells work, and teaches you **rc**, the Hydra's command language.

IN THIS CHAPTER: You'll learn how HyForth and rc share the keyboard, and all of rc's language: variables, quotes, sending input and output to files, pipes, patterns, tests, loops, functions and scripts. **You need:** Chapter 6.

8.1 Two shells in one

When the Hydra starts, its shell is **HyForth**. HyForth is a programming language (Part Four teaches it), but it's also a shell, because it hands every line that isn't Forth to a second shell, **rc**. The rule is simple:

- If the line's first word is a **Forth word** that HyForth knows, or a **number**, the line is Forth.
- Otherwise, the line is a **command** for rc.

```
/> 2 3 + .
5
/> echo Hello
Hello
```

`2` is a number, so the first line is Forth. `echo` isn't a Forth word, so rc runs the second line. You can type commands without thinking about Forth at all.

Sometimes a program has the same name as a Forth word. To make sure rc gets a line, start it with `%` and a space:

```
/> % echo This line is for rc
This line is for rc
```

You can also start rc itself, as a shell of its own, by typing `rc`. Its prompt is `%`. Type `exit` to leave it and go back to HyForth:

```
/> rc
% echo hello
hello
% exit
/>
```

The rest of this chapter's rc examples are typed at rc's `%` prompt, but they work just the same at HyForth's `/>` prompt.

8.2 Commands

A command is a **program's name**, then its **arguments**: words that tell it what to do. The words are separated by spaces:

```
% echo one two three
one two three
```

Many programs take **options**: arguments starting with –, which change what they do. `ls -l` asks `ls` for a long listing, with the files' sizes. Chapter 11 lists every program and its options.

Where programs come from. When you type a program's name, rc looks for it in the directories listed in the variable `$path`:

```
% echo $path
. /bin
```

`.` is the directory you're in, and `/bin` is where the Hydra's programs are. (`/bin` is really several directories joined together: the ROM's programs, the ROM disk's, the SD card's and the RAM disks'. Chapter 9 explains how.) A name with a `/` in it, like `/ram/myprog`, is found by its path, not by `$path`.

Stopping a program. CTRL-C stops the program that's running, and brings back the prompt.

8.3 rc: words and quotes

rc splits a line into words at spaces and tabs. Some characters have special jobs, and end a word too:

CHARACTERS	THEIR JOB
`;`	Ends a command, so another can follow on the same line
`&`	Runs a command in the background
`|`	A pipe: one program's output into another's input
`< >`	Redirection: input from, or output to, a file
`^`	Joins two words into one
`$`	A variable's value
`` ` ``	A command's output, as words
`{ } ( )`	Group commands, or words
`=`	Sets a variable
`#`	A comment: rc ignores the rest of the line

To use any of them in a word, **quote** the word with single quotes, `'...'`. Inside quotes, every character is itself. A quote mark inside quotes is written twice:

```
% echo 'it''s quoted: $x | ;'
it's quoted: $x | ;
```

A `\` at the very end of a line joins the next line to it, so a long command can be typed on two lines.

8.4 Variables

A **variable** is a name for some words. Set it with = (no spaces around the =), and use its value with `$`:

```
% x=(apple banana cherry)
% echo $x
apple banana cherry
```

In rc, every variable is a **list** of words. The parentheses make a list of several words; a single word needs none (`y=hello`). The list can be empty: `x=()`.

WRITTEN	GIVES
`$x`	All the words in `x`
`$#x`	How many words are in `x`
`$x(2)`	The second word (`$x(1 3)`: the first and third)
`$"x`	All the words, joined into one, with spaces between

```
% echo $#x
3
% echo $x(2)
banana
```

Every variable is part of the **environment**, which the programs you run can read too. Some variables have special jobs:

VARIABLE	WHAT IT HOLDS
`$path`	Where programs are found: `. /bin`
`$prompt`	The prompt: its first word, and its second, which is shown when a command goes on to another line
`$status`	How the last command ended: empty or `0` for success; anything else is a failure
`$task`	This shell's task number
`$apid`	The task number of the last command run in the background
`$window`	The number of this shell's window (Chapter 10)
`$home`	Where `cd` goes with no directory
`$*`	A script's or function's arguments; `$1`, `$2` ... are each one, and `$0` is the script's name

One more goes to each program rc starts, in its environment (`/env/prog`, or `getenv` in C), though it isn't a variable of rc's own: `prog`, the path the program was started from (`/bin/ls`, `/ram/round`). A program that starts copies of itself finds its own file by it.

8.5 Joining words

`^` joins two words into one. If either side is a list, it joins each word in turn:

```
% y=hello; echo $y^.txt
hello.txt
% echo a^(1 2 3)
a1 a2 a3
```

When a variable is right next to other text, rc puts in the `^` for you: `$y.txt` means the same as `$y^.txt`.

> **NOTE:** Joining with an empty list is an error: if `$z` is empty, `$z^.txt` makes rc say `null list`.

8.6 Input and output

Every program has three **standard files**, numbered 0, 1 and 2:

NUMBER	NAME	NORMALLY
0	Standard input	Your keyboard
1	Standard output	Your screen
2	Standard error	Your screen: for error messages

A **redirection** connects one of them to a file instead:

REDIRECTION	WHAT IT DOES
`>file`	Standard output into the file (made empty first)
`>>file`	Standard output added to the end of the file
`<file`	Standard input from the file
`>[2]file`	Standard error into the file (any number works: `>[1]` is the same as `>`)
`>[2=1]`	Standard error to wherever standard output goes
`>[2=]`	Standard error closed

```
% echo one >/ram/t.txt
% echo two >>/ram/t.txt
% cat /ram/t.txt
one
two
% wc -l </ram/t.txt
      2
```

The file `/dev/null` swallows whatever is written to it, which is handy for messages you don't want to see:

```
% ls /nothing
ls: /nothing: not found
% ls /nothing >[2]/dev/null
```

8.7 Pipes

A **pipe**, `|`, connects one program's standard output to the next one's standard input, so the first program's output becomes the second one's input. `wc -l` counts lines, so this counts the files on the ROM disk:

```
% ls /rom | wc -l
      5
```

A line can pipe several programs together. This one lists the ROM disk's programs, keeps the names with an `e` in them, sorts those backwards, and shows the first three:

```
% ls /rom/bin | grep e | sort -r | head -3
xmodem
tee
sleep
```

And `|[2]` pipes standard error instead of standard output.

> **NOTE:** Each program in a pipeline is a task, and the Hydra has sixteen tasks for everything (`ps` shows them). On the book's Hydra that leaves room for nine programs in one pipeline. A tenth isn't started: rc says `no free task` for it, and the others run without it. For a longer job, put the half-way result in a file in `/ram`, and carry on from there.

8.8 A command's output as words

`{command} runs the command, and puts its output into the line as words:

```
% echo today is `{date}
today is 2000-01-01 00:00:05
```

8.9 Success and failure

Every program ends with a **status**: success, or a number that says what went wrong. rc keeps it in `$status`:

```
% ls /nothing
ls: /nothing: not found
% echo $status
1
```

Commands can be joined by what happened:

WRITTEN	MEANS
`a && b`	Run `b` only if `a` succeeded
`a || b`	Run `b` only if `a` failed
`! a`	Run `a`, and turn success into failure, and failure into success

```
% ls /nothing >[2]/dev/null || echo not there
not there
```

8.10 Patterns

In a word that isn't quoted, `*`, `?` and `[` make a **pattern**, which rc replaces with the names of the files that match it:

PATTERN	MATCHES
`*`	Any letters, or none
`?`	Any one letter
`[abc]`	One of the letters a, b or c
`[a-z]`	One letter from a to z
`[~a-z]`	One letter that isn't from a to z

```
% ls /rom/lib/[ns]*
/rom/lib/namespace
/rom/lib/shell
```

If no file matches, the word is left as it was.

The ~ command matches words against patterns, without looking at files: `~ word patterns` succeeds if the word matches any of the patterns.

8.11 Decisions and loops

if runs a command if another succeeds. **if not** runs a command if the last `if`'s test failed:

```
% if(~ $x(1) apple) echo yes
yes
% if not echo no
```

for runs a command once for each word in a list, setting a variable to each word in turn:

```
% for(i in 1 2 3) echo number $i
number 1
number 2
number 3
```

while runs a command as long as a test succeeds. `{ }` groups several commands into one. (`calc` is the Hydra's calculator, Chapter 11.)

```
% i=3; while(! ~ $i 0) { echo $i; i=`{calc $i - 1} }
3
2
1
```

A `while()` with nothing in its parentheses has no test to fail, so it goes round for ever, until CTRL-C or an `exit` stops it. (An `if()` with nothing in them always runs its command.)

switch compares a word with patterns, and runs the commands after the first `case` that matches:

```
% switch($y){ case h*; echo starts with h; case *; echo other }
starts with h
```

8.12 Functions

A **function** is a command you make yourself, from other commands. Inside it, `$*` is its arguments, and `$1`, `$2` ... are each one:

```
% fn greet { echo Hello, $1! }
% greet Sam
Hello, Sam!
% fn count { echo $#* things: $* }
% count a b c
3 things: a b c
```

`whatis` shows what a name is to rc: a function, a variable, a built-in, or a program, and where it is:

```
% whatis greet
fn greet { echo Hello, $1! }
% whatis ls cd x
/bin/ls
builtin cd
x=(apple banana cherry)
```

`fn greet`, with no `{ }`, removes the function again.

8.13 Scripts

A **script** is a file of commands. Its first line can be `#!/bin/rc`, which tells HydraOS to run the file with rc, so you can run the script by its name, like a program. Here's a two-line script, made with `echo`:

```
% echo '#!/bin/rc' >/ram/hi
% echo 'echo hello from $0, you said $*' >>/ram/hi
% rc /ram/hi one two
hello from /ram/hi, you said one two
```

You can run a script three ways:

TYPED	WHAT HAPPENS
`rc file args`	A new rc runs the file's commands
`file args`	The same, if the file's first line is `#!/bin/rc`
`. file args`	This rc runs the file's commands itself, so the variables and functions the file sets stay set

8.14 rc's built-in commands

A few commands change the shell itself (its directory, its variables, its namespace), so they can't be programs: they're built into rc.

COMMAND	WHAT IT DOES
`cd [dir]`	Go to directory *dir* (with none: `$home`, or `/`)
`exit [status]`	End rc, with a status
`wait [task]`	Wait for the commands running in the background (or just one task) to end
`eval words`	Run the words as a command line
`. file [args]`	Run the file's commands in this shell
`builtin cmd args`	Run *cmd* as a built-in or a program, even if there's a function called *cmd*
`whatis names`	Say what each name is
`shift [n]`	Remove the first *n* (or 1) words from `$*`
`bind [-a|-b] [-c] new old`	Add a directory to the namespace (Chapter 9)
`mount [-a|-b] [-c] '#x' old [spec]`	Add a device's files to the namespace (Chapter 9)
`unmount [new] old`	Take something back out of the namespace (Chapter 9)
`newns`	Build the namespace again, as it was when the shell started (Chapter 9)
`exec cmd args`	Run *cmd* in the shell's place: the shell ends, and the program takes over its task

rc itself can be started with options:

TYPED	WHAT IT DOES
`rc`	Reads commands from the keyboard, with a prompt
`rc file args`	Runs the file's commands, then ends
`rc -c 'commands'`	Runs the commands, then ends
`rc -l`	A **login shell**: builds the namespace and runs `/rom/lib/profile` first
`rc -i`	Reads commands with a prompt, even from a file

8.15 HyForth's shell words

A command that rc runs, runs in a task of its own, so it can't change HyForth's own directory or namespace. So HyForth has its own words for those jobs, and a few more:

WORD	WHAT IT DOES
`% command`	The rest of the line is rc's
`cd dir`	Go to a directory (with none: `$home`)
`bind`, `mount`, `unmount`, `newns`	The same as rc's (section 8.14)
`status` (-- n)	The status of the last command (0: success)
`exit`	At the prompt: leave HyForth (in a definition, it's Forth's `EXIT`)
`exits` (n --)	Leave HyForth with status *n*
`wait` (task --)	Wait for a task started in the background
`getenv` (c-addr u -- c-addr2 u2)	An environment variable's value
`setenv` (c-addr u c-addr2 u2 --)	Set an environment variable
`unsetenv` (c-addr u --)	Remove an environment variable
`spawn` (c-addr u -- task)	Start a command in the background; its task number
`sh-out` (c-addr u -- c-addr2 u2)	Run a command line, and give its output as a string
`output-of` (xt -- c-addr u)	Run a Forth word, and give what it printed as a string
`xt | command`	What a Forth word prints, piped into a command
`piped` (xt c-addr u --)	The same, with the command as a string
`send n line`	Type a line into window *n*'s shell, as if you'd typed it there
`prompt` (c-addr u --)	Set the prompt
`prompt2` (c-addr u --)	Set the second prompt, shown while you're typing a definition

(The words in parentheses, like `( c-addr u -- )`, show what each word takes from Forth's stack, and leaves there. Chapter 19 explains them.)

```
/> ls /nothing
ls: /nothing: not found
/> status .
1
/> s" date" sh-out type
2000-01-01 00:00:05
/> : hi ." Hello" cr ." there" cr ;
/> ' hi | wc -l
      2
/> s" color" s" green" setenv
/> echo $color
green
```

The prompt. HyForth's prompt is made from a **format**: its letters are shown as they are, except for these:

CODE	SHOWS
`%d`	The directory you're in; on an SD card, the path on the card
`%v`	On an SD card, its number and a colon: `0:`
`%p`	The directory's whole path

CODE	SHOWS
`%t`	The shell's task number
`%w`	The window's number
`%%`	A `%`

The usual format is `%v%d>` , which shows `/rom/lib>` in `/rom/lib`, and `0:/games>` in `/sd/0/games`:

```
/> cd /rom/lib
/rom/lib> s" %t %p> " prompt
1 /rom/lib> s" %v%d> " prompt
/rom/lib>
```

8.16 Choosing your shell

Three of the Hydra's languages can be your shell, and each follows the same idea: a line in the language is the language's, and any other line is rc's.

SHELL	START IT	ITS LINES ARE
HyForth	`forth -l`	Lines starting with a Forth word or a number
hylang	`hylang -l`	Lines starting with `(`, `{` or `[`
HyBASIC	`basic -l`	BASIC's statements
rc	`rc -l`	Everything

The `-l` makes each one a **login shell**: it builds its namespace, and runs its **profile**, a file of commands for it to run as it starts. Each has its own:

SHELL	ITS PROFILE
HyForth	`/lib/forth/profile.fs`
hylang	`/lib/hylang/profile.hl`
HyBASIC	`/lib/basic/profile.bas`
rc	`/rom/lib/profile`

The shell init becomes in each window is named in the file `/lib/shell`. On the ROM disk it says:

```
/bin/forth -l
```

To choose another, put a file called `shell` in the `lib` directory of your SD card (`/sd/0/lib/shell`), with one line: `/bin/hylang -l`, say, or `rc -l`. Because `/lib` is a union (Chapter 9), the card's file is found before the ROM disk's. init reads the file again each time it starts a shell, so the simplest way to make the change is to reset the Hydra.

CHAPTER 9

Files and Namespaces

Everything the Hydra keeps, it keeps in **files**: programs, songs, your notes, its settings. Even its devices look like files. This chapter shows how the files are arranged, where they live, and how HydraOS builds the "tree" of names that you see, which it calls a **namespace**.

IN THIS CHAPTER: You'll learn paths, the disks, how devices appear as files, and how each task gets a tree of names of its own: binding, unions, and a namespace that you arrange yourself. **You need:** Chapter 6, and a little rc (sections 8.1 to 8.6).

9.1 Files and directories

A **file** is a named collection of bytes: a program, a picture, a story. A **directory** is a file that holds other files (it's like a folder on a PC). Directories can hold directories, which hold more files and directories, and so on, making a **tree**. The directory at the top, which holds everything, is the **root**, and is written `/`.

A file's **path** is the list of directories you go through to reach it from the root, with a `/` between each name. `/rom/lib/forth/startup.fs` is the file `startup.fs`, in the directory `forth`, in `lib`, in `rom`, in the root.

A name can be up to 31 characters long, and can use any character but `/`. Capital letters count: `Notes` and `notes` are two different names.

Where you are. Every task has a **current directory**: the directory it's "in". `pwd` shows it, and `cd` changes it. HyForth's prompt shows it too:

```
/> cd /rom/lib/forth
/rom/lib/forth> pwd
/rom/lib/forth
```

A path that doesn't start with `/` starts from the current directory. And every directory has two special names: `.` is the directory itself, and `..` is the directory it's in.

```
/rom/lib/forth> cd ..
/rom/lib> pwd
/rom/lib
/rom/lib> ls ../songs
scom.zsm
```

9.2 Looking at files

`ls` lists a directory's files, with a `/` after each directory. `ls -l` gives a **long** listing:

```
/> ls -l /rom
--r--r--r-- fx     1955 2000-01-01 00:00 README
d-r--r--r-- fx        0 2000-01-01 00:00 bin
d-r--r--r-- fx        0 2000-01-01 00:00 doc
d-r--r--r-- fx        0 2000-01-01 00:00 lib
d-r--r--r-- fx        0 2000-01-01 00:00 songs
```

Each line has five parts:

PART	EXAMPLE	MEANS
The mode	`d-r--r--r--`	`d` for a directory; `a` next if the file is append-only (writes always go on its end); then who may read (`r`), write (`w`) and run (`x`) it
The device	`fx`	Which driver keeps the file (`f`: HydraFS), and which disk (`x`: the ROM disk)
The length	`1955`	How many bytes it holds
The time	`2000-01-01 00:00`	When it was last changed
The name	`README`	

> **NOTE:** The mode's `r`, `w` and `x` come in three groups, for the file's owner, its group and everyone else, as on many other systems. HydraOS doesn't have separate users, so all three groups are the same.

To see what's in a file, `cat` it. `more` shows a long file a screenful at a time, and `xd` shows a file's bytes as hex. Chapter 11 has all the commands for files: `cp` copies a file, `mv` moves or renames one, `rm` removes one, `mkdir` makes a directory, and so on.

```
/> mkdir /ram/notes
/> cd /ram/notes
/ram/notes> echo remember the milk >list
/ram/notes> ls -l
--rw-rw-rw- fr       18 2000-01-01 00:00 list
```

9.3 The disks

The Hydra's files are kept on several **disks**. (A disk here means any place that keeps a file system, even if it isn't round!) Each one is part of the tree:

PATH	DISK	WHAT IT IS
`/rom`	`x`	The **ROM disk**: in the paged ROM. HydraOS's programs, libraries and settings. It can't be changed.
`/sd/0` … `/sd/f`	`0` … `f`	The **SD cards**, one for each SPI device that has a card (`/sd/0` is the card on SPI header J18)
`/sd/v`	`v`	The SD card in a Vera X
`/ram`	`r`	Your **RAM disk**: an area of RAM disk `r` that's only this shell's
`/sram`	`s`	The **shared RAM disk**: the same for every task
`/pc`		A folder on your PC, while the PC tool is running (section 9.9)

`df` lists the disks, and how much space each one has free:

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      13 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

The **RAM disks** are made of memory, so they're very fast, but they're emptied each time the Hydra starts. Use them for work in progress, and keep what matters on an SD card.

All of these disks use the Hydra's own way of arranging files on a disk, called **HydraFS**. To use a new SD card, you make an empty HydraFS on it with `mkfs` (Chapter 11). Chapter 14 tells you more about the disks and HydraFS.

9.4 Devices as files

On the Hydra, devices look like files too. The console, the sound chip, the SD cards' raw blocks, the clock, the tasks: each has a directory of files, and you use the device by reading and writing them. For example, the file `/dev/time` is the clock:

```
/> cat /dev/time
2000-01-01 00:00:04
```

Every driver gives its files a **device name** that starts with `#`, such as `#c` for the console and `#f` for HydraFS. Here are the Hydra's devices:

DEVICE	DRIVER	ITS FILES	USUALLY AT
`#/`	kdev	The root: an empty directory for each top-level name	`/`
`#c`	cons	The console: `cons`, `consctl`, `ser`, `serctl`, `wctl` ...	`/dev`
`#n`	kdev	`null`, `zero` and `kmesg`	`/dev`
`#t`	kdev	`ticks`, `time` and `rtc`: the clock	`/dev`
`#a`	snd	The sound chips: `snd`, `sndctl`, `bell`, `psg`	`/dev`
`#g`	kdev	The I/O header J27's pins	`/dev/gpio`
`#i`	kdev	The I²C bus's devices	`/dev/i2c`
`#v`	vid	The Vera X's screen	`/dev/vid`
`#d`	storage	The disks' raw data, and their control files	`/dev/sd`
`#S`	storage	The SPI devices	`/dev/spi`
`#m`	kdev	The modules in the paged ROM	`/dev/mod`
`#s`	kdev	Shared memory segments, by name	`/dev/seg`
`#e`	kdev	The environment: a file for each variable	`/env`
`#p`	kdev	The tasks: a directory for each	`/proc`
`#f`	storage	HydraFS: the files on the disks	`/sd`, `/rom`, `/ram`, `/sram`
`#P`	cons	A folder on the PC	`/pc`
`#|`	kdev	Pipes	

Part Three describes each device's files.

9.5 Namespaces

So far the tree looks like one big tree. But it's really put together from pieces, and each task has its own. The tree a task sees is its **namespace**. HydraOS builds namespaces the way the Plan 9 operating system does, with two operations:

- **bind** puts one part of the tree at another place. `bind /rom/songs /mnt` makes `/mnt` show the files of `/rom/songs`.
- **mount** puts a device's files at a place in the tree. `mount '#f' /rom x` puts HydraFS's disk `x`, the ROM disk, at `/rom`.

```
/> bind /rom/songs /mnt
/> ls /mnt
scom.zsm
/> unmount /mnt
/> ls /mnt
/>
```

`unmount` takes it away again.

Each task starts with a copy of the namespace of the task that started it. If it changes its namespace, only its own copy changes. That's why a `bind` typed in rc (which runs as a task of its own) doesn't change HyForth's namespace:

```
/> rc
% bind -a /rom/lib /mnt
% ls /mnt
as/
basic/
edit/
font/
forth/
hylang/
namespace
profile
shell
windows
% exit
/> ls /mnt
/>
```

HyForth has its own `bind`, `mount` and `unmount` words, for changing its own namespace (Chapter 8).

9.6 The Hydra's tree

When a shell starts, it builds its namespace from the file `/rom/lib/namespace`. Figure 9-1 shows the tree it makes.

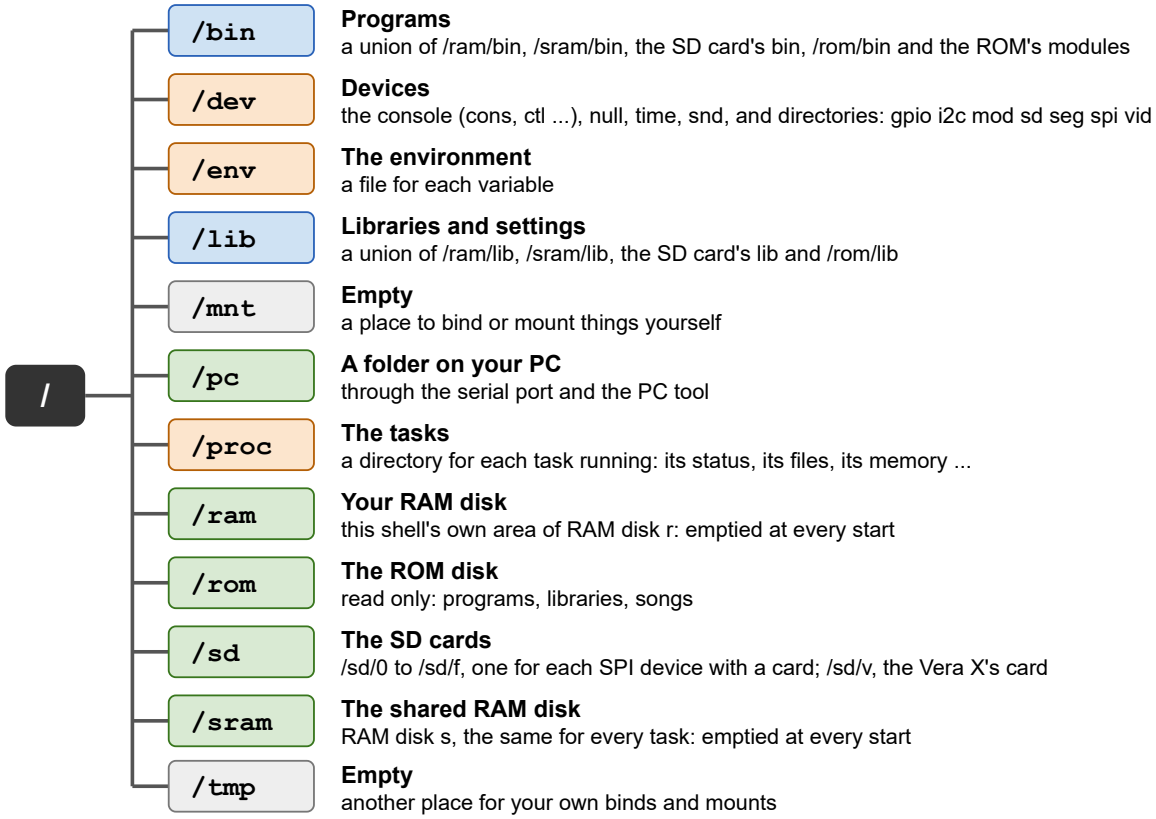


Figure 9-1. The Hydra's file tree.

Here is the namespace file. Each line is a `bind` or a `mount` (and `#` starts a comment):

```
# the root and the devices
bind '#/' /
bind -a '#c' /dev           # cons consctl ser serctl
bind -a '#n' /dev           # null zero kmesg
bind -a '#t' /dev           # ticks
bind '#g' /dev/gpio         # 0-7 port ctl ca1: the VIA's port A on J27
bind '#i' /dev/i2c          # ctl, and a file each I2C device on PA0 and PA1
bind -a '#a' /dev           # snd sndctl bell psg
bind '#v' /dev/vid          # ctl term vram pal sprites font frame psg pcm pcmctl
bind '#d' /dev/sd
bind '#S' /dev/spi
bind '#m' /dev/mod
bind '#s' /dev/seg          # shared segments by name
mount '#e' /env            # the environment
mount '#p' /proc

# the disks
mount '#f' /sd              # the cards: /sd/0 ... /sd/f
mount '#f' /rom x           # the ROM disk
mount '#f' /sram s          # the shared RAM disk
mount -c '#f' /ram r/$task  # this shell's own area of the RAM disk

# programs and libraries: unions, the caches first
bind -c /ram/bin /bin
bind -a /sram/bin /bin
bind -a /sd/0/bin /bin      # (added only if the card has one)
bind -a /rom/bin /bin       # programs on the ROM disk
bind -a '#m/bin' /bin       # the ROM's program modules
bind -c /ram/lib /lib
bind -a /sram/lib /lib
bind -a /sd/0/lib /lib
bind -a /rom/lib /lib

# the PC
mount '#P' /pc              # a folder on the PC, through the serial port
```

(It's shortened a little here: the real file has some more comments at its top.) Notice `$task` in the `/ram` line: each shell's `/ram` is its own directory of RAM disk `r`, named after its task. Your RAM disk is yours alone, and no other shell's files get in its way.

9.7 Unions

Look at the `/bin` lines. The first binds `/ram/bin` at `/bin`; each of the others adds (`-a`, for "after") another directory to it. Now `/bin` is a **union directory**: a directory made of several. When you look for a file in `/bin`, HydraOS looks in each of its members in order, and the first one that has the file wins. When you list `/bin`, you see the files of all of them.

So the Hydra's programs can come from five places: your RAM disk, the shared RAM disk, the SD card, the ROM disk, and the ROM's own modules. A program on the RAM disk or the SD card is found before the ROM's, so you can put a new version of a program there, and it takes the old one's place.

The `-c` on the first `/bin` line says where new files go: a file you make in `/bin` is made in `/ram/bin`.

```
/> echo '#!/bin/rc' >/bin/greet
/> echo 'echo Hello from a script in /bin' >>/bin/greet
/> greet
Hello from a script in /bin
/> ls /ram/bin
greet
```

`/lib` is built the same way, which is how the files that set up the system, like `/lib/shell` and `/lib/namespace`, can be replaced by your own: put your version on the SD card, and it's found first.

The options for `bind` and `mount` are:

OPTION	MEANS
(none)	Replace what's at the place
`-a`	Add after what's there: looked at last
`-b`	Add before what's there: looked at first
`-c`	New files made in the union go here

9.8 Your own namespace

You can change the namespace of any shell with `bind`, `mount` and `unmount`. To start again with the standard one, type `newns`. And `ns` shows your shell's namespace, as the binds and mounts that would make it:

```
/> ns
bind '#/' /
bind '#/dev' /dev
bind -a '#n' /dev
bind -a '#t' /dev
bind -a '#a' /dev
bind -a '#c' /dev
bind '#g' /dev/gpio
bind '#i' /dev/i2c
bind '#d' /dev/sd
bind '#S' /dev/spi
bind '#m' /dev/mod
bind '#s' /dev/seg
bind '#e' /env
bind '#p' /proc
bind '#f' /sd
mount '#f' /rom x
mount '#f' /sram s
mount -c '#f' /ram r/1
bind -c '#fr/1/bin' /bin
bind -a '#fs/bin' /bin
bind -a '#fx/bin' /bin
bind -a '#m/bin' /bin
bind -c '#fr/1/lib' /lib
bind -a '#fs/lib' /lib
bind -a '#fx/lib' /lib
bind '#P' /pc
```

It looks a little different from the namespace file: `ns` shows each part of the tree by its device's own name, like `'#fr/1/bin'` (directory `1/bin` of HydraFS disk `r`). The SD card's lines aren't there, because this card has no `bin` or `lib` directory. And the `/dev` directory starts with `#/dev`, which gives it its subdirectories: `gpio`, `i2c`, `mod`, `sd`, `seg`, `spi` and `vid`.

`ns 5` shows the namespace of task 5.

To change every shell's namespace, write your own namespace file, and put it on your SD card as `/sd/0/lib/namespace`. Each shell reads it after the ROM's, and does its lines too. For example, a card with this file:

```
bind -a /sd/0/games /bin
```

makes every shell look for programs in the card's `games` directory too.

9.9 /pc: a folder on your PC

When the Hydra is connected to a PC running the PC tool (section 5.2), the folder you gave the tool appears on the Hydra at `/pc`. You can copy files to and from it, and run programs from it, just like any other disk:

```
/> cp /pc/hello.hyx /ram/bin/hello2
/> cp /ram/notes/list /pc/list.txt
```

The files travel over the serial port, mixed in with the console's text (the PC tool takes them out, so you won't see them). They go no faster than the serial port allows, so switch to 115200 baud (section 5.5) before you copy big files. The PC tool's `--read-only` option stops the Hydra changing the folder's files.

The PC tool can be given an SD card's image file in place of a folder (section 5.2). Then `/pc` is the card in the image, and everything works the same way, except that a directory's files are listed in the card's own order, as they are on `/sd`, and not sorted by name.

Without the PC tool, the Hydra waits a second for an answer from the PC, then gives up:

```
/> ls /pc
ls: /pc: i/o error
```

With an ordinary terminal program, you may see a few odd characters just before the error: they're the Hydra's message to the PC tool, which isn't there to take it.

CHAPTER 10

The Console and Its Windows

The Hydra can run sixteen programs at once, but you have only one terminal. How do you talk to more than one of them? With **windows**. The console can keep up to sixteen windows, each with its own shell and its own screen, and show you any of them, or several side by side.

IN THIS CHAPTER: You'll open, switch, split, scroll and label the console's windows, and learn the files that control them. **You need:** Chapter 6, and the window key: CTRL-].

10.1 Windows on a terminal

Each window is like a separate terminal: it has its own screen, its own line being typed, its own history, and its own programs. One window at a time is **shown**, and gets your keys. The others carry on working, out of sight: what their programs print goes to their own screens, ready for when you look at them again, and a program that asks for a key waits until you show its window.

When the Hydra starts, there's just **window 0**, with HyForth in it. You make more, and move between them, with the **window key**, CTRL-], and then one more key. For example, CTRL-] ? shows a list of all the window keys:

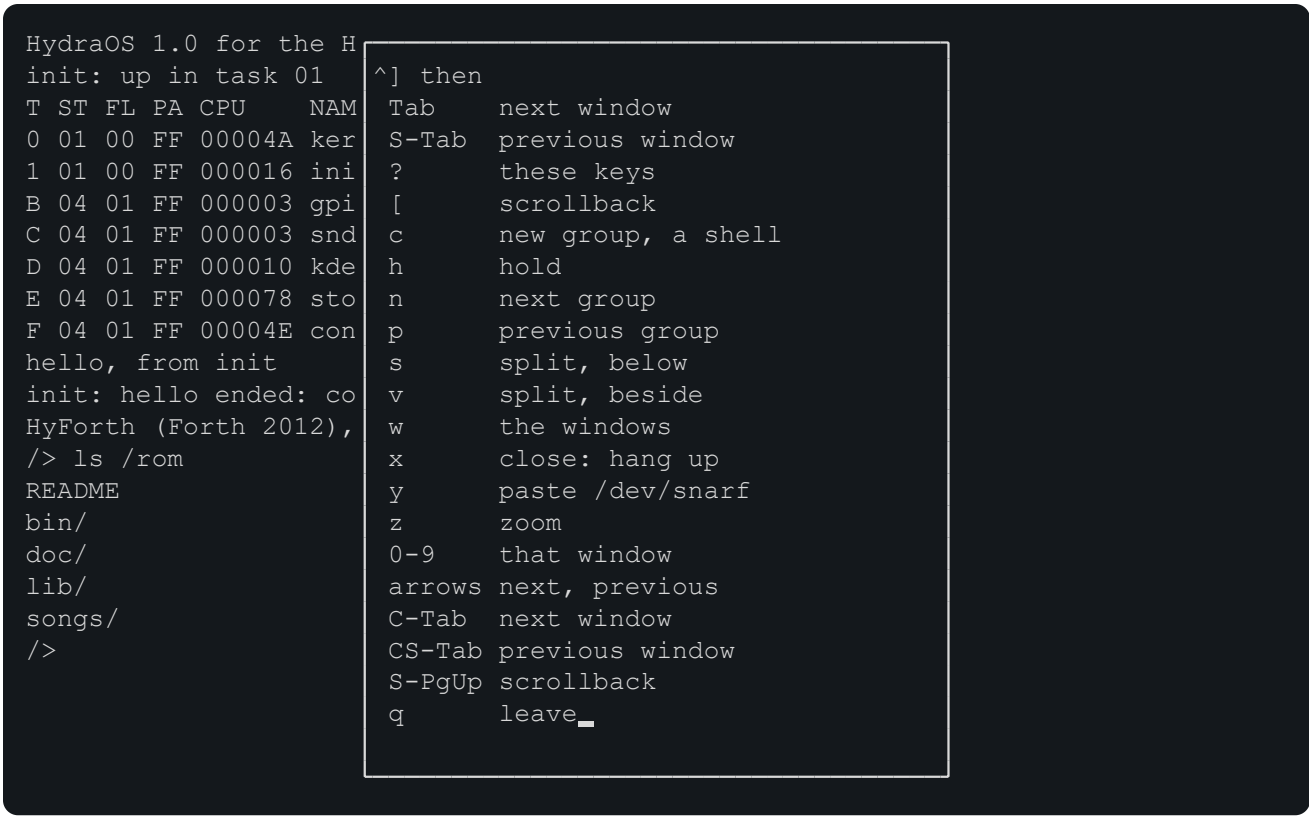


Figure 10-1. CTRL-] ?*: the window keys.*

Press Q to put it away.

10.2 A new window

CTRL-] C makes a new window (in a group of its own: section 10.3) with a new shell in it, and shows it. The screen clears, and the new shell says hello:

```
HyForth (Forth 2012), bye to end
/>
```

This is window 1. Everything you do here is separate from window 0. Type a command, then go back to window 0 with CTRL-] 0: window 0's screen comes back, just as you left it. CTRL-] 1 shows window 1 again.

To see all the windows, press CTRL-] W:

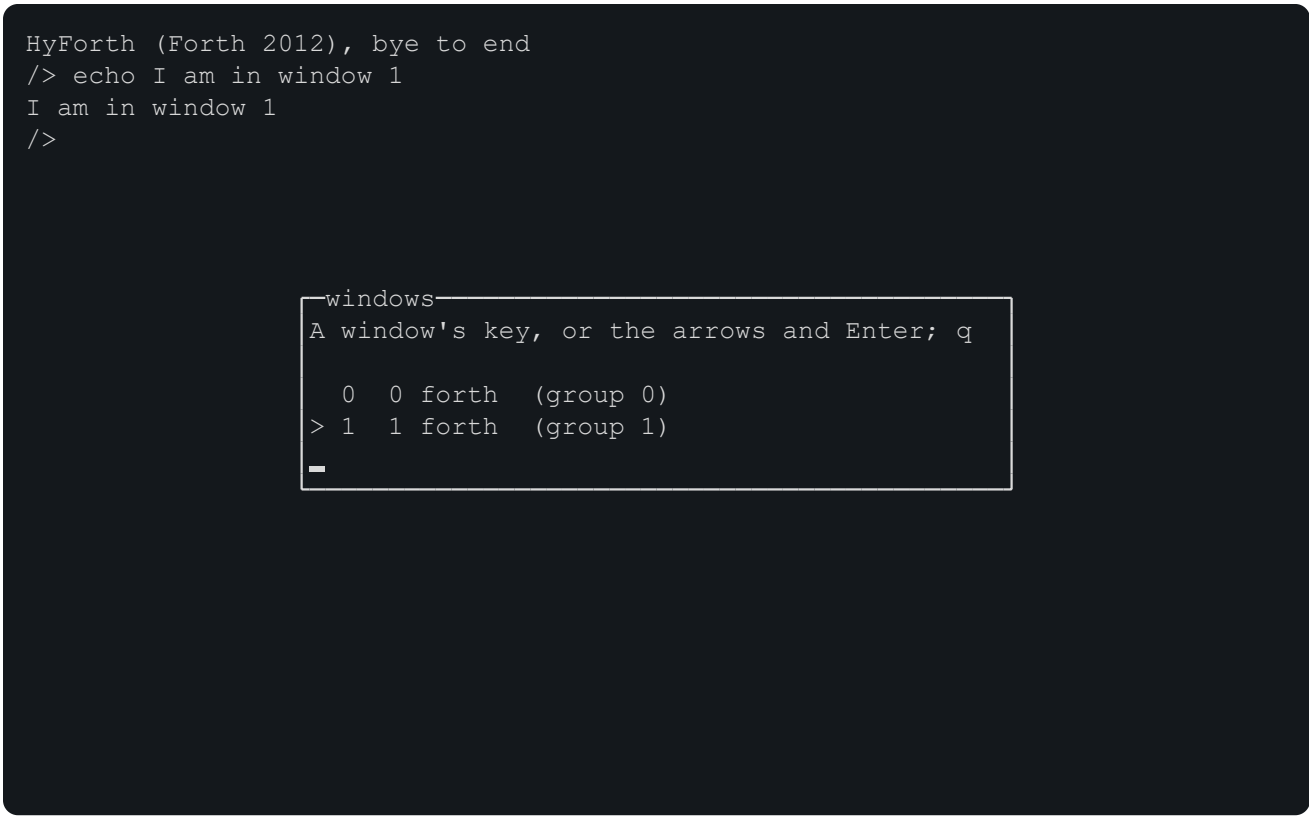


Figure 10-2. CTRL-] W*: the window list.*

The list shows each window's number, its group (section 10.3), and the program running in it. The > marks the window that was shown. Press a window's number to show it, or move with the arrow keys and press RETURN; Q leaves the list.

When a window's shell ends (type `bye` in HyForth, or `exit` in rc), the window goes away. Window 0 is special: it never goes away: the kernel starts init again, and init becomes a new shell in it.

10.3 Groups

Windows belong to **groups**. A group is like a desk you're working at: one shell session, with all the windows it has opened. CTRL-] C starts a new group, with a new shell. A program can open more windows in its own group, with the command `new-window` (Chapter 11).

KEYS	WHAT THEY DO
CTRL-] 0 ... 9	Show window 0 to 9
CTRL-] TAB	Show the group's next window
CTRL-] SHIFT-TAB	Show the group's previous window
CTRL-] and an arrow	The group's previous (↑ ←) or next (↓ →) window
CTRL-] N	Show the next group
CTRL-] P	Show the previous group
CTRL-] C	A new group, with a shell
CTRL-] X	Close the group shown: its programs are told to stop ("hang up")
CTRL-] W	The window list

KEYS	WHAT THEY DO
CTRL-] ?	The window keys
CTRL-] CTRL-]	Type a CTRL-]

CTRL-TAB and CTRL-SHIFT-TAB also move to the group's next and previous windows, if your terminal sends them. (With Windows Terminal, start the PC tool with `--win32-input`.)

10.4 Side by side

A group's windows can be shown together, **tiled**, each in its own part of the screen. CTRL-] S makes a new window with a shell, below the one shown, and CTRL-] V makes one beside it. Each tiled window has a heading, with its number and program at the left, and the group's windows at the right, the one with the keys marked:

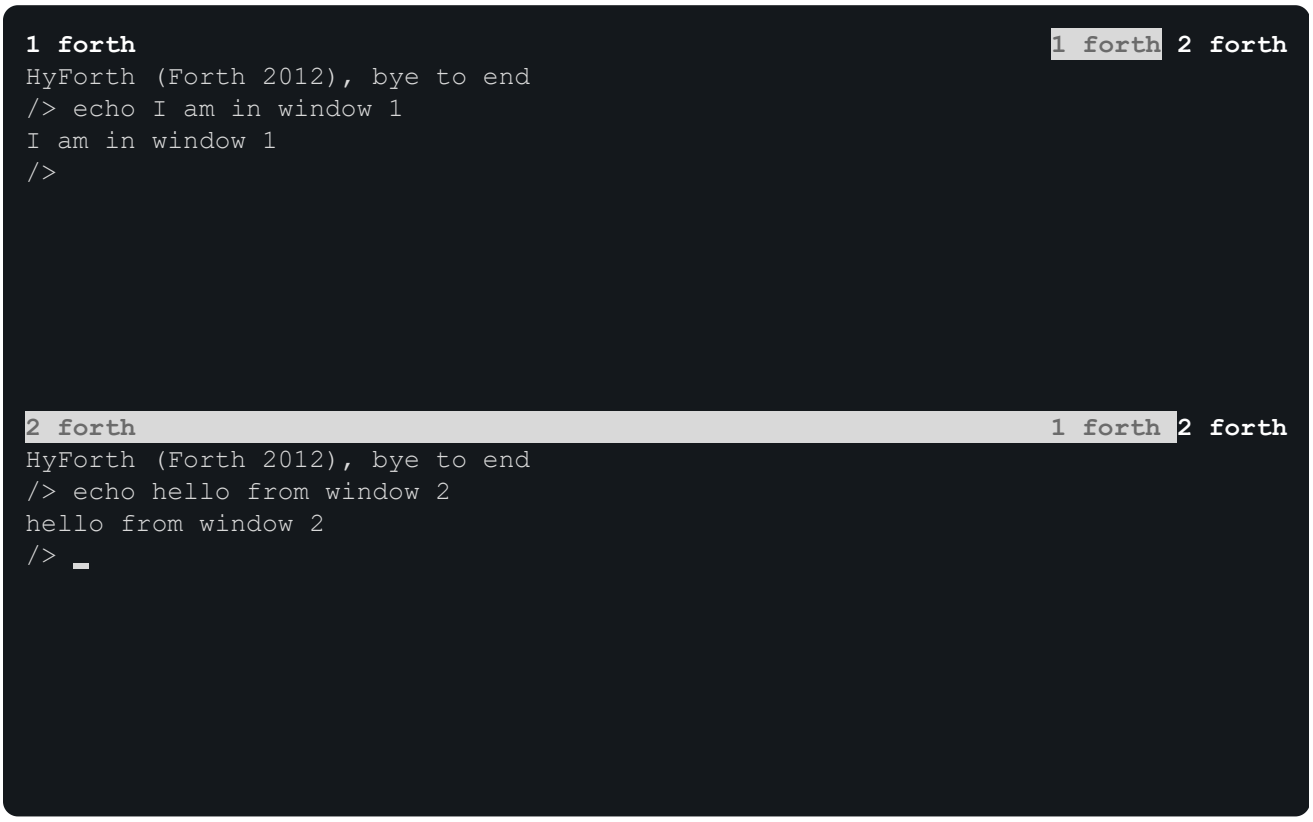


Figure 10-3. CTRL-] S*: a window below.*

The keys go to one window at a time: move between them with CTRL-] and the arrow keys. CTRL-] Z **zooms** the window with the keys, so it has the whole screen to itself; press it again to tile the group again.

The group's **layout** can be changed by writing to its `wctl` file:

COMMAND	LAYOUT
`echo layout tabs >/dev/wctl`	One window at a time
`echo layout rows >/dev/wctl`	One above another
`echo layout columns >/dev/wctl`	Side by side
`echo layout grid >/dev/wctl`	In a grid

10.5 Scrolling back

Each window remembers some of the lines that have scrolled off the top of its screen. To look back at them, press CTRL-[(or SHIFT-PAGE UP). The program in the window carries on; you're just looking. In the scrollback view:

KEYS	WHAT THEY DO
The arrows, PAGE UP, PAGE DOWN, HOME, END	Move around
SPACE	Mark the start of a piece to copy
RETURN	Copy the marked lines (or the line you're on) to the **snarf buffer**, and leave
Q, or ESC twice	Leave

The **snarf buffer**, `/dev/snarf`, is the console's clipboard. CTRL-[Y types whatever is in it into the window shown, as if you'd typed it. So you can copy a command or a result from one window, and paste it into another. Programs can read and write `/dev/snarf` too.

10.6 Holding the output

When a program prints faster than you can read, CTRL-[H **holds** the window: its program waits, mid-line, until you press CTRL-[H again.

10.7 Bars, headings and footers

The console can show extra lines around the windows, which it calls the **chrome**:

- the **bar**, a line at the top (or the bottom) of the screen, with the groups and the time;
- each window's **header**, a line above it, with its number and name, and the group's windows;
- each window's **footer**, a line below it, with its **status**: a message its program can set.

On the Vera X's screen they're all shown. On the serial terminal they're off, to leave you as many lines as possible, but you can turn them on for a window:

```
/> echo chrome serial on >/dev/wctl
```

```
 0 forth  1 forth                                                    09:41
0 forth                                                           0 forth
HydraOS 1.0 for the Hydra-16
init: up in task 01
T ST FL PA CPU    NAME
0 01 00 FF 00004A kernel
1 01 00 FF 000016 init
B 04 01 FF 000003 gpio
C 04 01 FF 000003 snd
D 04 01 FF 000010 kdev
E 04 01 FF 000078 storage
F 04 01 FF 00004E cons
hello, from init
init: hello ended: code $07 (bye)
HyForth (Forth 2012), bye to end
/> ls /rom
README
bin/
doc/
lib/
songs/
/> echo chrome serial on >/dev/wctl
/> _
```

Figure 10-4. The bar and the header, on the serial terminal.

The bar shows the groups (the one shown is in a different colour) and the time; the header shows the window's number and program, and the group's windows. What each line shows is set by a **format**, as HyForth's prompt is (section 8.15). The ROM disk's `/lib/windows` sets them up as the Hydra starts:

```
bar top
bar %[7] %G%=%t
default header %[1]%n %l%=%w
default footer %s
default chrome screen on
default chrome serial off
```

The codes in a format are:

CODE	SHOWS
`%n`	The window's number
`%l`	The window's **label**: its name (normally its program's name)
`%p`	Its program's name
`%s`	Its status line
`%g`	The group's number
`%w`	The group's windows
`%G`	The groups
`%c %r`	Its size: columns and rows
`%m`	Its modes

CODE	SHOWS
`%t %d`	The time and the date
`%L`	The keyboard's lights
`%y`	Where the scrollback view is
`%=`	Everything after this goes at the right-hand end
`%[...]`	A change of colour or style, as in a terminal's SGR sequence (Chapter 13): `%[7]` is reverse, `%[1;33;44]` bold yellow on blue, `%[0]` normal
`%%`	A `%`

To use your own, put a file called `windows` in the `lib` directory of your SD card: it's read before the ROM disk's.

10.8 A window's files

Each window has its own console files. A program running in a window finds them at `/dev`:

FILE	WHAT IT IS
`cons`	The window's console: read your typing from it, write to its screen
`consctl`	Control: raw mode, the keys' codes, where it's shown, and more (Chapter 13)
`wctl`	The windows' control: new windows, layouts, the chrome, the keys (Chapter 13)
`label`	The window's name: read it, or write a new one
`text`	Everything in the window's scrollback and screen, as text
`wnew`	Reading it waits for CTRL-] C, then makes the new window: for a program that wants to start the windows' shells itself. With nobody reading it, the console makes the window and the kernel starts init in it
`kbdin`	What's written here is typed into the window, as if from the keyboard

And these are the same in every window: `snarf` (the snarf buffer), `ser` and `serctl` (the serial port itself), and `kbin` (the Vera X keyboard's keys).

```
/> echo my window >/dev/label
/> cat /dev/label; echo
my window
```

(The label has no new line at its end, so `echo` adds one.)

Reading `wctl` lists the windows: each one's number, group, columns and rows, with a `*` after the one with your keys:

```
/> cat /dev/wctl
0 0 80 24 *
```

Each window's files are also the console device's, `#c`, with the window's number after it: window 2's console is `#c2/cons`. HyForth's `send` word uses `#cN/kbdin` to type a line into another window's shell:

```
/> send 1 ls /rom
```

types `ls /rom` and RETURN into window 1.

CHAPTER 11

Commands from A to Z

This chapter is a guide to every program on the Hydra, in alphabetical order, followed by the shell's built-in commands. Each entry shows how to type the command, what its options mean, and an example of it at work.

IN THIS CHAPTER: This is a chapter for looking things up: every program and every shell command, with its options and an example. **You need:** Chapter 6. Don't read it through; dip into it.

How to read the usage lines. Each entry starts with a line like this one:

```
ls [-ld] [name ...]
```

Words in `this type` are typed as they are. Words in *italics*, like *name*, stand for something you choose. Square brackets, `[ ]`, mark something you can leave out, and `...` means "as many as you like". So `ls`, `ls /rom`, `ls -l /rom /ram` and `ls -ld /rom` are all ways to use `ls`. Options can be joined: `-ld` is the same as `-l -d`. A program typed with no arguments, or with ones it doesn't understand, prints its usage line, so you never need to remember one.

When things go wrong. A program that can't do what you asked says why, with its own name first:

```
/> ls /nothing
ls: /nothing: not found
```

and ends with a **status** that isn't success, so a shell can tell (section 8.9).

Most of the examples use a few small files. To make them, type:

```
/> echo apple >/ram/fruits
/> echo cherry >>/ram/fruits
/> echo banana >>/ram/fruits
/> echo apple >>/ram/fruits
```

as: the assembler

```
as [-bl] file.s|- [out|-]
```

Turns a program written in 65C02 **assembly language** into a program the Hydra can run (`-`: standard input, or standard output, for a pipe). Chapter 27 is all about it.

basic: HyBASIC

```
basic [-l] [file]
```

Starts HyBASIC, the Hydra's BASIC. Chapters 23 and 24, and Appendix C, are all about it.

calc: the calculator

```
calc [-b base] [-d digits] [expression ...]
```

Works out an **expression**, exactly, and prints the answer. The expression is the rest of the line; with none, `calc` works out each line you type, until CTRL-D.

OPTION	WHAT IT DOES
`-b` *base*	Print the answer in another base: `x` for hex, `b` binary, `o` octal, and more (Chapter 18)
`-d` *digits*	How many digits to work out for answers like square roots, which never end (12 unless you say)

`calc` knows the Hydra's whole number system, which Chapter 18 explains: fractions stay exact, whole numbers can be as big as you like, and numbers can be written in any base. It has:

- the operators `+ - * /`, `%` (the remainder after dividing whole numbers), `^` (a power), and parentheses;
- the functions `sqrt exp log sin cos tan atan abs floor round truncate fib numerator denominator re im rational random`, and, with two numbers in parentheses, `gcd pow complex fixed`;
- the constants `pi`, `e` and `i`.

```
/> calc 2/3 + 0.5
7/6
/> calc 1/3 + 1/6
1/2
/> calc sqrt 2
1.41421356237
/> calc -d 30 pi
3.141592653589793238462643383328
/> calc '2^100'
1267650600228229401496703205376
/> calc 17 % 5
2
/> calc -b x 255
FF
/> calc '#xFF + #b101'
260
/> calc 'gcd(12, 18)'
6
/> calc 1/0
calc: division by zero
```

> **NOTE:** Characters that mean something to the shell, like `^`, `(`, `)` and `#`, must be inside quotes, as in `calc '2^100'`.

cat: show files

```
cat [file ...]
```

Copies each file to the standard output, one after another; its name means "con**cat**enate" (join together). With no file, it copies its standard input.

```
/> cat /rom/lib/shell
/bin/forth -l
/> cat /ram/fruits
apple
cherry
banana
apple
```

cmp: compare two files

```
cmp file1 file2
```

Compares two files, byte by byte. If they're the same, it says nothing, and ends with success; if not, it says where the first difference is.

```
/> echo hello >/ram/a
/> echo help >/ram/b
/> cmp /ram/a /ram/b
/ram/a /ram/b differ: byte 4
/> cmp /ram/a /ram/a
```

The first byte of a file is byte 1, so byte 4 is the fourth letter: `l` in one file, `p` in the other.

cp: copy files

```
cp [-r] from to
cp [-r] from ... dir
```

Copies the file *from* to a new file *to*. If the last name is a directory, it copies each file into it, keeping their names. A file that's already at *to* is replaced.

OPTION	WHAT IT DOES
`-r`	Copy whole directories too, with everything in them

```
/> cp /rom/lib/shell /ram/shell
/> cp -r /rom/lib/forth /ram/forth
/> ls /ram
bin/
lib/
shell
forth/
```

date: the date and time

```
date [-n]
```

Shows the date and time. With `-n`, it shows the time as the number of seconds since the start of January 1, 2000.

```
/> date
2000-01-01 00:00:04
/> date -n
5
```

To set the clock, write the date and time to `/dev/time`, in the same form:

```
/> echo 2026-10-12 09:41:00 >/dev/time
```

With a DS1747 clock chip fitted (section 4.5), this sets the chip's clock too, and the Hydra remembers the time when it's switched off. `cat /dev/rtc` says whether there's a clock chip: `running`, `stopped` (its clock needs setting), or `none`.

db: the debugger

```
db [-p task | program [arg ...]]
```

Runs a program one instruction at a time, so you can watch what it does. Chapter 29 is all about it.

df: free disk space

```
df
```

Lists the disks that have started, with each one's kind, size, free space and label:

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      13 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

The kinds are `sdhc` and `sdsc` (big and small SD cards), `rom` (the ROM disk), `ram` and `sram` (the RAM disks).

dis: the disassembler

```
dis [-cnw] [-l labels] [-o addr] file
```

Turns a program back into assembly language, so you can read it. Chapter 28 is all about it.

du: disk usage

```
du [-a] [name ...]
```

Shows how many kilobytes each directory takes, with everything in it, deepest first. With no name, it looks at the current directory.

OPTION	WHAT IT DOES
`-a`	A line for each file too

```
/> du /rom/lib
173     /rom/lib/as
1       /rom/lib/basic
2       /rom/lib/edit
4       /rom/lib/font
83      /rom/lib/forth
58      /rom/lib/hylang
327     /rom/lib
```

echo: print words

```
echo [-n] [word ...]
```

Prints its words, with a space between each, and a new line at the end.

OPTION	WHAT IT DOES
`-n`	No new line at the end

```
/> echo Hello there
Hello there
```

`echo` is most useful for writing small files, and for writing commands to devices' control files: `echo b115200 >/dev/serctl`.

ed: the line editor

```
ed [file]
```

`ed` is a **line editor**: you change a file by typing commands, each about some of its lines, by their numbers. It's simpler than a screen editor, and it works on any terminal, even a printing one. Its prompt is `*`. Lines you add are typed after an `a`, `i` or `c` command, and you end them with a line holding just a `.`.

COMMAND	WHAT IT DOES
`p` [*a*[,*b*]]	Print lines *a* to *b*, with their numbers (`p` alone: all of them)
`a` [*n*]	Add lines after line *n* (`a` alone: after the last)
`i` [*n*]	Insert lines before line *n* (`i` alone: before the first)
`c` *a*[,*b*]	Change lines *a* to *b*: they go, and the lines you type take their place
`d` *a*[,*b*]	Delete lines *a* to *b*
`w` [*file*]	Write the text to the file (or to another one, which then becomes the file)
`q`	Quit (if you haven't written your changes, it asks you to type `q` again)
`Q`	Quit at once, without writing
`h`	Help

A line number can be `$`, meaning the last line. When something goes wrong, `ed` prints a `?` and why.

```
/> ed /ram/poem
/ram/poem: new file
*a
Roses are red,
Violets are blue,
The Hydra has sixteen tasks,
And so could you.
.
*p
   1 Roses are red,
   2 Violets are blue,
   3 The Hydra has sixteen tasks,
   4 And so could you.
*c 4
Each one waiting for you.
.
*p 3,4
   3 The Hydra has sixteen tasks,
   4 Each one waiting for you.
*w
/ram/poem: 88 bytes
*q
```

edit: the screen editor

```
edit [+n] [file ...]
```

`edit` is a **screen editor**: it shows a screenful of the file, and you move around it with the arrow keys and type where the cursor is, as in a word processor. It works like the `nano` editor found on many other systems. *+n* starts at line *n*.

```
 edit  /ram/story.txt *                                    line 4/4  col 1
Once upon a time, a Hydra woke up.
It had sixteen heads, and each one
wanted to run a different program.
▬

^G Help    ^O Save    ^R Open    ^W Find    ^K Cut     ^U Paste   ^X Exit
M-A Mark   M-6 Copy   M-R Replace M-U Undo  M-E Redo   M-G Line   M-. File
```

Figure 11-1. The screen editor.

The top line shows the file's name (with a `*` if it has changes you haven't saved), and where the cursor is. The two lines at the bottom list the most useful keys: `^` means CTRL, so `^O` is CTRL-O, and `M-` means ESC and then the key (or ALT with it, if your terminal sends it), so `M-U` is ESC then U. CTRL-G shows all the keys:

KEYS	WHAT THEY DO
CTRL-G	Show all the keys
CTRL-O	Save, asking for the file's name (CTRL-S: save without asking)
CTRL-R	Open another file (up to six at once); `M-,` and `M-.` go between them
CTRL-X	Close the file (if it's the last one, `edit` ends)
CTRL-W	Find some text; `M-W` finds it again, `M-Q` backwards
`M-R`	Replace: find some text, and replace it with other text (asking each time: yes, no, or all)
`M-G`	Go to a line (`$`: the last)
CTRL-K	Cut the line (press it again to cut the next line too), or the marked block
`M-A`	Mark the start of a block
`M-6`	Copy the line, or the marked block
CTRL-U	Paste what was cut or copied (in any of the files)
`M-U`, `M-E`	Undo, and redo
The arrows, HOME, END, PAGE UP, PAGE DOWN	Move (also CTRL-B F P N, A E, Y V)
`M-\` and `M-/`	Go to the text's start, or end
DELETE or CTRL-D, BACKSPACE	Delete the letter at the cursor; rub out the one before it
`M-I`	Turn auto-indent on or off: a new line starts as far in as the one before
`M-X`	Turn the two help lines on or off
CTRL-L	Draw the screen again

When you close a file with changes, `edit` asks whether to save them:

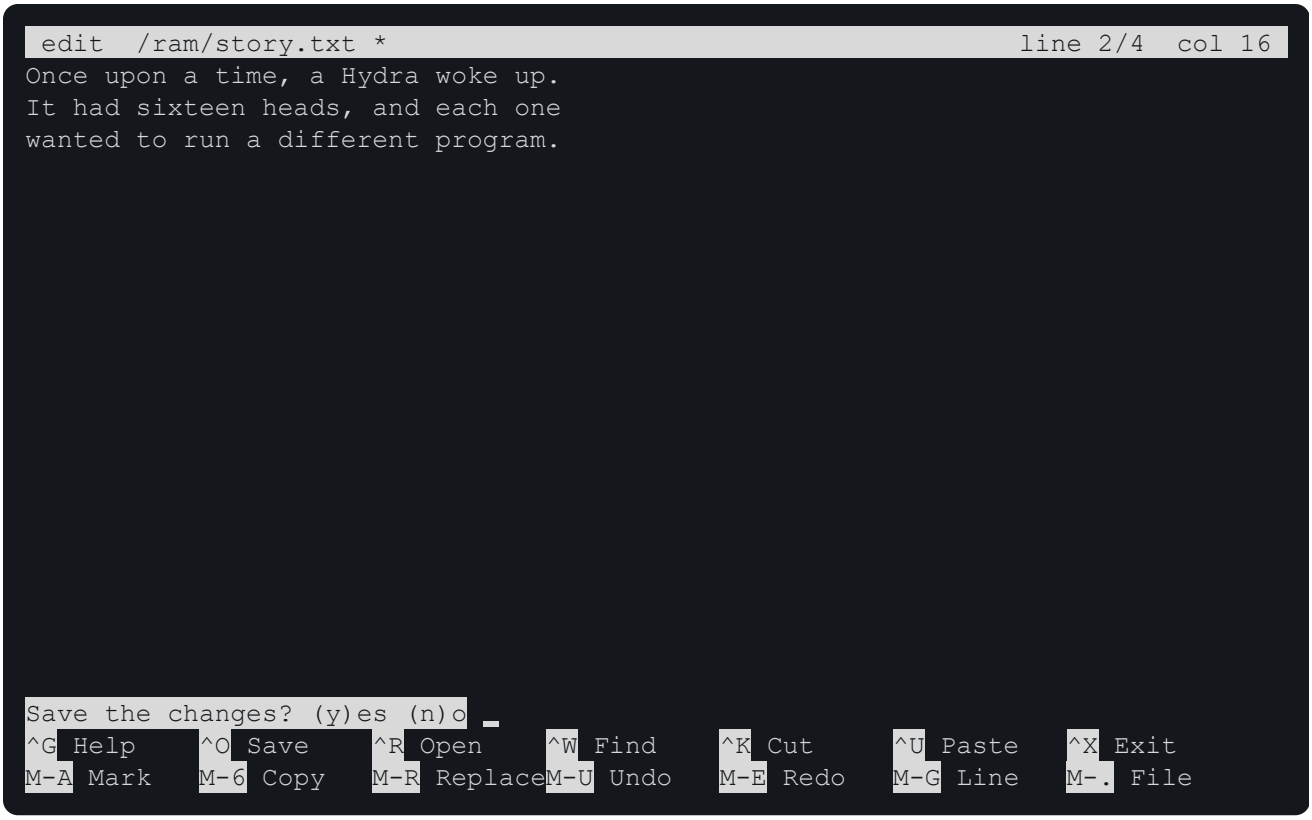


Figure 11-2. Closing a changed file.

Press Y, then RETURN to keep the file's name (or type another).

> **NOTE:** CTRL-C doesn't stop `edit`: it would lose your work. Use CTRL-X. And `edit` keeps the files' text in RAM banks, so it can edit files much bigger than a task's memory.

forth: HyForth

```
forth [-l] [file [arg ...]]
```

Starts HyForth. With a file, it runs the file as a Forth program. `-l` makes it a login shell (section 8.16). Chapters 19 and 20, and Appendix A, are all about it.

free: memory

```
free
```

Shows how much memory there is. The first line is each task's own RAM, in banks on the memory cards; the second is the shared RAM that tasks can share as **segments**, and how much of it is in use:

```
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

(Here, the one segment in use is the shared RAM disk.)

fsck: check a disk

```
fsck [-f] disk
```

Checks the HydraFS on a disk, and shows what it found. *disk* is the disk's name, as `df` shows it: `0`, `r`, `s` and so on.

OPTION	WHAT IT DOES
`-f`	Fix what it finds wrong

```
/> fsck s
hydrafs label=SRAM
free 510 KB of 511 KB
check: lost 0, unmarked 0, twice 0
segment 15
```

`lost` counts space that's marked as used but belongs to no file; `unmarked`, space a file uses that isn't marked as used; and `twice`, space that two files think is theirs. All three should be 0. Chapter 14 explains more.

grep: find lines

```
grep [-chilnsv] [-e] pattern [file ...]
```

Prints the lines of the files (or of its standard input) that match a **pattern**. The pattern is a **regular expression**, a small language for describing text:

PATTERN	MATCHES
`c`	The letter c (any letter but the ones below)
`\c`	The letter c, even one of these special ones: `\.` is a dot
`.`	Any letter
`[abc]`, `[a-z]`	One of the letters in the brackets
`[^abc]`	One letter that isn't in the brackets
`^`	The start of the line
`$`	The end of the line
e`*`	*e* any number of times (even none)
e`+`	*e* once or more
e`?`	*e* once, or not at all
e1`|`*e2*	Either *e1* or *e2*
`(`*e*`)`	*e*: parentheses group a pattern

OPTION	WHAT IT DOES
`-c`	Count the matching lines, instead of printing them
`-h`	Don't print the file's name before each line (when there are several files)
`-i`	Ignore the difference between capital and small letters
`-l`	Print only the names of the files with a matching line
`-n`	Print each line's number before it

OPTION	WHAT IT DOES
`-s`	Say nothing about files that can't be read
`-v`	Print the lines that *don't* match
`-e`	The pattern comes next (for a pattern that starts with –)

```
/> grep an /ram/fruits
banana
/> grep -n '^a' /ram/fruits
1:apple
4:apple
/> grep -c e /ram/fruits
3
/> grep -v e /ram/fruits
banana
```

Put patterns in quotes, so the shell leaves their special characters alone.

head: the first lines

```
head [-N] [file ...]
```

Prints the first 10 lines of each file (or of its standard input), or the first *N* (`head -3`).

```
/> ls /rom/bin | head -3
as
calc
cat
```

hello: say hello

```
hello [word]
```

A tiny program that greets its first argument. init runs it as the Hydra starts, to check that programs work.

```
/> hello there
hello, there
```

hwtest: the hardware test

```
hwtest
```

Starts the Hydra again, into the hardware test (section 5.7). Everything running stops.

hylang

```
hylang [-l] [file]
```

Starts hylang, the Hydra's Lisp-like language. Chapters 21 and 22, and Appendix B, are all about it.

kill: stop a task

```
kill [-i] task ...
```

Stops each task, by its number (`ps` shows them). The task can't refuse.

OPTION	WHAT IT DOES
`-i`	Interrupt the task instead, as CTRL-C does: a program can catch an interrupt and carry on

```
/> kill 99
kill: 99: no such task
```

label: a disk's name

```
label disk [text ...]
```

Shows a disk's **label** (its name), or sets it to the words after the disk's name.

```
/> label s
SRAM
/> label s scratch
/> label s
scratch
```

ls: list files

```
ls [-ld] [name ...]
```

Lists each directory's files, with a `/` after each directory's name, or shows each file. With no name, it lists the current directory.

OPTION	WHAT IT DOES
`-l`	A long listing: each file's mode, device, length and time (section 9.2)
`-d`	A directory itself, not its files

```
/> ls /rom
README
bin/
doc/
lib/
songs/
/> ls -l /rom/lib/shell
--r--r--r-- fx       14 2000-01-01 00:00 /rom/lib/shell
```

mkdir: make directories

```
mkdir [-p] dir ...
```

Makes each directory.

OPTION	WHAT IT DOES
`-p`	Make the directories it's in too, if they aren't there; and don't complain if it's there already

```
/> mkdir -p /ram/a/b/c
/> ls /ram/a/b
c/
```

mkfs: make a file system

```
mkfs [-fp] disk [label ...]
```

Makes a new, empty HydraFS on a disk, ready for files. Everything that was on the disk is lost! Use it on a new SD card before you use it with the Hydra (Chapter 14). The words after the disk's name are its label.

OPTION	WHAT IT DOES
`-f`	A full format: every part of the disk's map of free space is written now (slower)
`-p`	Put the file system in a **partition** of its own, after the card's others

```
/> mkfs s
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      13 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      511 KB
```

mods: the ROM's modules

```
mods
```

Lists the **modules** in the paged ROM: for each, the banks it's in, its kind, and its name. A small module shares a bank with others: `4+33` means bank 4, starting `$33` pages (256 bytes each) into it. A `boot` module is a driver the kernel starts as the Hydra starts; a `library` is code that programs share. (The tools, like `ls`, aren't modules: they're programs in files, on the ROM disk in `/rom/bin`.)

```
/> mods
bank   type     name
  4+33 program  init
  2- 4 boot     cons
  5- 6 boot     storage
 31    boot     kdev
  4+0A boot     snd
  0+0A boot     vid
 28    program  rc
  7- 8 program  play
 30    program  forth
  9-13 program  basic
 14-21 program  hylang
 22-23 library  hysnap
 24-25 library  numbers
 26-27 library  math
 29    library  asm
```

more: a screen at a time

```
more [file ...]
```

Shows each file (or its standard input) 22 lines at a time. After each screenful it shows `--more--`; press RETURN for the next, or Q and RETURN to stop.

```
/> ls /rom/bin | more
as
calc
cat
cmp
cp
date
db
df
dis
du
echo
ed
edit
free
fsck
grep
head
hello
kill
label
ls
mkdir
--more--
```

Figure 11-3. `ls /rom/bin | more`

mv: move or rename

```
mv from to
mv from ... dir
```

Moves a file or directory to *to*, or, if the last name is a directory, into it. Moved within the same directory, a file is just **renamed**. A directory can only be renamed, not moved to another disk.

```
/> echo hello >/ram/old
/> mv /ram/old /ram/new
/> cat /ram/new
hello
```

new-window: open a window

```
new-window [-g] [command ...]
```

Makes a new window in this window's group (Chapter 10), shows it, and runs the command in it (or, with no command, a shell). The window goes away when the command ends.

OPTION	WHAT IT DOES
`-g`	Put the window in a new group of its own

```
/> new-window edit /ram/notes
```

ns: show the namespace

```
ns [task]
```

Shows a task's namespace (or this shell's), as the `bind` and `mount` commands that would build it (section 9.8).

play: play a song

```
play [-l] song [n]
play -o score.mml song.zsm
play [-lx] -m channel mml ...
play [-lx] -c channel notes ...
```

Plays a song on the sound chips. *song* can be a ZSM file (the music format of the Commander X16, which music programs like Furnace can make), a score in MML, the Hydra's music language, or a WAV file (on a Vera X).

OPTION	WHAT IT DOES
`-l`	Play the song again and again, until you press CTRL-C
n	Play the song's repeating part *n* more times
`-o`	Turn an MML score into a ZSM file, instead of playing it
`-m` *channel*	Play a line of MML on one channel
`-c` *channel*	Play a chord: a note on each channel from *channel* on
`-x`	Use the Commander X16's version of MML

```
/> play /rom/songs/scom.zsm
```

Chapter 15 tells you all about the Hydra's sound, and MML.

ps: the tasks

```
ps [-a]
```

Lists the tasks running: each one's number, state, parent, how much processor time it has had (in seconds), its note group, and its program.

OPTION	WHAT IT DOES
`-a`	Show each program's arguments too

```
/> ps -a
task  state   parent     cpu group  name
   0  ready   -         1.6     0  kernel
   1  wait    -         0.5     1  forth -l
   2  ready    1        0.0     1  ps -a
  12  idle    -         0.0    12  snd
  13  idle    -         0.1    13  kdev
  14  idle    -         1.6    14  storage
  15  event   -         0.5    15  cons
```

STATE	MEANS
`ready`	Running, or ready to run
`wait`	Waiting for something: a key, a program to end ...
`call`	Waiting for a driver to answer it
`idle`	A driver, waiting for work
`sleep`	Asleep until a time
`blocked`	Waiting for a busy driver
`event`	Waiting for news from a driver

A **note group** is the tasks that get a note together: CTRL-C interrupts every task in the window's group. A command run in the background (`&`) gets a group of its own, so CTRL-C doesn't stop it.

pwd: where you are

```
pwd
```

Prints the current directory.

```
/> cd /rom/lib
/rom/lib> pwd
/rom/lib
```

rc: the shell

```
rc [-l] [-i] [-c command | file [arg ...]]
```

Starts the rc shell. Chapter 8 is all about it.

rm: remove files

```
rm [-rf] name ...
```

Removes each file, or empty directory. There's no wastebasket: a removed file is gone.

OPTION	WHAT IT DOES
`-r`	Remove directories with everything in them
`-f`	Say nothing about what can't be removed

```
/> mkdir -p /ram/junk/more
/> echo rubbish >/ram/junk/more/file
/> rm /ram/junk
rm: /ram/junk: directory not empty
/> rm -r /ram/junk
```

rmdir: remove directories

```
rmdir dir ...
```

Removes each directory, which must be empty.

```
/> mkdir /ram/x
/> rmdir /ram/x
```

scom: a song

```
scom [-l | n]
```

Plays a short tune, the first song the Hydra ever played. With `-l` it plays until you press CTRL-C. It's a script: look at it with `cat /bin/scom`.

slay: stop a program by its name

```
slay [-i] name ...
```

Like `kill`, but stops every task running a program of that name.

```
/> sleep 30 &
/> slay sleep
```

sleep: wait

```
sleep seconds
```

Does nothing for that many seconds. CTRL-C ends it sooner. It's useful in scripts, and for trying out background tasks.

sort: sort lines

```
sort [-bfnru] [file ...]
```

Prints the lines of the files (or of its standard input) in order.

OPTION	WHAT IT DOES
`-n`	Sort by the number each line starts with
`-f`	Treat capital and small letters the same
`-b`	Ignore spaces at the start of the lines
`-r`	Sort the other way round

OPTION	WHAT IT DOES
`-u`	Print each different line only once

```
/> echo 10 >/ram/n
/> echo 9 >>/ram/n
/> echo 100 >>/ram/n
/> sort /ram/n
10
100
9
/> sort -n /ram/n
9
10
100
```

Without `-n`, `sort` compares the lines letter by letter, so `100` comes before `9`, as "ab" comes before "b" in a dictionary.

tail: the last lines

```
tail [-N] [file]
```

Prints the last 10 lines of the file (or of its standard input), or the last *N* (`tail -20`).

```
/> tail -2 /ram/fruits
banana
apple
```

tee: copy to a file too

```
tee [-a] [file ...]
```

Copies its standard input to its standard output and to each file, like a T-shaped pipe fitting. It's handy in the middle of a pipeline, to keep a copy of what's passing through.

OPTION	WHAT IT DOES
`-a`	Add to the end of the files, instead of replacing them

```
/> ls /rom/bin | tee /ram/progs | wc -l
     45
/> head -2 /ram/progs
as
calc
```

top: the busiest tasks

```
top
```

Shows the tasks every second, with how much of the processor each one used in that second. Press CTRL-C to stop it.

```
task  state    cpu  name
   0  ready    93%  kernel
   1  wait      0%  init
   2  wait      0%  forth
   3  event     0%  wstart
   4  ready     2%  top
  11  idle      0%  gpio
  12  idle      0%  snd
  13  idle      0%  kdev
  14  idle      0%  storage
  15  idle      4%  cons
_
```

Figure 11-4. `top`. The kernel's share is mostly the idle task: time when nothing needed the processor.

touch: make a file, or change its time

```
touch name ...
```

Makes each file, empty, if it isn't there; if it is, sets its time to now.

```
/> touch /ram/empty
/> ls -l /ram/empty
--rw-rw-rw- fr        0 2000-01-01 00:00 /ram/empty
```

uniq: remove repeats

```
uniq [-c] [file]
```

Prints the lines of the file (or of its standard input), but each run of the same line only once. It's usually used after `sort`, which puts the same lines together.

OPTION	WHAT IT DOES
`-c`	Print how many times each line came, before it

```
/> sort /ram/fruits | uniq -c
      2 apple
      1 banana
      1 cherry
```

wc: count lines, words and bytes

```
wc [-lwc] [file ...]
```

Counts the lines, words and bytes in each file (or in its standard input), with a total when there are several.

OPTION	WHAT IT DOES
`-l`	Count only the lines
`-w`	Count only the words
`-c`	Count only the bytes

```
/> wc /ram/fruits
      4       4      26 /ram/fruits
/> wc -l /ram/fruits
      4 /ram/fruits
```

xd: hex dump

```
xd [file ...]
```

Shows each byte of the files (or of its standard input) as two hex digits, 16 to a line, after their position in the file; then the same bytes as letters, with a `.` for any that can't be shown. It's the way to see exactly what's in a file.

```
/> xd /ram/fruits
0000000  61 70 70 6c 65 0a 63 68 65 72 72 79 0a 62 61 6e  apple.cherry.ban
0000010  61 6e 61 0a 61 70 70 6c 65 0a                    ana.apple.
```

The `0a` bytes are the new lines.

xmodem: send and receive files

```
xmodem -r file
xmodem -s [-k] file
```

Sends or receives a file over the serial port, with **XMODEM**, a way of sending files that checks each block of the file as it arrives, and sends it again if it was damaged. Your terminal program must speak XMODEM too: Tera Term and minicom do, for example.

OPTION	WHAT IT DOES
`-r` *file*	Receive the file: start `xmodem -r`, then tell your terminal program to send
`-s` *file*	Send the file: start `xmodem -s`, then tell your terminal program to receive
`-k`	Send in 1K blocks, which is faster (your terminal program must allow it)

`xmodem` waits a minute for the PC to start, and says how many bytes it sent or received when it's done. With the PC tool, it's simpler to copy files through `/pc` (section 9.9).

The system's own programs

This program is started by the system itself, and you won't type it:

PROGRAM	WHAT IT DOES
`init`	The first program: sets up the system, and becomes the shell; the kernel starts it again for each new window (section 7.4)

The shell's own commands

These commands are built into rc (section 8.14). Each one changes the shell itself, which is why it can't be a program.

cd: change directory

```
cd [dir]
```

Makes *dir* the current directory. With no *dir*, it goes to `$home` (or `/`, if `$home` isn't set).

```
% cd /rom/lib
% pwd
/rom/lib
```

eval: run words as a command

```
eval word ...
```

Joins its words into a command line, and runs it. Variables in the words are looked at twice: once as `eval` is typed, and again as the command runs.

```
% eval echo '$path'
. /bin
```

exec: replace the shell

```
exec command [arg ...]
```

Runs the command in the shell's place: the shell ends, and the command takes over its task. Typed at an `rc` started from HyForth, the command runs and, when it ends, you're back at HyForth:

```
/> rc
% exec echo replaced
replaced
/>
```

exit: leave the shell

```
exit [status]
```

Ends the shell, with a status (success if there's none).

shift: drop arguments

```
shift [n]
```

In a script or function, removes the first *n* (or 1) words from `$*`.

```
% fn hi { echo hi $*; shift; echo now $* }
% hi one two three
hi one two three
now two three
```

wait: wait for the background

```
wait [task]
```

Waits for the commands running in the background (`&`) to end, or just for one task.

```
% sleep 2 &
% wait
% echo done
done
```

whatis: what is this name?

```
whatis name ...
```

Says what each name is to rc: a variable (with its value), a function (with its body), a built-in, or the program that would run (section 8.12).

builtin, ., bind, mount, unmount and newns

COMMAND	WHAT IT DOES
`builtin command [arg ...]`	Runs a built-in or a program, even when a function has the same name
`. file [arg ...]`	Runs the file's commands in this shell (section 8.13)
`bind`, `mount`, `unmount`, `newns`	Change the namespace (sections 9.5 to 9.8)



PART THREE

Devices

The Hydra's devices, and the drivers that look after them: the console and the terminal, the disks, sound, the Vera X's screen, keyboard and mouse, and the rest. Each chapter lists every file a driver gives you, and every command its control files understand.

The examples in this part are single lines typed at the shell. Some are HyForth and some are rc, and you can type them all just as they're printed, without knowing either language. If you'd rather learn a language first, read Part Four now and come back: the second chapter of each language shows how to work these same devices from a program.

- Chapter 12. How Drivers Work
- Chapter 13. The Console and the Terminal
- Chapter 14. Disks and HydraFS
- Chapter 15. Sound
- Chapter 16. The Vera X: Screen, Keyboard and Mouse
- Chapter 17. The Other Devices

CHAPTER 12

How Drivers Work

Every device on the Hydra is looked after by a **driver**: a program that knows how to talk to the device's chips. Other programs never touch the chips themselves. Instead, the driver makes the device look like a few files, and programs read and write those. This chapter explains how that works, and how to use any device from the shell.

IN THIS CHAPTER: You'll learn how a driver turns a device into files, how to command any device by writing to its control file, what the `#` device names mean, and what happens when a device is busy. **You need:** Chapter 6, and section 9.4.

12.1 Devices are files

Think about what a program wants from the serial port: to send some bytes, and to receive some. That's exactly what it wants from a file on a disk, too: to write some bytes, and read some. So HydraOS makes them the same. The serial port is the file `/dev/ser`; the sound chip is `/dev/snd`; the clock is `/dev/time`. To use a device, a program opens its file and reads or writes it, with the same few **system calls** it uses for every file:

CALL	WHAT IT DOES
`OPEN`	Open a file by its name, for reading, writing, or both; it gives a **file descriptor** (a small number for the open file)
`READ`, `WRITE`	Read or write bytes
`SEEK`	Move to a place in the file
`CLOSE`	Finished with the file
`CREATE`, `REMOVE`	Make a new file, or remove one
`STAT`, `WSTAT`	Read a file's details (its name, length, time ...), or change them

The idea comes from the Plan 9 operating system, and from Unix before it. It makes every device easy to use from any language, and from the shell, with nothing but `cat` and `echo`:

```
/> cat /dev/time
2000-01-01 00:00:04
```

12.2 Drivers are servers

Each driver runs in a task of its own (the boot drivers are in tasks F, E, D and C, and with a Vera X, B). A driver is a **file server**: it waits for **requests**, and answers them.

When a program calls `READ` on a device's file, the kernel turns the call into a request, and hands it to the driver's task: "read 100 bytes of file 3, from offset 0". The driver does the work, perhaps waiting for the device, and gives back the bytes. The kernel copies them into the program's memory, and the program carries on.

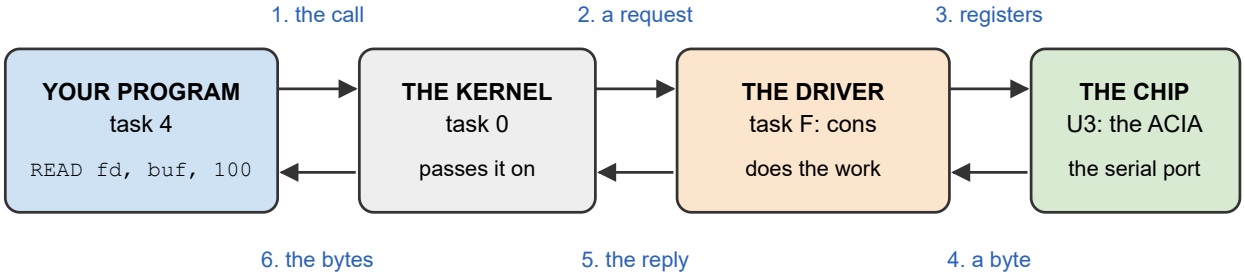


Figure 12-1. A program reads a device's file.

Because a driver is just a program serving files, a driver doesn't have to be in the ROM, or started as the Hydra starts: the PC tool, which serves `/pc`, is a program on your PC. And a small job doesn't need a driver, or a task, of its own. The Hydra has only sixteen tasks, so one driver often does several jobs: `kdev` serves the kernel's devices and the I/O header's pins, and between those requests it looks at a Vera X's keyboard and mouse.

> **HOW IT WORKS:** A driver also owns its device's **interrupt line**. When the device needs attention (a byte arrived, a timer ran out), the kernel runs the driver's **interrupt entry**: a few instructions that collect the byte or note the event, then tell the tasks that were waiting for it. So a program can wait for a key without using the processor at all.

12.3 Device names

Each driver gives its files a **device name**: a `#`, a letter, and sometimes a **spec** after it, which chooses one of the driver's devices. You can use a device name anywhere you'd use a path:

```
/> ls '#t'
ticks
time
rtc
/> cat '#fx/lib/shell'
/bin/forth -l
```

(`#` starts a comment in rc, so device names must be in quotes.) `#fx` is HydraFS's disk `x`, the ROM disk. But you'll rarely need device names, because the namespace (Chapter 9) puts every device at a friendlier path: `#t` is bound at `/dev`, so its `time` file is `/dev/time`.

DEVICE	DRIVER	USUALLY AT	CHAPTER
`#c`	cons	`/dev`: `cons`, `consctl`, `wctl` …	13
`#P`	cons	`/pc`	9, 13
`#S`	storage	`/dev/spi`	14, 17
`#d`	storage	`/dev/sd`	14
`#f`	storage	`/sd`, `/rom`, `/ram`, `/sram`	14
`#a`	snd	`/dev`: `snd`, `sndctl`, `psg`, `bell`	15
`#v`	vid	`/dev/vid`	16
`#g`	kdev	`/dev/gpio`	17

DEVICE	DRIVER	USUALLY AT	CHAPTER
`#i`	kdev	`/dev/i2c`	17
`#/`	kdev	`/`	9
`#n`	kdev	`/dev`: `null`, `zero`, `kmesg`	17
`#t`	kdev	`/dev`: `ticks`, `time`, `rtc`	17
`#m`	kdev	`/dev/mod`	17
`#s`	kdev	`/dev/seg`	17
`#e`	kdev	`/env`	17
`#p`	kdev	`/proc`	17
`#|`	kdev	(pipes: made by the shell's `|`)	17

12.4 Control files

Most devices have a **control file**, usually called `ctl` (the console's is `consctl`, the sound's `sndctl`). It works in two directions:

- **Write a command to it**, as a line of text, to tell the driver to do something or change a setting.
- **Read it**, to see the driver's settings, as lines of text.

For example, the serial port's control file sets its speed and its flow control, and reads as those two settings and two lines of counts, for a line that's giving trouble (section 13.6):

```
/> cat /dev/serctl
b9600
x1
pc 0 req, 0 cut short, 0 unanswered, 0 damaged, 0 nak, 0 gone
line 0 overruns, 0 framing errors
```

and `echo b115200 >/dev/serctl` would change the speed. Because commands are plain text, any program can send them, and you can try them out at the shell before you use them in a program. A command the driver doesn't understand gives the error `invalid argument`:

```
/> echo fly >/dev/gpio/ctl
echo: write error: invalid argument
```

The following chapters list every command each control file understands.

12.5 When it goes wrong

Every system call either succeeds, or fails with an **error code**: a number from 1 to 255, which says what went wrong. Programs print its message, like `not found` or `busy`. Appendix G lists them all. The ones you'll see most from devices are:

MESSAGE	MEANS
`invalid argument`	The driver didn't understand the command, or a number was out of range
`busy`	Someone else is using the device (or the channel, or the file)
`no such device`	The device isn't there: no Vera X, no SD card ...
`not allowed`	That file can't be read (or written)

MESSAGE	MEANS
`i/o error`	The device didn't answer, or answered wrongly
`not yet: try again`	Nothing is ready yet, and the program asked not to wait

12.6 Claims

Some devices can only do one thing at a time. A program can **claim** them: the sound driver's channels, the Vera X's chip, an SPI device. While a program has a claim, other programs' requests are refused (with `busy`), or, for sound, quietly ignored. The claim ends when the program releases it, or when it closes the device's files, which happens anyway when the program ends. So a program that crashes can never leave a device stuck.

CHAPTER 13

The Console and the Terminal

The **console** is how you and your programs talk: your keys in, and text out to your screen. Its driver, `cons`, looks after the serial port and the console's windows (Chapter 10), and serves `/pc` too. This chapter describes all of its files, and the **escape sequences** that programs use to move the cursor, clear the screen and change colours.

IN THIS CHAPTER: You'll learn the console's files, how a program reads lines or single keys, the escape sequences that move the cursor and set colours, the serial port's settings, and `/pc`. **You need:** Chapter 12. The examples are single lines of rc and HyForth that you can type just as they are: you don't need to know either language yet.

13.1 The console's files

`cons` is device `#c`. Each window has its own set of files (window *n*'s are `#c`*n*), and a shell finds its own window's at `/dev`. A few files are the whole console's, the same in every window.

FILE	WHOSE	WHAT IT IS
`cons`	The window's	Read: your typing. Write: text for the window's screen
`consctl`	The window's	The window's settings: raw mode, keys, where it's shown (section 13.4)
`wctl`	The window's	The windows: new ones, layouts, the chrome, the keys (section 13.5)
`label`	The window's	The window's name
`text`	The window's	The window's scrollback and screen, as text
`kbdin`	The window's	Bytes written here are typed into the window
`wnew`	The window's	A read waits for CTRL-] C, and gives the new window's number
`snarf`	The console's	The snarf buffer: the console's clipboard
`ser`	The console's	The serial port itself, byte by byte (section 13.6)
`serctl`	The console's	The serial port's speed
`kbin`	The console's	The Vera X keyboard's keys, from the `kdev` driver (Chapter 16)

13.2 Reading lines

Normally, the console is in **cooked mode**: a program that reads `cons` gets a whole line at a time, after you've typed it and pressed RETURN. Until then, the console itself lets you edit the line, with the keys in section 6.3, and keeps the last four lines you typed for ↑ and ↓ to bring back. A line can be up to 127 characters long; a line longer than the window is wide goes on to the next rows.

CTRL-D, typed at the start of a line, is the **end of the input**: the program's read gets nothing, which tells it there's no more. `cat`, typed alone, copies what you type to the screen until you press CTRL-D.

You don't have to wait for a prompt before you type. The console keeps the keys that come while a program is busy (about 300 of them from a terminal, 60 from the Vera X's keyboard), and hands them over when the program next reads: so you can type the next command while one is still running, or paste a whole program from your PC. When it's holding about half of what it can, it asks the terminal to wait (section 13.6), so a paste of any length arrives whole, as long as your terminal's flow control is set to XON/XOFF (section 5.2).

13.3 Raw mode and keys

A program like the screen editor needs each key as it's pressed, without waiting for RETURN. It writes `rawon` to `consctl`, and the console goes into **raw mode**: each read gets the keys typed so far, as they are, and the console doesn't show them (the program does that itself). Raw mode lasts until the program writes `rawoff`, or closes `consctl`: so when the program ends, the console goes back to cooked mode by itself.

In raw mode, the keys that send more than one byte, like the arrows, can come in two ways, which the program chooses with `consctl`'s `keys` command:

- `keys hydra` (the usual): each key is **one byte**. The ordinary keys are their characters, and the special keys are these codes:

CODE	KEY	CODE	KEY	CODE	KEY
`$80`	↑	`$87`	DELETE	`$8E`	F5
`$81`	↓	`$88`	PAGE UP	`$8F`	F6
`$82`	→	`$89`	PAGE DOWN	`$90`	F7
`$83`	←	`$8A`	F1	`$91`	F8
`$84`	HOME	`$8B`	F2	`$92`	F9
`$85`	END	`$8C`	F3	`$93`	F10
`$86`	INSERT	`$8D`	F4	`$94`, `$95`	F11, F12

And three more that aren't keys: `$96` means the window has changed size (read `consctl` for the new size); `$97` means you've moved to another window of the program's group, and the next byte is its number; and `$98` comes before a key typed with SHIFT, ALT or CTRL held down, if the program asked for them with `keys mods`: the next byte says which (1 SHIFT, 2 ALT, 4 CTRL, added together), and then comes the key.

- `keys vt`: each key comes as the bytes a VT100 terminal would send: ↑ is ESC `[A`, for example. This is for programs written for other terminals.

An ESC key on its own reaches the program a tenth of a second after it's pressed, once the console is sure it isn't the start of a longer sequence.

13.4 consctl: the window's settings

COMMAND	WHAT IT DOES
`rawon`	Raw mode: each key as it comes (section 13.3)
`rawoff`	Cooked mode: a line at a time
`keys hydra`	Raw mode's special keys as one byte each
`keys vt`	Raw mode's special keys as a VT100 sends them
`keys mods`	As `keys hydra`, and the keys typed with SHIFT, ALT or CTRL marked

COMMAND	WHAT IT DOES
`scroll smooth`	On the serial port, every byte the window's programs write is sent, and they wait for it
`scroll jump`	On the serial port, the programs don't wait: the terminal shows the screen as it is, skipping what it missed
`group`	CTRL-C in this window goes to the writer's note group (the shell's profile does this)
`serial`	Show the windows on the serial terminal
`screen`	Show the windows on the Vera X's screen
`both`	Show the windows on both
`seats`	Both terminals, each a **seat** of its own: its own window shown, and its own keys (Chapter 16)
`terminal size` *c r*	Tell the console the serial terminal's size: *c* columns, *r* rows
`terminal size`	Ask the serial terminal its size (with the sequence ESC `[18t`)

Reading `consctl` shows the window's settings, its number and its size:

```
/> cat /dev/consctl
rawoff
keys hydra
scroll smooth
group 1
window 0
size 80 24
terminal serial
```

The console starts with an 80 by 24 terminal, and asks the terminal for its real size. Terminals that answer (xterm, Windows Terminal, the PC tool) get windows that fit; the PC tool tells the Hydra each time you resize its window, too. A window is never more than 127 columns by 64 rows.

13.5 wctl: the windows

COMMAND	WHAT IT DOES
`new`	Make a window in the writer's group; a read of the same file then gives its number
`new group`	Make a window in a new group
`current` *n*	Show window *n*
`layout tabs` | `rows` | `columns` | `grid`	The group's windows one at a time, or tiled (section 10.4)
`float` *x y c r*	Show the window as a pop-up box, *c* by *r*, at column *x*, row *y*
`float` *c r*	The same, in the middle of the screen
`float off`	The window isn't a pop-up any more
`bar top` | `bottom` | `off`	Where the bar is (or none)
`bar` *format*	What the bar shows (section 10.7)
`header` *format*, `footer` *format*	What the window's header and footer show
`header on` | `off`, `footer on` | `off`	The window's header and footer on or off
`chrome screen` | `serial` | `both on` | `off` [`bar`] [`header`] [`footer`]	The chrome on or off, on a terminal (all of it, or the parts named)
`default header` *format*, `default footer` *format*, `default chrome ...`	The same, for new windows

COMMAND	WHAT IT DOES
`status` *text*	The window's status line (the footer's `%s`)
`monitor on` | `off`	Mark the window with `+` in the bar's list when it's written to while it isn't shown
`history` *n*	Keep *n* more rows of scrollback (64 at a time, up to 128)
`key prefix` *key*	Change the window key from CTRL-] to another control key, written `ctrl-X` or `^X` (not CTRL-C, CTRL-\, ESC, RETURN or CTRL-J)
`key` *key action*	What a key after the window key does (below)

The **actions** a key can have are `next`, `previous` (the group's windows), `next-group`, `previous-group`, `new` (a group), `list`, `hold`, `close`, `paste`, `scrollback`, `split`, `vsplit`, `zoom`, `help` and `none`. The key is a character, `ctrl-X`, `tab` or `shift-tab`; and `key ctrl-tab` *action*, `key ctrl-shift-tab` *action* and `key shift-pgup` *action* set the keys that work without the window key. For example, to make CTRL-] K close a group:

```
/> echo key k close >/dev/wctl
```

Reading `wctl` lists the windows, one a line: each one's number, group, columns and rows, with a `*` after the one with the keys (section 10.8).

13.6 The serial port

The console normally owns the serial port, sending each window's text to your terminal. Two files give programs the port itself:

- `ser` is the serial port, byte by byte, exactly as the bytes come and go. While a program has `ser` open for reading, every byte that arrives is its, even CTRL-C, and the windows wait. `xmodem` works this way.
- `serctl` sets the port's speed and its flow control. Writing a speed changes it at once (so change your terminal's speed too); reading shows both settings, a line each, and then two lines of counts that help when the line is unreliable: `/pc`'s requests and how many had to be asked again (and why), and the serial chip's overruns (a byte came before the last was taken) and framing errors (noise, or a speed that doesn't match).

COMMAND	SPEED	REALLY
`b300`	300 baud	291
`b600`	600 baud	583
`b1200`	1200 baud	1,165
`b2400`	2400 baud	2,330
`b4800`	4800 baud	4,661
`b9600`	9600 baud (as the Hydra starts)	9,322
`b19200`	19200 baud	18,644
`b115200`	115200 baud	111,861

(The serial chip's clock is a little slower than the speeds' names assume, section 5.5.) The port always sends 8 data bits, no parity and 1 stop bit.

Flow control keeps a fast sender from getting ahead of the Hydra. The console can hold about 300 keys that no program has read yet. With `x1`, which is how it starts, it sends the terminal an **XOFF** (the byte `$13`) when 96 bytes are waiting in its receive buffer, and an **XON** (`$11`) when a program has read them down to 32. A terminal that's set to *XON/XOFF* stops sending between the two, so nothing is lost, however much you paste.

COMMAND	FLOW CONTROL
`x1`	XON/XOFF (as the Hydra starts)
`x0`	None: the console never asks the terminal to wait

```
/> cat /dev/serctl
b9600
x1
pc 0 req, 0 cut short, 0 unanswered, 0 damaged, 0 nak, 0 gone
line 0 overruns, 0 framing errors
```

Three things to know about it:

- **A terminal that isn't set to XON/XOFF** takes no notice (it doesn't show the two bytes either), and then a long paste into a busy program loses its end, after the 300 keys the console can hold. What came first is kept.
- **A stopped terminal holds everything back**, even CTRL-C. So the console never leaves a terminal stopped for more than five seconds: if no program has read anything by then, it sends XON anyway, your CTRL-C arrives, and what was pasted behind it may be lost. (The PC tool sends CTRL-C at once, stopped or not.)
- **`xmodem` and `ser`** send and receive every byte as it is, with no XON or XOFF added. Turn your terminal's flow control off for a file transfer if it doesn't do so itself: the file's own `$11` and `$13` bytes are data.

> **HOW IT WORKS:** Many computers use two more wires for this, called RTS and CTS, and the Hydra's serial socket has both. But the Hydra's serial chip, the R65C51, turns its own transmitter off whenever it raises RTS to say "wait", so the Hydra could say nothing at all while it held the PC back: not even the echo of your keys. XON and XOFF are ordinary bytes sent on the ordinary wire, and have no such trouble.

13.7 The terminal

When a program writes to the console, most bytes are characters to show. But some are **control characters**, and some start **escape sequences**: commands to the terminal, to move the cursor, clear the screen, or change colours. Each window's screen is kept by the console itself, which understands the commands of the **VT100**, a famous terminal of 1978, and the VT102 after it; and each window's commands go on to your terminal program, which understands them too.

Escape sequences start with the ESC character (27, or `$1B`). Most of the useful ones are **CSI** sequences: ESC and `[`, then some numbers separated by *;*, then a letter that says what to do. In the tables, *n*, *r* and *c* stand for numbers; most can be left out, and then they're 1 (or 0, for `J`, `K` and `m`).

Here are the colours from HyForth (Chapter 19 explains the Forth):

```
: esc 27 emit ;
: rainbow 8 1 do esc ." [4" i 48 + emit ." m     " loop esc ." [0m" cr ;
: hello esc ." [1;33;44m Hello, Hydra! " esc ." [0m" cr ;
: styles esc ." [7m reversed " esc ." [0m " esc ." [4munderlined" esc ." [0m" cr ;
: colours esc ." [31mred " esc ." [32mgreen " esc ." [34mblue " esc ." [91mbright red" esc ." [0m" cr ;
: cls esc ." [2J" esc ." [H" ;
cls rainbow hello styles colours
```

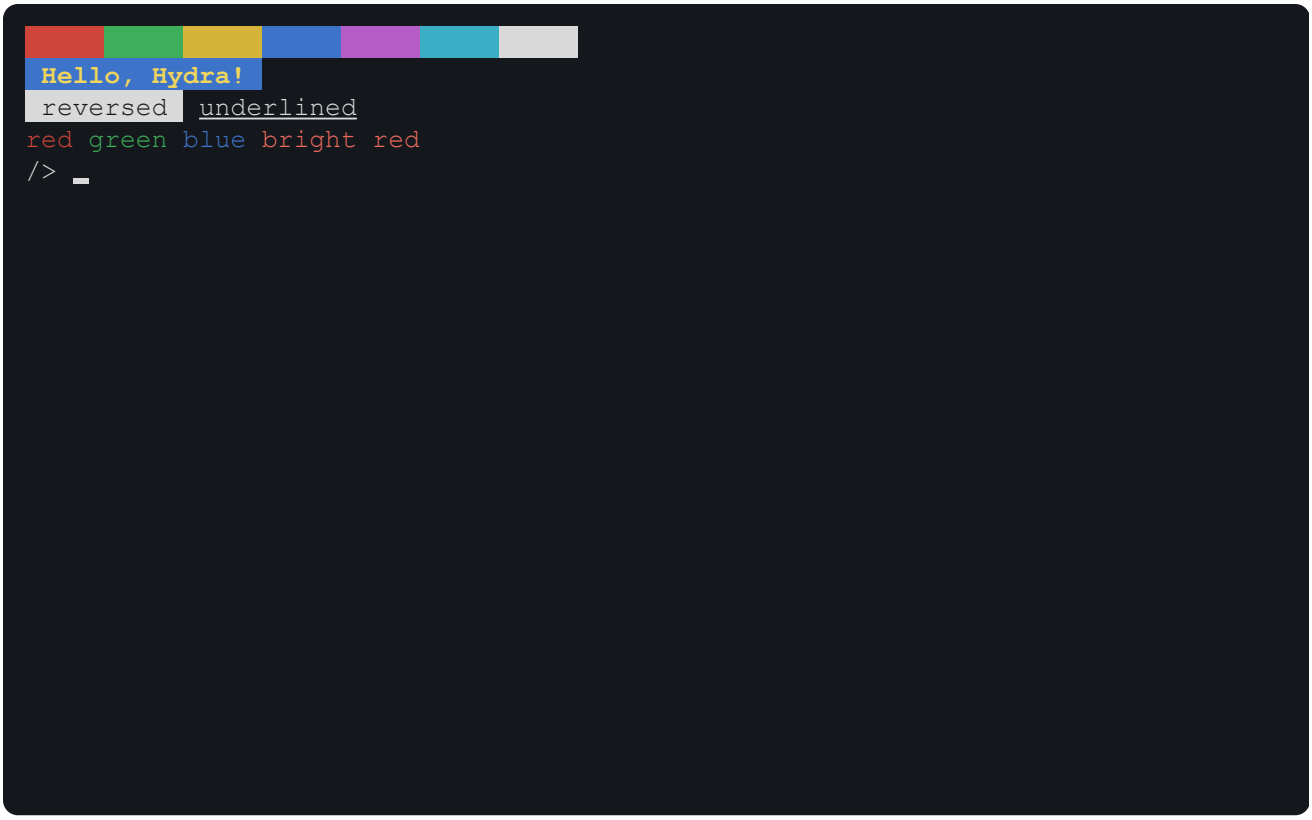


Figure 13-1. Colours and styles from escape sequences.

Control characters

BYTE	NAME	WHAT IT DOES
7	BEL	Rings the bell: a short beep from the sound chip
8	BS	Moves the cursor one column left
9	HT	Moves the cursor to the next tab stop (every 8 columns, unless they've been changed)
10	LF	A new line: down a row (scrolling at the bottom), to the first column
11, 12	VT, FF	As LF
13	CR	Back to the first column
14, 15	SO, SI	Use character set G1, or G0 (below)
24, 26	CAN, SUB	Cancel the escape sequence being sent
27	ESC	Starts an escape sequence

Moving the cursor

SEQUENCE	WHAT IT DOES
`ESC [` *n* `A`	Up *n* rows
`ESC [` *n* `B`	Down *n* rows
`ESC [` *n* `C`	Right *n* columns
`ESC [` *n* `D`	Left *n* columns

SEQUENCE	WHAT IT DOES
`ESC [` *n* `E`	Down *n* rows, to the first column
`ESC [` *n* `F`	Up *n* rows, to the first column
`ESC [` *c* `G`, `ESC [` *c* `` ` ``	To column *c*
`ESC [` *r* `d`	To row *r*
`ESC [` *r* `;` *c* `H`, `ESC [` *r* `;` *c* `f`	To row *r*, column *c* (counting from 1: `ESC [H` is the top left)
`ESC [` *n* `a`, `ESC [` *n* `e`	Right *n* columns; down *n* rows
`ESC [s`, `ESC 7`	Save the cursor (`ESC 7` saves its colours and character sets too)
`ESC [u`, `ESC 8`	Back to the saved cursor
`ESC D`	Down a row, scrolling at the bottom
`ESC M`	Up a row, scrolling down at the top
`ESC E`	To the first column of the next row

Erasing, inserting and scrolling

SEQUENCE	WHAT IT DOES
`ESC [` *n* `J`	Erase the screen: 0, from the cursor to the end; 1, from the start to the cursor; 2, all of it
`ESC [` *n* `K`	Erase the line: 0, from the cursor to its end; 1, from its start; 2, all of it
`ESC [` *n* `X`	Erase *n* characters from the cursor
`ESC [` *n* `@`	Insert *n* blanks at the cursor
`ESC [` *n* `P`	Delete *n* characters at the cursor
`ESC [` *n* `L`	Insert *n* blank lines at the cursor's row
`ESC [` *n* `M`	Delete *n* lines at the cursor's row
`ESC [` *n* `S`, `ESC [` *n* `T`	Scroll up, or down, *n* rows
`ESC [` *n* `b`	Repeat the last character *n* times
`ESC [` *t* `;` *b* `r`	The **scrolling region**: only rows *t* to *b* scroll (`ESC [r`: the whole screen)
`ESC #8`	Fill the screen with `E`s (the VT100's alignment test)

Tab stops

SEQUENCE	WHAT IT DOES
`ESC H`	Set a tab stop at the cursor's column
`ESC [g`, `ESC [3g`	Clear the tab stop at the cursor; clear them all
`ESC [` *n* `I`, `ESC [` *n* `Z`	Forward, or back, *n* tab stops

Colours and styles

`ESC [` *n* `;` *n* `...` `m` sets the **graphic rendition**: the colours and styles of the characters that follow. Each *n* is one of these:

N	MEANS	*N*	MEANS
0	Back to normal	22	Not bold or dim

N	MEANS	*N*	MEANS
1	Bold (on a screen, bright)	24	Not underlined
2	Dim	25	Not blinking
4	Underlined	27	Not reversed
5	Blinking	28	Not hidden
7	Reversed: the colours swapped	30-37	The text's colour (below)
8	Hidden	39	The text's normal colour
90-97	A bright text colour	40-47	The background colour
100-107	A bright background colour	49	The normal background
38;5;*n*, 48;5;*n*	One of 256 colours	38;2;*r*;*g*;*b*, 48;2;...	Any colour, by its red, green and blue

The colours are: 0 black, 1 red, 2 green, 3 yellow, 4 blue, 5 magenta, 6 cyan, 7 white. So `ESC [31m` is red text, `ESC [44m` a blue background, and `ESC [1;33;44m` bold yellow on blue. The window keeps 16 colours: a colour given by number or by red, green and blue is kept as the nearest of them.

Modes

`ESC [` *n* `h` sets a mode, and `ESC [` *n* `l` resets it; `ESC [?` *n* `h` and `ESC [?` *n* `l` are DEC's private modes.

MODE	WHEN SET
4	Insert: characters push the rest of the line right
20	Each LF is also a CR
?1	The cursor keys send application codes (`keys vt`)
?3	(132 columns: not on the Hydra; the screen is cleared)
?6	Origin mode: row numbers count from the scrolling region's top
?7	Auto-wrap: text reaching the last column goes on to the next line (set as it starts)
?25	The cursor is shown (set as it starts)
?47, ?1047	The **alternate screen**: a second screen, without scrollback, for full-screen programs
?1049	As ?1047, saving the cursor first (and back, when reset)
?1000	Report the mouse's buttons to the program (Chapter 16)
?2004	Bracketed paste: a paste comes between `ESC [200~` and `ESC [201~`

`ESC =` and `ESC >` set the keypad's application and numeric modes; `ESC [?2l` switches to the VT52's older sequences, and `ESC <` back.

Character sets

`ESC (` *x* chooses the character set G0, and `ESC )` *x* G1, where *x* is `B` (ASCII), `A` (British: `#` is £) or `0` (the **DEC Special Graphics**, with line-drawing characters). SO and SI switch between G1 and G0. In the Special Graphics, the lower-case letters draw lines and boxes:

BYTE	j	k	l	m	n	q	t	u	v	w	x
Draws	┘	┐	┌	└	┼	─	├	┤	┴	┬	│

So `ESC (0lqqqkESC (B` draws the top of a box, ┌──┐.

Questions and answers

A program can ask the terminal things; the console answers into the program's keys, as if they'd been typed.

SEQUENCE	ASKS	THE ANSWER
`ESC [c`, `ESC Z`	What are you?	`ESC [?6c`: a VT102
`ESC [5n`	Are you all right?	`ESC [0n`
`ESC [6n`	Where's the cursor?	`ESC [` *r* `;` *c* `R`

Titles, status lines and lights

SEQUENCE	WHAT IT DOES
`ESC ]0;` *text* BEL, `ESC ]2;` *text* BEL	Set the window's label (as writing `/dev/label` does)
`ESC [` *n* `$~`	The status line: 0 none, 1 the terminal's own, 2 the program's
`ESC [1$}`, `ESC [0$}`	Write into the status line from now on; back to the screen
`ESC [` *n* `q`	The keyboard's four lights: 0 all off, 1 to 4 one on, 21 to 24 one off

Resetting

SEQUENCE	WHAT IT DOES
`ESC c`	Reset the terminal completely, and clear the screen
`ESC [!p`	A soft reset: the modes, the margins and the rendition as they start, without clearing

13.8 /pc

`cons` also serves `/pc`, device `#P`, a folder on the PC, or a card's image there (section 9.9). Its requests and the PC tool's replies go over the serial port as **frames**, mixed in with the text; each frame starts with the byte `$1E`, and has a check number (a CRC) at its end, so a damaged frame is sent again. The PC tool takes the frames out, so you never see them.

CHAPTER 14

Disks and HydraFS

The **storage** driver, in task E, looks after everything that keeps files: the SD cards, the RAM disks and the ROM disk, and HydraFS, the way files are arranged on them. This chapter explains them all, and the files that control them.

IN THIS CHAPTER: You'll connect an SD card reader, get a card ready and check it, use the RAM disks and the ROM disk, learn how HydraFS arranges a disk, and move files to and from a PC. **You need:** Chapters 9 and 12.

14.1 Three devices

`storage` serves three devices, one inside the other:

DEVICE	AT	WHAT IT IS
`#S`	`/dev/spi`	The **SPI bus**: the eight SPI headers and the slots' SPI devices, byte by byte (Chapter 17)
`#d`	`/dev/sd`	The **disks**: each one as a long row of bytes, with a control file
`#f`	`/sd`, `/rom`, `/ram`, `/sram`	**HydraFS**: the files and directories on the disks

Most of the time you use `#f`, through the paths in the namespace, and never think about the others.

14.2 SD cards

An **SD card** is the little memory card used in cameras and phones. The Hydra can use up to sixteen of them: one on each of its eight SPI headers, J18 to J25, and eight more on expansion cards in the slots. A Vera X card has an SD card socket of its own, too.

The SPI headers have six pins:

PIN	NAME	JOB
1	CS	Chip select: the Hydra chooses this device
2	CLK	The clock: one pulse for each bit
3	MOSI	Data from the Hydra to the device
4	MISO	Data from the device to the Hydra
5	+5V	Power
6	GND	Ground

The Hydra's SPI bus works at 5 volts, but SD cards work at 3.3 volts, so a card needs an **SD card module** between them: a small board with a card socket, a voltage regulator and a level shifter. Modules sold for 5-volt boards, like the Arduino, are just right. Connect each of the module's pins to the header's pin with the same job:

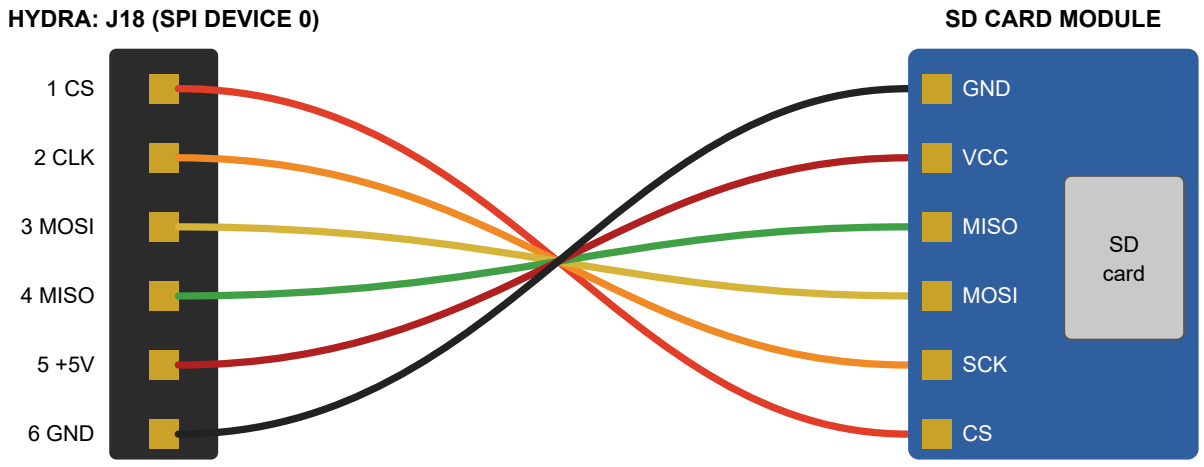


Figure 14-1. An SD card module on SPI header J18.

SPI HEADER	SPI DEVICE	THE CARD'S NAME	ITS FILES
J18	0	`0`	`/sd/0`
J19	1	`1`	`/sd/1`
J20 ... J25	2 ... 7	`2` ... `7`	`/sd/2` ... `/sd/7`
On cards in the slots	8 ... 15	`8` ... `f`	`/sd/8` ... `/sd/f`
The Vera X's socket		`v`	`/sd/v`

The card in device 0 is special: the namespace looks there for programs (`/sd/0/bin`), libraries and settings (`/sd/0/lib`), so it's the best place for your own.

Card types. SD cards up to 2 GB are **SDSC** cards; bigger ones are **SDHC** (up to 32 GB) or **SDXC** (the Hydra shows both as `sdhc`). A card of any size works, and HydraFS can use all of it, but a small one is plenty: the Hydra's files are small. A card has to have a HydraFS on it before the Hydra can keep files there.

A new card. To use a new card, put it in the module, and make an empty HydraFS on it:

```
/> mkfs 0 MYCARD
```

This erases everything that was on the card! If the card has partitions (a PC's card usually has one), `mkfs -p` puts HydraFS in a partition of its own after the others, and leaves the rest of the card alone.

You can also prepare a card on your PC, with the tools in Chapter 2: `hydrafs.js` makes a card image with your files in it, and `sdwrite.js` writes an image onto a card.

Taking a card out. The Hydra keeps the last piece of a file it's writing until the file is closed, so make sure no program is writing to a card before you take it out. A card can be changed while the Hydra runs: write `init` to the card's control file (section 14.4) after you put the new one in.

Speed. The Hydra's SPI bus is worked by its processor, one bit at a time, so it isn't fast: under 20 kilobytes a second for reading, and about half that for writing. A card in a Vera X is faster, as the Vera X has SPI hardware of its own.

14.3 The RAM disks and the ROM disk

The RAM disks keep files in memory. init starts both as the Hydra starts (section 7.4):

DISK	SIZE	KEPT IN	AT
`r`	256K	RAM banks on the memory cards (the storage driver's own)	`/ram`: each shell has its own directory of it
`s`	512K	A segment of the shared RAM	`/sram`: one for everybody

With less memory, they're made smaller. They're very fast, but they're empty again each time the Hydra starts.

The ROM disk, `x`, is part of the paged ROM, after the modules. It's made when HydraOS is built, and can't be changed (a program that tries is told `read-only`). It holds HydraOS's own files: the programs in `/rom/bin`, the libraries and settings in `/rom/lib`, a song, and a README.

14.4 The disks' control files

Each disk that has started is a directory in `#d` (bound at `/dev/sd`), with two files: `data`, the disk as one long file of bytes (its first 4 GB), and `ctl`, its control file.

```
/> ls /dev/sd
0/
x/
r/
s/
/> cat /dev/sd/0/ctl
sdhc 17 MB 34816 blocks
hydrafs label=HYDRA-16
partition at block 2048
free 15956 KB of 16380 KB
```

Reading a disk's `ctl` gives:

- its kind, size and number of **blocks** (pieces of 512 bytes: a disk is read and written a block at a time);
- `hydrafs label=`, and the disk's label, if it has a HydraFS;
- where its HydraFS partition starts, if it's in one;
- how much space is free;
- for a RAM disk, which banks or which segment it's in;
- after a check, what the check found.

The commands it understands are:

COMMAND	WHAT IT DOES
`init`	Start the card again: after it's been changed for another
`sync`	Write the block the driver is keeping back now, without waiting for its file to close
`format` [`-f`] [`-p`] [`-s` *size*] [*label*]	Make a new, empty HydraFS (as `mkfs`): `-f` a full format; `-p` in a partition of its own; `-s` *size* only that many megabytes of the card (or gigabytes, with a G: `-s 4G`)
`label` *text*	Set the disk's label (as `label`)
`check`	Check the HydraFS (as `fsck`)
`check fix`	Check it, and fix what's wrong (as `fsck -f`)
`start` *size* [*from-to*]	Start a RAM disk of *size* 8K banks (or *size*`K`, *size*`M`), with an empty HydraFS
`stop`	Stop a RAM disk, and give its memory back (not while one of its files is open)

A **full format** writes the disk's whole map of free space at once; a **quick format** (the usual) writes it a piece at a time, as the space is first used. A full format of a 32 GB card takes about a minute and a half.

> **CAUTION:** Writing to a disk's `data` file changes its blocks directly, underneath HydraFS. It's a quick way to spoil a disk. Reading it is harmless, though: `xd /dev/sd/0/data | more` shows a card's first blocks.

14.5 HydraFS

HydraFS is the Hydra's own way of arranging files on a disk. Every disk uses it: the cards, the RAM disks and the ROM disk. Here's how it works.

A disk is a row of **blocks**, of 512 bytes each. The first block of the HydraFS (block 0 of the card, or of its partition) is the **superblock**. It starts with the word `HYDRAFS1`, and says where everything else is: the **free map**, which has one bit for each **cluster** of the disk, saying whether it's in use; and the **data area**, where the files and directories are. Space is given to files a cluster at a time: 4K (8 blocks) on a card, or 1K (2 blocks) on the RAM disks and the ROM disk, so a disk with lots of small files wastes less.

A **directory** is a file of **entries**, 64 bytes each. An entry holds a file's:

- name (up to 31 characters);
- **mode**: whether it's a directory, append-only or read-only;
- length, in bytes (up to 4 GB);
- time it was last changed, in seconds since the start of 2000;
- where its clusters are: up to two **extents** (runs of clusters, one after another) in the entry itself, and, for a file in many pieces, more in **extent blocks**, of 84 extents each.

A file can have **holes**: parts that were never written (because the program writing it skipped ahead with `SEEK`). A hole takes no space on the disk, and reads as zeros.

HYDRAFS'S LIMITS	
A name	31 characters: any but `/`; capital and small letters are different
A path (on any disk)	63 characters
A file	4 GB
Files open at once	32, on all the disks together

Partitions. A PC's SD card has a **partition table** in its first block, dividing it into parts. HydraFS can live in a partition of its own (its type number is `$7F`), so a card can have a PC's files in one partition and the Hydra's in another. `mkfs -p` makes one, starting on a 1 MB boundary.

14.6 Checking a disk

If the power goes off while a disk is being written, or a card is taken out too soon, its HydraFS may be left with small mistakes. `fsck` (or the `check` command) looks at every directory and every file, and compares the space they use with the free map:

```
/> fsck s
hydrafs label=SRAM
free 510 KB of 511 KB
check: lost 0, unmarked 0, twice 0
segment 15
```

COUNT	WHAT IT IS	WHAT `FSCK -F` DOES
`lost`	Clusters marked in use, that no file has	Marks them free
`unmarked`	Clusters a file has, that aren't marked in use	Marks them in use
`twice`	Clusters that two files both claim	(Reports them: copy the files, and remove one)

All three should be 0. `lost` space is harmless, just wasted; `unmarked` space is dangerous, because a new file could be given it and overwrite the old one, so fix it straight away.

14.7 Moving files to and from a PC

There are four ways:

WAY	HOW
`/pc`	With the PC tool running, copy files to and from `/pc` (section 9.9)
`xmodem`	With a terminal program that speaks XMODEM (Chapter 11)
A card image	On the PC, put files into a card image with `hydrafs.js`, and write it to a card with `sdwrite.js` (Chapter 2)
The emulator	The emulator can use a card image, or a folder on the PC as `/pc` (Chapter 2)

CHAPTER 15

Sound

The Hydra has a real music synthesizer chip on its board: the **Yamaha YM2151**, the chip that played the music in many 1980s arcade games and keyboards. With a Vera X card, it has a second sound chip too: the Vera X's **PSG**, with sixteen simple voices, and **PCM**, which plays recorded sounds. This chapter shows how to play notes, songs and sounds, and describes the sound driver's files.

IN THIS CHAPTER: You'll play notes, chords, drums and whole songs on the YM2151 (and on a Vera X's sound chips), write music as text, and learn the sound driver's files. **You need:** Chapter 12, and headphones or a speaker (in the emulator, the **Sound** button).

15.1 The sound hardware

The **YM2151** (U38) makes sound by **FM synthesis**. It has 8 **channels**, so it can play 8 notes at once. Each channel has 4 **operators**: little oscillators, each making a tone, which change each other's sound as they're combined. Different settings of the operators make sounds like a piano, a flute, a bass guitar or a drum. A set of settings is called a **patch**, or an instrument.

The YM2151 doesn't make sound directly: it sends numbers to the **YM3012** (U41), which turns them into a voltage for the left and right speakers. The **TL074** (U42) smooths and boosts them, and the **mixer** adds the sound from slot 0 (the Vera X's) and from **Audio In** (J29). The **NJM4556A** (IC9) is the headphone amplifier, and the sound comes out of J26, a 3.5 mm stereo jack, for headphones or powered speakers.

The **Vera X's PSG** (programmable sound generator) has 16 voices. Each makes a simple wave (a pulse, a sawtooth, a triangle or noise) at any pitch and volume, in the left or right speaker or both. Its **PCM** plays sampled sounds: recordings, like a WAV file.

The sound driver, `snd` (task C), calls these all **channels**:

CHANNELS	CHIP
0 to 7	The YM2151
8 to 23	The Vera X's PSG voices 0 to 15 (only with a Vera X)

15.2 Playing notes from the shell

The simplest way to make a sound is to write commands to `/dev/sndctl`. Choose an instrument, play a note, and stop it:

```
/> echo patch 0 0 >/dev/sndctl
/> echo note 0 60 >/dev/sndctl
/> echo off 0 >/dev/sndctl
```

`patch 0 0` gives channel 0 patch 0, the acoustic piano. `note 0 60` plays note 60, **middle C**, on channel 0. `off 0` lets it go. Notes are numbered the way electronic instruments number them (the **MIDI** numbers): each number is a **semitone** (one key on a piano, black or white) higher than the one before, and every 12 numbers is an **octave**:

NOTE	C	C#	D	D#	E	F	F#	G	G#	A	A#	B
Octave 3	48	49	50	51	52	53	54	55	56	57	58	59
Octave 4	**60**	61	62	63	64	65	66	67	68	**69**	70	71
Octave 5	72	73	74	75	76	77	78	79	80	81	82	83

Note 60 is middle C, and note 69 is the A above it, which sounds at 440 vibrations a second (440 Hz): the note an orchestra tunes to.

It's quicker to play a tune with `play -m`, which takes a line of **MML** (section 15.6):

```
/> play -m 0 'o4 l8 c d e f g a b > c'
/> play -m 0 'I40 o5 l4 c c g g a a g2'
```

The first plays a scale of C on channel 0. The second plays "Twinkle, Twinkle, Little Star" on patch 40, the violin. `play -c` plays a **chord**, a note on each channel from the one given:

```
/> play -c 0 c e g
```

15.3 /dev/sndctl: the sound driver's commands

COMMAND	WHAT IT DOES
`patch` *ch p*	Give channel *ch* patch *p* (section 15.4)
`note` *ch n*	Play note *n* on channel *ch*
`off` *ch*	Let channel *ch*'s note go
`level` *ch v*	Channel *ch*'s volume, 0 to 127
`pan` *ch* `left` | `right` | `both`	Which speakers channel *ch* plays in (or 0 to 3: none, left, right, both)
`bend` *ch b*	Bend channel *ch*'s pitch, -128 to 127: 64 is a semitone up
`drum` *ch n*	Play General MIDI drum *n* on channel *ch* (section 15.4)
`freq` *ch hz*	Play a tone of *hz* vibrations a second on channel *ch* (0: stop)
`glide` *ch n*	Change channel *ch*'s note to *n* smoothly, without playing it again (**legato**)
`wave` *ch* `pulse` | `saw` | `triangle` | `noise` [*width*]	A PSG channel's wave (or 0 to 3); a pulse's width, 0 to 63 (63: a square wave)
`sens` *ch pms ams*	How much the LFO (below) changes channel *ch*'s pitch (0-7) and volume (0-3)
`lfo` *rate pmd amd wave*	The YM2151's **LFO**, a slow wobble for vibrato and tremolo: its rate (0-255), its pitch and volume depths (0-127), and its wave (0 saw, 1 square, 2 triangle, 3 noise)
`noise` *n* | `off`	The YM2151's noise (on channel 7), at frequency *n* (0-31), or off
`reg` *r v* [*r v* ...]	Write value *v* to YM2151 register *r*
`volume` *n*	The master volume of everything, 0 to 200: 100 is normal, more is louder
`reset`	Silence the chips, and clear every setting
`claim` *n* [*p*]	Claim channels: *n*'s bits are channels 0-7, *p*'s channels 8-23

COMMAND	WHAT IT DOES
`release` *n* [*p*]	Release them

Numbers can be written in hex with `$`: `claim $FF` claims all eight YM2151 channels.

Reading `sndctl` shows the master volume, how many channels there are (8, or 24 with a Vera X), and which channels are claimed:

```
/> cat /dev/sndctl
volume 100
channels 8
claimed
```

Claims. Any program can play on any channel, which is fine until two of them play on the same one. A program that's playing a song claims the channels it uses; while they're claimed, other programs' commands for them are ignored. A claim belongs to the program that made it, and ends when it releases the channels, or closes its last sound file. So a claim made with `echo` ends straight away, when `echo` ends!

15.4 Patches and drums

The sound driver has 163 patches, the same as the Commander X16's. Patches 0 to 127 are the instruments of **General MIDI**, a standard list used by electronic instruments everywhere:

NO.	INSTRUMENT	NO.	INSTRUMENT	NO.	INSTRUMENT	NO.	INSTRUMENT
0	Acoustic Piano	32	Acoustic Bass	64	Soprano Sax	96	FX 1 Rain
1	Bright Acoustic Piano	33	Electric Finger Bass	65	Alto Sax	97	FX 2 Soundtrack
2	Electric Grand Piano	34	Electric Picked Bass	66	Tenor Sax	98	FX 3 Crystal
3	Honky-Tonk Piano	35	Fretless Bass	67	Baritone Sax	99	FX 4 Atmosphere
4	Electric Piano 1	36	Slap Bass 1	68	Oboe	100	FX 5 Brightness
5	Electric Piano 2	37	Slap Bass 2	69	English Horn	101	FX 6 Goblins
6	Harpsichord	38	Synth Bass 1	70	Bassoon	102	FX 7 Echoes
7	Clavinet	39	Synth Bass 2	71	Clarinet	103	FX 8 Sci-Fi
8	Celesta	40	Violin	72	Piccolo	104	Sitar
9	Glockenspiel	41	Viola	73	Flute	105	Banjo
10	Music Box	42	Cello	74	Recorder	106	Shamisen
11	Vibraphone	43	Contrabass	75	Pan Flute	107	Koto
12	Marimba	44	Tremolo Strings	76	Blown Bottle	108	Kalimba
13	Xylophone	45	Pizzicato Strings	77	Shakuhachi	109	Bagpipes
14	Tubular Bells	46	Harp	78	Whistle	110	Fiddle
15	Dulcimer	47	Timpani	79	Ocarina	111	Shanai
16	Drawbar Organ	48	String Ensemble 1	80	Lead 1 Square	112	Tinkle Bell
17	Percussive Organ	49	String Ensemble 2	81	Lead 2 Saw	113	Agogo
18	Rock Organ	50	Synth Strings 1	82	Lead 3 Triangle	114	Steel Drum
19	Church Organ	51	Synth Strings 2	83	Lead 4 Chiff Sine	115	Woodblock

NO.	INSTRUMENT	NO.	INSTRUMENT	NO.	INSTRUMENT	NO.	INSTRUMENT
20	Reed Organ	52	Choir Aahs	84	Lead 5 Charang	116	Taiko Drum
21	Accordion	53	Doo	85	Lead 6 Voice	117	Melodic Tom
22	Harmonica	54	Synth Choir	86	Lead 7 Fifths	118	Synth Drum
23	Bandoneon	55	Orch Hit	87	Lead 8 Solo	119	Reverse Cymbal
24	Acoustic Nylon Guitar	56	Trumpet	88	Pad 1 Fantasia	120	Guitar Fret Noise
25	Acoustic Steel Guitar	57	Trombone	89	Pad 2 Warm	121	Breath Noise
26	Electric Jazz Guitar	58	Tuba	90	Pad 3 Poly	122	Seashore
27	Electric Clean Guitar	59	Muted Trumpet	91	Pad 4 Choir	123	Bird Tweet
28	Electric Muted Guitar	60	French Horn	92	Pad 5 Bowed	124	Telephone
29	Electric Overdriven Guitar	61	Brass Section	93	Pad 6 Metallic	125	Helicopter
30	Electric Distorted Guitar	62	Synth Brass 1	94	Pad 7 Halo	126	Applause
31	Electric Guitar Harmonics	63	Synth Brass 2	95	Pad 8 Sweep	127	Gunshot

Patches 128 to 162 are drum sounds:

NO.	DRUM SOUND	NO.	DRUM SOUND	NO.	DRUM SOUND
128	Silent	140	Pedal Hi-Hat	152	Long Guiro
129	Snare Roll	141	Open Hi-Hat	153	Mute Cuica
130	Snap	142	Crash	154	Open Cuica
131	High Q	143	Ride Cymbal	155	Mute Triangle
132	Scratch	144	Splash Cymbal	156	Open Triangle
133	Square Click	145	Tambourine	157	Jingle Bell
134	Kick	146	Vibraslap	158	Bell Tree
135	Rim	147	Bongo	159	Mute Surdo
136	Snare	148	Maracas	160	Pure Sine
137	Clap	149	Short Whistle	161	Timbale
138	Tom	150	Long Whistle	162	Open Surdo
139	Closed Hi-Hat	151	Short Guiro		

On a PSG channel (8 to 23), a patch below 4 chooses a wave: 0 pulse, 1 sawtooth, 2 triangle, 3 noise.

Drums. `drum` *ch n* plays a drum by its **General MIDI drum number**, which picks the right patch and pitch. The most useful are:

NUMBER	DRUM	NUMBER	DRUM
35, 36	Bass drum (kick)	49, 57	Crash cymbal
37	Side stick (rim)	51, 59	Ride cymbal
38, 40	Snare drum	54	Tambourine
39	Hand clap	55	Splash cymbal
41, 43, 45, 47, 48, 50	Toms, low to high	56	Cowbell

NUMBER	DRUM	NUMBER	DRUM
42	Closed hi-hat	60, 61	Bongos
44	Pedal hi-hat	69	Cabasa
46	Open hi-hat	70	Maracas

15.5 The other sound files

/dev/snd is the YM2151 itself. Each pair of bytes written to it is a **register** number and a value: the chip has 256 registers, and every setting of every operator is in one. The driver keeps a copy of what was written, which a read gives back (the chip's own registers can't be read). The driver adjusts the levels for the channels' and the master volume, so a song written as register writes still follows the volume settings.

The YM2151's main registers are:

REGISTER	WHAT IT HOLDS
`$08`	Key on and off: bits 6-3 the operators to start (0: stop), bits 2-0 the channel
`$0F`	Noise: bit 7 on, bits 4-0 its frequency
`$18`	The LFO's rate
`$19`	The LFO's depth: bit 7 0 for volume, 1 for pitch; bits 6-0 the depth
`$1B`	Bits 1-0 the LFO's wave; bits 7-6 the CT1 and CT2 outputs (J9)
`$20-$27`	For each channel: bits 7-6 the speakers (R, L), bits 5-3 the feedback, bits 2-0 the **algorithm** (how the operators are joined)
`$28-$2F`	Each channel's pitch: its octave (bits 6-4) and note (bits 3-0)
`$30-$37`	Each channel's pitch, finer: in 64ths of a semitone (bits 7-2)
`$38-$3F`	Each channel's LFO sensitivity: pitch (bits 6-4), volume (bits 1-0)
`$40-$5F`	Each operator's detune 1 (bits 6-4) and frequency multiple (bits 3-0)
`$60-$7F`	Each operator's **total level**: its loudness, 0 loudest to 127 silent
`$80-$9F`	Each operator's key scaling (bits 7-6) and attack rate (bits 4-0)
`$A0-$BF`	Each operator's LFO volume on (bit 7) and first decay rate (bits 4-0)
`$C0-$DF`	Each operator's detune 2 (bits 7-6) and second decay rate (bits 4-0)
`$E0-$FF`	Each operator's first decay level (bits 7-4) and release rate (bits 3-0)

In the operators' registers, the 32 registers of each block are 8 channels for each of 4 operators: register `$40` + 8 × *operator* + *channel*. The full story is in Yamaha's YM2151 application manual. The register numbers the chip doesn't have, from `$02` to `$0E` (but `$08`, key on), are the driver's own commands, the same ones as `sndctl`'s, for programs that want to send them as bytes.

/dev/psg is the Vera X's PSG: pairs of a register (0 to 63) and a value, four registers for each voice:

REGISTER	WHAT IT HOLDS
4*v* + 0, 4*v* + 1	Voice *v*'s frequency, low byte then high byte
4*v* + 2	Bit 7 the right speaker, bit 6 the left; bits 5-0 the volume (0 to 63)
4*v* + 3	Bits 7-6 the wave (0 pulse, 1 saw, 2 triangle, 3 noise); bits 5-0 a pulse's width

/dev/bell rings the bell: a short beep on channel 7, unless a program has claimed channel 7. The console writes it when a program prints the BEL character (7).

15.6 MML: writing music

MML, the **music macro language**, is a short way of writing a tune as text. Each letter is a note, and numbers say how long. `play -m` plays a line of it, and a **score** file holds a whole song.

MML	WHAT IT DOES
`c d e f g a b`	A note; `+` or `#` after it makes it sharp, `-` flat: `c#`, `b-`
note length	A length after a note: 1 a whole note, 2 a half, 4 a quarter, 8 an eighth, 16, 32, 64 (and 3, 6, 12, 24, 48 for triplets): `c4`
`.`	After a length: half as long again (`c4.`)
`^` *length*	Tie more on: `c4^16`
`r` [*length*]	A rest: silence
`o` *n*	The octave: `o4 c` is middle C
`> <`	Up an octave; down an octave
`l` *n*	The length for notes that don't give one
`v` *n*	The volume, 0 to 127
`q` *n*	How much of each note is held, in eighths (1 to 8; usually 7)
`p l` | `r` | `c` | `0`	The speakers: left, right, both (centre), none
`@`*name*	Use the score's instrument *name*
`I` *n*	Use patch *n* as the instrument
`x` *n* [*length*]	A General MIDI drum
`&`	Between notes: join them (no new attack)
`_`	Before a note: slide to it from the last
`k` *n*	Transpose by *n* semitones
`D` *n*	Detune by *n* 64ths of a semitone
`M` *pms,ams*	The channel's LFO sensitivity
`L` *rate,pmd,amd,wave*	The LFO
`N` *n*, `N-`	The noise on, at frequency *n*; off
`y` *reg,val*	Write any register
`[ ... ]` *n*	Repeat what's inside *n* times
`|`	A bar line (it does nothing, but helps you read)

A **score** is a text file whose name ends in `.mml`. Each line is one of these:

LINE	WHAT IT IS
`; ...`	A comment
`#tempo` *n*	The speed: *n* quarter notes a minute (120 unless you say)

LINE	WHAT IT IS
`#rate` *n*	The song's ticks a second (200, the Hydra's own tick)
`#title` *text*	The song's title
`@`*name* `{` ... `}`	An instrument (below)
`A` *mml* ... `H` *mml*	MML for channel 0 ... 7 (the YM2151); a channel's lines are joined, in order
`I` *mml* ... `X` *mml*	MML for channel 8 ... 23 (the PSG)

An instrument is either `gm` *n*, one of the patches, or the YM2151 settings themselves: `alg` *a* `fb` *f* `pms` *p* `ams` *a*, then for each operator, `m1`, `m2`, `c1` and `c2`, its `mul`, `dt1`, `dt2`, `tl`, `ks`, `ar`, `d1r`, `d2r`, `d1l`, `rr` and `am` (the registers' fields, as in section 15.5). A PSG instrument is `wave` *w* [*width*] and `env` *a d s r*, a volume **envelope**: *a* ticks rising to the note's volume, *d* ticks falling by *s* steps, held till the note is let go, then *r* ticks falling to silence.

Here's a little score, with a tune, a bass line and drums:

```
; A little tune
#tempo 120
@piano { gm 0 }
@bass { gm 33 }
A @piano o4 l8 c d e f g4 g4 a a a a g2
B @bass o2 l4 c c f c g g c2
C x35 4 x38 4 x35 4 x38 4 x35 4 x38 4 x35 4 x38 4
```

Save it as `/ram/tune.mml` (with `edit`), and play it:

```
/> play /ram/tune.mml
```

15.7 play

`play` plays scores, and **ZSM** files: the song format of the Commander X16, which Furnace, a free music program for PCs, can write. A ZSM file is a song already turned into register writes, so it takes less work to play.

COMMAND	WHAT IT DOES
`play` *song*	Play a score (`.mml`), a ZSM file, or a WAV file (on a Vera X)
`play` *song n*	Play it, and its repeating part *n* more times
`play -l` *song*	Play it over and over, until you press CTRL-C
`play -o` *score.mml song.zsm*	Turn a score into a ZSM file, instead of playing it
`play -m` *ch mml* ...	Play a line of MML on channel *ch*
`play -c` *ch notes* ...	Play a chord: each note on a channel, from *ch* on
`play -x` ...	With `-m` or `-c`: the Commander X16's MML (from its BASIC's `FMPLAY` and `PSGPLAY`)

```
/> play -o /ram/tune.mml /ram/tune.zsm
/> ls -l /ram/tune.zsm
--rw-rw-rw- fr      703 2000-01-01 00:00 /ram/tune.zsm
```

While it plays, `play` claims the channels the song uses, so other programs can't spoil it. The SD card that comes with HydraOS has more songs, in `/sd/0/songs`.

> **TIP:** On a PC, the `hysong.js` tool in `reborn/sim/tools` turns a score into a ZSM file too, and into a VGM file you can listen to on the PC.

15.8 Recorded sound

A Vera X can play **samples**: recorded sound, as numbers. `play` plays a WAV file, of 8 or 16 bits, mono or stereo, at up to 48,828 samples a second. Programs can play samples themselves through the Vera X's `pcm` and `pcmctl` files (Chapter 16).

15.9 Audio In

J29, **Audio In**, mixes sound from another device into the Hydra's: pin 1 is one channel, and pin 2 the other. Each goes into the mixer through a 10K resistor, as the YM2151's and slot 0's sound do. The other device's ground must be connected to the Hydra's ground too: at the I/O header, J27 pin 1, for example. Keep the sound's level to a line level, like a PC's line output: never connect a speaker output.

CHAPTER 16

The Vera X: Screen, Keyboard and Mouse

With a **Vera X** card, the Hydra has a screen of its own, a keyboard and a mouse, more sound, and another SD card. This chapter shows how to use them, and describes the files of the two drivers that look after them: `vid`, for the screen, and `kdev`, for the keyboard and mouse.

IN THIS CHAPTER: You'll put the console on a screen of its own, draw on it, and read its keyboard and mouse, through the files of the `vid` and `kdev` drivers. **You need:** Chapter 12, and a Vera X: the emulator has one.

16.1 The Vera X

The **VERA** (Video Embedded Retro Adapter) is the graphics chip of the Commander X16, a modern 8-bit computer. It's built in an **FPGA**, a chip that can be programmed to be any kind of circuit, and it does a lot:

- a picture of 640 by 480 pixels, for a VGA monitor (or a TV, through its composite or S-video output);
- 128K of **video RAM** (VRAM), which holds everything on the screen;
- two **layers**, each a screen of text (tiles) or a **bitmap** (a picture made of dots), one over the other;
- 128 **sprites**: small pictures that can be moved around the screen on their own;
- a **palette** of 256 colours, chosen from 4,096;
- the **PSG**, with 16 voices, and **PCM**, for recorded sound (Chapter 15);
- an SD card socket.

The **Vera X** is a VERA on a small card of its own. On the Hydra, it's wired to **slot 0**, through the bus breakout card, as I/O ports 2 and 3 (`$FF20` to `$FF3F`), with interrupt line 2. A PS/2 keyboard and mouse connect to the Commander X16's **SMC**, a little microcontroller that reads them, which talks to the Hydra on the I²C bus, at address `$42`. (The wiring is in the Vera X's schematic, in the `board/vera-x` folder.)

The emulator has a Vera X too (Chapter 2). In the web emulator it's there unless you turn it off in Setup; in the terminal emulator, add `--vera`.

> **NOTE:** The Vera X isn't part of the Hydra-16 kit, and this book doesn't show how to build one or wire it up: the reference book does (`docs/02-hardware/cards.md` has the breakout card's wiring, pin by pin, and how to bring the card up). If you don't have one, use the emulator's: everything in this chapter works there.

16.2 The console on the screen

When the Hydra starts with a Vera X, the console's windows are shown on its screen as well as on the serial terminal. The screen shows 80 columns and 60 rows of text, and the chrome: the bar at the top, with the groups and the time, and the window's header:

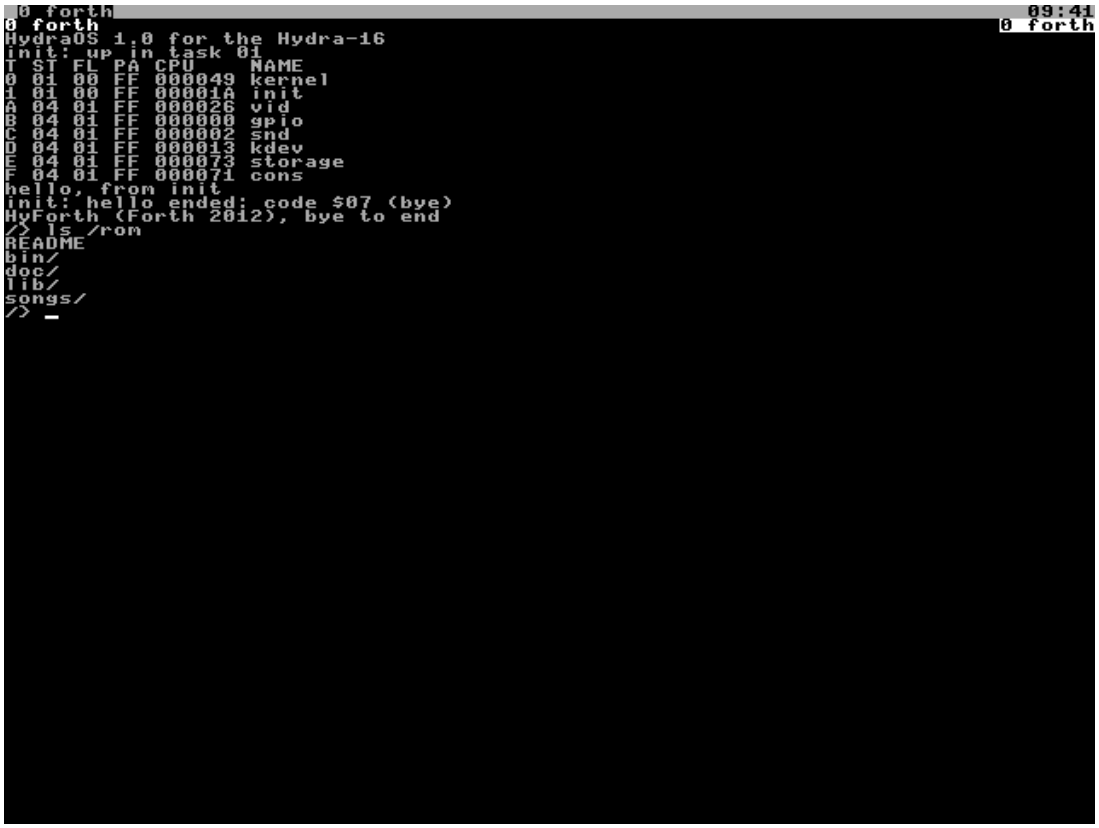


Figure 16-1. The console on the Vera X's screen, in its 80 by 60 mode.

With a keyboard, you can type at the screen just as at the terminal. `consctl` chooses where the windows are shown:

COMMAND	THE WINDOWS ARE SHOWN
`echo screen >/dev/consctl`	Only on the Vera X's screen
`echo serial >/dev/consctl`	Only on the serial terminal
`echo both >/dev/consctl`	On both (as the Hydra starts)
`echo seats >/dev/consctl`	On both, each a **seat** of its own (section 16.9)

16.3 /dev/vid/ctl: the screen's settings

COMMAND	WHAT IT DOES
`mode 80x60`	80 columns and 60 rows of text (the screen cleared)
`mode 80x30`	80 columns and 30 taller rows
`mode 40x30`	40 columns and 30 rows: big letters
`cursor blink` | `on` | `off`	The cursor blinking, steady, or hidden
`border` *n*	The border's colour, 0 to 255

COMMAND	WHAT IT DOES
`bitmap 320` *d*	A bitmap under the text: 320 by 240 dots, *d* bits a dot (1, 2, 4 or 8: 2, 4, 16 or 256 colours); the text becomes 40 by 30
`bitmap 640` *d*	A bitmap of 640 by 480 dots (*d*: 1 or 2); the text 80 by 60
`bitmap off`	No bitmap
`output vga`	The picture for a VGA monitor (as it starts)
`output ntsc` [`mono`] [`240p`]	For a TV: `mono` without colour, `240p` without interlace
`output rgb` [`240p`]	For an RGB monitor
`claim`	Take the chip for this program alone (section 16.6)
`claim all`	The same, and all of VRAM too, even the console's text and font
`release`	Give the chip back
`reset`	Set the chip up again for the console, and clear the screen

Reading it shows the settings:

```
/> cat /dev/vid/ctl
vera 47.0.2
mode 80x60
cursor blink
border 0
bitmap off
output vga
claimed
```

The first line is the version of the VERA's FPGA program (its **gateware**). The last line, `claimed`, names the program that has the chip, if one has (section 16.6).

16.4 Drawing

`/dev/vid/draw` draws on the bitmap, under the text. Turn a bitmap on with `ctl`'s `bitmap` command, then write commands to `draw`:

COMMAND	WHAT IT DOES
`pen` *c*	Draw in colour *c* from now on
`plot` *x y*	A dot
`line` *x0 y0 x1 y1*	A line
`box` *x0 y0 x1 y1*	A rectangle's outline
`bar` *x0 y0 x1 y1*	A filled rectangle
`circle` *x y r*	A circle's outline, of radius *r*
`disc` *x y r*	A filled circle
`clear` [*c*]	Fill the whole bitmap with colour 0, or *c*
`text` *x y words*	Write the words, in the console's font, at *x*, *y*

The dots are numbered from 0, with *x* across from the left and *y* down from the top: in a 320 bitmap, *x* goes from 0 to 319 and *y* from 0 to 239. Anything off the edge isn't drawn. Reading `draw` shows the pen's colour.

```
/> echo bitmap 320 8 >/dev/vid/ctl
/> echo clear 0 >/dev/vid/draw
/> echo pen 12 >/dev/vid/draw
/> echo circle 160 110 60 >/dev/vid/draw
/> echo pen 10 >/dev/vid/draw
/> echo box 40 30 280 190 >/dev/vid/draw
/> echo pen 14 >/dev/vid/draw
/> echo disc 160 110 20 >/dev/vid/draw
/> echo pen 11 >/dev/vid/draw
/> echo line 40 30 280 190 >/dev/vid/draw
/> echo line 40 190 280 30 >/dev/vid/draw
/> echo pen 15 >/dev/vid/draw
/> echo text 108 210 Hello, Hydra! >/dev/vid/draw
```

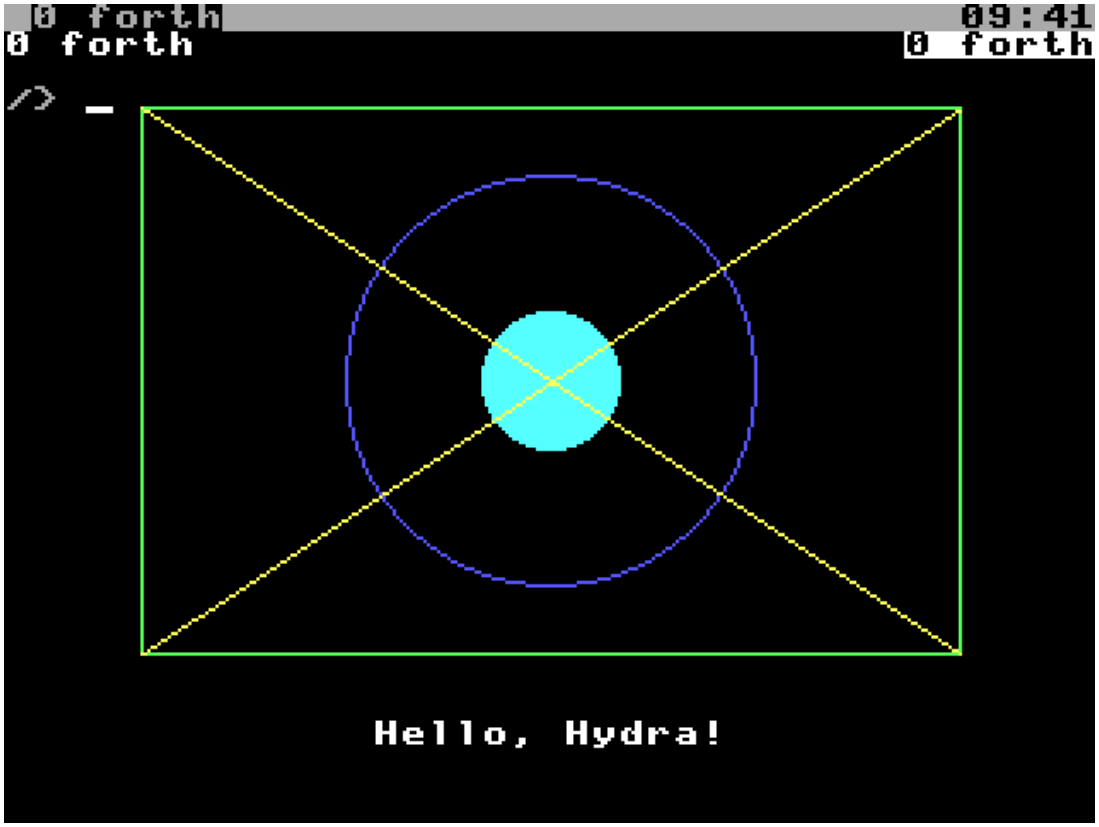


Figure 16-2. Drawing on the bitmap (the text screen cleared first).

The colours are the palette's: 0 to 15 are the console's 16 colours (in the order of section 13.7's colours, then their bright versions: 8 grey, 9 bright red, 10 bright green, 11 bright yellow, 12 bright blue, 13 bright magenta, 14 bright cyan, 15 white), and 16 to 255 the VERA's own.

16.5 The screen's other files

FILE	WHAT IT IS
`term`	The screen as a terminal: what's written is shown there, as the console does (below). A read gives the screen's text
`vram`	The VERA's 128K of video RAM: the file's offset is the address
`pal`	The palette: 256 colours, 2 bytes each (the first: green in the high 4 bits, blue in the low; the second: red)
`sprites`	The 128 sprites' settings, 8 bytes each. Sprite 0 is the console's cursor, and sprite 1 the mouse's pointer

FILE	WHAT IT IS
`font`	The console's font: 256 characters, 8 bytes each, a byte a row of dots
`frame`	A read waits for the next **frame** (the screen is drawn 59.5 times a second), then gives the number of frames so far
`psg`	The PSG's registers (the sound driver uses it: section 15.5)
`pcm`, `pcmctl`	Recorded sound (section 16.7)
`mouse`, `mousein`, `mousectl`	The mouse (section 16.8)
`draw`	Drawing on the bitmap (section 16.4)

The console's font is ISO-8859-15: ASCII, and the letters of Western European languages. The ROM disk has the PC's font too, with its line-drawing characters; to use it:

```
/> cat /lib/font/cp437 >/dev/vid/font
```

The screen's terminal. The console draws its windows on the screen through `term`, which understands this part of the terminal's language (section 13.7): the printable characters, CR, LF, BS, TAB and FF; `ESC 7`, `ESC 8`, `ESC c`, `ESC D`, `ESC E` and `ESC M`; the scrolling region (`ESC [`*t*`;`*b*`r`); the character sets (`ESC (` and `ESC )` with `0` or `B`, SO and SI); the cursor moves (`ESC [` with `A` to `H`, `d` and `f`); erasing (`J` and `K`); the colours (`m`: 0, 1, 7, 22, 27, 30-37, 39, 40-47, 49, 90-97, 100-107); saving the cursor (`s`, `u`); and showing or hiding the cursor (`?25h`, `?25l`). Anything else is ignored.

VRAM. The console uses the top of the video RAM; the rest is free for programs:

ADDRESSES	WHAT'S THERE
`$00000-$1AFFF`	Free for programs (a bitmap starts at `$00000`)
`$1B000-$1EFFF`	The console's text: 128 by 64 characters, 2 bytes each
`$1F000-$1F7FF`	The console's font
`$1F800-$1F9BF`	The cursor's and the pointer's pictures
`$1F9C0-$1F9FF`	The PSG's registers
`$1FA00-$1FBFF`	The palette
`$1FC00-$1FFFF`	The sprites' settings

16.6 Taking over the chip

A program that wants to drive the VERA itself, the way an X16 program does (for a game, perhaps), writes `claim` to `/dev/vid/ctl`. While it has the chip, the driver leaves it alone, and the console stops drawing on the screen; the program can write the VERA's registers at `$FF20` to `$FF3F` directly. `claim all` lets it use all of the video RAM too. When the program writes `release` (or ends), the driver sets the chip up for the console again, and the console draws its window again. The VERA's registers have the same names and meanings as on the Commander X16, so the X16's documentation (its *Programmer's Reference*, in its VERA chapter) describes them; only their address is different.

16.7 Recorded sound: pcm and pcmctl

The VERA's PCM plays samples from a **FIFO**: a 4K queue that it empties at the sample rate. A program writes its samples to `pcm`; the driver puts them in the FIFO as it has room, and the program waits when it's full. Only one program at a time can have `pcm` open.

PCMCTL COMMAND	WHAT IT DOES
`rate` *hz*	The sample rate, up to 48,828 a second (in steps of 381; 0 stops)
`bits 8` | `16`	8-bit or 16-bit samples (signed: 16-bit ones low byte first)
`mono` | `stereo`	One channel, or two (left first)
`volume` *n*	0 to 15
`reset`	Empty the FIFO
`drain`	Wait until the FIFO has played out

Reading `pcmctl` shows the settings, and who has `pcm`. `play` plays WAV files this way (Chapter 15).

16.8 The mouse

The `kdev` driver reads the mouse (it's on the I²C bus, which is `kdev`'s) and writes its moves to `/dev/vid/mousein`. The video driver moves the **pointer**, sprite 1, and keeps the mouse's state in `/dev/vid/mouse`, which works like Plan 9's: a read waits until the mouse changes, then gives five numbers, each 11 characters wide:

NUMBER	WHAT IT IS
`m`	(Always `m`: it's a mouse record)
x, *y*	The pointer's position, in the screen's dots (from the top left)
buttons	The buttons held down: 1 left, 2 middle, 4 right, added together; 8 and 16 the wheel, up and down
ms	The time, in milliseconds

Writing `m` *x y* to `mouse` moves the pointer. `mousectl` has two commands: `pointer on` or `off` (the pointer is shown once the mouse has moved), and `swap on` or `off` (swap the left and right buttons).

The mouse works in the console too: clicking a tiled window gives it the keys, and a program that asks (with `ESC [?1000h`, section 13.7) gets each click as a report, as xterm sends them.

16.9 The keyboard

`kdev` also reads the keyboard, and writes each key to the console's keyboard file, `/dev/kbin`, as the bytes a PC's terminal would send for it. So everything works the same from the keyboard as from the serial terminal: CTRL-C, the window keys, the arrows, the function keys.

KEYS	SEND
Letters, digits, symbols	Their characters, for the US keyboard layout, with SHIFT and CAPS LOCK
CTRL and a letter	Its control character: CTRL-C is 3
ALT and a key	ESC, then the key
ENTER, BACKSPACE, TAB	CR, DEL, TAB (SHIFT-TAB: `ESC [Z`)
The arrows, HOME, END, INSERT, DELETE, PAGE UP, PAGE DOWN, F1-F12	As xterm sends them
The keypad	Digits with NUM LOCK on, the cursor keys with it off
SCROLL LOCK	Holds the window (as CTRL-] H)

The keyboard's lights follow its lock keys. `kdev` looks for the keyboard controller as the Hydra starts (three times, a second apart); finding none, it reads no keys, and doesn't look again.

Seats. Normally the screen and the serial terminal show the same window, and keys from either go to it. With `echo seats >/dev/consctl`, they become two **seats**: each shows its own window, and has its own keys (the keyboard's go to the screen's seat). Two people can use one Hydra at once, each with their own shell.

CHAPTER 17

The Other Devices

This chapter covers the rest of the Hydra's devices: the I/O header and its pins, the I²C bus, the SPI devices, the clock, and the kernel's own devices, which show the tasks, the environment, the shared memory and the ROM's modules as files.

IN THIS CHAPTER: You'll read switches and light LEDs on the I/O header, talk to I²C and SPI chips, set the clock, and look at the system through the kernel's own files: the tasks, the environment, shared memory and the ROM's modules. **You need:** Chapter 12; and for sections 17.1 to 17.4 on a real Hydra, a few parts (an LED, some resistors and wire).

17.1 The I/O header, J27

J27 brings eight of the VIA's pins out to a header, for your own experiments: switches, lights, sensors and small circuits. They're the VIA's **port A**, PA0 to PA7, and its two **handshake** lines, CA1 and CA2.

PIN	NAME	PIN	NAME
1	Ground	2	+5 V
3	PA0, and the I²C bus's SCL	4	PA1, and the I²C bus's SDA
5	PA2	6	PA3
7	PA4	8	PA5
9	PA6	10	PA7
11	CA1 (input only)	12	CA2

Each pin is protected by J28, an SP720 chip, which stops sparks of static and voltages beyond the supply's from reaching the VIA. Even so, keep everything you connect between 0 and 5 volts.

> **CAUTION:** A pin can only give out a tiny current: enough for another chip's input, but not enough to light an LED properly. To light an LED, connect it the other way round: from +5 V, through a 2.2K resistor and the LED, to the pin. Then a 0 on the pin lights it. For anything bigger, like a motor or a relay, use a transistor.

PA0 and PA1 are also the **I²C bus** (section 17.3): while a program is using the I²C bus, they're the bus's, and `/dev/gpio`'s commands for them don't work.

17.2 /dev/gpio: the pins

The `kdev` driver serves the pins as `#g`, at `/dev/gpio`:

FILE	WHAT IT IS
`0` … `7`	A pin: reads as `0` or `1`; writing `0` or `1` sets it (and makes it an output)

FILE	WHAT IT IS
`port`	All eight pins as one byte: bit *n* is pin *n*. Writing a byte sets the outputs (the inputs stay inputs)
`ctl`	The pins' directions, and CA1 and CA2
`ca1`	A read waits for a change on CA1, then gives the number of changes so far

Each pin is an **input** (the Hydra reads it) or an **output** (the Hydra sets it). All are inputs as the Hydra starts.

CTL COMMAND	WHAT IT DOES
`in` *n*	Make pin *n* an input
`out` *n*	Make pin *n* an output
`ddr` *n*	Set all eight at once: bit *k* of *n* is 1 for an output, 0 for an input
`ca1 rise` | `fall`	Count CA1's rises (0 to 1) or falls (1 to 0)
`ca2 0` | `1`	Make CA2 an output, 0 or 1
`ca2 in`	Make CA2 an input

```
/> echo out 2 >/dev/gpio/ctl
/> echo 1 >/dev/gpio/2
/> cat /dev/gpio/2
1
/> echo 0 >/dev/gpio/2
/> cat /dev/gpio/ctl
0 in 1
1 in 1
2 out 0
3 in 1
4 in 1
5 in 1
6 in 1
7 in 1
ca1 fall 0
ca2 in
```

Reading `ctl` gives a line for each pin (its number, its direction, and its level), then CA1's edge and how many it has counted, then CA2's setting. An input with nothing connected reads 1.

Waiting for a button. CA1 is an input that the VIA watches for you. Connect a push button from CA1 (pin 11) to ground (pin 1), and a 10K resistor from CA1 to +5 V (pin 2). Then each press makes CA1 fall, and a read of `/dev/gpio/ca1` waits for the next press:

```
/> cat /dev/gpio/ca1
1
```

(Press CTRL-C to stop it.) A button's contacts bounce when it's pressed, so one press may count as several.

17.3 /dev/i2c: the I²C bus

I²C (say "eye-squared-see") is a way of connecting small chips with just two wires, a clock (SCL) and data (SDA), as well as power and ground. Each chip on the bus has an **address**, a number from `$08` to `$77`, so many chips can share the two wires. Memory chips, clocks, temperature sensors, small displays and the Vera X's keyboard controller all use it. The Hydra's I²C bus is on J27: SCL on pin 3, SDA on pin 4, with the pull-up resistors already on the board.

`kdev` serves the bus as `#i`, at `/dev/i2c`. Its directory lists a file for each chip that answers, named by its address in hex, and `ctl`:

```
/> ls /dev/i2c
ctl
42
```

(Here, `42` is the Vera X's keyboard controller.) Writing to a chip's file sends it bytes; reading reads bytes from it: up to 64 at a time. Many chips have **registers**, numbered places inside them; with `ctl`'s `subaddress`, the file's offset is the register to read or write.

`CTL` COMMAND	WHAT IT DOES
`speed` *n*	The bus's speed, in kilohertz (1 to 100; the Hydra can only go about 40)
`subaddress 0`	Just bytes
`subaddress 1`	The file's offset is a one-byte register number, sent first (most chips)
`subaddress 2`	A two-byte register number, high byte first (big memory chips)

A chip that doesn't answer gives `i/o error`.

17.4 /dev/spi: the SPI devices

The storage driver serves the SPI bus itself as `#S`, at `/dev/spi`: a directory for each of the sixteen SPI devices (0 to 7 on the headers J18 to J25, section 14.2; 8 to f on the slots' cards), each with two files:

FILE	WHAT IT IS
`data`	Writing sends bytes to the device, and keeps what it sends back; reading gets those bytes. With none kept, a read clocks in new bytes (sending `$FF`): up to 256 a read
`ctl`	`mode 0` (the clock rests low) or `mode 3` (it rests high); both send and read on the clock's rise

Each write is one **transaction**: the device is selected, the bytes go back and forth, and it's let go. One program at a time can have a device's `data` open, and not while it's an SD card the storage driver is using.

> **NOTE:** Don't `cat` a `data` file: it never ends, as a read always clocks in more bytes.

17.5 The clock

The kernel counts **ticks**, 200 a second, and keeps the time. Three files, from `#t`, show them:

FILE	WHAT IT IS
`/dev/ticks`	The ticks since the Hydra started (counting up to 65,535, then from 0 again)
`/dev/time`	The date and time, as `2026-10-12 09:41:00`; write one in the same form to set the clock
`/dev/rtc`	Whether there's a DS1747 clock chip: `running`, `stopped` (it needs setting), or `none` (with `battery low` if its battery is running down)

A **DS1747**, fitted in U7's socket in place of the task RAM chip, is task RAM and a clock in one, with a battery that keeps the clock running while the Hydra is off. With one, the Hydra sets its clock from the chip as it starts, and setting `/dev/time` sets the chip too.

17.6 null, zero and kmesg

FILE	WHAT IT IS
`/dev/null`	Swallows whatever is written to it; reads as empty
`/dev/zero`	Reads as zeros, for ever; swallows writes
`/dev/kmesg`	The kernel's messages, from the start: the boot's, and anything else the kernel has had to say (section 7.4)

17.7 /env: the environment

Each task has an **environment**: a set of variables, which programs can read. A task starts with a copy of its parent's, so it's how a shell passes settings like `$path` to the programs it runs. `#e`, at `/env`, shows the environment of the task that's looking, a file for each variable:

```
/> ls /env
prompt
path
task
prog
```

(`prog` is the path of the program that's looking, `/bin/ls` here: the shell gives it to each program it starts, section 8.4.) A variable is a list of words, with a zero byte after each one:

```
/> cat /env/path | xd
0000000  2e 00 2f 62 69 6e 00                           ../bin.
```

Writing a file sets the variable, and making a file makes a new variable, in the writer's own environment. A program that sets a variable doesn't change its parent's.

17.8 /proc: the tasks

`#p`, at `/proc`, has a directory for each task in use, named by its number:

```
/> ls /proc
0/
1/
2/
12/
13/
14/
15/
```

In each are these files:

FILE	WHAT IT IS
`status`	One line: the task's name, state, parent, processor time (in ticks) and note group
`ctl`	Commands for the task (below)
`args`	Its program's arguments
`cwd`	Its current directory
`fd`	Its current directory, then its open files, a line each: the number, how it's open (`r`, `w`, `rw`), the device, the offset, and the name
`ns`	Its namespace, as `ns` shows it

FILE	WHAT IT IS
`env`	Its environment
`regs`	The processor's registers, as it was when it last stopped running
`mem`	The 64K of memory it sees: its task RAM, and its RAM and ROM banks as it has them chosen
`ram`	Its RAM banks
`note`	Writing a note's name (`interrupt`, `kill`, `hangup`, `alarm`) or number sends it the note

```
/> cat /proc/1/status
forth wait - 107 1
/> cat /proc/1/fd
/
0 rw #c 368 #c/cons
1 rw #c 368 #c/cons
2 rw #c 368 #c/cons
/> cat /proc/1/regs
PC=EAA0 A=00 X=02 Y=01 S=E7 P=37 W=00 U=00 RAM=00 ROM=1E
```

CTL COMMAND	WHAT IT DOES
`kill`	End the task (as `kill`)
`interrupt`	Interrupt it (as CTRL-C)
`note` *n*	Send it note *n*
`stop`	Stop it: it isn't given any more turns until it's started again
`start`	Start it again
`step`, `next`, `break`, `nobreak`	For the debugger, `db` (Chapter 29)

`ps` and `top` read `/proc`; so does the debugger, which reads and writes a program's memory through `mem`.

17.9 /dev/seg: shared memory segments

The shared RAM can be divided into **segments**: pieces that several tasks can use together. A program makes one with a system call, and gets its number. `#s`, at `/dev/seg`, lets a segment have a **name**, so other programs can find it, and keeps it alive as long as it's named.

CTL COMMAND	WHAT IT DOES
`name` *name n*	Give segment *n* a name
`free` *name*	Take the name away

Each name is a file, reading as the segment's number and size (in 8K banks):

```
/> echo name disk-s 15 >/dev/seg/ctl
/> ls /dev/seg
ctl
disk-s
/> cat /dev/seg/disk-s
15 64
```

(Segment 15 is the shared RAM disk: 64 banks, 512K.)

17.10 /dev/mod: the ROM's modules

`#m`, at `/dev/mod`, has a file for each module in the paged ROM, holding its program, and a directory, `bin`, of the modules that are programs (which the namespace adds to `/bin`). `mods` lists them in more detail (Chapter 11).

17.11 Pipes, and raw memory

`#|` makes **pipes**: each time it's opened, a new pipe, 512 bytes long, whose two ends are two files. What's written to one end can be read from the other. The shell makes one for each `|` in a command line. There can be 8 at once.

`#r` is the whole of the Hydra's RAM, for init alone: `task` is all sixteen tasks' 32K, one after another, and `shared` is the shared RAM. It isn't in the namespace.



PART FOUR

Programming Languages

The Hydra has three programming languages built in: **HyForth**, **hylang** and **HyBASIC**. This part starts with the numbers they all share, then teaches each language in two chapters: the first for learning it, the second for using it to work the Hydra's devices, files and sound. The last chapter writes one program in all three, side by side. Appendices A, B and C are each language's complete reference.

Which language first? If you've never programmed, start with **HyBASIC** (Chapter 23): it reads almost like English. **HyForth** (Chapter 19) is the Hydra's shell, so it's the one you'll use every day, and it's the best way to feel how a small computer works. **hylang** (Chapter 21) has the most powerful ideas, and suits people who like mathematics and lists. You don't have to learn all three, and you can read each language's two chapters without the others'. Whichever you choose, read sections 18.1 to 18.3 first: all three count the same way.

- Chapter 18. Numbers and Bases
- Chapter 19. Learning HyForth
- Chapter 20. Programming the Hydra in HyForth
- Chapter 21. Learning hylang
- Chapter 22. Programming the Hydra in hylang
- Chapter 23. Learning HyBASIC
- Chapter 24. Programming the Hydra in HyBASIC
- Chapter 25. One Program, Three Languages

CHAPTER 18

Numbers and Bases

Most small computers can only count to 255, or 65,535, and get fractions slightly wrong. The Hydra's numbers are different: they can be as big as you like, fractions are exact, and they can be written in any **base**, from binary to base 80 and beyond. The same numbers work in hylang, in HyBASIC, in HyForth (with its numbers library), in the `calc` command, and in assembly language. This chapter explains them, using `calc` for the examples, since it works at the shell; each language's own chapter shows how it writes them.

IN THIS CHAPTER: You'll meet the Hydra's four kinds of number (whole numbers of any size, exact fractions, decimals and complex numbers), and learn to write and print them in any base. **You need:** Chapter 6, and section 1.6. If you're in a hurry to program, read sections 18.1 to 18.3, and come back for the rest when you need it.

18.1 Four kinds of number

The Hydra has four kinds of number.

Integers are whole numbers: 0, 42, -17. They can have as many digits as you need (up to about 600).

```
/> calc '2^100'
1267650600228229401496703205376
/> calc 'fib 100'
354224848179261915075
```

(`fib 100` is the hundredth **Fibonacci number**: each is the sum of the two before it, 1, 1, 2, 3, 5, 8 ...)

Fixed decimals are numbers with a decimal point, like 3.14 or 0.5. They're exact: each has a fixed number of places after the point, and adding or multiplying them never loses any. So, unlike most computers, the Hydra knows that 0.1 and 0.2 make exactly 0.3:

```
/> calc '0.1 + 0.2'
0.3
/> calc '3.14 * 2'
6.28
```

Rationals are fractions: one integer over another, like 2/3. Dividing two integers that don't divide exactly gives a rational, never a rounded decimal, and rationals are always kept in their lowest terms:

```
/> calc '1/3 + 1/6'
1/2
/> calc '100 / 7'
100/7
/> calc '1/3 * 3'
1
```

Complex numbers have a real part and an imaginary part, which is written with `i`. `i` is the square root of -1, a number that mathematicians invented so that every number has a square root:

```
/> calc 'i^2'
-1
/> calc 'sqrt(-4)'
2i
/> calc '(1+2i) * (3-i)'
5+5i
```

When numbers of different kinds meet, the answer is the simplest kind that can hold it exactly. An integer divided by an integer is a rational, unless it comes out whole; a rational and a decimal make a rational (`1/3 + 0.5` is `5/6`); and anything with a complex number is complex, unless its imaginary part turns out to be 0.

Each part of a complex number can be any real kind, and the two needn't match: `4/3+3.7i`, `3.9+1/2i`. Where `/` divides, in `calc` and HyBASIC, a fraction written right before `i` is still one number: `1/2i` is half of `i`, and `1/(2i)` divides.

18.2 Functions and precision

Some numbers can't be written exactly in any of these ways: the square root of 2, or pi, has digits that go on for ever. For these, the **math library**'s functions (square roots, powers, logarithms, sines ...) work to a **precision**: 12 digits unless you ask for more. `calc`'s `-d` asks for more:

```
/> calc 'sqrt 2'
1.41421356237
/> calc -d 40 'sqrt 2'
1.4142135623730950488016887242096980785
/> calc -d 50 pi
3.14159265358979323846264338327950288419716939937510
```

The answers are fixed decimals, rounded at the last place. Each language has its own way to set the precision (up to 100 digits): Chapters 19, 21 and 23 show how.

18.3 Bases

Section 1.6 showed three ways to write a number. In **decimal** (base 10) each place is worth ten times the one to its right. In **binary** (base 2) each is worth twice. In **hexadecimal** (base 16) each is worth sixteen times, and the digits run on from 9 through A to F. If any of that is hazy, read section 1.6 again before going on.

Nothing limits the idea to those three. Any whole number from 2 up can be a base: a base needs that many digits, and each place is worth that many times the one to its right.

- **Base 8**, **octal**, has the digits 0 to 7, and its places are worth 1, 8, 64, 512 and so on. One octal digit is exactly three bits, as one hex digit is four, and some older computers used octal where the Hydra uses hex.
- **Base 3** has the digits 0, 1 and 2, and its places are worth 1, 3, 9, 27, 81 and so on.
- **Base 36** uses every digit and every letter: 0 to 9, then A to Z for ten to thirty-five.

Here is one number, a hundred, in six bases:

BASE	ITS NAME	A HUNDRED	BECAUSE
2	Binary	1100100	64 + 32 + 4
3		10201	81 + 2 × 9 + 1
8	Octal	144	64 + 4 × 8 + 4

BASE	ITS NAME	A HUNDRED	BECAUSE
10	Decimal	100	100
16	Hexadecimal	64	6 × 16 + 4
36		2S	2 × 36 + 28 (S is the digit for 28)

`calc -b` shows its answer in another base. The base is named by a **base string**: `b` for binary, `x` for hex, `o` for octal and more (the next section lists them), or a number and `r` for any base at all (section 18.5):

```
/> calc -b b 100
1100100
/> calc -b x 100
64
/> calc -b o 100
144
/> calc -b 3r 100
10201
/> calc -b 36r 100
2S
```

18.4 The named bases

The Hydra has nineteen **named bases**, each named by a letter (capital or small: `X` and `x` are the same):

LETTER	BASE	DIGITS	CALLED
`b`	2	01	Binary
`t`	3	012	Ternary
`q`	4	0123	Quaternary
`v`	5	01234	Quinary
`f`	6	012345	Senary
`s`	7	0123456	Septenary
`o`	8	01234567	Octal
`n`	9	012345678	Nonary
`d`	10	0123456789	Decimal
`x`	16	0123456789ABCDEF	Hexadecimal
`z`	36	0-9 and A-Z	Base 36 (all the digits and letters)
`c`	3	`- 0 +`	Balanced ternary, least digit first
`i`	3	`U O N`	Balanced ternary in letters, least digit first
`e`	5	`= - 0 + #`	Balanced quinary, least digit first
`j`	5	`W U O N M`	Balanced quinary in letters, least digit first
`g`	7	`~ = - 0 + # *`	Balanced septenary, least digit first
`k`	27	`ZYXWVUTSRQPON 0 ABCDEFGHIJKLM`	Balanced base 27
`y`	53	`zyx...a 0 ABC...Z`	Balanced base 53
`m`	-13	`DanielStphrus`	Balanced base -13, least digit first

(Base `m` is a joke with a point: its thirteen digits are the letters of the Hydra's designer's name, and it shows that a base can be negative.)

```
/> calc -b t 10
101
/> calc -b o 10
12
/> calc -b z 1000000
LFLS
```

The first eleven are ordinary bases: their digits start at 0, and the biggest digit is one less than the base. The others are stranger, and need some explaining.

Balanced bases. In a **balanced** base, the digits go below zero as well as above it. In balanced ternary, base `c`, the three digits are `-` (minus one), `0` (zero) and `+` (plus one). There's no need for a minus sign: a negative number just uses its digits differently. The places are still ones, threes, nines, twenty-sevens ...

Least digit first. The bases `c`, `i`, `e`, `j`, `g` and `m` write their digits the other way round: the ones digit first, on the left. So in base `c`, 5 is `--+`:

PLACE	ONES	THREES	NINES
Digit	`-` (-1)	`-` (-1)	`+` (+1)
Worth	-1	-3	+9

and -1 - 3 + 9 = 5. Base `i` is the same, with the letters `U` (minus one), `O` (zero) and `N` (plus one) as its digits, so 5 is `UUN`.

```
/> calc -b c 5
--+
/> calc -b c 10
+0+
/> calc -b i 5
UUN
```

Balanced ternary isn't just a curiosity: some early Russian computers, like the Setun of 1958, worked in it, and it's the neatest way to weigh things with a balance scale.

Negative bases. In a **negative** base, the places are worth 1, the base, the base squared ..., and since the base is negative, the places alternate between positive and negative. Base `m` is base -13, balanced, with thirteen letters as its digits: `D a n i e l` are -6 to -1, `S` is 0, and `t p h r u s` are 1 to 6. Capital and small letters are different digits here, as `S` and `s` both appear.

18.5 Radixes and digits of your own

Besides the named bases, a base string can name a **radix**, or a set of **digits of your own**.

A radix is a number from 2 to 80, then `r`: `16r` is base 16, the same as `x`. A radix's digits are the first of these eighty:

```
0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz-=+`~!@#$%^&*,;:|?
```

So `36r` is the same as `z`, and `80r` uses them all:

```
/> calc -b 16r 255
FF
/> calc -b 80r 1000000
1;K0
```

Digits of your own are written in square brackets: from 2 to 80 characters, each used once, the first standing for zero, the next for one, and so on. Any character can be a digit, except a space, a control character, and these: `< > [ ] ( ) . / _ \ ' "`.

```
/> calc -b '[ABCDEFGHIJ]' 123
BCD
```

(With the digits A to J for 0 to 9, 123 is BCD.) If no two of the digits differ only in being capital or small, then capital and small letters are read as the same.

18.6 The modifiers

A base string can start with **modifiers**, which change how the base works:

MODIFIER	WHAT IT DOES
`#`	Numbers written in this base show the base, as a prefix: `#xFF` instead of `FF`
`<`	Write the digits least first (ones on the left)
`>`	Write the digits most first (the usual way)
`=`	Balanced: the middle digit is zero (for a radix or digits of your own; a named base is balanced already, or not)
`-`	A negative base
`+`	A positive base (the usual way)

They go in that order: `#`, then `<` or `>`, then `=`, then `+` or `-`, then the base.

```
/> calc -b '#x' 255
#xFF
/> calc -b '<x' 4660
4321
/> calc -b '-d' 10
190
```

4660 is `1234` in hex, so least digit first it's `4321`. In base -10, 190 is 1 × 100 + 9 × -10 + 0: 10.

So one base can have several names, which all mean the same:

BASE	SOME OF ITS NAMES
Binary	`b`, `2r`, `[01]`
Octal	`o`, `8r`, `[01234567]`
Decimal	`d`, `10r`, `[0123456789]`
Hexadecimal	`x`, `16r`, `[0123456789ABCDEF]`
Base 36	`z`, `36r`
Balanced ternary	`c` (least first, with `- 0 +`), `>c` (most first), `=3r` (most first, with `0 1 2` for -1, 0 and +1)

18.7 Writing a number in a base

Inside a program (or a `calc` expression), a number can say its own base: a `#`, the base string, and the digits.

WRITTEN	IS
`#b1010`	10, in binary

WRITTEN	IS
`#o17`	15, in octal
`#xFF`	255, in hex
`#16rFF`	255, base 16 as a radix
`#z1Z`	71, in base 36
`#c+--`	-11, in balanced ternary, least digit first
`#[ABCDEFGHIJ]BCD`	123, with digits of its own
`#<x4321`	4660, in hex, least digit first

```
/> calc '#b1010 + #x0F'
25
/> calc '#[ABCDEFGHIJ]BCD'
123
```

Numbers can have _s in them, to make them easier to read: `1_000_000` is a million. And decimal numbers can be written in all the forms of section 18.1: `-42`, `3.14`, `2/3`, `2i`, `1+2i`.

18.8 Fractions in other bases

A fraction can be written with a point in any base: in binary, `0.1` is a half, and `0.01` a quarter.

```
/> calc -b b 0.75
0.11
/> calc '#x0.8'
1/2
```

But just as 1/3 never ends in decimal (0.3333...), some fractions never end in other bases. Then they're written as a fraction, with the base's digits:

```
/> calc -b b '1/3'
1/11
```

(1 over 11: 1 over three, in binary.)

18.9 The base setting

Each program has a **base setting**: the base it reads numbers in, and writes them in, when they don't say their own. It starts as decimal. Every language can change it, and when it does, every number the program reads or writes follows it:

LANGUAGE	SETTING THE BASE	READING IT
hylang	`(base "x")`	`(base)`
HyBASIC	`BASE "x"`	`BASE$`
HyForth	`s" x" set-base` (or `hex`, `decimal`, and `n base !`)	`get-base`
`calc`	`calc -b x ...` (for its answer)	

In hylang, for example:

```
/> hylang
hylang (danlang on the Hydra-16)
Type 'exit' to Exit

hylang> (base "x")
=> "d"
hylang> (+ 9 1)
=> A
hylang> (base "d")
=> "x"
```

In hex, 9 + 1 is A. Setting the base gives back the one before, so a program can put it back when it's done.

18.10 Bits

Integers can be treated as rows of **bits** (binary digits), however big they are. Each language has the operations **and**, **or**, **xor**, **not**, and **shifts**, which move the bits left (multiplying by 2 each place) or right (dividing by 2, rounding down). A negative integer acts as if it had an endless row of 1s on its left, as in most computers.

OPERATION	EACH BIT OF THE ANSWER IS 1 WHEN
and	Both bits are 1
or	Either bit is 1
xor	The bits differ
not	The bit is 0 (and so not *x* is -*x* - 1)

18.11 When numbers go wrong

MESSAGE	MEANS
`division by zero`	Dividing by 0
`too big`	An integer of more than 255 bytes (about 600 digits), or a decimal with more than 65,535 places
`not a number`	Text that isn't a number
`outside the function's domain`	A function given a number it can't take, like the logarithm of 0
`not a base`	A base string that names no base
`an integer is needed`	A fraction where only a whole number will do
`a real number is needed`	A complex number where only a real one will do

```
/> calc 1/0
calc: division by zero
/> calc 'abs(3-4i)'
calc: a real number is needed
```

CHAPTER 19

Learning HyForth

Forth is a programming language invented by Charles Moore around 1970, to control radio telescopes. It's small, fast and simple enough to fit in a tiny computer, which made it popular on the home computers of the 1980s. **HyForth** is the Hydra's Forth: it follows the 2012 standard for the language, and adds words for the Hydra's own features. This chapter teaches you Forth from the beginning. You've met HyForth already: it's the shell you type at.

IN THIS CHAPTER: You'll learn Forth from the beginning: the stack, making new words, decisions, loops, variables, strings and memory. **You need:** Chapter 6. You don't need to have programmed before.

19.1 Talking to HyForth

At the `/>` prompt, any line that starts with a Forth word, or a number, is Forth (section 8.1); everything else is a command for rc. If you'd rather every line was Forth, type `forth` to start a plain HyForth, which says `ok` after each line it has done; type `bye` to leave it.

Forth reads a line as **words**, separated by spaces, and does each one in turn, from left to right. A word is any group of characters without a space: `+`, `dup`, `."` and `2/` are all words. Capital and small letters are the same, so `DUP` and `dup` are the same word. And a word that's a number puts that number on the **stack**.

19.2 The stack

The **stack** is the heart of Forth. It's a pile of numbers: each number you type goes on the top, and most words take their numbers off the top, and put their answers back there. `.s` shows the whole stack, without changing it; the number in angle brackets is how many are on it, and the top is on the right. `.` ("dot") takes the top number off, and prints it:

```
/> 1 2 3 .s
<3> 1 2 3
/> . . .
3 2 1
/> .s
<0>
```

The numbers come off in the opposite order to the one they went on: the last one on is the first one off. Think of a pile of plates: you put plates on the top, and take them off the top.

19.3 Arithmetic

Since the numbers are on the stack, a word like `+` doesn't need anything after it: it takes two numbers off the stack, adds them, and puts the answer back. So in Forth, you write the numbers first, and then what to do with them:

```
/> 2 3 + .
5
/> 10 3 - .
7
/> 6 7 * .
42
/> 20 4 / .
5
/> 7 2 mod .
1
```

`/` divides, and throws away any remainder; `mod` gives the remainder; and `/mod` gives both, the remainder under the answer:

```
/> 7 2 /mod . .
3 1
```

This way of writing sums is called **postfix**, or **reverse Polish**, notation. It never needs parentheses, because the order of the words says what's done first:

```
/> 2 3 + 4 * .
20
/> 2 3 4 * + .
14
```

The first adds 2 and 3, then multiplies by 4. The second multiplies 3 by 4, then adds 2. When a line of Forth puzzles you, do what Forth does: go along it a word at a time, and keep track of the stack. Here's the first line, that way:

THE WORD	WHAT IT DOES	THE STACK AFTERWARDS
`2`	Goes on the stack	2
`3`	Goes on the stack	2 3
`+`	Takes 2 and 3, puts back 5	5
`4`	Goes on the stack	5 4
`*`	Takes 5 and 4, puts back 20	20
`.`	Takes 20, and prints it	(empty)

To turn a sum from school into Forth, work from the inside out: do the parts in parentheses first. For (3 + 4) × (5 – 2), make the 7, then make the 3, then multiply them:

```
/> 3 4 + 5 2 - * .
21
```

TRY THIS: Work these out in your head, then ask HyForth: `10 2 3 + * .` and `10 2 * 3 + .` Now write (8 – 2) × (1 + 1) in Forth. (It should print 12.) If a line goes wrong half way, `.s` shows what's left on the stack, and a few `drop`s clear it.

NOTE: Each number on Forth's stack is a **cell**: 16 bits, so it holds -32,768 to 32,767 (or 0 to 65,535). A sum that goes past them wraps round: `32767 1 +` . prints `-32768`. Section 20.8 shows how HyForth's numbers library works with the Hydra's big numbers, fractions and decimals instead.

19.4 Moving things on the stack

These words rearrange the stack. Each comes with a **stack diagram**, in parentheses: what it takes from the stack, `--`, then what it leaves, with the top on the right.

WORD	STACK	WHAT IT DOES
`dup`	(*a* -- *a a*)	Copy the top
`drop`	(*a* --)	Throw away the top
`swap`	(*a b* -- *b a*)	Swap the top two
`over`	(*a b* -- *a b a*)	Copy the second to the top
`rot`	(*a b c* -- *b c a*)	Bring the third to the top
`nip`	(*a b* -- *b*)	Throw away the second
`tuck`	(*a b* -- *b a b*)	Copy the top under the second
`2dup`	(*a b* -- *a b a b*)	Copy the top two
`2drop`	(*a b* --)	Throw away the top two

```
/> 1 2 swap .s
<2> 2 1
/> drop drop
/> 1 2 3 rot .s
<3> 2 3 1
```

19.5 Making new words

This is where Forth comes alive. You can make new words out of old ones, and use them just like the built-in ones. A **definition** starts with `:` and the new word's name, and ends with `;`:

```
/> : square dup * ;
/> 5 square .
25
```

Now `square` is a word, as good as `dup` or `*`. You can use it to make more words:

```
/> : cube ( n -- n^3 ) dup square * ;
/> 3 cube .
27
```

The `( n -- n^3 )` is a **comment**: Forth skips everything from `(` to `)`. It's good to give each word a comment with its stack diagram, so you (and others) know what it takes and leaves. A `\` makes the rest of the line a comment.

`see` shows how a word was defined:

```
/> see cube
: cube dup square * ;
```

A Forth program is just a collection of words, each built from the ones before, until the last one does the whole job. Give your words names that say what they do, and keep each one short: a line or two. If you define a word again, the new definition is used from then on, but the words already made with the old one keep using it.

A definition on several lines. A definition doesn't have to fit on one line. Until its `;`, HyForth is still **compiling** it: adding each word you type to the new word, not doing it. To show that, the prompt changes to a row of spaces:

```
/> : half
        2 /
        ;
/> 10 half .
5
```

So if the prompt ever turns into spaces when you didn't expect it, you've left out a `;`. Type one, and the normal prompt comes back. (A line can be 127 characters long, at most; a long definition *has* to be on several lines.)

TRY THIS: Define `double` (*n* -- *2n*), and then `quadruple`, using `double` twice. Check them: `5 quadruple .` should print 20. Then ask `see quadruple`.

19.6 Printing

WORD	STACK	WHAT IT DOES
`.`	(*n* --)	Print a number, and a space
`."` *text*`"`	(--)	Print the text (in a definition)
`.(` *text*`)`	(--)	Print the text, at once (outside a definition)
`cr`	(--)	Start a new line
`emit`	(*char* --)	Print the character whose code is *char*
`space`	(--)	Print a space
`spaces`	(*n* --)	Print *n* spaces
`type`	(*c-addr u* --)	Print a string (section 19.11)

```
/> : hello ." Hello, world!" cr ;
/> hello
Hello, world!
```

The space after `."` isn't part of the text: it just separates the word from the text.

19.7 True and false

A **comparison** takes two numbers, and leaves a **flag**: -1 for true, or 0 for false.

```
/> 5 3 > .
-1
/> 5 3 < .
0
/> 5 5 = .
-1
```

WORD	STACK	TRUE IF
`=`	(*a b* -- *flag*)	*a* equals *b*
`<>`	(*a b* -- *flag*)	They differ
`<`, `>`	(*a b* -- *flag*)	*a* is less, or more, than *b*
`0=`	(*a* -- *flag*)	*a* is 0

WORD	STACK	TRUE IF
`0<`, `0>`	(*a* -- *flag*)	*a* is below, or above, 0
`within`	(*n lo hi* -- *flag*)	*lo* ≤ *n* < *hi*

`and`, `or` and `xor` combine flags (they work on each bit of the numbers), and `0=` turns true into false and back.

19.8 Making decisions

In a definition, `if` takes a flag from the stack. If it's true (anything but 0), the words up to `then` are done; if it's false, they're skipped. With an `else`, the words between `else` and `then` are done when it's false:

```
/> : test ( n -- ) 0< if ." negative" else ." not negative" then cr ;
/> -4 test
negative
/> 7 test
not negative
```

`then` is where the paths join up again: "if this, do that, *then* carry on". `if`, `else` and `then` only work inside definitions, and so do the loops of the next section. Typed at the prompt, they get this answer:

```
/> if
if: compile only
```

To choose between several cases, use `case`:

```
: day ( n -- )
  case
    0 of ." Sunday" endof
    6 of ." Saturday" endof
    ." a weekday"
  endcase cr ;
```

19.9 Loops

`do` ... `loop` counts. It takes two numbers: the end, and the start. `i` puts the count on the stack each time round:

```
/> : count-up 10 0 do i . loop cr ;
/> count-up
0 1 2 3 4 5 6 7 8 9
```

The loop stops before it reaches the end: 10 0 counts from 0 to 9. `+loop` counts in steps of the number it takes:

```
/> : evens 10 0 do i . 2 +loop cr ;
/> evens
0 2 4 6 8
```

`begin` ... `until` repeats until a flag is true:

```
/> : countdown ( n -- ) begin dup . 1- dup 0= until drop cr ;
/> 5 countdown
5 4 3 2 1
```

And `begin` ... `while` ... `repeat` tests at the start of each time round: it goes on while the flag is true.

LOOP	HOW IT WORKS
end start `do` ... `loop`	Counts from *start* to *end* - 1; `i` is the count (`j` the outer loop's)
end start `do` ... *n* `+loop`	Counts in steps of *n*
`begin` ... *flag* `until`	Repeats until the flag is true
`begin` ... *flag* `while` ... `repeat`	Repeats while the flag is true
`begin` ... `again`	Repeats for ever (press CTRL-C)
`leave`	Leaves a `do` loop at once

A `do` loop always goes round at least once, and if its start and its end are the same, it goes round 65,536 times: all the way round the numbers, and back to where it began. `?do` is `do` with a check: it skips the loop when they're the same. Use `?do` whenever the count might be 0.

> **TRY THIS:** `+loop` can count down: `: threes 0 30 do i . -3 +loop cr ;` prints 30 27 24 ... 0. (Counting down, the loop *does* reach its end.) Write `countdown-by` (*n* --), which counts down from 100 in steps of *n*.

19.10 Variables and constants

A **variable** is a named place in memory that holds a number. `variable score` makes one. Saying its name puts its **address** (where it is) on the stack; `!` ("store") puts a number there, and `@` ("fetch") gets it back:

```
/> variable score
/> 10 score !
/> score @ .
10
/> 5 score +!
/> score ?
15
```

`+!` adds to a variable, and `?` prints it (it's the same as `@ .`).

A **constant** has a fixed value, which its name puts on the stack:

```
/> 42 constant answer
/> answer .
42
```

A **value** is between the two: its name gives its value, like a constant, but `to` changes it:

```
/> 0 value lives
/> 3 to lives
/> lives .
3
```

19.11 Characters and strings

Every character has a number, its **code** (Appendix E has them all). `char` gives a character's code, and `emit` prints the character for a code:

```
/> char A .
65
/> 65 emit cr
A
```

In a definition, `[char] A` does the same as `char A`:

```
/> : stars ( n -- ) 0 do [char] * emit loop cr ;
/> 5 stars
*****
```

A **string** is a row of characters in memory. Forth keeps a string as two numbers on the stack: its address, and its length (written *c-addr u* in stack diagrams). `s"` makes one, and `type` prints one:

```
/> s" hello" type cr
hello
/> : greet ( c-addr u -- ) ." Hello, " type ." !" cr ;
/> s" Sam" greet
Hello, Sam!
```

19.12 Memory

A variable is one cell of memory. For more, `create` makes a name for the place where the next things go, and `,` ("comma") puts a number there, in the next cell:

```
/> create primes 2 , 3 , 5 , 7 , 11 ,
/> primes 3 cells + @ .
7
```

`primes` gives the address of the first number; `3 cells +` moves three cells on (a cell is two bytes); `@` fetches the fourth prime. Counting starts at 0, so that's item number 3.

WORD	STACK	WHAT IT DOES
`@`, `!`	(*addr* -- *x*), (*x addr* --)	Fetch, store a cell
`c@`, `c!`	(*addr* -- *char*), (*char addr* --)	Fetch, store a byte
`+!`	(*n addr* --)	Add to a cell
`,`, `c,`	(*x* --), (*char* --)	Put a cell, a byte, in the next place
`allot`	(*n* --)	Leave room for *n* bytes
`cells`	(*n* -- *n×2*)	How many bytes *n* cells take
`here`	(-- *addr*)	The next free place
`fill`, `erase`	(*addr u char* --), (*addr u* --)	Fill memory with a byte, or zeros
`move`	(*from to u* --)	Copy *u* bytes
`dump`	(*addr u* --)	Show memory, in hex

19.13 Numbers in other bases

HyForth reads and prints numbers in the base in the variable `base`, usually 10. `hex` sets it to 16, and `decimal` back to 10:

```
/> 255 hex . decimal
FF
```

A number can also say its own base: `$` for hex, `%` for binary, `#` for decimal, and a character in single quotes is its code:

```
/> $FF . %1010 . #99 . 'A' .
255 10 99 65
```

(With the numbers library, section 20.8, HyForth reads all of Chapter 18's numbers too.)

19.14 Words that call themselves

A word can use itself, with `recurse`. The **factorial** of a number is it times every number below it: 5 × 4 × 3 × 2 × 1. Or, put another way, the factorial of *n* is *n* times the factorial of *n* - 1:

```
/> : fact ( n -- n! ) dup 1 > if dup 1- recurse * then ;
/> 5 fact .
120
```

19.15 When things go wrong

When HyForth doesn't know a word, it prints the word and a `?`:

```
/> : oops nothere ;
nothere ?
```

Other mistakes give a message, and HyForth empties its stack and carries on with the next line:

```
/> 1 0 / .
division by zero
/> drop
stack underflow
```

Stack underflow means a word wanted more numbers than the stack had. Nothing is harmed.

Here are the mistakes everyone makes when they start, and what to do about each:

WHAT YOU SEE	WHAT HAPPENED	WHAT TO DO
`3+ ?`	No space: `3+` was read as one word	Put a space between every two words
`rc: sqare: not found`	The line's first word wasn't Forth's, so the whole line went to rc as a command (section 8.1)	Check the first word's spelling
A prompt of spaces	You're inside a definition: there was a `:` with no `;`	Type `;` (then define the word again, properly)
`if: compile only`	`if`, a loop or `."` typed outside a definition	Put it in a definition
`stack underflow`	A word needed more numbers than there were	Count what each word takes: write its stack diagram
A wrong answer, with no message	Numbers left on the stack by an earlier line got used	Type `.s` to look, and `drop` what shouldn't be there
`-32768` after `32767 1 +`	The number went past the end of a 16-bit cell	Use the big numbers (section 20.8)
A loop that prints for ever	`do` with equal start and end, or a `begin` loop whose flag never comes true	Press CTRL-C; use `?do`

A program can catch errors itself. `catch` runs a word (given by its **execution token**, which `'` or `[']` gives), and leaves 0 if it worked, or the error's number if it **threw** one:

```
/> : safe-div ( a b -- ) ['] / catch if 2drop ." can't divide" cr else . cr then ;
/> 10 2 safe-div
5
/> 10 0 safe-div
can't divide
```

`throw` (*n* --) throws error *n* (unless it's 0). The standard's errors are -1 to -255, and HyForth's system errors -512 and below, so give your own errors positive numbers.

19.16 Keeping your words

Words you type at the prompt are gone when HyForth ends. To keep them, write them in a file (with `edit`, Chapter 11), and **include** it:

```
/> echo ': twice ( n -- 2n ) 2 * ;' >/ram/my.fs
/> echo '21 twice . cr' >>/ram/my.fs
/> include /ram/my.fs
42
/> 4 twice .
8
```

`include` reads the file as if you'd typed it. `require` does the same, but only once: a file that's been included already isn't read again (one that couldn't be opened doesn't count, so you can fix the problem and `require` it again).

A file of Forth can also be run as a program, a **script**, with `forth file.fs`, followed by any arguments. Its first line can be `#!/bin/forth`, and then it runs by its name, like any program. In a script, `argc` and `arg` give its arguments (section 20.9), and `bye` (or reaching the end) ends it.

Put your Forth files on an SD card, so they're there next time.

19.17 Whole programs

Here are three small programs that use what this chapter has taught. Type them in, run them, and then change them: that's how a language is learned.

A times table

`.r` prints a number at the right-hand end of a column: `4 .r` uses four characters, so the columns line up. `table` prints one row, and `tables` uses it for nine:

```
/> : table ( n -- ) 11 1 do dup i * 4 .r loop drop cr ;
/> 7 table
   7  14  21  28  35  42  49  56  63  70
/> : tables ( -- ) 10 1 do i table loop ;
/> tables
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
   6  12  18  24  30  36  42  48  54  60
   7  14  21  28  35  42  49  56  63  70
   8  16  24  32  40  48  56  64  72  80
   9  18  27  36  45  54  63  72  81  90
```

Look at how `table` treats its stack. Its diagram, (*n* --), promises to take one number and leave none. Inside the loop, `dup i *` makes a copy of *n* to multiply, so *n* itself is still there for the next time round. After the loop, *n* is no longer wanted, and `drop` keeps the promise. A word that leaves the stack different from what its diagram says is the commonest bug in Forth.

Dice

The `random` library has `random` (*n* -- *r*), which gives a number from 0 to *n* – 1, chosen by chance. A die has six faces, numbered from 1:

```
/> lib random
/> : die ( -- n ) 6 random 1+ ;
/> : roll ( -- ) die die 2dup . . + ." makes " . cr ;
/> roll
6 2 makes 8
/> roll
3 6 makes 9
```

(Yours will fall differently: the library shakes its dice with the clock as it loads.) `roll` throws two dice, copies them both with `2dup`, prints the copies, and then adds the two that are left.

FizzBuzz, in a file

In the game of FizzBuzz, you count aloud, but say "Fizz" instead of every number that 3 goes into, "Buzz" for those that 5 goes into, and "FizzBuzz" for both. This program is long enough to belong in a file. Go to `/ram`, start `edit fizz.fs`, and type:

```
\ fizz.fs - the game of FizzBuzz: count, but say Fizz for the threes,
\ Buzz for the fives, and FizzBuzz for both

: fizz? ( n -- flag ) 3 mod 0= ;
: buzz? ( n -- flag ) 5 mod 0= ;

: say ( n -- )
  dup fizz? over buzz? and if drop ." FizzBuzz " exit then
  dup fizz? if drop ." Fizz " exit then
  dup buzz? if drop ." Buzz " exit then
  . ;

: fizzbuzz ( n -- ) 1+ 1 do i say loop cr ;
```

Save it, leave the editor, and `include` it:

```
/ram> include fizz.fs
/ram> 15 fizzbuzz
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
/ram> 6 say
Fizz
```

The program is built the Forth way, from the bottom up:

- `fizz?` and `buzz?` are tests: each takes a number, and leaves a flag. (A `?` at the end of a name is the custom for words that ask a question.)
- `say` says one number. It tries the three special cases in turn; in each, `drop` throws the number away, the word is printed, and `exit` leaves `say` at once. Only if none of them fits does it reach the `.` at the end.
- `fizzbuzz` is the loop. `1+ 1 do` counts from 1 up to the number it was given.

Because each word does one small job, each can be tried alone, as `6 say` was. When a Forth program doesn't work, test its smallest words first.

> **TRY THIS:** Change `fizz.fs` so that numbers 7 goes into say "Whizz". (You'll need a `whizz?`, and one more line in `say`. What should 21 say? And 35?) After each change, `include fizz.fs` again.

Chapter 20 shows how to use HyForth to work the Hydra's devices, and Appendix A lists every word.

CHAPTER 20

Programming the Hydra in HyForth

Chapter 19 taught the Forth language. This chapter shows how to use HyForth to work the Hydra: its files, the terminal, the devices, sound, the screen and the big numbers. Most of it comes from **libraries**: sets of words that HyForth loads when you ask for them.

IN THIS CHAPTER: You'll use HyForth's libraries to work the Hydra's files, terminal, devices, sound and screen, and its big numbers. **You need:** Chapter 19. Each section leans on the chapter about its device (Chapters 13 to 17), which you can read alongside.

20.1 Libraries

HyForth's core is small: just the standard's Core words. Everything else is in libraries on the ROM disk, in `/lib/forth`. As it starts, HyForth loads four of them (from `/lib/forth/startup.fs`): the Core Extension, Exception, File Access and Programming-Tools word sets. The shell adds a fifth, `shell`. `libs` lists the libraries loaded:

```
/> libs
forth coreext exception file tools shell
```

`lib` *name* loads another (or, if it's loaded already, does nothing):

```
/> lib hydra
/> libs
forth coreext exception file tools shell hydra
```

LIBRARY	WHAT IT HAS
`coreext`	The Core Extension word set: `nip`, `tuck`, `?do`, `case`, `value`, `defer`, `:noname`, `pick` ...
`exception`	`catch` and `throw`
`file`	The File Access word set: files (section 20.3)
`tools`	The Programming-Tools word set: `.s`, `?`, `dump`, `see`, `words`, `[if]` ..., and `lib`, `libs`, `-lib`
`facility`	The terminal: the cursor, colours, keys, the time; structures (section 20.4)
`string`	Strings: `compare`, `search`, `/string`, `-trailing`, `substitute` ...
`search`	Word lists and the search order
`double`	Double-cell (32-bit) numbers: `d+`, `d.`, `2constant` ...
`memory`	`allocate`, `free`, `resize`
`locals`	Local variables: `{: a b -- c :}`
`block`	Forth's blocks (section 20.10)

LIBRARY	WHAT IT HAS
`hydra`	The Hydra's own words: system calls, commands, directories, banks, notes (section 20.9)
`numbers`	The Hydra's numbers: any size, fractions, decimals, complex (section 20.8)
`sound`	Sound (section 20.6)
`bits`	`tbit`, `sbit`, `cbit`: testing, setting and clearing bits
`random`	Random numbers: `rand`, `random`, `rseed`
`disasm`	The disassembler: `disasm` (Chapter 28)
`gpio`, `i2c`, `spi`, `clock`, `disk`, `cons`, `proc`, `pc`, `video`	The devices, in Forth (sections 20.5 to 20.7)

`-lib` *name* hides a library's words again (its code stays, for words that use it), and `require` *file* loads any file of Forth just once. To have a library loaded every time HyForth starts, put your own `startup.fs` (a copy of the ROM disk's, with more `require` lines) in `/sd/0/lib/forth`.

20.2 Running commands

The `hydra` library has words that run commands, and other programs, from Forth:

WORD	STACK	WHAT IT DOES
`sh`	(*c-addr u -- status*)	Runs a command line with rc, and waits: its status (0 for success)
`run`	(*c-addr u -- status*)	Runs a program and its arguments (without rc), and waits

And the shell library has more (section 8.15):

WORD	STACK	WHAT IT DOES
`sh-out`	(*c-addr u -- c-addr2 u2*)	Runs a command line, and gives its output as a string
`spawn`	(*c-addr u -- task*)	Starts a command line, without waiting: its task
`wait`	(*task* --)	Waits for it to end
`piped`	(*xt c-addr u* --)	What a word prints, into a command line's input

```
/> s" ls /nothing" sh .
ls: /nothing: not found
1
/> s" /bin/echo hi there" run drop
hi there
```

20.3 Files

The File Access words work on any file: on a disk, a device, or `/pc`. A file is opened with a **file access method**, `r/o` (read only), `w/o` (write only) or `r/w` (both), and gives a **file id** for the other words to use. Each word that can fail leaves an **ior**: 0 for success, or an error number, which `throw` turns into an error.

WORD	STACK	WHAT IT DOES
`open-file`	(*c-addr u fam -- fileid ior*)	Open a file
`create-file`	(*c-addr u fam -- fileid ior*)	Make a new file (or empty an old one), and open it

WORD	STACK	WHAT IT DOES
`close-file`	(*fileid -- ior*)	Close it
`read-file`	(*c-addr u1 fileid -- u2 ior*)	Read up to *u1* bytes: *u2* is how many came (0 at the end)
`read-line`	(*c-addr u1 fileid -- u2 flag ior*)	Read a line: *flag* is false at the end of the file
`write-file`	(*c-addr u fileid -- ior*)	Write bytes
`write-line`	(*c-addr u fileid -- ior*)	Write a line, and a new line
`file-size`	(*fileid -- ud ior*)	Its length, a double number
`file-position`, `reposition-file`	(*fileid -- ud ior*), (*ud fileid -- ior*)	Where reading or writing happens next
`delete-file`	(*c-addr u -- ior*)	Remove a file
`rename-file`	(*c-addr1 u1 c-addr2 u2 -- ior*)	Rename one
`bin`	(*fam -- fam*)	Read and write bytes, as they are

Writing a file:

```
/> s" /ram/out.txt" w/o create-file throw constant g
/> s" first line" g write-line throw
/> s" second line" g write-line throw
/> g close-file throw
/> cat /ram/out.txt
first line
second line
```

And reading one, a line at a time. This definition is long, so it's typed over several lines: HyForth's second prompt, a tab, shows that the definition isn't finished.

```
/> create buf 80 allot
/> : show-file ( c-addr u -- )
        r/o open-file throw >r
        begin buf 80 r@ read-line throw while
        buf swap type cr
        repeat drop r> close-file throw ;
/> s" /rom/lib/shell" show-file
/bin/forth -l
```

The `hydra` library adds words for directories: `open-dir` (*c-addr u -- dirid ior*), `read-dir` (*c-addr u1 dirid -- u2 flag ior*), which reads a directory's next name, and `close-dir` (*dirid -- ior*); and `=mkdir`, `get-dir` and `set-dir`.

20.4 The terminal

The `facility` library has the words for the terminal's screen and keys (they send the escape sequences of section 13.7 for you):

WORD	STACK	WHAT IT DOES
`page`	(--)	Clear the screen
`at-xy`	(*col row* --)	Move the cursor (from 0, 0 at the top left)
`color`, `bgcolor`	(*c* --)	The text's colour, the background's (0 to 15)
`black` … `white`	(-- *c*)	The colours: `black red green yellow blue magenta cyan white`
`bright`	(*c* -- *c'*)	A colour's bright version

WORD	STACK	WHAT IT DOES
`bold`, `dim`, `underline`, `blink`, `reverse`, `plain`	(--)	Styles; `plain` turns them all off
`cursor-up` … `cursor-left`	(*n* --)	Move the cursor *n* places
`clear-line`, `clear-below`	(--)	Clear the rest of the line, or of the screen
`cursor-on`, `cursor-off`	(--)	Show or hide the cursor
`form`	(-- *rows cols*)	The window's size
`key?`	(-- *flag*)	Has a key been pressed?
`ekey`	(-- *u*)	Wait for a key, special keys too: compare with `k-up`, `k-f1` …
`ms`	(*u* --)	Wait *u* milliseconds
`time&date`	(-- *sec min hour day month year*)	The date and time
`beep`	(--)	Ring the bell

```
lib facility
: at ( col row -- ) at-xy ;
: banner ( -- )
  yellow color blue bgcolor bold
  10 4 at ."  +--------------------+ "
  10 5 at ."  |   Hydra-16 Forth   | "
  10 6 at ."  +--------------------+ "
  plain ;
: bars ( -- ) 8 1 do i bgcolor 12 i 8 + at-xy 20 spaces loop plain ;
page banner bars 0 17 at-xy
```

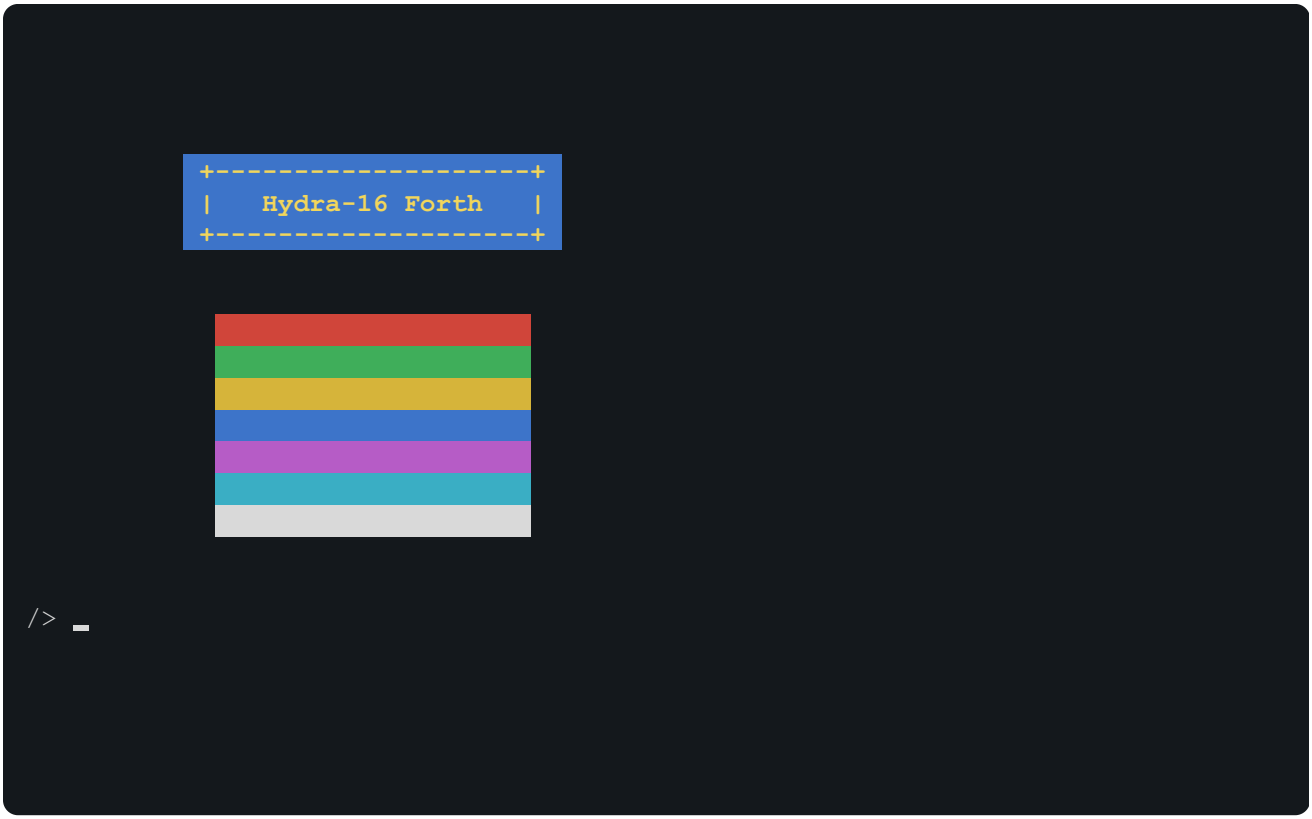


Figure 20-1. The `facility` library's colours and cursor moves.

20.5 The devices

The device libraries work the devices of Part Three through their files, so you don't need to remember the files' names or commands.

The pins: `lib gpio` (section 17.2).

WORD	STACK	WHAT IT DOES
`gpio`	(*pin* -- *level*)	A pin's level, 0 or 1
`gpio!`	(*pin level* --)	Make a pin an output, and set it
`gpio-in`, `gpio-out`	(*pin* --)	Make a pin an input, or an output
`gpio-port`	(-- *byte*)	All eight pins at once
`gpio-port!`	(*byte* --)	Set all the outputs at once
`gpio-ddr!`	(*byte* --)	Set all eight directions (bit *n* 1: pin *n* an output)
`gpio-ca1!`	(*rise?* --)	CA1 counts rises (true) or falls (false)
`gpio-ca2!`	(*n* --)	CA2: 0 low, 1 high, -1 an input
`gpio-wait`	(-- *count*)	Wait for CA1's next change: the changes so far
`gpio-state`	(-- *c-addr u*)	The `ctl` file's text

With an LED on pin 2 (section 17.1), this flashes it ten times:

```
lib gpio  lib facility
: flash ( -- ) 10 0 do 2 0 gpio! 250 ms 2 1 gpio! 250 ms loop ;
flash
```

(0 lights the LED, if it's wired from +5 V to the pin.)

The I²C bus: `lib i2c` (section 17.3): `i2c-read` (*addr reg c-addr u* --) and `i2c-write` (*addr reg c-addr u* --) read and write a chip's registers; `i2c-reg-size` (*n* --) sets how many bytes a register's number has (0, 1 or 2); `i2c-speed` (*khz* --); `i2c?` (*addr* -- *flag*) asks whether a chip is there; and `i2c-devices` lists the chips that answer.

SPI: `lib spi` (section 17.4): `spi` (*dev c-addr u* --) sends the bytes to SPI device *dev* and puts what it sent back in their place; `spi-read` (*dev c-addr u* --) reads bytes; `spi-mode` (*dev mode* --) sets mode 0 or 3.

The clock: `lib clock`: `set-date` (*c-addr u* --) sets the date and time, given as text like `2026-10-12 09:41:00`, and `rtc` (-- *c-addr u*) says whether there's a clock chip. (`time&date`, in `facility`, reads it.)

The disks: `lib disk`: `disk-ctl` (*disk* -- *c-addr u*) is a disk's control file's text, `disk-cmd` (*c-addr u disk* --) sends one a command, `disk-start` and `disk-stop` (*disk* --) start a card again or stop a RAM disk, and `cards` (-- *mask*) says which SD cards are there (bit *n* for SPI device *n*). A disk is named by its character: `char 0 disk-ctl type`.

The windows: `lib cons`: `window` (-- *n*) is this window's number; `windows` (-- *c-addr u*) lists them; `new-window` and `new-group` (*c-addr u* --) open a window running a command line; `show-window` (*n* --) shows one; and `window-label`, `window-status` and `window-ctl` (*c-addr u* --) set its label, its status line, or send its `wctl` a command.

The tasks: `lib proc`: `task-args`, `task-cwd`, `task-env`, `task-ns` and `task-regs` (*task* -- *c-addr u*) read a task's `/proc` files; `task-mem` (*task addr c-addr u* --) reads its memory, and `task-ram` (*task bank offset c-addr u* --) its RAM banks.

The PC: `lib pc`: `pc?` (-- *flag*) is true if the PC tool is there to serve `/pc`.

20.6 Sound

`lib sound` has words for each of the sound driver's commands (section 15.3), and for playing tunes:

WORD	STACK	WHAT IT DOES
`snd-patch`	(*ch p* --)	Give channel *ch* patch *p*
`snd-note`	(*ch n* --)	Play note *n* (60: middle C)
`snd-off`	(*ch* --)	Let it go
`snd-level`	(*ch v* --)	The channel's volume, 0 to 127
`snd-pan`	(*ch pan* --)	Its speakers: 1 left, 2 right, 3 both
`snd-bend`, `snd-glide`	(*ch n* --)	Bend its pitch; glide to note *n*
`snd-drum`	(*ch n* --)	Play General MIDI drum *n*
`snd-freq`	(*ch hz* --)	Play a frequency (0: stop)
`snd-wave`	(*ch w width* --)	A PSG channel's wave
`snd-sens`, `snd-lfo`, `snd-noise`, `snd-reg`		The LFO, the noise, a register (as `sndctl`'s commands)
`snd-volume`	(*v* --)	The master volume, 0 to 200
`snd-claim`, `snd-release`	(*mask* --)	Claim or release channels 0 to 7 (bit *n*: channel *n*); `snd-claim-psg` and `snd-release-psg` for 8 to 23

WORD	STACK	WHAT IT DOES
`snd-reset`	(--)	Silence everything
`note-of`	(*c-addr u* -- *n*)	A note's number from its name: `s" A4" note-of` is 69
`tune`	(*c-addr u ch tempo* --)	Play a tune: note names and their beats
`snd-mml`	(*ch c-addr u* -- *status*)	Play a line of MML (section 15.6)
`snd-chord`	(*ch c-addr u* -- *status*)	Play a chord
`snd-play`	(*c-addr u times* -- *status*)	Play a song file *times* times (0: until CTRL-C)

```
/> lib sound
/> s" A4" note-of .
69
/> 0 0 snd-patch
/> s" C4 1 D4 1 E4 1 C4 1" 0 120 tune
```

A tune is note names and beats: `C4 1` is middle C for one beat, and a – is a rest. `0 120` plays it on channel 0 at 120 beats a minute.

20.7 The screen

With a Vera X, `lib video` draws on the screen (section 16.4):

WORD	STACK	WHAT IT DOES
`bitmap`	(*width depth* --)	A bitmap under the text: 320 or 640 across, 1, 2, 4 or 8 bits a dot
`bitmap-off`	(--)	No bitmap
`pen`	(*colour* --)	The colour to draw in
`plot`	(*x y* --)	A dot
`line`, `box`, `bar`	(*x0 y0 x1 y1* --)	A line, a rectangle, a filled rectangle
`circle`, `disc`	(*x y r* --)	A circle, a filled circle
`clear`	(--)	Clear the bitmap
`text`	(*x y c-addr u* --)	Write text on it
`border`	(*colour* --)	The border's colour

It has **turtle graphics** too, as the Logo language has: an imaginary turtle, carrying a pen, that walks around the screen. `cs` clears the screen and puts the turtle in the middle, facing up; `fd` (*n* --) walks it forward *n* dots, drawing a line; `bk` (*n* --) walks backward; `rt` and `lt` (*degrees* --) turn it right and left; `pu` and `pd` lift its pen up and put it down; and `heading` (-- *degrees*) and `seth` (*degrees* --) read and set the way it faces.

```
lib video
320 8 bitmap
: square ( n -- ) 4 0 do dup fd 90 rt loop drop ;
: flower 36 0 do i 16 mod 1+ pen 60 square 10 rt loop ;
cs flower
```

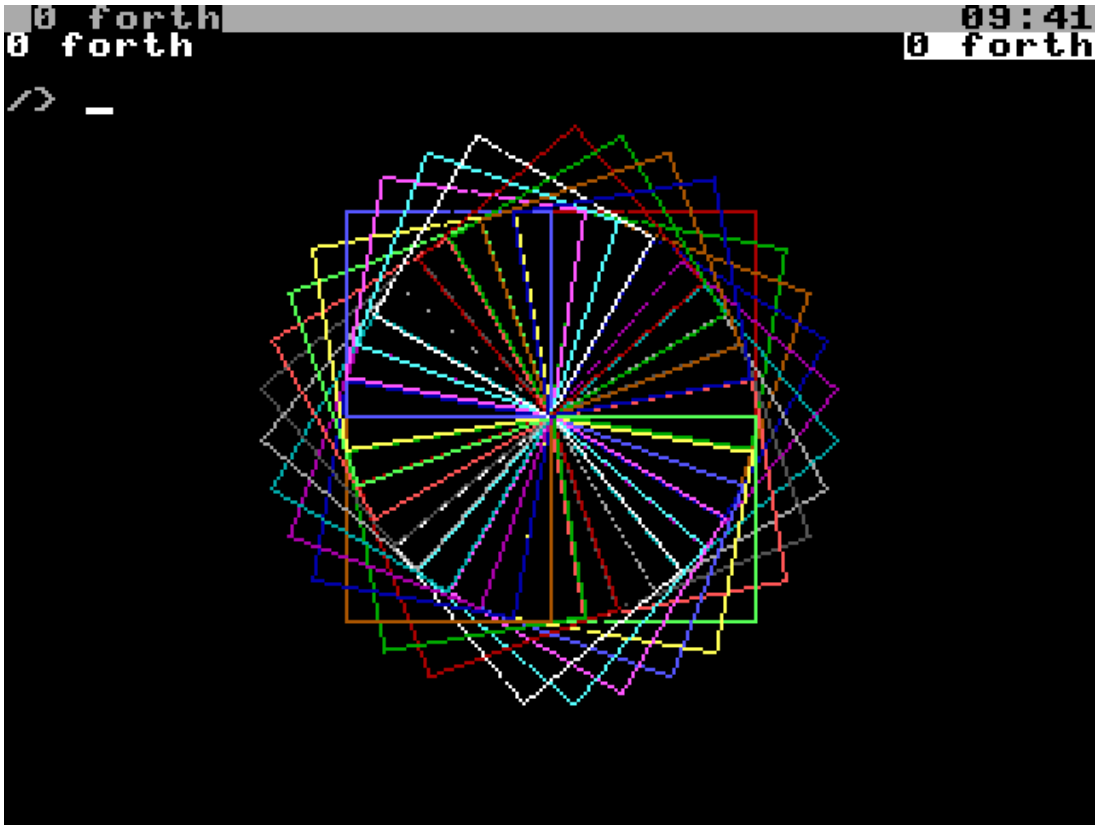


Figure 20-2. Turtle graphics: thirty-six squares make a flower.

For programs that need more, `lib video` reaches the VERA itself: `vpoke` (*addr bank c* --) and `vpeek` (*addr bank* -- *c*) read and write a byte of video RAM (*bank* is the address's 17th bit), `vram!` and `vram@` (*addr bank c-addr u* --) move several, `palette!` (*index rgb* --) sets a colour (as $RGB, 4 bits each), `sprite!` (*n c-addr* --), `sprite-at` (*n x y* --) and `sprite-off` (*n* --) work the sprites, `vsync` (--) waits for the next frame, and `mouse` and `mouse-wait` (-- *x y buttons*) read the mouse.

20.8 Big numbers

Forth's own numbers are 16-bit cells. `lib numbers` brings in all of Chapter 18's numbers: integers of any size, fixed decimals, fractions and complex numbers. They live on a stack of their own, the **number stack**, and the words that work on them have an `n` in front of Forth's name: `n+`, `n*`, `n.`, `ndup`. In the stack diagrams, `N:` shows the number stack.

With the library loaded, any number Forth can't fit in a cell goes on the number stack: `2/3`, `1.25`, `#xFF`, `2i`, `10000000000000000000`. `s>n` moves a cell from Forth's stack to the number stack, and `n>s` moves one back.

```
/> lib numbers
/> 2/3 n.
2/3
/> 1/3 1/6 n+ n.
1/2
/> 100 s>n 10 s>n npow n.
10000000000000000000
/> 7 s>n 2 s>n n/ n.s
<1> 7/2
```

> **NOTE:** Load the library before you type one of its numbers. Without it, the shell takes a line that starts with `2/3` for a command, not for Forth, and hands it to rc.

The math functions, `nsqrt`, `nexp`, `nlog`, `nsin`, `ncos`, `ntan`, `natan` and `npi`, work to the precision in the variable `digits`, 12 unless you change it:

```
/> lib numbers
/> 50 digits ! 2 s>n nsqrt n.
1.41421356237309504880168872420969807856967187537694
```

`set-base` (*c-addr u* --) sets the base for all numbers, with any base string from Chapter 18: `s" b" set-base` works in binary. Appendix A lists all the library's words.

20.9 The Hydra library

`lib hydra` has HyForth's words for the Hydra's system.

System calls. For each of HydraOS's **system calls** (Appendix K), there's a word named `sys-` and the call's name: `sys-getpid`, `sys-ticks`, `sys-open` and so on. Each takes the call's inputs from the stack, in the order Appendix K lists them (the first deepest), and leaves its outputs; a call that can fail leaves an ior on top too. A file name is given as a zero-terminated string: `>z` (*c-addr u* -- *z-addr*) makes one.

```
/> lib hydra
/> sys-getpid .
1
```

(This shell is task 1.)

Arguments. In a script (section 19.16), `argc` (-- *n*) is how many arguments it was given, counting its own name, and `arg` (*n* -- *c-addr u*) is one of them: `0 arg` is the script's name.

Banks. `bank!` (*bank* --) puts one of the task's RAM banks at `bank-window` (`$8000`), and `bank@` (-- *bank*) says which is there; `seg-bank!` (*seg n* -- *ior*) puts a shared segment's bank there. HyForth keeps compiled words in RAM banks of its own, so a word that changes the bank must be compiled with `false code-banks` first.

Notes. `note` (*task n* --) sends a task a note, `note-group` (*group n* --) sends one to every task in a note group, and `on-note` (*xt* --) gives a word to handle the notes that come to HyForth.

Machine code. `sys` (*addr a x y* -- *a x y p*) calls machine code at *addr*, with the processor's A, X and Y registers set, and gives back the registers, and its flags, after it (Chapter 27).

Odds and ends. `ctl` (*c-addr1 u1 c-addr2 u2* --) writes the text *c-addr2 u2* to the file *c-addr1 u1*, as `echo` does (`s" /dev/sd/0/ctl" s" sync" ctl`); `pause` (--) lets the other tasks have a turn; and `ior>text` (*ior* -- *c-addr u*) gives an error's message.

20.10 Blocks

Forth's oldest way of keeping programs is the **block**: a numbered 1K piece of a disk, shown as 16 lines of 64 characters. `lib block` brings it back: `s" /sd/0/my.blk" open-blocks` chooses a file of blocks, `list` (*u* --) shows block *u*, `load` (*u* --) runs it, `thru` (*u1 u2* --) runs several, and `block` (*u* -- *addr*), `update`, `save-buffers` and `flush` work on them as memory. Files are easier to edit, but blocks are a part of Forth's history, and many old Forth programs come as blocks.

CHAPTER 21

Learning hylang

hylang is the Hydra's version of **danlang**, a language in the family of **Lisp**, one of the oldest programming languages there is (it was invented in 1958). In Lisp, a program is made of lists, and the language can work on its own programs as easily as on any other list. hylang has the Hydra's whole number system, strings, lists and hashes, and functions for the Hydra's files, devices and programs. This chapter teaches you the language.

IN THIS CHAPTER: You'll learn hylang from the beginning: expressions in parentheses, values, names, functions, lists, decisions, loops, strings and hashes. **You need:** Chapter 6. You don't need to have programmed before.

21.1 Starting hylang

Type `hylang`. It says hello, and shows its prompt, `hylang>`. Type an **expression**, and hylang works out its value, and shows it after `=>`:

```
/> hylang
hylang (danlang on the Hydra-16)
Type 'exit' to Exit

hylang> (+ 1 2)
=> 3
```

`exit`, or `(exit)`, leaves hylang. (`(exit 3)` leaves it with the status 3, for a shell to see.)

21.2 Expressions

In hylang, a **call** is written in parentheses: the function first, then its **arguments**, separated by spaces. So `(+ 1 2)` calls the function `+` with the arguments 1 and 2. Calls can be **nested**: each argument can be a call of its own, which is worked out first:

```
hylang> (* 2 (+ 3 4))
=> 14
hylang> (/ 10 4)
=> 5/2
```

Because the function always comes first, there's never any doubt about the order things are done in, and `+` can take as many arguments as you like: `(+ 1 2 3 4)` is 10. hylang has all of Chapter 18's numbers, so `(/ 10 4)` is exactly 5/2.

To turn a sum from school into hylang, find the last thing you'd do, and write that first. In (3 + 4) × (5 – 2), the last thing is the multiplying, so the `*` goes on the outside, with the two parts as its arguments:

```
hylang> (* (+ 3 4) (- 5 2))
=> 21
```

Count the parentheses: every `(` needs its `)`. If you press RETURN before they're all closed, hylang doesn't complain. It waits for the rest, with a prompt of spaces, until you type the missing `)`.

> **TRY THIS:** Work these out in your head, then ask hylang: `(- 10 (* 2 3))` and `(* (- 10 2) 3)`. Then write (8 – 2) × (1 + 1) in hylang. (It should say 12.) What does `(/ 1 3)` give? And `(+ (/ 1 3) (/ 1 6))`?

21.3 Values

Every expression has a **value**. The kinds of value are:

KIND	LOOKS LIKE	TYPE-OF SAYS
Numbers	`42`, `-3`, `2/3`, `3.14`, `1+2i`, `#xFF`	`:number`
Strings	`"Hello"`	`:string`
Characters	`\a`, `\space`, `\newline`	`:char`
Atoms	`:name`, `:red`	`:atom`
Symbols	`x`, `square`, `+`	`:symbol`
Lists	`{1 2 3}`	`:list`
Hashes	`#({{:name "Sam"}})`	`:hash`
Functions	`square`, `(fn {x} {* x x})`	`:function`
True	`T`	`:t`
False, and nothing	`NIL`	`:list` (it's the empty list)
Errors	`(error "oops")`	`:error`

A **string** is text in double quotes. Inside it, `\n` is a new line, `\t` a tab, `\"` a double quote and `\\` a backslash. A **here string**, in three double quotes, `"""` ... `"""`, can hold anything, quotes and new lines too.

An **atom** is a name that stands for itself: `:red` is just `:red`. Atoms are used for names of things, like the keys of a hash.

`T` means true and `NIL` means false: every test gives one or the other, and anything that isn't `NIL` counts as true. `NIL` also means "nothing", or an empty list. hylang always prints these two in capitals, so that they stand out; you can type them either way.

```
hylang> (> 5 3)
=> T
hylang> (> 3 5)
=> NIL
```

21.4 Names

`def` gives a value a **name**. The name is written in braces (section 21.6 says why):

```
hylang> (def {x} 10)
=> NIL
hylang> x
=> 10
hylang> (* x x)
=> 100
```

`def` makes **global** names, which can be used anywhere. A name is in small letters (hylang turns capitals into small letters as it reads them), and it can have digits, and signs like - and `?`, in it: `high-score`, `even?`.

`def` is used so often that there's a shorter way to write it: `:(y 20)` means exactly the same as `(def {y} 20)`. The `:` right against the parenthesis is a **prefix**. There are a few more prefixes, for other things that are used a lot. You'll meet them as you go, and section 21.15 lists them all.

```
hylang> :(y 20)
=> NIL
hylang> (+ x y)
=> 30
```

A name that hasn't been given a value is an error. This is the message you'll see most often while you're learning, usually because of a slip of the finger:

```
hylang> (+ x z)
=> Error: Unbound Symbol 'z'
```

21.5 Functions

`fun` makes a new function. Its first argument is a list of the function's name and its **formals** (the names its arguments will have); the second is its **body**: what it does.

```
hylang> (fun {square n} {* n n})
=> NIL
hylang> (square 7)
=> 49
```

A function's value is the value of its body. Functions can use other functions:

```
hylang> (fun {greet name} {print (+ "Hello, " name "!")})
=> NIL
hylang> (greet "Sam")
Hello, Sam!
=> NIL
```

`print` prints its arguments, with spaces between them, and starts a new line; its value is `NIL`. `+` joins strings, if its first argument is a string.

A function doesn't need a name. `fn` makes one without, which can be given a name later, or used straight away:

```
hylang> (def {cube} (fn {n} {* n (square n)}))
=> NIL
hylang> (cube 3)
=> 27
hylang> (@({n} {* n 2}) 21)
=> 42
```

Any number of arguments. In a function, `&_` is a list of any arguments it was given beyond its formals, so a function can take as many as you like:

```
hylang> (fun {add-all} {sum &_})
=> NIL
hylang> (add-all 1 2 3)
=> 6
```

Too many, and too few. Give a function more arguments than it has formals, and hylang says so. Give it fewer, and something more interesting happens: you get back a *new function*, which is waiting for the rest.

```
hylang> (square 1 2)
=> Error: The function takes 1 argument, not 2
hylang> (square)
=> <function>(fn {n} {* n n})
```

So if an answer comes back as `<function>` when you expected a number, count your arguments.

> **TRY THIS:** Write `(fun {double n} ...)`, and then `quadruple`, which uses `double` twice. Then write `average`, which takes two numbers and gives the number half way between them. `(average 3 4)` should be 7/2.

21.6 Lists, and code as lists

A list in braces, `{1 2 3}`, is a **Q-expression**: a list that's left as it is, not worked out. A list in parentheses is an **S-expression**: a call, which is worked out. That's why names and bodies are written in braces: `{* n n}` is kept as it is, to be run later, when the function is called.

```
hylang> {1 2 3}
=> {1 2 3}
hylang> (head {1 2 3})
=> {1}
hylang> (tail {1 2 3})
=> {2 3}
```

`list`, or square brackets, makes a list of values that *are* worked out:

```
hylang> [1 (+ 1 1) 3]
=> {1 2 3}
```

And `eval` runs a Q-expression as code:

```
hylang> (eval {+ 1 2})
=> 3
```

So a hylang program can build code as a list, and then run it. Lisp programmers say "code is data".

FUNCTION	WHAT IT GIVES
`(len l)`	The number of items
`(fst l)`, `(snd l)`, `(thd l)`, `(last l)`	The first, second, third, last item
`(nth n l)`	Item *n*, counting from 0
`(head l)`, `(tail l)`	A list of the first item; all but the first
`(cons x l)`	*l* with *x* in front
`(join l1 l2 ...)`	The lists joined
`(take n l)`, `(drop n l)`	The first *n* items; all but them
`(reverse l)`	Backwards
`(range n)`, `(range a b [step])`	The numbers 0 to *n* - 1, or *a* up to (not including) *b*
`(sort l)`	In order
`(elem? x l)`	Is *x* in the list?

21.7 Doing something to every item

Lisp's great strength is working on whole lists at once. `map` calls a function for each item, and makes a list of the answers; `filter` keeps the items a test is true for; and `foldl` combines them all, from the left:

```
hylang> (map square {1 2 3 4})
=> {1 4 9 16}
hylang> (filter (fn {n} {> n 2}) {1 2 3 4})
=> {3 4}
hylang> (foldl + 0 {1 2 3 4})
=> 10
hylang> (sum (range 101))
=> 5050
```

That last one adds up the numbers from 0 to 100, which the mathematician Gauss is said to have done in his head when he was a schoolboy.

21.8 Decisions

`if` takes a test, a value for when it's true, and (if you like) one for when it's false:

```
hylang> (if (> x 5) "big" "small")
=> "big"
hylang> ?((> x 50) "big" "small")
=> "small"
```

The comparisons are == (equal), `!=` (not equal), <, >, <= and >=, and `and`, `or` and `not` combine tests. <=> chooses by whether a number is below, equal to, or above zero:

```
hylang> (<=> (cmp 3 7) {< "less"} {= "same"} {> "more"})
=> "less"
```

21.9 Loops

LOOP	WHAT IT DOES
`(dotimes {i n} body ...)`	Runs the body *n* times, with *i* 0, 1 ... *n* - 1
`(each {x list} body ...)`	Runs the body for each item of a list (or each character of a string)
`(while test body ...)`	Runs the body as long as the test is true
`(do x y ...)`	Works out each expression in turn; its value is the last one's

```
hylang> (each {i (range 3)} (print "item" i))
item 0
item 1
item 2
=> NIL
hylang> (dotimes {i 3} (print i))
0
1
2
=> NIL
```

But a Lisp programmer often uses **recursion** instead: a function that calls itself.

```
hylang> (fun {fact n} {if (< n 2) 1 (* n (fact (- n 1)))})
=> NIL
hylang> (fact 20)
=> 2432902008176640000
```

21.10 Names inside functions

A function's formals are its own: they don't change any global name. For more names of its own, a function uses `=(` (that's `set`): a name in the function's own **scope**, which is gone when it ends. `let` makes a scope of its own too:

```
hylang> (let {{a 1} {b 2}} (+ a b))
=> 3
```

Loops make a new scope each time round, so to change a name from inside a loop, use `set!`, which changes the name where it was made:

```
hylang> (fun {counter} {do =(c 0) (dotimes {i 5} (set! {c} (+ c i))) c})
=> NIL
hylang> (counter)
=> 10
```

21.11 Strings and characters

FUNCTION	WHAT IT DOES
`(+ s ...)`	Joins strings (and anything else, as `print` shows it)
`(len s)`	Its length
`(substring s i [n])`	*n* characters from position *i* (counting from 0)
`(char-at s i)`	The character at *i*
`(index-of s sub)`	Where *sub* first appears (or `NIL`)
`(str-upper s)`, `(str-lower s)`, `(str-trim s)`	Capitals, small letters, spaces trimmed off
`(str-split s sep)`, `(str-join l sep)`	A string to a list, and back
`(str-replace s old new)`	Every *old* replaced by *new*
`(format text x ...)`	The text, with each `{}` replaced by the next value
`(to-str x)`, `(val s)`	A value as a string; the number a string holds

```
hylang> (str-split "a,b,c" ",")
=> {"a" "b" "c"}
hylang> (substring "Hydra-16" 0 5)
=> "Hydra"
hylang> (format "{} plus {} is {}" 2 3 (+ 2 3))
=> "2 plus 3 is 5"
```

21.12 Hashes

A **hash** is a set of **entries**, each a **key** and its **value**, like a dictionary: look up a word, and find its meaning. The keys are usually atoms. `#(` makes one, from a list of entries, and the hash can be called like a function, with a key, to look the key up:

```
hylang> (def {h} #({{:name "Sam"} {:age 12}}))
=> NIL
hylang> (h :name)
=> "Sam"
hylang> (hash-keys h)
=> {:name :age}
hylang> (hash-put h {:city "Leeds"})
=> NIL
hylang> h
=> <hash>{{:name "Sam"} {:age 12} {:city "Leeds"}}
```

A value in a hash can be a function: a **method**. Calling the hash with its key calls it, and inside it, `&0` is the hash itself. So hashes can be **objects**, with data and methods together:

```
hylang> (def {dog} #({{:name "Rex"} {:speak (fn {} {print (+ (&0 :name) " says woof")})}}))
=> NIL
hylang> (dog :speak)
Rex says woof
=> NIL
```

21.13 Errors

An error is a value too. When something goes wrong, the error goes back through every call, until something catches it, or it reaches the prompt:

```
hylang> (error "my own error")
=> Error: my own error
hylang> (try (/ 1 0) (+ "oops: " &err))
=> "oops: Division by zero."
```

`try` works out its first argument; if that's an error, it works out its second instead, with `&err` the error's message (and `&code` its code). `(error "message")` makes an error of your own.

Here are the mistakes everyone makes when they start, and what to do about each:

WHAT YOU SEE	WHAT HAPPENED	WHAT TO DO
`Error: Unbound Symbol 'squre'`	A name hylang doesn't know: nearly always a slip of the finger	Check the spelling
A prompt of spaces	A `(`, a `{` or a `"` hasn't been closed yet	Type what's missing
`<function>(fn ...)` where you expected an answer	The function was given too few arguments	Count them
`Error: The function takes 1 argument, not 2`	It was given too many	Count them
`Error: Unbound Symbol 'n'`, as you define a function	The body is in parentheses, so hylang tried to work it out at once	Put the body in braces: `{* n n}`
`Error: '+' adds numbers, or joins values to a string`	`(+ 1 "a")`: a number first, then a string	To join, put a string first: `(+ "" 1 "a")`
`Error: 'head' expects a QExpr`	A list was wanted (a "QExpr" is a Q-expression: a list in braces)	Give it a list
A changed name that changes back	`=(` made a new name inside a loop or a function	Use `set!` (section 21.10)

21.14 Files of hylang

Write longer programs in a file (with `edit`), with a name ending in `.hl`, and run it with `hylang` and the file's name, and any arguments: in the program, `args` is a list of the file's name and its arguments.

```
/> echo '(print "args are" args)' >/ram/a.hl
/> hylang /ram/a.hl one two
args are {"/ram/a.hl" "one" "two"}
```

At the prompt, `(load "file")` runs a file's expressions, keeping its definitions. A file's first line can be `#!/bin/hylang`, to run it by its name. Comments start with `;`.

21.15 Prefixes: writing less

A **prefix** is a character right against an opening parenthesis, which stands for a function that's used a lot. You've met `:(`, `=(`, `?(`, `@(` and `#(`. Here they all are:

PREFIX	MEANS	EXAMPLE
`:(`*name value*`)`	`(def {`*name*`}` *value*`)`	`:(lives 3)`
`=(`*name value*`)`	`(set {`*name*`}` *value*`)`: a name in this function only	`=(total 0)`
`?(`*test then else*`)`	`(if` *test then else*`)`	`?((> x 5) "big" "small")`
`@(`*formals body*`)`	`(fn` *formals body*`)`: a function	`@({n} {* n 2})`
`#(`*entries*`)`	`(hash-create` *entries*`)`: a hash	`#({{:a 1}})`
`.(`*f list*`)`	`(unpack` *f list*`)`: *f* called with the list's items	`.(+ {1 2 3})`
`~(`*text values*`)`	`(format` *text values*`)`	`~("{} and {}" 1 2)`
`[`*a b*`]`	`(list` *a b*`)`	`[1 (+ 1 1) 3]`
`$`*name*	`(env "`*name*`")`: an environment variable	`$path`

```
hylang> .(+ {1 2 3})
=> 6
hylang> ~("{} and {}" 1 2)
=> "1 and 2"
hylang> $path
=> {"." "/bin"}
```

Prefixes are only a shorter spelling. While you're learning, the long names say more clearly what's going on, and you can always use them instead.

21.16 Whole programs

Here are three small programs that use what this chapter has taught. Type them in, run them, and then change them.

A times table

`range` makes the numbers 1 to 10, and `map` multiplies each of them by *n*: a whole row of the table at once, with no loop in sight. Then `each` prints a row for each *n*:

```
hylang> (fun {table n} {map (fn {i} {* n i}) (range 1 11)})
=> NIL
hylang> (table 7)
=> {7 14 21 28 35 42 49 56 63 70}
hylang> (each {n (range 1 4)} (print (table n)))
{1 2 3 4 5 6 7 8 9 10}
{2 4 6 8 10 12 14 16 18 20}
{3 6 9 12 15 18 21 24 27 30}
=> NIL
```

`table` doesn't *print* a row: it *gives* one, as a list, which is more useful. You could add its numbers up with `(sum (table 7))`, or ask for `(nth 3 (table 7))`. In hylang, try to write functions that give values, and leave the printing to whoever calls them.

Dice

`(random n)` gives a whole number from 0 to $n - 1$, chosen by chance:

```
hylang> (fun {die} {+ 1 (random 6)})
=> NIL
hylang> (fun {roll} {do =(a (die)) =(b (die)) (print a b "makes" (+ a b))})
=> NIL
hylang> (roll)
3 4 makes 7
=> NIL
hylang> (roll)
6 6 makes 12
=> NIL
```

(Yours will fall differently.) `roll` has to remember each die, to print it and then to add it, so it gives them the names `a` and `b`, with `=(`. A function's body is one expression; `do` makes one out of several.

FizzBuzz, in a file

In the game of FizzBuzz, you count aloud, but say "Fizz" instead of every number that 3 goes into, "Buzz" for those that 5 goes into, and "FizzBuzz" for both. Go to `/ram`, start `edit fizz.hl`, and type:

```
; fizz.hl - the game of FizzBuzz: count, but say Fizz for the threes,
; Buzz for the fives, and FizzBuzz for both

(fun {fizz n} {cond
  {(== 0 (mod n 15)) "FizzBuzz"}
  {(== 0 (mod n 3)) "Fizz"}
  {(== 0 (mod n 5)) "Buzz"}
  {T n}})

(fun {fizzbuzz n} {do
  (each {i (range 1 (+ n 1))} (write (fizz i) " "))
  (print)})

(fizzbuzz 15)
```

Save it, leave the editor, and run it:

```
/ram> hylang fizz.hl
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

- `cond` chooses between several cases. Each case is a test and a value, in braces; the first test that's true wins. The last case's test is `T`, which is always true: it catches everything the others didn't.

- `mod` gives a remainder. If the remainder of *n* divided by 15 is 0, then both 3 and 5 go into *n*. That test has to come first. (Why?)
- `fizz` gives a string or a number. It doesn't print anything, so it can be tried alone at the prompt: `(load "fizz.hl")`, then `(fizz 9)`.
- `fizzbuzz` does the printing. `write` prints like `print`, but doesn't start a new line; the `(print)` at the end does.

> **TRY THIS:** Change `fizz.hl` so that numbers 7 goes into say "Whizz". What should 21 say? And 35? And 105? Then, at the `hylang>` prompt, try `(map fizz (range 1 16))`: the whole game as a list.

Chapter 22 shows how to use hylang to work the Hydra, and Appendix B lists every function.

CHAPTER 22

Programming the Hydra in hylang

hylang was made for the Hydra: files, programs, the clock, memory and the system are built into the language, and libraries add the devices, sound and the screen. This chapter shows them.

IN THIS CHAPTER: You'll use hylang to work the Hydra's files, programs, clock, memory, devices, sound and screen. **You need:** Chapter 21. Each section leans on the chapter about its device (Chapters 13 to 17), which you can read alongside.

22.1 Libraries

hylang starts with its own library (`globals`) already loaded. The others are on the ROM disk, in `/lib/hylang`: `(use "name")` loads one (once: using it again does nothing), and `(load "name")` loads a file every time it's asked.

LIBRARY	WHAT IT HAS
`gpio`, `i2c`, `spi`	The I/O header's pins, the I²C bus, the SPI devices (section 22.5)
`clock`, `disk`, `cons`, `proc`, `pc`	The clock chip, the disks, the windows, the tasks, `/pc`
`snd`	Sound (section 22.6)
`video`	The Vera X's screen: drawing and turtle graphics (section 22.7)
`screen`	The terminal's screen: clearing, the cursor, colours
`dice`	Dice for games: `(d6)`, `(dice 3 6)` ...
`dev`	What the device libraries share

22.2 Files and directories

These are built in:

FUNCTION	WHAT IT DOES
`(read-file p)`	The file's contents, a string
`(read-lines p)`	Its lines, a list of strings
`(write-file p x ...)`, `(append-file p x ...)`	Write the values to a file (made, or emptied first), or add them to its end
`(ls [p])`	The names in a directory, sorted
`(dir [p])`, `(stat p)`	The details of each file, or of one: a hash of `:name`, `:length`, `:mtime` ...
`(exists? p)`, `(dir? p)`, `(file? p)`	Is it there? Is it a directory, or a file?
`(mkdir p)`, `(remove p)`, `(rename p q)`, `(copy-file p q)`	Make a directory; remove, rename, copy a file

FUNCTION	WHAT IT DOES
`(cd [p])`, `(cwd)`	Change the current directory; what it is
`(glob pattern)`	The paths that match a pattern, like `/rom/lib/*`

```
hylang> (write-file "/ram/t.txt" "line one\nline two")
=> NIL
hylang> (read-lines "/ram/t.txt")
=> {"line one" "line two"}
hylang> (ls "/rom")
=> {"README" "bin" "doc" "lib" "songs"}
hylang> (exists? "/rom/lib")
=> T
```

Streams. For reading or writing a file a piece at a time, open a **stream**: `(open p [mode])`, where the mode is `:read` (the usual), `:write`, `:append` or `:update`. Then `(read-line s)`, `(read-byte s)`, `(read-all s)` and `(read-bytes s n)` read it, giving `NIL` at its end; `(print-to s x ...)`, `(write-to s x ...)` and `(write-bytes s bytes)` write it; `(seek s n)` and `(tell s)` move around it; and `(close s)` closes it.

```
hylang> (def {s} (open "/ram/u.txt" :write))
=> NIL
hylang> (print-to s "first")
=> NIL
hylang> (print-to s "second")
=> NIL
hylang> (close s)
=> NIL
hylang> (def {r} (open "/ram/u.txt"))
=> NIL
hylang> (read-line r)
=> "first"
hylang> (read-line r)
=> "second"
hylang> (read-line r)
=> NIL
```

`read-line`, `read-byte` and `read-all` with no stream read the standard input: your typing.

22.3 Programs

FUNCTION	WHAT IT DOES
`(run prog x ...)`	Run a program, with the values as its arguments; wait, and give its exit code
`(sh line [input])`	Run a command line with rc: its exit code
`(sh-out line [input])`	The same, but give its output, as a string
`(spawn prog x ...)`	Start a program without waiting: its task
`(wait task)`	Wait for it to end: its exit code
`(kill task)`	Stop a task
`(pid)`	This task's number
`(env [name])`, `(setenv name value)`, `(unsetenv name)`	The environment's variables (`$name` is `(env "name")`)

```
hylang> (sh-out "date")
=> "2000-01-01 00:00:05\n"
hylang> (run "echo" "hi" "there")
hi there
=> 0
hylang> (env "path")
=> {"." "/bin"}
```

22.4 Time, memory and the system

FUNCTION	WHAT IT DOES
`(date [t])`	The date and time, as a string
`(date-parts [t])`	A hash of its `:year`, `:month`, `:day`, `:hour`, `:minute`, `:second` and `:weekday`
`(time)`	The clock: seconds since the start of 2000
`(seconds-of y m d [h mi s])`	The seconds since 2000 of a date
`(ticks)`, `(tick-rate)`	The tick count (0 to 32,767); ticks a second (200)
`(sleep s)`, `(sleep-until tick)`	Sleep for *s* seconds (`(sleep 1/10)`: a tenth); until a tick
`(clock)`	Seconds since hylang started
`(key)`, `(key?)`	The next key, as it's pressed (an atom like `:up` for a special key); is one waiting?
`(free)`	Memory, as `free` shows it: a hash
`(sysinfo)`, `(mods)`, `(ps)`, `(task-info task)`	The system, the ROM's modules, the tasks, a task
`(peek addr)`, `(poke addr byte)`, `(peek-word addr)`, `(poke-word addr n)`	Read and write the task's memory
`(banks)`, `(bank-alloc n)`, `(bank-free bank n)`, `(bank-read bank off n)`, `(bank-write bank off bytes)`	The task's RAM banks
`(seg-create banks)`, `(seg-attach seg)`, `(seg-read ...)`, `(seg-write ...)`	Shared segments
`(bind new old ...)`, `(mount dev old ...)`, `(unmount old ...)`, `(ns)`, `(newns)`	The namespace
`(note task n)`, `(note-group group n)`, `(on-note f)`	Notes
`(hold body ...)`	Run the body without letting other tasks have a turn (for exact timing)

```
hylang> (date-parts)
=> <hash>{{:year 2000} {:month 1} {:day 1} {:hour 0} {:minute 0} {:second 5} {:weekday 6}}
hylang> (free)
=> <hash>{{:ram 384} {:shared 1024} {:used 512} {:free 512} {:segments 1}}
```

System calls. Every system call has a function: `sys-` and its name, taking the call's inputs as Appendix K lists them, and giving its output (a list, if it has several). `(sys :name ...)` does the same.

```
hylang> (sys-getpid)
=> 2
hylang> (sys :ticks)
=> 1186
```

22.5 The devices

The pins: `(use "gpio")` (section 17.2): `(gpio pin)` reads a pin, `(gpio! pin level)` sets one (making it an output), `(gpio-in pin)` and `(gpio-out pin)` set its direction, `(gpio-port)` and `(gpio-port! byte)` read and set all eight, `(gpio-ddr! byte)` sets all their directions, `(gpio-ca1! edge)` and `(gpio-ca2! v)` set the handshake lines, `(gpio-wait)` waits for CA1, and `(gpio-state)` reads the `ctl` file.

```
hylang> (use "gpio")
=> NIL
hylang> (gpio-out 2)
=> NIL
hylang> (gpio! 2 1)
=> NIL
hylang> (gpio 2)
=> 1
```

The I²C bus: `(use "i2c")`: `(i2c-devices)` lists the chips that answer; `(i2c-read addr n)` reads *n* bytes from a chip, and `(i2c-write addr bytes)` writes a list of bytes; `(i2c-reg-size n)` and `(i2c-speed khz)` set the bus up.

SPI: `(use "spi")`: `(spi d bytes)` sends a list of bytes to SPI device *d*, and gives what came back; `(spi-read d n)` reads *n* bytes; `(spi-mode d mode)` sets mode 0 or 3.

The disks: `(use "disk")`: `(df)` lists the disks, each as a hash; `(cards)` lists the SD cards; `(disk-start d)` and `(disk-stop d)` start a card again, or stop a RAM disk.

```
hylang> (use "disk")
=> NIL
hylang> (cards)
=> {0}
```

And the rest. `(use "clock")`: `(set-date text)` sets the clock, and `(rtc)` says whether there's a clock chip. `(use "cons")`: `(window)`, `(windows)`, `(new-window)`, `(show-window n)`, `(window-label)`, `(window-status s)`, `(raw-on)`, `(raw-off)`, `(snarf)` and `(snarf! s)` work the console's windows. `(use "proc")`: `(task-args task)`, `(task-cwd task)`, `(task-regs task)`, `(task-mem task addr n)` ... read a task's `/proc` files. `(use "pc")`: `(pc?)` is `T` if the PC tool is there.

The terminal: `(use "screen")`: `(cls)` clears the screen, `(at col row)` moves the cursor, `(color fg)` sets the text's colour, by its name (`:black`, `:red`, `:green`, `:yellow`, `:blue`, `:magenta`, `:cyan` or `:white`), and `(color fg bg)` the background's too; `(bold)` and `(plain)` change the style; and `(clear-line)`, `(cursor-off)` and `(cursor-on)` do what they say.

22.6 Sound

`(use "snd")` has the same words as HyForth's sound library (section 20.6): `snd-patch`, `snd-note`, `snd-off`, `snd-level`, `snd-pan`, `snd-bend`, `snd-drum`, `snd-freq`, `snd-glide`, `snd-wave`, `snd-lfo`, `snd-sens`, `snd-noise`, `snd-reg`, `snd-volume`, `snd-claim`, `snd-release`, `snd-reset`, `note-of`, `snd-mml`, `snd-chord` and `play` (a song file). `tune` plays a list of notes, each a name (or a number, or `NIL` for a rest) and its beats, with the channel and the tempo after it if you like:

```
hylang> (use "snd")
=> NIL
hylang> (note-of "A4")
=> 69
hylang> (tune {{"C4" 1} {"E4" 1} {"G4" 2}})
=> NIL
```

22.7 Drawing

`(use "video")` has the same drawing words as HyForth's (section 20.7): `bitmap`, `bitmap-off`, `pen`, `plot`, `line`, `box`, `bar`, `circle`, `disc`, `clear`, `text`, `border`; the turtle's `cs`, `home`, `fd`, `bk`, `rt`, `lt`, `pu`, `pd`, `heading` and `seth`; and `vpoke`, `vpeek`, `vram!`, `vram@`, `palette!`, `sprite!`, `sprite-at`, `sprite-off`, `vsync`, `mouse` and `mouse-wait`.

```
(use "video")
(use "screen")
(bitmap 320 8)
(cs)
(fun {spiral n} {dotimes {i n} (do (pen (+ 1 (mod i 15))) (fd (* i 2)) (rt 91))})
(spiral 100)
(cls)
```

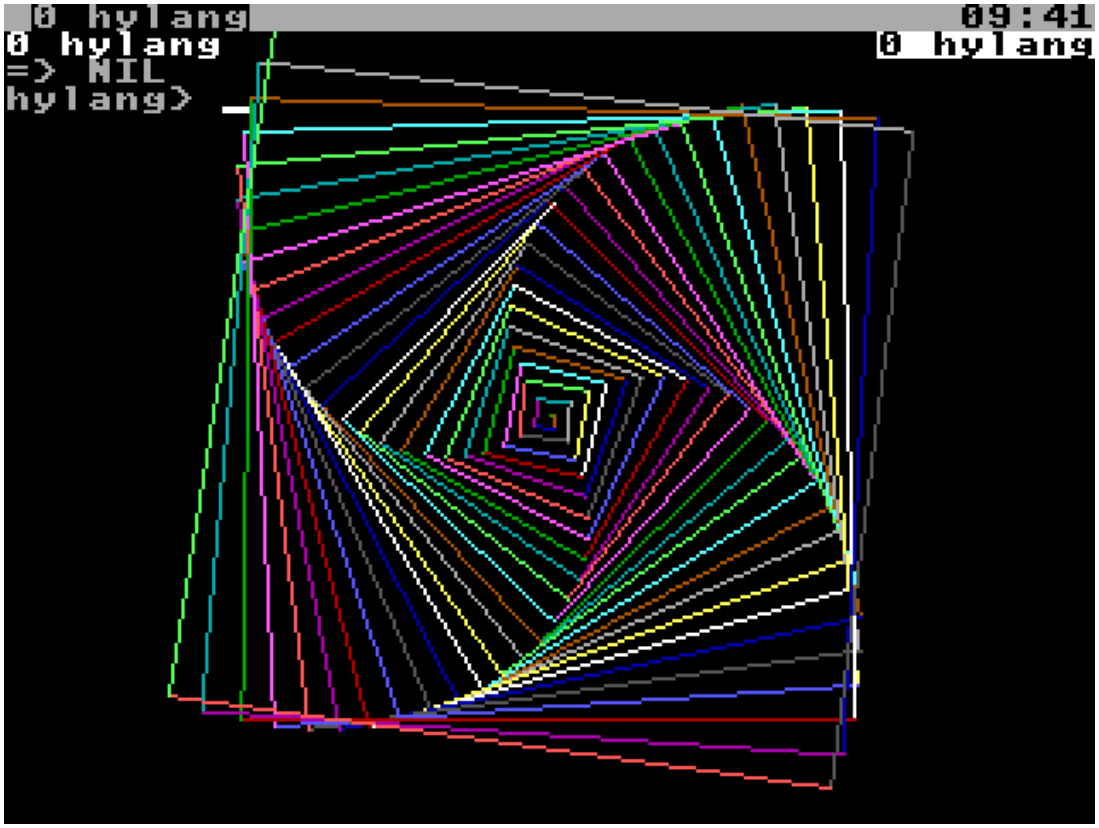


Figure 22-1. A spiral from hylang: turning 91 degrees, not 90, makes the squares twist.

22.8 Numbers

hylang's numbers are Chapter 18's, written as Chapter 18 writes them. `(base)` and `(base "x")` read and set the base; `(digits)` and `(digits n)` the precision of `sqrt`, `exp`, `log`, `sin`, `cos`, `tan`, `atan`, `pi` and `pow`:

```
hylang> (digits 30)
=> 12
hylang> (sqrt 2)
=> 1.41421356237309504880168872421
```

(Setting the precision gives back the one before.) For bits: `bit-and`, `bit-or`, `bit-xor`, `bit-not`, `shl`, `shr` and `bit?`; `(hex n)` and `(bin n [width])` write a number's digits; and `lo`, `hi`, `word`, `bytes` and `from-bytes` work with bytes.

22.9 hylang as your shell

`hylang -l` makes hylang a login shell (section 8.16): any line that doesn't start with `(`, `{` or `[` (or a prefix) is a command for rc, and the prompt shows where you are, like HyForth's. Put `/bin/hylang -l` in `/sd/0/lib/shell` to make it yours, and your own settings in `/sd/0/lib/hylang/profile.hl`.

CHAPTER 23

Learning HyBASIC

BASIC (Beginner's All-purpose Symbolic Instruction Code) was invented in 1964, at Dartmouth College, to let students who weren't scientists use a computer. In the 1980s nearly every home computer had a BASIC built in: it was the first language of millions of people. **HyBASIC** is the Hydra's BASIC, in the style of Microsoft's QuickBASIC: it has line numbers if you want them, but also named procedures, records and structured loops, so programs can be written clearly. And it has the Hydra's numbers.

IN THIS CHAPTER: You'll learn HyBASIC from the beginning: `PRINT`, variables, programs, `INPUT`, decisions, loops, strings, arrays, and procedures of your own. **You need:** Chapter 6. You don't need to have programmed before: this is the gentlest of the three languages.

23.1 Starting HyBASIC

Type `basic`. HyBASIC's prompt is `>`. A line typed without a number is done at once (**direct mode**); `SYSTEM` (or `BYE`) leaves HyBASIC.

```
/> basic
> PRINT "Hello, world!"
Hello, world!
> PRINT 2 + 3
 5
```

You can type keywords in capitals or small letters: HyBASIC turns them into capitals. `?` is a short way to type `PRINT`. This book prints HyBASIC in capitals, as the books of the 1980s did, but `print 2 + 3` works just as well.

TRY THIS: Use HyBASIC as a calculator. What's `PRINT 365 * 24`: the hours in a year? How many seconds is that? And what does `PRINT 1 / 3 + 1 / 6` give? (HyBASIC keeps fractions exact: Chapter 18.)

23.2 PRINT

`PRINT` shows its items. A number is shown with a space in front (where a minus sign would go). Items separated by `;` are shown one after another; items separated by `,` are shown in **zones**, 14 characters wide, like a table's columns. A `;` at the end of a `PRINT` keeps the next `PRINT` on the same line.

```
> PRINT 2 ^ 64
 18446744073709551616
> ? 10 / 4
 5/2
> A = 5
> B = A * 2
> PRINT A; B
 5  10
> PRINT A, B
 5             10
```

HyBASIC uses Chapter 18's numbers, so 2 to the 64th is exact, and 10 / 4 is exactly 5/2.

23.3 Variables

A **variable** is a name for a value. A name is letters and digits (and _ or .), starting with a letter. A name that ends in $ holds a **string**: text, in double quotes. Any other holds a number.

```
> NAME$ = "Sam"
> PRINT "Hello, "; NAME$
Hello, Sam
```

(`LET NAME$ = "Sam"` means the same: `LET` is optional, as in all modern BASICs.) A number variable that hasn't been given a value is 0, and a string one is `""`, the empty string.

23.4 Programs

A line typed with a **line number** in front isn't done at once: it's kept, as part of the **program**. The lines are kept in order of their numbers, so you can type them in any order, and put a line between two others by giving it a number between theirs. That's why lines are usually numbered in tens. `RUN` runs the program, and `LIST` shows it:

```
> 10 FOR I = 1 TO 5
> 20 PRINT I; I * I
> 30 NEXT I
> RUN
 1  1
 2  4
 3  9
 4  16
 5  25
> LIST
10 FOR I = 1 TO 5
20 PRINT I; I * I
30 NEXT I
```

COMMAND	WHAT IT DOES
`RUN`	Run the program
`LIST` [*from*][-*to*]	Show the program (or some of its lines)
`NEW`	Throw the program away (and its variables)
`SAVE` "*file*"	Keep it in a file
`LOAD` "*file*"	Bring one back
`DELETE` *from*-*to*	Remove lines
`EDIT`	Edit the program in the screen editor (Chapter 11)

COMMAND	WHAT IT DOES
`CONT`	Carry on after a `STOP`, or CTRL-C

To change a line, type it again with the same number. To put a line between two others, give it a number between theirs. To remove one, type just its number:

```
> 10 PRINT "one"
> 20 PRINT "two"
> 15 PRINT "one and a half"
> LIST
10 PRINT "one"
15 PRINT "one and a half"
20 PRINT "two"
> 20
> LIST
10 PRINT "one"
15 PRINT "one and a half"
```

A program in memory is gone when you leave HyBASIC, or type `NEW`. `SAVE` keeps it in a file, and `LOAD` brings it back:

```
> SAVE "/ram/count.bas"
> NEW
> LIST
> LOAD "/ram/count.bas"
> RUN
one
one and a half
```

(Remember section 6.7: `/ram` forgets. Save the programs you want to keep on the SD card, as `"/sd/0/work/count.bas"`.) Program files usually have names ending in `.bas`.

Programs without line numbers. Line numbers are only needed for typing a program in at the `>` prompt. A program in a file doesn't need them, and is easier to read without them. Write it with the screen editor, and run it from the shell with `basic` and the file's name:

```
/> edit /ram/dice.bas
```

```
' dice.bas - throw two dice
RANDOMIZE TIMER
A = INT(RND * 6) + 1
B = INT(RND * 6) + 1
PRINT A; B; "makes"; A + B
```

```
/ram> basic dice.bas
 5  6 makes 11
```

A line that starts with `'` (or `REM`) is a **comment**: a note for people, which HyBASIC skips. `RND` gives a number between 0 and 1, chosen by chance, so `INT(RND * 6) + 1` is a whole number from 1 to 6: a die. And `RANDOMIZE TIMER` shakes the dice with the clock, so that they don't fall the same way every time.

> **TIP:** For longer programs, `EDIT` is much easier than typing numbered lines: it opens the program in the screen editor, and when you close the editor, HyBASIC takes the program back. In the editor, you don't need line numbers at all.

23.5 INPUT

`INPUT` asks a question, and waits for the answer. The answer goes into the variable:

```
> 10 INPUT "What is your name"; N$
> 20 PRINT "Hello, "; N$; "!"
> RUN
What is your name? Sam
Hello, Sam!
```

`LINE INPUT` takes a whole line, commas and all.

23.6 Arithmetic

OPERATOR	MEANS	OPERATOR	MEANS
`+ - * /`	Add, subtract, multiply, divide	`^`	A power
`\`	Divide, keeping only the whole part	`MOD`	The remainder
`= <> < > <= >=`	Comparisons	`AND OR XOR NOT`	Logic

The **order** follows school mathematics: `^` first, then `*` and `/`, then `+` and `-`. Parentheses change it.

```
> PRINT INT(7.8), FIX(-7.8), 7 MOD 3, 7 \ 2
 7             -7             1              3
> PRINT SQR(2), SQR(-1)
 1.41421356237                1i
> PRINT 1/3 + 1/6
 1/2
> PRINT FIB(50)
 12586269025
```

A comparison gives -1 for true and 0 for false.

23.7 Decisions

`IF` ... `THEN` does something only if a test is true, and `ELSE` gives something to do if it's false. On one line:

```
> A = 5
> IF A > 3 THEN PRINT "big" ELSE PRINT "small"
big
```

For more than one line, an `IF` **block** ends with `END IF`, and can have `ELSEIF`s:

```
IF SCORE >= 90 THEN
  PRINT "Excellent!"
ELSEIF SCORE >= 50 THEN
  PRINT "Well done."
ELSE
  PRINT "Try again."
END IF
```

`SELECT CASE` chooses between many cases:

```
> 10 FOR D = 0 TO 6
> 20   SELECT CASE D
> 30     CASE 0, 6: PRINT D; "weekend"
> 40     CASE ELSE: PRINT D; "weekday"
> 50   END SELECT
> 60 NEXT D
> RUN
 0 weekend
 1 weekday
 2 weekday
 3 weekday
 4 weekday
 5 weekday
 6 weekend
```

A `CASE` can be a list of values, a range (`CASE 1 TO 5`), or a test (`CASE IS > 100`).

23.8 Loops

`FOR` ... `NEXT` counts, as in section 23.4; `STEP` gives the size of each step (`FOR I = 10 TO 0 STEP -2`). `DO` ... `LOOP` repeats while (or until) a test is true; the test can be at the start or the end:

```
> X = 1 : DO WHILE X < 100 : X = X * 2 : LOOP : PRINT X
 128
```

(A `:` puts several statements on one line.)

LOOP	REPEATS
`FOR` *v* = *a* `TO` *b* [`STEP` *s*] ... `NEXT`	With *v* from *a* to *b*
`DO WHILE` *test* ... `LOOP`	While the test is true (tested first)
`DO UNTIL` *test* ... `LOOP`	Until it's true
`DO` ... `LOOP WHILE` *test*	While it's true (tested at the end: always at least once)
`DO` ... `LOOP UNTIL` *test*	Until it's true (tested at the end)
`WHILE` *test* ... `WEND`	While it's true (the old way)
`EXIT FOR`, `EXIT DO`	Leave the loop at once

23.9 Strings

FUNCTION	GIVES
`LEN(s$)`	Its length
`LEFT$(s$, n)`, `RIGHT$(s$, n)`	Its first, or last, *n* characters
`MID$(s$, i [, n])`	*n* characters from the *i*th (counting from 1)
`INSTR([i,] s$, t$)`	Where *t$* is in *s$* (0 if it isn't)
`UCASE$(s$)`, `LCASE$(s$)`	In capitals, in small letters
`LTRIM$(s$)`, `RTRIM$(s$)`	Without spaces at its start, at its end
`CHR$(n)`, `ASC(s$)`	The character with code *n*; the code of the first character
`STR$(x)`, `VAL(s$)`	A number as a string; the number a string holds

FUNCTION	GIVES
`SPACE$(n)`, `STRING$(n, c)`	*n* spaces; *n* of a character

```
> PRINT LEN("Hydra"), LEFT$("Hydra", 3), MID$("Hydra", 2, 3), UCASE$("hi")
 5            Hyd           ydr           HI
> PRINT CHR$(65); ASC("A")
A 65
```

`+` joins strings: `"Hydra" + "-16"` is `"Hydra-16"`.

23.10 Arrays

An **array** is a numbered list of values, under one name. `DIM` makes one, giving its highest number:

```
> DIM SCORES(5)
> SCORES(1) = 10 : SCORES(2) = 20
> PRINT SCORES(1) + SCORES(2)
 30
```

`DIM SCORES(5)` makes `SCORES(0)` to `SCORES(5)`. Arrays can have more than one dimension, like a grid: `DIM BOARD(8, 8)`. `REDIM` changes an array's size (`REDIM PRESERVE` keeps what's in it).

23.11 DATA and READ

`DATA` lines hold values inside the program; `READ` takes them, one by one:

```
> 10 READ N$, A
> 20 IF N$ = "end" THEN END
> 30 PRINT N$; " is"; A
> 40 GOTO 10
> 50 DATA "Sam", 12, "Ana", 13, "end", 0
> RUN
Sam is 12
Ana is 13
```

`GOTO` jumps to a line, and `END` ends the program. (`RESTORE` starts the `DATA` again from the beginning.)

23.12 SUBs and FUNCTIONs

Big programs are easier to write, and to read, in pieces. A **SUB** is a named piece of program; a **FUNCTION** is one that gives back a value. Each has **parameters**: names for the values it's given.

```
> 10 PRINT SQUARE(7)
> 20 CALL GREET("Sam")
> 30 END
> 40 FUNCTION SQUARE(N)
> 50   SQUARE = N * N
> 60 END FUNCTION
> 70 SUB GREET(WHO$)
> 80   PRINT "Hello, "; WHO$; "!"
> 90 END SUB
> RUN
 49
Hello, Sam!
```

A FUNCTION's value is what's given to its name. The variables in a SUB or FUNCTION are its own: they don't change the main program's variables of the same names. (`SHARED` lets it use them, and `STATIC` keeps its own between calls.) `GOSUB` and `RETURN`, the older way, jump to a line and back.

> **TIP:** A program with `SUB`s and `FUNCTION`s reads better without line numbers (section 23.14). Keep it in a file, and run it from the shell with `basic` and its name.

23.13 Errors

When something goes wrong, HyBASIC stops the program, and says where. A program can catch its errors with `ON ERROR GOTO`: the error jumps to a **handler**, where `ERR` is its number and `ERR$` its message, and `RESUME NEXT` carries on after the statement that failed:

```
> 10 ON ERROR GOTO 100
> 20 PRINT 1 / 0
> 30 PRINT "after"
> 40 END
> 100 PRINT "error"; ERR; ERR$
> 110 RESUME NEXT
> RUN
error 11 division by zero
after
```

Here are the mistakes everyone makes when they start, and what to do about each:

```
> PRNT 5
syntax error
> PRINT "five" + 5
type mismatch
> GOTO 999
label not defined
> NEXT
NEXT without FOR
```

WHAT YOU SEE	WHAT HAPPENED	WHAT TO DO
`syntax error`	HyBASIC couldn't make sense of the line: a keyword spelled wrong, a missing quote or parenthesis	Read the line again, a word at a time
`type mismatch`	A string where a number should be, or the other way round	A string's name ends in `$`; `VAL` and `STR$` change one into the other
`label not defined`	A `GOTO` or `GOSUB` to a line that isn't there	`LIST` the program, and check the number
`NEXT without FOR`	A `NEXT` with no `FOR` before it, or loops that cross each other	Each `FOR` needs its own `NEXT`, inside any loop it's in
`?Redo from start`	`INPUT` wanted a number, and you typed something else	Type a number
`duplicate definition`	Two `SUB`s or `FUNCTION`s with the same name	Give one of them another name
`0`, or an empty line, where you expected a value	A variable's name spelled two ways: `SCORE` and `SCROE` are two variables, and the new one is 0	Check the names
A program that never stops	A loop whose test never comes true	Press CTRL-C; then `CONT` carries on, or `LIST` shows the program

When a program stops with an error, HyBASIC says which line it was in. `LIST` that line, and look at it hard.

23.14 Whole programs

Here are three small programs that use what this chapter has taught. Type them in, run them, and then change them.

A times table

A loop inside a loop: for each row `T`, the inner loop prints ten numbers. `PRINT USING "####"` prints each number in a column four characters wide, so the columns line up, and the `;` at the end keeps them on one line. The `PRINT` at line 50 ends each row:

```
> 10 FOR T = 1 TO 9
> 20   FOR I = 1 TO 10
> 30     PRINT USING "####"; T * I;
> 40   NEXT I
> 50   PRINT
> 60 NEXT T
> RUN
   1   2   3   4   5   6   7   8   9  10
   2   4   6   8  10  12  14  16  18  20
   3   6   9  12  15  18  21  24  27  30
   4   8  12  16  20  24  28  32  36  40
   5  10  15  20  25  30  35  40  45  50
   6  12  18  24  30  36  42  48  54  60
   7  14  21  28  35  42  49  56  63  70
   8  16  24  32  40  48  56  64  72  80
   9  18  27  36  45  54  63  72  81  90
```

TRY THIS: Change the program to ask which table you want (`INPUT "Which table"; T`), and print only that one, as "7 times 3 is 21", a line for each.

Dice with a score

`dice.bas` threw two dice. This version throws them until they make a double, and counts the throws. It uses a `DO` loop, because nobody knows how many times it will go round:

```
> 10 RANDOMIZE TIMER
> 20 THROWS = 0
> 30 DO
> 40   A = INT(RND * 6) + 1 : B = INT(RND * 6) + 1
> 50   THROWS = THROWS + 1
> 60   PRINT A; B
> 70 LOOP UNTIL A = B
> 80 PRINT "A double, after"; THROWS; "throws."
> RUN
 1  2
 1  3
 4  4
A double, after 3 throws.
```

(Yours will fall differently.)

FizzBuzz, in a file

In the game of FizzBuzz, you count aloud, but say "Fizz" instead of every number that 3 goes into, "Buzz" for those that 5 goes into, and "FizzBuzz" for both. Go to `/ram`, start `edit fizz.bas`, and type:

```
' fizz.bas - the game of FizzBuzz: count, but say Fizz for the threes,
' Buzz for the fives, and FizzBuzz for both
FOR N = 1 TO 15
  IF N MOD 15 = 0 THEN
    PRINT "FizzBuzz ";
  ELSEIF N MOD 3 = 0 THEN
    PRINT "Fizz ";
  ELSEIF N MOD 5 = 0 THEN
    PRINT "Buzz ";
  ELSE
    PRINT N;
  END IF
NEXT N
PRINT
```

Save it, leave the editor, and run it:

```
/ram> basic fizz.bas
 1  2 Fizz  4 Buzz Fizz  7  8 Fizz Buzz  11 Fizz  13  14 FizzBuzz
```

`N MOD 3` is the remainder when `N` is divided by 3: if it's 0, then 3 goes into `N`. The test for 15 has to come first. (Why?) The lines inside the loop and inside the `IF` are moved in by two spaces. HyBASIC doesn't mind either way, but **indenting** like this shows at a glance which lines belong to which loop or test.

> **TRY THIS:** Change `fizz.bas` so that numbers 7 goes into say "Whizz". What should 21 say? And 35? Then make it ask how far to count, with `INPUT`.

Chapter 24 shows how to use HyBASIC to work the Hydra: files, the terminal, sound and graphics. Appendix C lists every keyword.

CHAPTER 24

Programming the Hydra in HyBASIC

This chapter shows how HyBASIC works the Hydra: its files, the terminal, other programs, sound, graphics, the numbers, and machine code. Everything here is built into HyBASIC: there are no libraries to load.

IN THIS CHAPTER: You'll use HyBASIC to work the Hydra's files, terminal, programs, sound and graphics, and to call machine code. **You need:** Chapter 23. Each section leans on the chapter about its device (Chapters 13 to 17), which you can read alongside.

24.1 Files

A file is opened with a **file number**, and then read or written by that number:

```
OPEN "name" FOR INPUT AS #1     ' read it
OPEN "name" FOR OUTPUT AS #1    ' write it (made, or emptied first)
OPEN "name" FOR APPEND AS #1    ' write on to its end (made if it isn't there)
OPEN "name" FOR BINARY AS #1    ' read and write bytes, anywhere in it
```

A `'` starts a comment, as `REM` does.

```
> 10 OPEN "/ram/data.txt" FOR OUTPUT AS #1
> 20 PRINT #1, "apple"
> 30 PRINT #1, "banana"
> 40 CLOSE #1
> 50 OPEN "/ram/data.txt" FOR INPUT AS #1
> 60 DO UNTIL EOF(1)
> 70   LINE INPUT #1, L$
> 80   PRINT "read: "; L$
> 90 LOOP
> 100 CLOSE #1
> RUN
read: apple
read: banana
```

STATEMENT OR FUNCTION	WHAT IT DOES
`PRINT #n, items`, `WRITE #n, items`	Write to a file (`WRITE` puts strings in quotes, and commas between)
`INPUT #n, v, ...`, `LINE INPUT #n, v$`	Read items, or a whole line
`INPUT$(k, #n)`	Read *k* bytes
`GET #n, [pos], v`, `PUT #n, [pos], v`	Read or write a value at a place in a binary file
`SEEK #n, pos`, `SEEK(n)`	Move to a place; where it is
`EOF(n)`, `LOF(n)`, `LOC(n)`	At the end? Its length; the bytes before the place
`CLOSE [#n, ...]`	Close files (none named: all of them)

STATEMENT OR FUNCTION	WHAT IT DOES
`FREEFILE`	A file number not in use
`KILL f$`, `NAME old$ AS new$`	Remove, rename a file
`MKDIR d$`, `RMDIR d$`, `CHDIR d$`	Make, remove, change to a directory
`FILES [path$]`	List a directory
`DIR$(path$)`, `DIR$`	A directory's first name; then each next one ("" after the last)

```
> FILES "/rom/lib"
as/             basic/          edit/           font/           forth/
hylang/         namespace       profile         shell           windows
```

24.2 The terminal

STATEMENT OR FUNCTION	WHAT IT DOES
`CLS`	Clear the screen
`LOCATE row, col`	Move the cursor (from 1, 1 at the top left)
`COLOR fg [, bg]`	The text's colour and the background's (0 to 15)
`CSRLIN`, `POS(0)`	The cursor's row and column
`WIDTH n`	The screen's width, for `PRINT`'s zones
`INKEY$`	The key pressed, or `""` if none (it doesn't wait)
`INPUT$(n)`	Wait for *n* keys
`SLEEP [s]`	Wait *s* seconds (none: until a key)
`BEEP`	Ring the bell
`TIMER`, `TIME$`, `DATE$`	Seconds since midnight; the time; the date

`PRINT USING` prints numbers in a fixed form: each `#` is a digit, `.` places the decimal point, `,` puts commas in, and `$$` a dollar sign:

```
> PRINT USING "###.##"; 3.14159
  3.14
> PRINT USING "Total: $$#,###.##"; 1234.5
Total:  $1,234.50
```

24.3 Other programs

STATEMENT OR FUNCTION	WHAT IT DOES
`SHELL [line$]`	Run a command line with rc, and wait (none: an rc of its own)
`SHELL$(line$)`	Run it, and give its output, as a string
`STATUS`	The last `SHELL`'s status
`ENV$(name$)`	An environment variable
`ENVIRON "name=value"`	Set one
`ARG$(n)`, `COMMAND$`	In a program run from a file: its argument *n* (0: its name); all its arguments

STATEMENT OR FUNCTION	WHAT IT DOES
`SYSTEM [n]`, `BYE [n]`	Leave HyBASIC, with status *n*

```
> PRINT SHELL$("date")
2000-01-01 00:00:05
> PRINT ENV$("path")
. /bin
```

A program run from a file gets its arguments:

```
/> echo 'PRINT "args:"; COMMAND$' >/ram/args.bas
/> echo 'PRINT ARG$(0); " "; ARG$(1)' >>/ram/args.bas
/> basic /ram/args.bas red blue
args:red blue
/ram/args.bas red
```

`INCLUDE "file"` puts another file's lines into the program, so procedures can be kept in a file of their own and shared between programs.

24.4 Records

`TYPE` makes a new kind of variable, a **record**, with named **fields**:

```
> 10 TYPE PLAYER
> 20   NAME AS STRING * 8
> 30   SCORE AS INTEGER
> 40 END TYPE
> 50 DIM P AS PLAYER
> 60 P.NAME = "Sam" : P.SCORE = 42
> 70 PRINT P.NAME; P.SCORE
> RUN
Sam      42
```

`STRING * 8` is a string of exactly 8 characters. Records are useful for files of fixed-size entries, with `GET` and `PUT`, and arrays of them: `DIM TEAM(10) AS PLAYER`.

24.5 Sound

STATEMENT	WHAT IT DOES
`SOUND ch, note [, patch [, vol]]`	Play a note (a MIDI number: 60 is middle C) on channel *ch*
`SOUND ch`	Let it go
`SOUND "text"`	Send the sound driver a command, as `sndctl` takes them (section 15.3)
`PLAY [ch,] mml$`	Play a line of MML (section 15.6) on channel 0 (or *ch*), and wait for it to finish
`PLAY "file"`	Play a song file: a ZSM file or a score

```
SOUND 0, 60, 0, 100   ' middle C, on the piano, loud
SLEEP 1
SOUND 0               ' let it go
PLAY "o4 l8 c d e f g a b > c"
PLAY "/rom/songs/scom.zsm"
```

24.6 Graphics

With a Vera X, HyBASIC has QuickBASIC's graphics statements, drawn on the bitmap under the text. `SCREEN` chooses a mode first:

MODE	DOTS	COLOURS
`SCREEN 0`	(Text only: no bitmap)	
`SCREEN 1`	320 by 240	4
`SCREEN 2`	640 by 480	2
`SCREEN 7`	320 by 240	16
`SCREEN 12`	640 by 480	4
`SCREEN 13`	320 by 240	256

STATEMENT	WHAT IT DOES
`PSET (x, y) [, c]`, `PRESET (x, y)`	Set a dot (in colour *c*); clear one
`LINE (x1, y1)-(x2, y2) [, c [, B | BF]]`	A line; with `B` a box, with `BF` a filled box
`CIRCLE (x, y), r [, c]`	A circle
`PAINT (x, y) [, c [, border]]`	Fill the area around a point
`DRAW s$`	Draw with a little language of moves (below)
`GPRINT (x, y), text$`	Write text on the bitmap
`PALETTE n, rgb`	Change colour *n*
`POINT(x, y)`	A dot's colour
`WINDOW (x1, y1)-(x2, y2)`	Use your own coordinates
`VIEW (x1, y1)-(x2, y2)`	Draw in only part of the bitmap
`SPRITE n, x, y [, image$]`, `SPRITE n OFF`	Show, move or hide a sprite
`COLOR c`, `CLS`	With a bitmap: the pen's colour; clear the bitmap too

`STEP (x, y)` instead of `(x, y)` means "this far from the last point drawn".

`DRAW`'s language moves a pen from the last point drawn: `U`, `D`, `L` and `R` (up, down, left, right) and `E`, `F`, `G` and `H` (the diagonals: up and right, down and right, down and left, up and left), each followed by a distance; `M x,y` (to a point, or by so much with + or –); `B` before a move (don't draw it) and `N` (come back after it); `C n` (the colour); `S n` (the scale: *n* quarter-dots a step); `A n` (turn *n* quarter turns) and `TA n` (turn *n* degrees). So `DRAW "C2 R40 D40 L40 U40"` draws a green square.

```
SCREEN 13
CLS
FOR R = 10 TO 100 STEP 10 : CIRCLE (160, 110), R, R / 10 + 1 : NEXT
LINE (20, 20)-(300, 200), 14, B
LINE (20, 200)-(300, 20), 11
GPRINT (96, 216), "HyBASIC graphics"
```

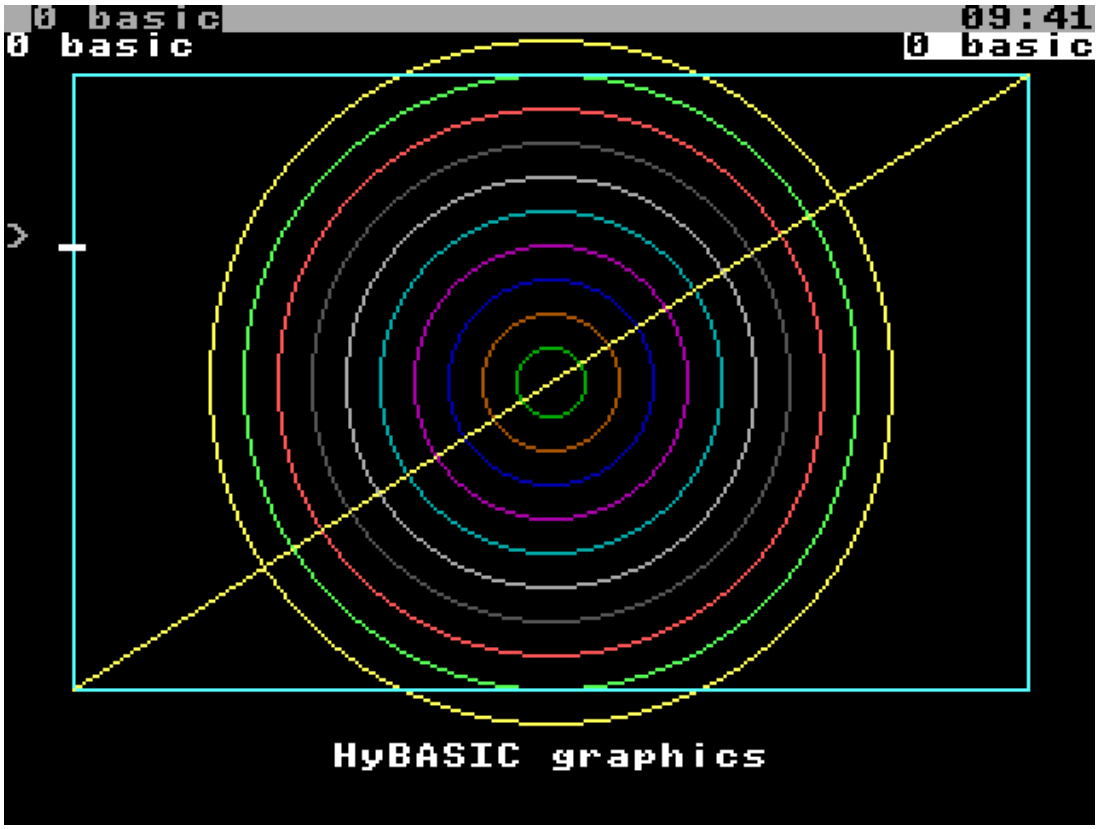


Figure 24-1. HyBASIC's graphics, in SCREEN 13.

24.7 Numbers

HyBASIC's numbers are Chapter 18's. A number in a program can be written in any of its forms (`2/3`, `1.5`, `2i`, `1_000`, `#xFF`), and also QuickBASIC's way: `&HFF` (hex), `&O17` (octal) and `&B101` (binary), and `1E3` (1000).

STATEMENT OR FUNCTION	WHAT IT DOES
`BASE b$`, `BASE$`	Set the base for reading and showing numbers; what it is
`DIGITS n`, `DIGITS()`	Set the precision of `SQR`, `EXP`, `LOG`, `SIN` ...; what it is
`FIXED(x, n)`	*x* as a fixed decimal of *n* places
`RATIONAL(x)`, `NUMERATOR(x)`, `DENOMINATOR(x)`	As a fraction; its top; its bottom
`COMPLEX(re, im)`, `REAL(z)`, `IMAG(z)`	A complex number; its parts
`FIB(n)`, `GCD(a, b)`	A Fibonacci number; the greatest common divisor
`BIT(x, n)`, `SHL(x, n)`, `SHR(x, n)`	A bit of *x*; *x* shifted left, right
`HEX$(x)`, `OCT$(x)`, `STR$(x [, base$])`	A number's text in hex, octal, or any base
`MKN$(x)`, `CVN(s$)`	A number as bytes (to keep in a file), and back
`RND`, `RANDOMIZE n`	A random number from 0 to 1; start the random numbers from *n*
`INT(x)`, `FIX(x)`, `SGN(x)`, `ABS(x)`	Round down; drop the fraction; -1, 0 or 1; the size
`SQR`, `EXP`, `LOG`, `SIN`, `COS`, `TAN`, `ATN`, `PI`	The math functions

```
> PRINT &HFF, &B101, &O17
 255           5             15
> PRINT FIXED(1/3, 4), RATIONAL(0.25), NUMERATOR(6/8)
 0.3333        1/4           3
> PRINT COMPLEX(1, 2) * COMPLEX(1, -2)
 5
> DIGITS 30 : PRINT SQR(2)
 1.41421356237309504880168872421
```

24.8 Memory and machine code

STATEMENT OR FUNCTION	WHAT IT DOES
`PEEK(addr)`, `POKE addr, b`	Read or write a byte of the task's memory
`BANK n`, `BANK()`	Which of the task's RAM banks `PEEK` and `POKE` see at `$8000`
`SYS addr [, a [, x [, y]]]`	Call machine code at *addr*, with the processor's A, X and Y registers set
`SYS "NAME" [, a [, x [, y]]]`	Make a system call by its name (Appendix K)
`RREG a, x, y, p`	The registers after the last `SYS`
`ASM` ... `END ASM`	Assembly language, inside the program (Chapter 27)
`CALL ASM label`, `CALL ABSOLUTE (addr)`	Call an `ASM` block's code, or machine code

24.9 HyBASIC as your shell

`basic -l` makes HyBASIC a login shell (section 8.16). A line is BASIC's if it starts with a line number, `?`, a keyword, an assignment, or the name of one of the program's SUBs; any other line is a command for rc, and `%` at the start makes any line rc's. The prompt shows where you are. Put `/bin/basic -l` in `/sd/0/lib/shell` to make it yours, and your own settings in `/sd/0/lib/basic/profile.bas`.

CHAPTER 25

One Program, Three Languages

You've now met three programming languages. Which one is best? None of them: each has its own way of thinking about a problem, and a problem that's awkward in one is easy in another. The clearest way to see the difference is to write the same program in all three, and put them side by side. That's what this chapter does.

IN THIS CHAPTER: You'll read one small game written three times, compare how each language does the same seven jobs, measure how fast each one runs, and get some help in choosing between them. **You need:** the "Learning" chapter of at least one of the languages (Chapter 19, 21 or 23). You'll follow your own language's version closely, and the other two well enough.

25.1 The game

The Hydra thinks of a number from 1 to 100, and you try to guess it. After each guess, it tells you whether you were too small or too big, and when you get it right, it tells you how many guesses you took.

It's a small game, but it has a bit of everything. Whatever the language, the program has to:

1. choose a number by chance, and remember it;
2. print a message;
3. read a number that you type;
4. compare two numbers, and decide what to say;
5. go round again until you're right;
6. count the guesses.

Each program is in a file. Type in the one for your language (in `/ram`, with `edit`), or all three.

25.2 In HyBASIC

```
' guess.bas - guess the number, in HyBASIC
RANDOMIZE TIMER
SECRET = INT(RND * 100) + 1
TRIES = 0
PRINT "I'm thinking of a number from 1 to 100."
DO
  INPUT "Your guess"; GUESS
  TRIES = TRIES + 1
  IF GUESS < SECRET THEN
    PRINT "Too small."
  ELSEIF GUESS > SECRET THEN
    PRINT "Too big."
  END IF
LOOP UNTIL GUESS = SECRET
PRINT "Right! You took"; TRIES; "guesses."
```

```
/ram> basic guess.bas
I'm thinking of a number from 1 to 100.
Your guess? 50
Too big.
Your guess? 25
Too big.
Your guess? 12
Too big.
Your guess? 6
Too small.
Your guess? 9
Too big.
Your guess? 7
Right! You took 6 guesses.
```

HyBASIC's version reads from top to bottom, like a recipe: do this, then this, then go back and do it again `UNTIL` the guess is right. `INPUT` does three jobs in one: it prints the question, waits for you, and turns what you typed into a number. The values live in **variables** that any line can change: `TRIES = TRIES + 1` means "work out `TRIES + 1`, and put the answer back in `TRIES`".

(Your secret number will be different from the book's: `RANDOMIZE TIMER` starts the random numbers from the clock.)

25.3 In hylang

```
; guess.hl - guess the number, in hylang

(def {secret} (+ 1 (random 100)))
(def {tries} 0)

(fun {ask} {do
  (write "Your guess? ")
  (val (read-line))})

(fun {play} {do
  =(guess (ask))
  (set! {tries} (+ tries 1))
  (cond
    {(< guess secret) (do (print "Too small.") (play))}
    {(> guess secret) (do (print "Too big.") (play))}
    {T (print "Right! You took" tries "guesses.")}))})

(print "I'm thinking of a number from 1 to 100.")
(play)
```

```
/ram> hylang guess.hl
I'm thinking of a number from 1 to 100.
Your guess? 50
Too small.
Your guess? 75
Too small.
Your guess? 88
Too big.
Your guess? 81
Too small.
Your guess? 84
Too big.
Your guess? 83
Right! You took 6 guesses.
```

hylang's version is made of **functions**. `ask` prints the question, reads a line, and gives back its value as a number. `play` plays one turn: it asks, counts, and then `cond` picks one of three cases. Look at how it goes round again: there's no loop. When the guess is wrong, `play` simply calls `play`, to play another turn. That's **recursion** (section 21.9), and it's the way a Lisp programmer says "again".

`=(guess (ask))` makes a name that belongs to this turn only. The count has to last for the whole game, so `tries` is a global name, made with `def`, and changed with `set!`.

25.4 In HyForth

```
\ guess.fs - guess the number, in HyForth
lib random

variable secret
variable tries

: ask ( -- n )                  \ A number, typed at the keyboard
  ." Your guess? " pad 8 accept
  0 0 rot pad swap >number 2drop drop ;

: judge ( n -- flag )           \ Say how a guess did; true if it's right
  dup secret @ < if drop ." Too small." cr false exit then
      secret @ > if      ." Too big." cr false exit then
  true ;

: play ( -- )
  100 random 1+ secret !  0 tries !
  ." I'm thinking of a number from 1 to 100." cr
  begin  ask  1 tries +!  judge  until
  ." Right! You took " tries @ . ." guesses." cr ;

play
```

```
/ram> forth guess.fs
I'm thinking of a number from 1 to 100.
Your guess? 50
Too big.
Your guess? 25
Too small.
Your guess? 37
Too big.
Your guess? 31
Too big.
Your guess? 28
Too big.
Your guess? 26
Too small.
Your guess? 27
Right! You took 7 guesses.
```

HyForth's version is made of small **words**, each built on the ones before, and each passing its values to the next on the stack. `ask` leaves a number on the stack. `judge` takes it, says how the guess did, and leaves a flag, which is just what `until` wants: so the loop in `play` reads almost like a sentence, "begin: ask, count, judge, until right".

Forth has no `INPUT`, so `ask` does the job in two steps. `pad 8 accept` reads up to eight characters that you type into a scratch area called `pad`, and leaves how many there were. `>number` then turns those characters into a number. (Its stack diagram is in Appendix A; the `0 0` gives it a zero to start from, and the `2drop drop` throws away the parts of its answer that this program doesn't need.) Every Forth programmer writes a word like `ask` once, and keeps it.

25.5 Side by side

Here's each of the game's jobs, in each language:

THE JOB	HYBASIC	HYLANG	HYFORTH
A number from 1 to 100, by chance	`INT(RND * 100) + 1`	`(+ 1 (random 100))`	`100 random 1+`
Remember it	`SECRET = ...`	`(def {secret} ...)`	`variable secret` and `secret !`
Print a message	`PRINT "Too big."`	`(print "Too big.")`	`." Too big." cr`
Read a number	`INPUT "Your guess"; GUESS`	`(val (read-line))`	`pad 8 accept`, then `>number`
Decide	`IF` ... `ELSEIF` ... `END IF`	`(cond {test value} ...)`	`if` ... `then`
Go round again	`DO` ... `LOOP UNTIL`	`play` calls `play`	`begin` ... `until`
Count	`TRIES = TRIES + 1`	`(set! {tries} (+ tries 1))`	`1 tries +!`

And here's what the differences come to:

	HYBASIC	HYLANG	HYFORTH
A program is	A list of **statements**, done in order	**Expressions**, each with a value	**Words**, each made from other words
Values are kept	In variables	In names, and passed to functions	On the stack
It's written	Like a sentence: `PRINT A + B`	Function first: `(print (+ a b))`	Function last: `a b + .`
Its numbers	The Hydra's: any size, exact fractions	The Hydra's: any size, exact fractions	16 bits; the Hydra's with a library
It's best at	Getting something working quickly, and being read by someone else	Lists, text and anything that's built up from pieces	Speed, small size, and getting close to the machine
The hard part	Big programs get tangled, unless you use `SUB`s	Counting the parentheses	Keeping track of the stack

25.6 How fast?

Here's a race. Each language counts to ten thousand, one at a time, and says how long it took.

HyBASIC's `TIMER` is the number of seconds since midnight:

```
> 10 T = TIMER : C = 0
> 20 FOR I = 1 TO 10000
> 30   C = C + 1
> 40 NEXT I
> 50 PRINT C; "in"; TIMER - T; "seconds"
> RUN
 10000 in 2.73 seconds
```

hylang's `(clock)` is the number of seconds since hylang started:

```
hylang> (fun {count-up n} {do =(c 0) (dotimes {i n} (set! {c} (+ c 1))) c})
=> NIL
hylang> (def {t0} (clock))
=> NIL
hylang> (count-up 10000)
=> 10000
hylang> (- (clock) t0)
=> 2.435
```

And HyForth's `sys-ticks`, from the `hydra` library, counts the Hydra's **ticks**: there are 200 in a second.

```
/> lib hydra
/> : count-up ( n -- count ) 0 swap 0 do 1+ loop ;
/> : timed ( -- ) sys-ticks 10000 count-up . sys-ticks swap - . ." ticks" cr ;
/> timed
10000 40 ticks
```

HyForth takes about a fifth of a second; the other two take more than ten times as long. That's the price of their numbers. Each time HyBASIC or hylang adds 1, it's working with a number that could have a hundred digits, or be a fraction, and it has to check. HyForth just adds two 16-bit cells, which the 65C02 can do in a few instructions.

But don't choose a language by one race. For most programs, all three are fast enough, and the time that matters is *yours*: how long the program takes to write, and to get right. And when a program really does need speed, there's something faster than all three: the processor's own language, which is the subject of Part Five. A loop like this one, in machine code, takes less than a tenth of a second.

25.7 Choosing

IF YOU WANT	TRY
Your first program, today	HyBASIC
A calculation with huge numbers or exact fractions	HyBASIC or hylang
A program a friend can read without learning anything first	HyBASIC
To work on lists, words and text	hylang
To build something big out of small, tested pieces	hylang or HyForth
Speed, without learning machine code	HyForth
To control the Hydra's devices as you type	HyForth: it's the shell, so it's always there
To understand how a computer really runs a program	HyForth, then Part Five

All three can read and write files, run other programs, play music and draw on the Vera X's screen: the second chapter of each (Chapters 20, 22 and 24) shows how. And you don't have to choose only one. Programmers who know several languages write better programs in each of them, because each language teaches a different habit: HyBASIC, to say plainly what happens next; hylang, to think in values and functions; HyForth, to build big things from small ones.



PART FIVE

Machine Language

Underneath every program on the Hydra, HyForth, hylang and HyBASIC included, the processor is doing one simple thing over and over: reading an instruction from memory and doing it. This part goes down to that level. It explains the 65C02 processor, then the three tools for programming it directly: the **assembler**, which turns your instructions into machine code; the **disassembler**, which turns machine code back into instructions you can read; and the **debugger**, which runs a program one instruction at a time while you watch. Appendix D lists every instruction.

- Chapter 26. The 65C02
- Chapter 27. The Assembler
- Chapter 28. The Disassembler
- Chapter 29. The Debugger

CHAPTER 26

The 65C02

The Hydra's processor is a **W65C02S**, made by the Western Design Center (WDC). It's the newest member of a famous family. Its ancestor, the **6502**, was designed in 1975 by a small team at MOS Technology, among them Chuck Peddle and Bill Mensch. It was cheap, simple and quick, and it became the heart of the home computer revolution: the Apple II, the Commodore PET, VIC-20 and 64 (a 6510, a 6502 with a port added), the BBC Micro and the Atari 400 and 800 all had one, and the Atari 2600 and the Nintendo Entertainment System used versions of it. Bill Mensch went on to start WDC, which still makes the 65C02 today. The "C" means it's made in CMOS, which needs very little power; the "S" means it's **static**: its clock can be slowed down, or even stopped, without losing anything.

IN THIS CHAPTER: You'll learn what machine code is, the 65C02's registers and flags, its addressing modes, every group of its instructions, and how interrupts work. **You need:** hexadecimal (section 1.6), and some programming in any language (Part Four). There's nothing to type in this chapter: Chapter 27 puts it all to work.

This chapter explains how the 65C02 works, as you need to know it to write machine language for the Hydra. WDC's data sheet for the W65C02S has every detail of the chip itself (section 26.13 says where to find it), and Appendix D lists every instruction.

26.1 Machine code

The processor does one thing, over and over, millions of times a second:

1. It reads the byte at the address in its **program counter** (PC). That byte is an **opcode**: the number of an instruction.
2. If the instruction needs more, it reads the byte or two after it: its **operand**, such as a number or an address.
3. It does the instruction, and moves the program counter on to the next one (or, for a jump, to somewhere else).

A program in this form, as bytes, is **machine code**. The bytes `A9 41` mean "load the number `$41` into the A register": `$A9` is the opcode of that instruction, and `$41` is its operand. Nobody wants to write programs as numbers, so each instruction has a short name, its **mnemonic**: `$A9` is `LDA #`, "load A with this number". Programs written with the names are **assembly language**, and a program that turns them into machine code is an **assembler**. The Hydra's is called `as` (Chapter 27). `lda #$41` is assembled to `A9 41`.

Each 65C02 instruction is one, two or three bytes long: its opcode, then nothing, a byte, or an address. The 65C02 has 70 instructions (212 opcodes, counting each instruction's different ways of finding its data). They're all very simple: each moves a byte, adds, compares, tests, or jumps. Big programs are made of many small steps.

26.2 Bytes, hex, and negative numbers

Section 1.6 explained bits, bytes and hexadecimal. The 65C02 works on **bytes**: eight bits, holding a number from 0 to 255 (`$00` to `$FF`). An address is 16 bits, from `$0000` to `$FFFF`, so it takes two bytes. The 65C02 always keeps the two bytes of a 16-bit number **low byte first**: the address `$F950` is stored as the bytes `50 F9`. (This is called **little-endian**.) The

assembler writes < for a number's low byte and > for its high byte: `<$F950` is `$50`, and `>$F950` is `$F9`.

A byte can also hold a **negative number**. The trick is called **two's complement**: the bytes `$80` to `$FF` stand for -128 to -1. So `$FF` is -1, `$FE` is -2, and `$80` is -128. Bit 7 is set in every negative number, which makes them easy to spot. The nice thing about two's complement is that adding works the same either way: `$FF` + `$01` is `$00` (with a carry out of the top), and -1 + 1 is 0.

BYTE	AS A NUMBER FROM 0 TO 255	AS A SIGNED NUMBER
`$00`	0	0
`$01`	1	1
`$7F`	127	127
`$80`	128	-128
`$FE`	254	-2
`$FF`	255	-1

The processor doesn't know which you mean: it's the program that decides how to read a byte.

26.3 The registers

Inside the 65C02 are six **registers**: tiny memories that the processor works with directly.

REGISTER	SIZE	WHAT IT'S FOR
A	8 bits	The **accumulator**: arithmetic and logic are done here
X	8 bits	An **index register**: counting, and pointing into tables
Y	8 bits	Another index register
S	8 bits	The **stack pointer**: where the next byte pushed on the stack goes (`$0100` + S)
PC	16 bits	The **program counter**: the address of the next instruction
P	8 bits	The **processor status**: eight flags, set by what the last instructions did

The status register's bits are its **flags**:

BIT	7	6	5	4	3	2	1	0
Flag	N	V	-	B	D	I	Z	C

- **N**, negative: bit 7 of the last result (1 if it was negative, as a signed number).
- **V**, overflow: 1 if the last addition or subtraction went past the signed range (-128 to 127).
- **B**, break: only seen in a copy of P pushed on the stack: 1 if a `BRK` or `PHP` pushed it.
- **D**, decimal: 1 makes `ADC` and `SBC` work in decimal (section 26.8).
- **I**, interrupt disable: 1 holds interrupts off (section 26.11).
- **Z**, zero: 1 if the last result was zero.
- **C**, carry: the bit carried out of an addition (or 1 for no borrow in a subtraction), or the bit shifted out of a shift.
- Bit 5 isn't used.

Most instructions that change a register set N and Z by the new value. Appendix D shows which flags each instruction changes.

26.4 Try it: watching the registers

The debugger, `db`, can run a program one instruction at a time and show the registers after each one. Chapter 29 is all about it, but you can try it now. Here's a tiny program, `try.s`:

```
; try - a few instructions, to watch in db
.include "hydra.inc"
.include "hyx2.inc"

            HYX2_PROGRAM "try", main

.code
main:       lda #$7F            ; A = $7F (127)
            clc                 ; C = 0
            adc #1              ; A = A + 1 + C
            ldx #$FF            ; X = $FF
            inx                 ; X = X + 1
            rts                 ; The end
```

Go to `/ram` (`cd /ram`) and type it into the file `try.s` with the screen editor, `edit try.s` (Chapter 11 tells you how to use it). Copy it exactly, but the comments, after the `;`s, can be left out. Then assemble it with `as`, and start it in `db`. `s` steps one instruction:

```
/ram> as try.s
/ram> db ./try
task 3
PC=0830 A=30 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830  A9 7F     lda #$7F
db> s
PC=0832 A=7F X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0832  18        clc
db> s
PC=0833 A=7F X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0833  69 01     adc #$01
db> s
PC=0835 A=80 X=FF Y=00 S=FD P=NV--dizc W=0 U=0 RAM=00 ROM=00
0835  A2 FF     ldx #$FF
db> s
PC=0837 A=80 X=FF Y=00 S=FD P=NV--dizc W=0 U=0 RAM=00 ROM=00
0837  E8        inx
db> s
PC=0838 A=80 X=00 Y=00 S=FD P=nV--diZc W=0 U=0 RAM=00 ROM=00
0838  60        rts
db> q
```

Each line starting `PC=` shows the registers, and the line under it the next instruction: its address, its bytes, and the instruction. In `P`, a capital letter is a flag that's 1, and a small one a flag that's 0. Follow it through:

- `lda #$7F` put `$7F` in A. (The registers started with what was left in them: they're not cleared.)
- `adc #1` added 1 (and the carry, 0): `$7F` + 1 = `$80`. The result has bit 7 set, so **N** became 1. And 127 + 1 is 128, too big for a signed byte, which wrapped round to -128: so **V**, overflow, became 1 too.
- `ldx #$FF` put `$FF` in X. N stayed 1, because `$FF` has bit 7 set.
- `inx` added 1 to X: `$FF` + 1 wraps round to `$00`. So **Z** became 1 (the result is zero), and N became 0.

`q` quits the debugger (and the program). The other numbers on the line are the Hydra's: `W` and `U` are two of its registers (section 7.2), and `RAM` and `ROM` the task's bank registers.

26.5 Memory

The 65C02 sees 65,536 bytes: addresses `$0000` to `$FFFF`. Memory is thought of in **pages** of 256 bytes: page `$00` is `$0000-$00FF`, page `$01` is `$0100-$01FF`, and so on. The high byte of an address is its page.

Two pages are special:

- **Page 0**, the **zero page** (`$0000-$00FF`). Its addresses need only one byte, so instructions that use them are shorter and quicker. The zero page is also the only place a pointer can be kept (section 26.6). Programs use the zero page for their most important variables.
- **Page 1**, the **stack** (`$0100-$01FF`). The 65C02 keeps its stack here (section 26.10).

On the Hydra each task has its own 32K at `$0000-$7FFF` (section 7.3), so every program has a zero page and a stack of its own. A program in assembly language sees its memory like this:

ADDRESSES	WHAT'S THERE	FOR THE PROGRAM?
`$00`, `$01`	The task's RAM and ROM bank registers	Read them; write them to choose a bank
`$02-$21`	r0 to r15, the **call registers**: values going to and from HydraOS	Yes, but every system call can change them
`$22-$7F`	The program's zero page: 94 bytes	Yes: all its own. HydraOS never touches them
`$80-$FF`	HydraOS's zero page for this task	No
`$0100-$01FF`	The stack	Through the stack instructions
`$0200-$03FF`	HydraOS's area for this task (its open files, its name ...)	No
`$0800-$7FFF`	The program: its code, its data, and its BSS (its variables, cleared as it starts)	Yes
`$8000-$9FFF`	The RAM bank chosen by `$00`	Yes (section 27.15)
`$A000-$DFFF`	The paged ROM bank chosen by `$01` (the libraries, and the ROM disk)	Read only
`$E000-$FFFF`	The kernel, the I/O ports and the Hydra's registers	No: through system calls

A program file is loaded at `$0800`. Its first 48 bytes are its **header** (who it is, how long it is, where it starts), so the first instruction is usually at `$0830`. After the program comes its BSS, then free memory up to the **break**, which a program can move with the `BREAK` system call to get more.

26.6 Addressing modes

An instruction that works on memory has to say which byte. The ways it can say are its **addressing modes**. Here they all are, as the assembler writes them. *M* means the byte the instruction works on.

MODE	WRITTEN	BYTES	*M* IS
Implied	`inx`	1	(No memory: the instruction says it all)
Accumulator	`asl a`	1	The A register itself
Immediate	`lda #$41`	2	The number written: `$41`
Zero page	`lda $22`	2	The byte at `$0022`
Zero page, X	`lda $22,x`	2	The byte at `$22` + X (staying on the zero page)
Zero page, Y	`ldx $22,y`	2	The byte at `$22` + Y (`LDX` and `STX` only)
Absolute	`lda $1234`	3	The byte at `$1234`

MODE	WRITTEN	BYTES	*M* IS
Absolute, X	`lda $1234,x`	3	The byte at `$1234` + X
Absolute, Y	`lda $1234,y`	3	The byte at `$1234` + Y
Zero page indirect	`lda ($22)`	2	The byte at the address kept in `$22` and `$23`
Indirect, Y	`lda ($22),y`	2	The byte at (the address kept in `$22` and `$23`) + Y
Indexed indirect	`lda ($22,x)`	2	The byte at the address kept in `$22` + X and the byte after
Absolute indirect	`jmp ($1234)`	3	(`JMP` only) Jump to the address kept at `$1234` and `$1235`
Absolute indexed indirect	`jmp ($1234,x)`	3	(`JMP` only) Jump to the address kept at `$1234` + X
Relative	`bne loop`	2	(Branches) Go to `loop`: up to 128 bytes back, or 127 forward
Zero page, relative	`bbr0 $22,loop`	3	(`BBR`, `BBS`) Test a bit of `$22`, and maybe go to `loop`

The `#` is important: `lda #$41` loads the number `$41`, but `lda $41` loads the byte at address `$0041`. Forgetting the `#` is the commonest mistake in assembly language.

Indexing adds X or Y to an address, which is how a program steps through a table or a string. This loop prints the string at `text`, a byte at a time, until it reaches the 0 at its end:

```
            ldy #0              ; Y = 0: the first character
loop:       lda text,y          ; A = the character at text + Y
            beq done            ; (0: the end)
            jsr PUTC            ; Print it
            iny                 ; The next one
            bne loop
done:
```

Indirect modes use a **pointer**: an address kept in two bytes of the zero page. `lda ($22),y` reads the address kept at `$22` and `$23`, adds Y, and loads the byte there. Since the pointer can be changed, one piece of code can work on any string or table. The Hydra's system calls pass pointers this way, in r0 to r15.

26.7 Moving bytes

INSTRUCTIONS	WHAT THEY DO
`LDA`, `LDX`, `LDY`	**Load** A, X or Y from memory (or with a number)
`STA`, `STX`, `STY`	**Store** A, X or Y into memory
`STZ`	Store zero into memory
`TAX`, `TAY`, `TXA`, `TYA`	**Transfer**: copy A to X, A to Y, X to A, Y to A
`TSX`, `TXS`	Copy S to X, X to S

Loads and transfers (except `TXS`) set N and Z by the value. Stores don't change any flags. There's no instruction to copy one byte of memory straight to another: a program loads it into a register, then stores it.

```
            lda #5              ; A = 5
            sta count           ; count = A
            ldx count           ; X = count
            stz total           ; total = 0
```

26.8 Arithmetic

The 65C02 can add and subtract, and nothing more: multiplying and dividing are done by programs (or, on the Hydra, the numbers library, section 27.16).

INSTRUCTIONS	WHAT THEY DO
`ADC`	A = A + *M* + C (**add with carry**)
`SBC`	A = A - *M* - (1 - C) (**subtract with borrow**)
`INC`, `DEC`	Add 1 to, or take 1 from, a byte of memory (or A)
`INX`, `INY`, `DEX`, `DEY`	Add 1 to, or take 1 from, X or Y
`CLC`, `SEC`	Make C 0, or 1

There's no plain add: `ADC` always adds the carry too. So before adding, a program clears it with `CLC`; and before subtracting, it sets it with `SEC` (C = 1 means "no borrow"). Why? Because the carry is how bigger numbers are added, a byte at a time. This adds 1000 to the 16-bit number in `score` and `score + 1` (low byte first):

```
            clc                 ; No carry into the low bytes
            lda score
            adc #<1000          ; The low bytes ($E8) ...
            sta score
            lda score + 1
            adc #>1000          ;   then the high bytes ($03), and the carry from the low
            sta score + 1
```

If the low bytes' sum is more than 255, C becomes 1 and the second `ADC` adds it in, exactly as you carry a 1 when you add on paper. `INC`, `DEC` and the others don't change C; they set N and Z.

Decimal mode. After `SED` (set D), `ADC` and `SBC` work in **binary-coded decimal**: each half of a byte holds a digit from 0 to 9, so `$19` + `$01` gives `$20`, not `$1A`. Scores on old arcade games were often kept this way, because they're easy to print. `CLD` goes back to binary. Always `CLD` when you're done: HydraOS expects D to be 0.

26.9 Logic, bits and shifts

INSTRUCTIONS	WHAT THEY DO
`AND`	A = A AND *M*: a bit is 1 only where both are 1 (to clear bits)
`ORA`	A = A OR *M*: a bit is 1 where either is 1 (to set bits)
`EOR`	A = A XOR *M*: a bit is 1 where they're different (to flip bits)
`BIT`	Test: Z = 1 if A AND *M* is 0; N and V get bits 7 and 6 of *M*
`TSB`, `TRB`	**Test and set** or **test and reset**: set or clear in *M* the bits that are 1 in A
`RMB`*n*, `SMB`*n*	Clear or set bit *n* (0 to 7) of a byte on the zero page
`ASL`, `LSR`	Shift left or right one bit: the bit shifted out goes into C, a 0 comes in
`ROL`, `ROR`	Rotate left or right through C: the bit shifted out goes into C, and C comes in

`AND #$DF` clears bit 5, which turns a small letter into a capital (the `upper` sample on the SD card does just that). Shifting left multiplies by 2, and shifting right divides by 2:

```
            lda #6
            asl a               ; A = 12
            asl a               ; A = 24
            lsr a               ; A = 12
```

26.10 Comparing, branching and the stack

Comparing. `CMP`, `CPX` and `CPY` compare A, X or Y with *M*. They subtract, but keep only the flags:

AFTER `CMP`	Z	C	BRANCH WITH
A equals *M*	1	1	`BEQ`
A is not *M*	0		`BNE`
A is less than *M* (from 0 to 255)	0	0	`BCC`
A is *M* or more		1	`BCS`

Branching. A branch goes somewhere else if a flag is set (or clear), and on to the next instruction if not:

INSTRUCTION	GOES IF
`BEQ`, `BNE`	Z = 1 (equal, or zero); Z = 0
`BCS`, `BCC`	C = 1; C = 0
`BMI`, `BPL`	N = 1 (minus); N = 0 (plus)
`BVS`, `BVC`	V = 1; V = 0
`BRA`	Always
`BBR`*n*, `BBS`*n*	Bit *n* of a zero page byte is 0; is 1

A branch can reach 128 bytes back or 127 bytes forward. For further, `JMP` goes anywhere. This loop counts X down from 10, doing its work 10 times:

```
            ldx #10             ; 10 times
again:      ...                 ; (The work)
            dex                 ; X = X - 1, and Z = 1 when it's 0
            bne again           ; Again, until X is 0
```

Subroutines. `JSR` (jump to subroutine) jumps to a piece of code and remembers where it came from; `RTS` (return from subroutine) goes back there. The return address is kept on the **stack**: a pile of bytes in page 1. Pushing a byte puts it at `$0100` + S, then takes 1 from S; pulling adds 1 to S, then reads it. So the stack grows downwards, and the last byte pushed is the first pulled. `JSR` pushes two bytes (the address of its own last byte: `RTS` adds 1), and `RTS` pulls them.

INSTRUCTIONS	WHAT THEY DO
`JSR`, `RTS`	Call a subroutine; return from one
`JMP`	Jump
`PHA`, `PHX`, `PHY`, `PHP`	Push A, X, Y or P
`PLA`, `PLX`, `PLY`, `PLP`	Pull A, X, Y or P

Pushing is the way to keep a register safe while something else uses it. A system call may change A, X and Y, so a loop that counts with X keeps it on the stack over the call:

```
            ldx #3              ; 3 times
loop:       phx                 ; Keep X ...
            lda #'*'
            jsr PUTC            ;   (PUTC may change it)
            plx                 ;   and have it back
            dex
            bne loop
```

Every push needs a pull. A subroutine that returns with more (or fewer) bytes on the stack than it found will return to the wrong place. The stack has room for 256 bytes; a program starts with S = `$FD`.

26.11 Interrupts

An **interrupt** is a signal from a chip that something has happened: a key has come in, a timer has run out. The 65C02 finishes its instruction, pushes PC and P on the stack, sets I, and jumps to an **interrupt handler**; the handler deals with the chip, and `RTI` (return from interrupt) pulls P and PC, so the program carries on as if nothing had happened.

INSTRUCTIONS	WHAT THEY DO
`SEI`, `CLI`	Set I (hold interrupts off), clear I (let them in)
`BRK`	A **software interrupt**: like an interrupt, but from an instruction
`RTI`	Return from an interrupt
`WAI`	Wait (asleep) for an interrupt
`STP`	Stop the processor until the next reset

The 6502 has one interrupt line, so its handler has to ask every chip "was it you?". The Hydra does better: its sixteen interrupt lines (line 15 has nothing on it) go to a **priority encoder** (U17 and U18), and its **vector RAM** gives each line its own handler's address, so the 65C02 goes straight to the right one. Line 0 (the VIA, which makes the tick) comes first, then the serial port, the Vera X, the YM2151 and the slots.

On the Hydra, the interrupts are HydraOS's. They're how it shares the processor between the tasks (the tick, two hundred times a second), and how its drivers hear from their chips. So a program:

- leaves the I flag alone. (To keep the processor to itself for a moment, it uses the `PREEMPT_OFF` and `PREEMPT_ON` system calls.)
- never writes the Hydra's registers T, U, V and W, the vector RAM, or HydraOS's memory.
- never uses `STP`, which stops the whole Hydra, every task, until you press RESET.

A `BRK` in a program doesn't crash the Hydra: HydraOS sends the program a **note**, `sys: brk` (section 27.16 has more on notes), which ends it unless it has a note handler. The debugger uses `BRK` for its breakpoints (Chapter 29).

26.12 How fast?

The 65C02 does its work in **cycles**, ticks of its clock. On the Hydra, the clock is 3.58 MHz: 3,579,545 cycles a second (the jumper J7, section 4.6). Each instruction takes from 2 to 7 cycles. `LDA #` takes 2, about half a millionth of a second; `JSR` takes 6. Appendix D gives every instruction's cycles. So the Hydra does very roughly a million instructions a second, shared between its tasks. Between two ticks of HydraOS's clock (200 a second) there are about 17,900 cycles.

Machine code is the fastest a program can be. HydraOS's twenty benchmark programs (on the SD card, in `/sd/0/bench`) run about 60 times faster written in assembly language, in HyBASIC's `ASM` blocks, than in HyBASIC itself.

26.13 Learning more

These are good places to go on from here:

- **WDC's W65C02S data sheet** (from wdc65xx.com): every instruction, its cycles and its flags, and the chip's pins and timing.
- *Programming the 65816* by David Eyes and Ron Lichty: the 65C02's instructions explained in depth, with the 6502's and the 65816's.
- *6502 Assembly Language Programming* by Lance Leventhal, and *Programming the 6502* by Rodnay Zaks: two classics from the 1980s, written for the 6502 (the 65C02 adds to it, and has fixed its few bugs).
- *Machine Language for the Commodore 64* by Jim Butterfield: a gentle start, for Commodore owners, but the 6502 is the same.

The 65C02 has a few instructions the first 6502 didn't: `BRA`, `PHX`, `PHY`, `PLX`, `PLY`, `STZ`, `TRB`, `TSB`, `WAI`, `STP`, the bit instructions `RMB`, `SMB`, `BBR` and `BBS`, the zero page indirect mode (`lda ($22)`), `INC A` and `DEC A`, more modes for `BIT`, and `JMP ($1234,x)`. So a book about the 6502 teaches you everything but these.

CHAPTER 27

The Assembler

An **assembler** turns assembly language into machine code. The Hydra's is called `as`. It reads a **source file** (a text file of instructions, usually with a name ending in `.s`) and writes a **program**: a file that the Hydra can run like any other command. Its language is the one of **ca65**, the assembler of the cc65 package that's used for 6502 programs on PCs, so programs (and books about ca65) work on both.

IN THIS CHAPTER: You'll write, assemble and run programs in assembly language, learn the whole of `as`'s language, call HydraOS's system calls, and put machine code inside HyBASIC and HyForth programs. **You need:** Chapter 26, and the screen editor (section 6.6).

```
as [-bl] file.s|- [out|-]
```

OPTION	WHAT IT DOES
`-l`	Write a **labels file** too, *out*`.lbl`, for the debugger and the disassembler (section 27.17)
`-b`	Write only the bytes, without a program's header (section 27.17)
out	The program's name (without it: *file.s*'s name without its `.s`)
`-`	In place of *file.s*: read the source from standard input, a pipe or a < file. In place of *out*: write the program to standard output (section 27.17)

27.1 Your first program

Here's a whole program in assembly language: about twenty instructions that print a triangle of stars. Go to `/ram` (`cd /ram`), start `edit stars.s`, and type it in. The columns don't have to line up as neatly as these do: one space is as good as ten.

```
; stars - a triangle of stars
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "stars", main

ROWS        = 5                 ; How many rows

.zeropage
row:        .res 1              ; The row it's on: 1 to ROWS
count:      .res 1              ; The stars still to print on it

.code
main:       lda #1              ; The first row
            sta row
@row:       lda row             ; As many stars as the row's number
            jsr line
            inc row             ; The next row, till the last is done
            lda row
            cmp #ROWS + 1
            bne @row
            rts

; A line of stars: .A of them (1 or more)
line:       sta count
@star:      lda #'*'
            jsr PUTC
            dec count
            bne @star
            lda #LF             ; The line's end
            jsr PUTC
            rts
```

Save it (CTRL-O, then RETURN), leave `edit` (CTRL-X), and assemble and run it:

```
/ram> as stars.s
/ram> ./stars
*
**
***
****
*****
```

`as stars.s` said nothing, which means it worked: it made the program `stars`. `./stars` runs the `stars` in this directory. Now look at it in pieces:

- A `;` starts a **comment**: everything after it on the line is for people, and `as` skips it.
- `.include "hydra.inc"` reads in another file of source: `hydra.inc` gives the names of HydraOS's system calls and constants, such as `PUTC`; `hyx2.inc` has `HYX2_PROGRAM`; and `macros.inc` has `LF`, and the macros that the next section's program uses. They're all in `/lib/as`.
- `HYX2_PROGRAM "stars", main` makes the program's **header**: its name is `stars`, and it starts at `main`.
- `ROWS = 5` makes a **constant**: from then on, `ROWS` means 5.
- `.zeropage` and `.code` say which **segment** the lines after them go into (section 27.8): the variables on the zero page, and the code. `row` and `count` are the variables: `.res 1` **reserves** a byte for each.
- `main:`, `line:` and `@row:` are **labels**: names for places in the program (section 27.5).
- `main` puts 1 in `row`, then goes round a loop: it calls the subroutine `line` with the row's number in A, adds 1 to `row`, and goes round again (`bne @row`) until `row` is `ROWS + 1`.

- `line` prints A stars. `jsr PUTC` is a **system call**: it asks HydraOS to do something, here to print the character in A (section 27.13). A system call may change A, X and Y, so `line` keeps its count in `count`, not in X. `dec count` takes 1 from it and sets Z when it reaches 0, so `bne @star` goes round again until it does.
- `LF` is the code of a new line, 10.
- A program ends when its `main` returns: the last `rts`.

If you make a mistake, `as` tells you where (section 27.18). Fix it in `edit`, and assemble again.

> **TRY THIS:** Change `ROWS` to 10, assemble again, and run it. Then change the `'*'` to another character. Can you make the triangle point the other way, with the longest row first? (Start `row` at `ROWS`, and count down.)

27.2 A bigger program: hi.s

The SD card has some sample programs, with their sources, in `/sd/0/sample/as`. Copy `hi.s` into `/ram`, assemble it, and run it:

```
/> cd /ram
/ram> cp /sd/0/sample/as/hi.s .
/ram> as hi.s
/ram> ./hi Ann Bob
Hello, Ann!
Hello, Bob!
I'm task 2, in /ram.
/ram> ./hi
Hello, you!
I'm task 2, in /ram.
```

Run with names, `hi` greets each one; without, it greets you. Then it says which task it's running in, and its current directory. (`./hi` runs the `hi` in this directory: just `hi` would look for it in `/bin` too.)

27.3 Inside hi.s

Here is `hi.s`. It's longer than `stars.s`, and it uses more of what `as` can do. Don't worry if parts of it look strange: the rest of this chapter explains every one.

```
; ****************************************************************************
; hi [name ...] - a sample RAM program (the assembly SDK's: sdk/asm/README.md).  It greets each name
it's given (or
; you), then says where it runs: its task (GETPID), its current directory (GETCWD) and its window
($window, from its
; environment: ENV_GET).
;   % hi Ann Bob
;   Hello, Ann!
;   Hello, Bob!
;   I'm task 3, in /ram, in window 0.

.include "hydra.inc"                                    ; The calls and constants (made from
spec/api.def)
.include "hyx2.inc"                                     ; The header: HYX2_PROGRAM
.include "macros.inc"                                   ; LDR, MOVR, PRINT, CALL, CHECK

            HYX2_PROGRAM "hi", main                     ; Its name, and where it starts

.zeropage
arg:        .res        2                               ; An argument (its zero page: $22-$7F)

.bss
cwd:        .res        PATH_MAX + 1                    ; Its current directory
window:     .res        8                               ;   and window

.code

; Its arguments at r0: strings one after another, each zero-terminated, an empty one after the last.
Returning
; ends it, with code 0 (EXITS)
main:
            MOVR        arg, r0
            lda         (arg)
            bne         @name
            LDR         arg, s_you                      ; None: you
@name:
            PRINT       "Hello, "                       ; "Hello, NAME!"
            MOVR        r0, arg
            CALL        PUTS
            PRINT       s_bang
:
            lda         (arg)                           ; The next: past this one's 0
            inc         arg
            bne         :+
            inc         arg + 1
:
            cmp         #0
            bne         :--
            lda         (arg)
            bne         @name
            PRINT       "I'm task "                     ; Its task: 0-15, in decimal
            CALL        GETPID
            cmp         #10
            bcc         :+
            pha
            lda         #'1'
            CALL        PUTC
            pla
            sec
            sbc         #10
:
            ora         #'0'
            CALL        PUTC
            PRINT       ", in "                         ; Its current directory
            LDR         r0, cwd
            CALL        GETCWD
```

```
            PRINT       cwd
            LDR         r0, s_window                    ; Its window, if its environment has one
            LDR         r1, window
            LDR         r2, 7
            stz         r3
            stz         r3 + 1
            lda         #$FF                            ; (This task's)
            CALL        ENV_GET
            CHECK       @end                            ; (Not there: C = 1, .A = E_NOENT)
            tax
            stz         window,X                        ; (Its value: zero-terminated)
            PRINT       ", in window "
            PRINT       window
@end:
            PRINT       s_end
            rts

.rodata
s_you:      .byte       "you", 0, 0
s_bang:     .byte       "!", LF, 0
s_window:   .byte       "window", 0
s_end:      .byte       ".", LF, 0
```

What's new here:

- `macros.inc`'s **macros** `LDR`, `MOVR`, `PRINT`, `CALL` and `CHECK` (section 27.11). A macro is a name for several instructions: `PRINT "Hello, "` is the instructions that print that string.
- `.bss` and `.rodata` are two more segments (section 27.8): the BSS's buffers `cwd` and `window`, and the read-only data (the strings).
- A label alone on a line, `:`, has no name: `:+` means the next one, and `:--` the one two back.
- A program starts with its arguments in r0: `main` walks along them, greeting each.
- `CALL PUTS` (the same as `jsr PUTS`) is another system call: it prints the string at r0.
- The comment at the top shows a run that ends `in window 0.hi` prints that part only when its environment has a variable called `window`, and looks for it with the system call `ENV_GET`. In the run above there was none, so `CHECK @end` skipped the last two `PRINT`s.

27.4 What's on a line

Each line of a source can have, in this order, any of:

1. a **label**, ending with a colon: `main:`
2. a **statement**: an instruction (`lda #1`), a **directive** (`.byte 1, 2, 3`, starting with a dot), the use of a macro (`PRINT "Hello"`), or a constant (`ROWS = 5`)
3. a **comment**, from a `;` to the end of the line.

Spaces and tabs separate the parts; how many doesn't matter, so line your columns up as you like. Instruction names and directives can be written in capitals or small letters (`LDA`, `lda`). Names are different: `Count` and `count` are two different names. A line can be 255 characters long.

Instructions are written as in section 26.6: `lda #$41`, `sta $22,x`, `lda (r0),y`, `jmp ($1234,x)`, `asl a`. When an address is under `$100`, `as` uses the shorter zero page form. To force the 3-byte form, write `a:` before it (`lda a:$12`); to insist on the zero page form, `z:`.

27.5 Labels

A label names the place it's at. There are three kinds:

KIND	WRITTEN	ITS NAME IS SEEN
A **label**	`main:`	Everywhere in the program
A **cheap local** label	`@loop:`	Only between the two normal labels around it
An **unnamed** label	`:`	Through `:+` (the next one), `:++` (the one after that), `:-` (the last one), `:--` ...

Cheap locals let every routine have its own `@loop` and `@done`, without inventing a new name each time. Unnamed labels are for little hops of a line or two:

```
            lda count
            bne :+              ; (Not zero: past the inx)
            inx
:           rts
```

A name is letters, digits and _ (and a cheap local's `@` in front), starting with a letter or _, and up to 32 characters long.

27.6 Numbers and expressions

Numbers can be written in four ways:

WRITTEN	MEANS
`65`	Decimal
`$41`	Hexadecimal
`%01000001`	Binary
`'A'`	A character's code: 65

Wherever a number goes, an **expression** can go instead: numbers, names and operators. `as` works them out in 32 bits, as signed numbers. `*` alone means the address the statement is at.

OPERATOR	MEANS	EXAMPLE
`+ - * /`	Add, subtract, multiply, divide (whole numbers)	`ROWS + 1`
`.mod`	The remainder	`100 .mod 7` is 2
< (before)	The low byte	`<$1234` is `$34`
> (before)	The high byte	`>$1234` is `$12`
^ (before)	The bank byte (bits 16-23)	
`& | ^`	AND, OR, XOR of the bits	`$F0 & $3C` is `$30`
~	NOT of the bits	
<< >>	Shift left, right	`1 << 4` is 16
= <> < > <= >=	Compare: 1 if it's so, 0 if not	`2 < 3` is 1
`&& || !`	And, or, not (of true and false)	
( )	Brackets	`(ROWS + 1) * 2`

The operators are worked out in this order (highest first): the ones before a number (`-`, `~`, `<`, `>`, `^`, `!`); then `*`, `/`, `.mod`, `&`, `^`, `<<`, `>>`; then `+`, `-`, `|`; then the comparisons; then `&&` and `.xor`; and last `||`. The words `.bitand`, `.bitor`, `.bitxor`, `.bitnot`, `.shl`, `.shr`, `.and`, `.or` and `.not` can be used instead of the signs.

Functions work on what's in brackets after them:

FUNCTION	GIVES
`.lobyte(x)`, `.hibyte(x)`, `.bankbyte(x)`	Byte 0, 1 or 2 of *x*
`.loword(x)`, `.hiword(x)`	The low or high 16 bits of *x*
`.strlen("text")`	A string's length
`.defined(name)`, `.def(name)`	1 if *name* has been defined, else 0
`.blank(x)`	1 if nothing's in the brackets (for macros)
`.match(a, b)`	1 if *a* and *b* are the same kinds of thing (for macros)

27.7 Constants

`name = expression` makes a **constant**: a name for a number. `name := expression` is the same. Constants make programs easier to read and to change:

```
ROWS        = 5                 ; How many rows
SCREEN_W    = 80
MIDDLE      = SCREEN_W / 2
```

A constant isn't stored in the program: `as` puts its value wherever its name is used. `hydra.inc` is mostly constants: `PUTC = $F950` is the address of the `PUTC` system call.

27.8 Segments: where everything goes

A program has several parts, called **segments**. `as` keeps each one's lines together, wherever they are in the source, and puts the segments in order when it writes the program:

SEGMENT	DIRECTIVE	WHAT IT'S FOR	WHERE IT GOES
`ZEROPAGE`	`.zeropage`	Variables on the zero page (`.res` only)	From `$22`, up to `$7F`
`HEADER`	(`HYX2_PROGRAM`)	The program's header	From `$0800` (48 bytes)
`CODE`	`.code`	Instructions	After the header
`RODATA`	`.rodata`	Data that doesn't change: strings, tables	After the code
`DATA`	`.data`	Data that starts with a value but may change	After the read-only data
`BSS`	`.bss`	Variables that start as 0 (`.res` only)	After the data: not in the file

A source starts in `CODE`. `.segment "NAME"` is the long way to switch, and `.pushseg` and `.popseg` switch to another segment for a while and back. The BSS isn't kept in the program's file: HydraOS clears it to zeros as it loads the program, so a program with a big buffer is still a small file. (`hi`'s file is 326 bytes; its BSS, 72 bytes more, isn't in it.)

`as` also defines the names a linker would, for programs that want to know where their parts went: `__DATA_LOAD__`, `__DATA_RUN__` and `__DATA_SIZE__` (the data's place and size), `__BSS_RUN__` and `__BSS_SIZE__`, `__RAM_START__` (`$0800`), `__RAM_SIZE__` and `__RAM_LAST__` (the end of the program's memory), `__ZP_START__`, `__ZP_SIZE__` and `__ZP_LAST__`. `HYX2_RAM` is defined too, which tells `hyx2.inc` that this is a program in a file.

A program can be 30K at most: from `$0800` to `$7FFF`, its BSS included. It has 94 bytes of zero page (`$22` to `$7F`).

27.9 Data

DIRECTIVE	WHAT IT PUTS IN THE PROGRAM
`.byte` *x*, ... (or `.byt`)	Bytes: numbers from -128 to 255, and strings (`"text"`), a byte for each character
`.asciiz "text"`, ...	Strings, each with a 0 after it
`.word` *x*, ... (or `.addr`)	16-bit numbers, two bytes each, low byte first
`.dword` *x*, ...	32-bit numbers, four bytes each, lowest first
`.res` *count* [, *fill*]	*count* bytes, all *fill* (0 if it's not given)
`.incbin "file"` [, *start* [, *count*]]	A file's bytes (from *start*, *count* of them)

```
.rodata
greeting:   .byte "Hello!", LF, 0   ; A string, a new line, and the 0 that ends it
squares:    .byte 0, 1, 4, 9, 16, 25
jumps:      .word main, line        ; Two addresses
.bss
buffer:     .res 256                ; 256 bytes, cleared as it starts
```

A string in HydraOS ends with a 0 byte: that's how `PUTS` knows where it stops. A string can't have a `"` in it; use its code, `34`, in a `.byte` instead.

27.10 Including files

`.include "file"` reads another source file in, as if its lines were in this one. `as` looks for the file as it's named, then in the directory of the file that includes it, and then in `/lib/as`, which has the **SDK**'s files (the software development kit's: the files for writing programs):

FILE	WHAT IT HAS
`hydra.inc`	Every system call's address, every error code, and HydraOS's constants (Appendix K)
`hyx2.inc`	The header macros: `HYX2_PROGRAM` for a program, `HYX2_DRIVER` for a driver, `HYX2_LIBRARY`
`macros.inc`	`LDR`, `MOVR`, `PRINT`, `CALL`, `CHECK`; and `CR`, `LF`, `TAB`
`toollib.inc`, `toollib.s`	The tools' library: flags, files, output (section 27.16)
`numbers.inc`, `numlib.s`	The numbers and math libraries (section 27.16)
`asmlib.inc`	The assembler library itself, for programs that assemble or disassemble
`srvlib.inc`, `srvlib.s`	For writing a **server**: a driver with files of its own
`nslib.s`	Building a namespace from a namespace file, as init does

A `.s` file of the SDK is included at the **end** of a program's source (its code is put after the program's own), and its `.inc` at the top.

27.11 Macros

A **macro** is a piece of source with a name. Using the name puts the piece in, as if you'd typed it. Macros can have **parameters**: names that are replaced by what's written after the macro's name when it's used.

```
; mac - a macro of its own
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "mac", main

; SAY char, times: the character, times times
.macro SAY char, times
            ldx #times
:           lda #char
            phx                 ; (PUTC may change .X: kept on the stack)
            jsr PUTC
            plx
            dex
            bne :-
.endmacro

.code
main:       SAY 'H', 1
            SAY 'o', 5
            SAY '!', 3
            SAY LF, 1
            rts
```

```
/ram> as mac.s
/ram> ./mac
Hooooo!!!
```

`.macro NAME param, ...` starts one (up to 9 parameters), and `.endmacro` ends it. `.exitmacro` leaves it early (in an `.if`, section 27.12). An unnamed label inside a macro is fine, as here; but a normal label would be defined twice if the macro were used twice.

The SDK's macros, in `macros.inc`, make the commonest jobs shorter:

MACRO	WHAT IT DOES	THE SAME AS
`LDR r, value`	Put a 16-bit value (an address, often) in a call register	`lda #<value`, `sta r`, `lda #>value`, `sta r+1`
`MOVR to, from`	Copy one zero page word (a call register) to another	Two `lda`s and two `sta`s
`PRINT label`	Print the string at *label*	`LDR r0, label` and `jsr PUTS`
`PRINT "text"`	Print the text (kept in the code, with a jump over it)	
`CALL name`	A system call	`jsr name`
`CHECK label`	After a call: if it failed (C = 1), go to *label*	`bcc` past a `jmp label`

Note that `LDR`, `MOVR` and `PRINT` change A (and `PRINT` changes Y and r0 too).

27.12 Conditions and checks

Conditional assembly assembles some lines only if something's so:

```
.if ROWS > 9
            .warning "a big triangle"
.elseif ROWS = 0
            .error "no rows"
.else
            ...
.endif
```

DIRECTIVE	ASSEMBLES THE LINES AFTER IT IF
`.if` *expr*	*expr* isn't 0
`.ifdef` *name*, `.ifndef` *name*	*name* is defined; isn't
`.ifblank` ..., `.ifnblank` ...	Nothing follows it on the line; something does (for a macro's parameters)
`.elseif` *expr*	The ones before weren't, and *expr* isn't 0
`.else`	The ones before weren't
`.endif`	(The end)

`.if`s can be inside each other, 16 deep. Three directives check things:

DIRECTIVE	WHAT IT DOES
`.assert` *expr*, `error`, "message"	Says the message as an error if *expr* is 0 (or `warning`, as a warning)
`.error "message"`	Says the message as an error: `as` makes no program
`.warning "message"`	Says it as a warning: `as` goes on

`hyx2.inc` uses `.assert` to check that a program's name is short enough. `as` also takes, and ignores, ca65's directives that don't matter here (`.setcpu`, `.import`, `.export`, `.global`, `.macpack`, `.feature`, `.list`, `.case`, `.smart` and others), so ca65's sources assemble.

27.13 System calls

A **system call** asks HydraOS to do something: print, open a file, start a program, wait. Each has an address in HydraOS's **jump table**, from `$F800` up, and a program calls it with `jsr`. The rules are the same for all of them:

- The inputs go in A, X, Y and the call registers r0 to r15 (each 2 bytes, low byte first, at `$02` to `$21`).
- The answers come back in the same places. A 16-bit answer comes back in A (its low byte) and X (its high byte).
- **C = 0** means it worked. **C = 1** means it failed, and A holds the **error code** (Appendix G).
- A call may change A, X, Y, r0 to r15 and the flags. It never changes `$22` to `$7F`: the program's zero page is safe.

Appendix K lists every call. The ones a first program needs:

CALL	INPUTS	WHAT IT DOES
`PUTC`	A = a character	Write it to the standard output (the screen)
`PUTS`	r0 = a string	Write it (up to its 0)
`GETC`		Read a byte from the standard input (a key): A = it
`OPEN`	r0 = a file's name; A = `O_READ`, `O_WRITE` or `O_RDWR`	Open a file: A = its **fd** (file descriptor)
`READ`	A = an fd; r0 = a buffer; r1 = how many bytes	Read: A/X = how many came (0 at the end)
`WRITE`	A = an fd; r0 = the bytes; r1 = how many	Write them
`CLOSE`	A = an fd	Close it
`ERRSTR`	A = an error code; r0 = a 32-byte buffer	The error's message, in the buffer
`SLEEP`	A/X = how many ticks (200 a second)	Wait
`EXITS`	A = the exit code; r0 = a message, or 0	End the program

Every program has three fds open as it starts: 0, the **standard input** (the keyboard, usually); 1, the **standard output** (the screen); and 2, the **standard error** (for error messages, usually the screen too).

Arguments. A program starts at its `main` with r0 pointing at its arguments: each one a string ending in 0, one after the other, with an empty string (a lone 0) after the last. `hi.s` shows how to walk along them.

Ending. When `main` returns (`rts`), the program ends with exit code 0, which means success. To end with another code, and a message, put them in A and r0 and `jmp EXITS`. rc's `$status` shows the message (section 8.9).

27.14 Reading a file

`show.s` is a little `cat`: it prints the file it's given. It opens the file, then reads 256 bytes at a time and writes them to fd 1 until a read gives 0 bytes, which means the end. If any call fails, it prints the error's message and ends with code 1.

```
; show file - a file's text, on the screen
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "show", main

.zeropage
fd:         .res 1              ; The file, open

.bss
buf:        .res 256            ; A piece of it
msg:        .res 32             ; An error's text

.code
main:       lda (r0)            ; The file's name: r0's first argument
            bne @open
            PRINT "usage: show file"
            lda #LF
            jsr PUTC
            rts
@open:      lda #O_READ         ; Open it, to read
            jsr OPEN
            bcs @fail           ; (C = 1: it failed, .A the error)
            sta fd
@read:      LDR r0, buf         ; 256 bytes of it at most
            LDR r1, 256
            lda fd
            jsr READ
            bcs @fail
            sta r1              ; The count read: .A (low), .X (high)
            stx r1 + 1
            ora r1 + 1
            beq @done           ; (0: the file's end)
            LDR r0, buf         ; Those bytes, to fd 1: the screen
            lda #1
            jsr WRITE
            bcs @fail
            bra @read
@done:      lda fd
            jsr CLOSE
            rts
@fail:      pha                 ; The error's text (LDR changes .A: kept)
            LDR r0, msg
            pla
            jsr ERRSTR
            PRINT "show: "
            PRINT msg
            lda #LF
            jsr PUTC
            lda #1              ; The end, with code 1
            stz r0
            stz r0 + 1
            jmp EXITS
```

```
/ram> as show.s
/ram> echo Roses are red >poem
/ram> echo Violets are blue >>poem
/ram> ./show poem
Roses are red
Violets are blue
/ram> ./show nothing
show: not found
/ram> ./show
usage: show file
```

Look at the start of `@fail`. `LDR` uses A, so the error code in A would be lost: `pha` keeps it on the stack, and `pla` brings it back for `ERRSTR`. Mistakes like this are easy to make, and the debugger (Chapter 29) is how you find them.

27.15 More memory: RAM banks

A program's 30K isn't much. But each task also has its own **RAM banks**, 8K each: 16 on each RAM module of the memory card (48 on the book's Hydra). A program asks HydraOS for banks with `BANKS_ALLOC`, and writes a bank's number to address `$00` to put it at `$8000`-`$9FFF`. `BANKS_FREE` gives banks back (HydraOS takes them back anyway when the program ends).

```
; bank - a RAM bank of its own
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "bank", main

.zeropage
mine:       .res 1              ; The bank it's given

.code
main:       lda #1              ; One bank, please
            jsr BANKS_ALLOC
            bcs @none           ; (C = 1: none free)
            sta mine
            sta $00             ; It's at $8000-$9FFF now
            ldx #0              ; The message copied into it ...
:           lda message,x
            sta $8000,x
            beq :+              ;   (its 0 too)
            inx
            bra :-
:           LDR r0, $8000       ;   and printed from there
            jsr PUTS
            lda mine            ; The bank given back
            ldx #1
            jsr BANKS_FREE
            rts
@none:      PRINT "no bank free"
            rts

.rodata
message:    .byte "Hello from a RAM bank!", LF, 0
```

```
/ram> as bank.s
/ram> ./bank
Hello from a RAM bank!
```

Only one bank is at `$8000` at a time; to use another, write its number to `$00`. The `BANKS` call says how many banks the task has. For more memory in the program's own 32K, `BREAK` moves the program's **break** (the end of its memory, after its BSS) further up.

27.16 The SDK's libraries

The SDK has libraries that save writing everything yourself. Each sample on the SD card shows one:

The tools' library. `toollib.s` does what every command needs: its options (`-x`), its files (or the standard input, with none), its output (written 256 bytes at a time, which is much quicker than `PUTC`), and its error messages, in the same form as the Hydra's own tools. The `upper` sample (`/sd/0/sample/as/upper.s`) is a whole tool in a few lines: it prints its files in

capitals.

ROUTINE	WHAT IT DOES
`tl_start`	Reads the options (the letters in `tl_flagset`) into `tl_flags`; `tl_arg` points at the first argument after them
`tl_next`	On to the next argument (A = its first byte: 0 if there's none)
`tl_eachin`	For each file named (or the standard input), calls the routine at `tl_ivec`, which reads it with `tl_getc`
`tl_getc`	The next byte of the file in A (C = 1 at its end)
`tl_putc`, `tl_puts`, `tl_nl`, `tl_space`	Output: a character, the string at r0, a new line, a space
`tl_dec`	Print the 4-byte number in `tl_num` in decimal
`tl_err`, `tl_warn`	Say an error about a name, or a message: "*tool*: ..." on fd 2, and set the exit code to 1
`tl_end`	Write the output, and end, with the exit code
`tl_badusage`	Say "usage: ..." (`tl_usage`), and end

A tool defines `tl_name`, `tl_flagset` and `tl_usage`, includes `toollib.inc` at its top and `toollib.s` at its end.

The numbers libraries. `numbers.inc` and `numlib.s` give assembly programs Chapter 18's numbers: integers of any size, fractions, decimals and complex numbers, read and written in any base. The `nsum` sample adds up the numbers it's given, and gives the square root too:

```
/> /sd/0/sample/nsum 1/3 0.5 2
17/6
sqrt 1.68325082306
```

Notes. A **note** is a message to a task from HydraOS or another task: CTRL-C sends the `interrupt` note to the programs in its window. Most notes end a program, but a program can catch them with a **note handler**, set with the `NOTIFY` call. The `tick` sample prints a dot a second until you press CTRL-C, and then, instead of just stopping, says how many seconds it ran.

Programs in C. The SDK has a C side too: with the cc65 compiler on your PC (section 2.7), `reborn/sdk/c` builds C programs for the Hydra, and the samples in `/sd/0/sample/c` (`chorus`, `race`, `jukebox` and the rest) were made that way. Its `README.md` says how.

27.17 as's options and the labels file

The program's name. `as stars.s` makes `stars`; `as stars.s /bin/stars` puts it straight in `/bin` (in fact in `/ram/bin`, where new files in `/bin` go: section 9.7), so the shell finds it as plain `stars`, from any directory. A source whose name doesn't end with `.s` needs the output's name given.

Pipes. A `-` in place of the source reads it from standard input, so `as` can take the end of a pipe: `dis stars | as - stars2` makes `stars2` from `stars`'s disassembly (Chapter 28), without a source file in between. With a source `-` the program goes to standard output unless you name it, and a `-` in place of *out* sends it there anyway: `as stars.s - > stars`. `-l` needs a named output, since the labels file takes its name from it.

-l: the labels file. `as -l stars.s` also writes `stars.lbl`, listing the address of every label:

```
/ram> as -l stars.s
/ram> cat stars.lbl
al 000023 .count
al 000830 .main
al 000022 .row
al 000842 .line
```

Each line is `al`, the address in hex, and the label's name after a dot. (It's the form the ld65 linker writes with its `-Ln` option, so the same tools read both.) The debugger and the disassembler read it to show names instead of addresses. Cheap local labels aren't in it.

-b: bytes alone. `as -b` writes only the machine code, with no header, as a **raw** file. Its code starts at the address given by `.org` (or `$0800` if there's no `.org`), which must come before anything else that makes bytes. A raw file isn't a program you can run by name: it's machine code for something else to load, such as a ROM, or a routine that another program reads in.

```
; A routine on its own, at $1000: no header (as -b)
.org $1000
double:     asl a               ; .A times 2 ...
            bcc :+
            inx                 ;   (its carry into .X)
:           rts
```

```
/ram> as -b raw.s
/ram> xd raw
0000000  0a 90 01 e8 60                                    ....`
```

The five bytes are `asl a` (`$0A`), `bcc` (`$90`) and its distance (1: past one byte), `inx` (`$E8`) and `rts` (`$60`).

27.18 When as finds a mistake

`as` says each mistake as *file*:*line*: *what*, and makes no program:

```
; bad - a program with mistakes in it, to see what as says
.include "hydra.inc"
.include "hyx2.inc"

            HYX2_PROGRAM "bad", main

.code
main:       lda #300            ; A byte is 0 to 255
            ldx #10
@loop:      jsr PUTCH           ; (PUTC, spelt wrong)
            dex
            bne @loop
            rts
```

```
/ram> as bad.s
as: bad.s:10: undefined: PUTCH
```

`PUTCH` should be `PUTC`. Fix it and assemble again, and `as` finds the next mistake:

```
/ram> as bad.s
as: bad.s:8: not a byte
```

Line 8 is `lda #300`: a byte only goes up to 255. `as` reads a source three times (section 27.19), and it stops after the first reading that finds a mistake, so a mistake can hide another until it's fixed. The messages you'll see most:

MESSAGE	WHAT'S WRONG
`undefined: NAME`	A name that isn't defined anywhere: often a spelling mistake, or a missing `.include`
`not an instruction, directive or macro: NAME`	A misspelt instruction, or a line that doesn't start where you think
`not a byte`	A number that should be a byte (from -128 to 255) isn't
`an addressing mode the instruction doesn't have`	Like `stz ($22),y`: the 65C02 has no such instruction (Appendix D)
`a bad operand`	What follows the instruction isn't an address, a number or a mode it knows
`a branch too far`	A branch to more than 128 bytes away: use `jmp` (or a branch over a `jmp`)
`defined twice`	Two labels with the same name
`not found: FILE`	An `.include` (or the source) that isn't there
`a string with no closing quote`	A `"` missing
`more on the line than it takes`	Something extra at the end of a line
`an .if with no .endif, a .macro with no .endmacro`	A block not closed
`data in BSS or the zero page`	Only `.res` can go in those segments
`too big: past $8000`	The program (with its BSS) is bigger than 30K
`the zero page's $5E bytes overrun`	More zero page variables than fit in `$22` to `$7F`
`a segment's size changed between passes`	A size that depends on a name defined later (section 27.19)
`an assertion failed`	An `.assert` (`HYX2_PROGRAM`'s, say: a name longer than 11 characters)

27.19 How as works

`as` reads the source three times, called **passes**:

1. The first pass finds how big each segment is. It can't know yet where every label will be.
2. The second finds every label's and constant's value.
3. The third makes the bytes.

Why three? Because a program can use a name before it's defined: `bne @star` where `@star` comes later, or `jsr line` before `line`. The first pass can't know where `line` is, so it leaves room for it; the second fills in the places; the third has everything.

There's one catch. An instruction with a zero page address is a byte shorter than one with an absolute address, so `as` must decide which to use the same way in every pass. It uses the zero page form only for an address it already knows (from earlier in the source) is under `$100`. So define zero page variables at the top of the program, before the code that uses them, as the samples do; one defined later is reached with the longer form, which works but is slower.

`as` keeps its symbols, its macros, the files it reads and the program it makes in RAM banks, so a big source fits. It has some limits: lines of 255 characters, names of 32, files and macros 16 inside each other, `.if`s 16 deep, 9 parameters to a macro, and 30 errors (then it stops).

`as` is a small program; the assembler itself is the **asm library**, a module in the ROM. HyBASIC uses the same library for its `ASM` blocks (section 27.20), and the disassembler and debugger use its other half to show instructions, so all of them write instructions exactly the same way.

27.20 Assembly language in HyBASIC

HyBASIC can have assembly language inside a program. The lines between `ASM` and `END ASM` (each alone on its line) are given to the assembler when the program runs, and `CALL ASM label` calls the code at one of their labels:

```
> 10 REM Stars, from an ASM block
> 20 ASM
> 30 .include "hydra.inc"
> 40 stars:  lda #'*'          ; X stars, then a new line
> 50         phx
> 60         jsr PUTC
> 70         plx
> 80         dex
> 90         bne stars
> 100        lda #10
> 110        jsr PUTC
> 120        rts
> 130 twice:  lda n + 1         ; N's low byte, times 2
> 140         asl a
> 150         rts
> 160 END ASM
> 170 FOR N = 1 TO 5
> 180   CALL ASM stars, 0, N
> 190 NEXT
> 200 N = 21
> 210 CALL ASM twice
> 220 RREG A, X, Y, P
> 230 PRINT "twice"; N; "is"; A
> RUN
*
**
***
****
*****
twice 21 is 42
```

- `CALL ASM label, a, x, y` sets A, X and Y (all optional) before calling. Here X is the number of stars.
- `RREG a, x, y, p` copies the registers, as they were when the code returned, into four variables.
- An `ASM` block can use the program's **variables**: each number variable's name (in capitals or in small letters) is the address of its value (the main program's variables, not a `SUB`'s own), and each integer `CONST`'s name is its value. A whole number's value is 5 bytes: a 0, then the number, 4 bytes, low byte first. So `n + 1` is the low byte of `N`.
- `.include "hydra.inc"` gives the system calls' names, as in a program.
- All of a program's `ASM` blocks are assembled together into 8K of their own, a RAM bank, at `$8000`. A mistake in one is said as a HyBASIC error, at its line.

`SYS address [, a [, x [, y]]]` calls machine code anywhere in memory, as `CALL ASM` does.

27.21 Machine code from HyForth

HyForth's `sys` (*addr a x y -- a x y p*) calls machine code at *addr* with A, X and Y set, and gives back the registers and the flags after it (`lib hydra` has it). The machine code can be anywhere: here it's made with `c,` right in the dictionary, a byte at a time, and `disasm` (Chapter 28) shows it:

```
/> lib hydra
/> hex
/> create hi  a9 c, 48 c, 20 c, 50 c, f9 c,
/> a9 c, 69 c, 20 c, 50 c, f9 c, 60 c,
/> decimal
/> hi 0 0 0 sys . . . .
Hi48 0 0 105
/> lib disasm
/> hi 5 disasm
 4658  A9 48     lda #$48
 465A  20 50 F9  jsr $F950
 465D  A9 69     lda #$69
 465F  20 50 F9  jsr $F950
 4662  60        rts
```

The bytes are `lda #'H'`, `jsr PUTC`, `lda #'i'`, `jsr PUTC` and `rts`. After it ran, `.` printed the flags (48), Y, X and A (105, the code of `i`, which it was left holding).

A file made with `as -b` can be read into memory with HyForth's file words, and called the same way. Here is `raw.s`'s routine from section 27.17, which doubles A (its carry going into X), given 200:

```
/ram> lib hydra
/ram> create code 16 allot
/ram> s" /ram/raw" r/o open-file throw value fd
/ram> code 16 fd read-file throw .
5
/ram> fd close-file throw
/ram> code 200 0 0 sys . . . .
49 0 1 144
```

Twice 200 is 400, which is `$190`: A is `$90` (144), and X is 1. The flags, 49, have C set: the carry out of the top. The routine only uses a relative branch, so it works wherever it's loaded; code with `jmp`s or `jsr`s to its own labels has to be assembled (with `.org`) for the address it will run at.

hylang has no way to call machine code, but `(peek addr)` and `(poke addr byte)` reach memory, and `(sys name ...)` makes any system call (Chapter 22).

CHAPTER 28

The Disassembler

A **disassembler** does an assembler's job backwards: it reads machine code and writes it out as assembly language. It's how you look inside a program you don't have the source of, check what the assembler really made, or find out how the Hydra's own tools work. The Hydra has three: the `dis` command, HyForth's `disasm`, and the debugger's `d` command. All three use the same part of the asm library, so they all write instructions exactly as `as` reads them.

IN THIS CHAPTER: You'll take programs apart with `dis`, tell code from data, put a program back together, and read the Hydra's own tools. **You need:** Chapters 26 and 27.

```
dis [-cnw] [-l labels] [-o addr] file
```

OPTION	WHAT IT DOES
`-l` *labels*	Use the names in a labels file (`as -l`'s, section 27.17)
`-n`	No names for the system calls and r0 to r15: just their addresses
`-w`	Wide: the instructions in the SDK's columns, as its sources are written
`-c`	Every byte as code: none taken for data
`-o` *addr*	The file is raw bytes (`as -b`'s), to be loaded at *addr* (in hex)

28.1 Taking a program apart

Here is Chapter 27's `stars`, taken apart:

```
/ram> dis stars
; stars, as dis read it: as assembles this to its bytes again
.include "hydra.inc"

.include "hyx2.inc"

        HYX2_PROGRAM "stars", main
.code
main:
        lda #$01
        sta $22
L0834:
        lda $22
        jsr L0842
        inc $22
        lda $22
        cmp #$06
        bne L0834
        rts
L0842:
        sta $23
L0844:
        lda #$2A
        jsr PUTC
        dec $23
        bne L0844
        lda #$0A
        jsr PUTC
        rts
```

Compare it with `stars.s` (section 27.1). The instructions are all there, but much has gone, because it was never in the program: the comments, the constant `ROWS` (just `#$06` is left, `ROWS + 1`), the names of the variables (just their addresses, `$22` and `$23`) and of the labels. `dis` makes up a name for each place an instruction jumps or branches to: `L` and its address, so `L0842` is the routine at `$0842`, `line`. It knows the system calls' names (`PUTC`), and `main`, where the program starts. Numbers are in hex, and characters as their codes (`#$2A` is `'*'`).

28.2 Names from a labels file

With the labels file `as -l` wrote, `dis` has the names back:

```
/ram> dis -l stars.lbl stars
; stars, as dis read it: as assembles this to its bytes again
.include "hydra.inc"
row             = $22
count           = $23

.include "hyx2.inc"

        HYX2_PROGRAM "stars", main
.code
main:
        lda #$01
        sta row
L0834:
        lda row
        jsr line
        inc row
        lda row
        cmp #$06
        bne L0834
        rts
line:
        sta count
L0844:
        lda #$2A
        jsr PUTC
        dec count
        bne L0844
        lda #$0A
        jsr PUTC
        rts
```

Names for places outside the program, like the zero page's, become constants at the top (`row = $22`). The two cheap local labels, `@row` and `@star`, weren't in the labels file, so they're still `L0834` and `L0844`.

28.3 And back again

The first line says it: `as` assembles what `dis` writes back into the same bytes. So you can take a program apart, change it, and put it back together, even without its source:

```
/ram> dis stars >stars2.s
/ram> as stars2.s stars2
/ram> cmp stars stars2 && echo the same
the same
```

`cmp` compares two files, and says nothing if they're the same. Edit `stars2.s` (change `#$2A` to `#$2B`, say), assemble it again, and `stars2` prints a triangle of plus signs.

When you don't need the source file in between, a pipe does it in one line: `dis stars | as - stars2`. The `-` tells `as` to read its source from standard input (section 27.17).

28.4 Code and data

A program file has data in it as well as code: strings, tables, numbers. A byte doesn't say which it is, so `dis` has to work it out. It starts where the program starts, at `main`, and follows the code: each instruction leads to the next one, and a branch, jump or `jsr` leads to another place too. It stops following at an instruction that doesn't go on to the next: `rts`, `rti`, `jmp`, `bra`, `brk` or `stp`. Every byte it reached is code; the rest is data, which it writes as `.byte` lines, with any text in quotes.

Look at `hi` from Chapter 27. Its `PRINT "Hello, "` keeps the text in the code, with a jump over it, and `dis` sees that:

```
/ram> cp /sd/0/sample/hi .
/ram> dis hi | head -32
; hi, as dis read it: as assembles this to its bytes again
.include "hydra.inc"

.include "hyx2.inc"

        HYX2_PROGRAM "hi", main
.code
main:
        lda r0
        sta $22
        lda r0+1
        sta $23
        lda ($22)
        bne L0844
        lda #$34
        sta $22
        lda #$09
        sta $23
L0844:
        lda #$52
        sta r0
        lda #$08
        sta r0+1
        jsr PUTS
        jmp L085A
        .byte "Hello, ", $00
L085A:
        lda $22
        sta r0
        lda $23
        sta r0+1
        jsr PUTS
```

`lda #$52`, `sta r0`, `lda #$08` and `sta r0+1` are how `PRINT` puts `$0852`, the address of the text just after the `jmp`, in r0, for `PUTS`. The end of the program has its strings, and its BSS:

```
/ram> dis hi | tail -12
        lda #$43
        sta r0
        lda #$09
        sta r0+1
        jsr PUTS
        rts
        .byte $79, $6F, $75, $00, $00, $21, $0A, $00, "window", $00, $2E
        .byte $0A, $00
.bss
        .res 64
B0986:
        .res 8
```

`dis` writes text in quotes only when there are at least four printable characters in a row, so `$79, $6F, $75` (`you`) stays as numbers. The BSS isn't in the file, so `dis` writes its size, with a `B` label where the code uses it.

Following the code can miss some: code that's only reached through a table of addresses, or a `jmp (addr)`, looks like data. `-c` takes every byte as code, from the start to the end, which shows it (and shows data as nonsense instructions, so use it with care).

28.5 The other options

`-w` writes the SDK's columns, which are easier to read:

```
/ram> dis -w -l stars.lbl stars | tail -10
line:
            sta         count
L0844:
            lda         #$2A
            jsr         PUTC
            dec         count
            bne         L0844
            lda         #$0A
            jsr         PUTC
            rts
```

`-n` leaves out the names of the system calls and the call registers, so `jsr PUTC` is `jsr $F950`, and `lda r0` is `lda $02`.

`-o` reads a raw file, as `as -b` makes them, and says where it's loaded, since a raw file doesn't:

```
/ram> dis -o 1000 raw
; raw, as dis read it: as assembles this to its bytes again
.include "hydra.inc"

.org $1000
.code
        asl a
        bcc L1004
        inx
L1004:
        rts
```

28.6 Inside the Hydra's tools

The Hydra's own tools are programs too, on the ROM disk in `/rom/bin`, and `dis` can take any of them apart. Here is the start of `echo`:

```
/> dis /rom/bin/echo | head -40
; /rom/bin/echo, as dis read it: as assembles this to its bytes again
.include "hydra.inc"

.include "hyx2.inc"

        HYX2_PROGRAM "echo", main
.code
main:
        lda r0
        sta $22
        lda r0+1
        sta $23
        lda #$01
        sta B0A40
        sta B0A41
        ldx #$00
        lda ($22)
        cmp #$2D
        bne L085B
        ldy #$01
        lda ($22),y
        cmp #$6E
        bne L085B
        iny
        lda ($22),y
        bne L085B
        stz B0A40
        jsr L088D
L085B:
        lda ($22)
        beq L087D
        lda B0A41
        bne L0869
        lda #$20
        jsr L08A0
L0869:
        stz B0A41
        ldy #$00
L086E:
        lda ($22),y
```

You can read it like a puzzle. It keeps its arguments' address (r0) on the zero page at `$22`. Then it checks for `-n`: the first argument's first character is `$2D` (`-`), its second `$6E` (`n`), and then it ends. If so, it clears the variable at `$0A40`, which is whether to print a new line at the end, and moves on to the next argument (`L088D`). Then, at `L085B`, it prints each argument (`L08A0` puts a character into its output), with a space (`$20`) before each but the first.

28.7 Disassembling in HyForth

HyForth's `lib disasm` has `disasm` (*addr u* --), which disassembles *u* instructions from *addr*, in any memory the task can see. And while it's loaded, `see` shows a **code word** (a word written in machine code, as HyForth's own words are) as its instructions:

```
/> lib disasm
/> see dup
code dup
 A299  CA        dex
 A29A  B5 23     lda $23,x
 A29C  95 22     sta $22,x
 A29E  B5 43     lda $43,x
 A2A0  95 42     sta $42,x
 A2A2  60        rts
end-code
/> see 2dup
code 2dup
 A318  20 CD A2  jsr $A2CD  \ over
 A31B  4C CD A2  jmp $A2CD  \ over
end-code
```

This shows how HyForth keeps its stack: on the zero page, the low bytes of its cells from `$22` and their high bytes from `$42`, with X pointing at the top. `dup` takes 1 from X, to make room, and copies the top cell's two bytes into the new top. `2dup` is just `over over`: a `jsr` to `over`, then a `jmp` to it (which is quicker than another `jsr` and an `rts`). A `jsr` or `jmp` to a word has its name after it.

`disasm` reaches anywhere. Here is the start of HydraOS's jump table, at `$F800`, and `PUTC` and `PUTS`, at `$F950`:

```
/> hex f800 4 disasm decimal
 F800  4C 8D EB  jmp $EB8D
 F803  4C C6 EB  jmp $EBC6
 F806  4C DD EE  jmp $EEDD
 F809  4C 41 EB  jmp $EB41
/> hex f950 2 disasm decimal
 F950  4C 61 E2  jmp $E261
 F953  4C 06 E3  jmp $E306
```

Each system call's address holds a `jmp` to the code that does it, in the kernel's ROM. Programs call the jump table, not the code, so a new HydraOS can move its code about and every program still works.

28.8 In the debugger

The debugger's `d` command disassembles a running program's memory, with names from its labels file and `hydra.inc`. Chapter 29 has it.

CHAPTER 29

The Debugger

A **bug** is a mistake in a program, and **debugging** is finding it. The hardest part is usually seeing what the program is really doing, as opposed to what you meant it to do. A **debugger** shows you: it runs a program an instruction at a time, or up to a place you choose, and lets you look at its registers and memory, and change them, on the way. The Hydra's debugger is `db`.

IN THIS CHAPTER: You'll run a program an instruction at a time, set breakpoints, look at its registers and memory and change them, and hunt down a real bug. **You need:** Chapters 26 and 27: it uses their programs `stars` and `show`.

```
db program [arg ...]
db -p task
```

`db program` starts the program (with its arguments), stopped at its first instruction, in a task of its own. A name with no `/` in it is looked for in `/bin`, so a program in this directory is `./name`. `db -p task` stops a task that's already running, wherever it is, and debugs that.

29.1 The commands

`db` shows the registers, then waits at its prompt, `db>`, for a command:

COMMAND	WHAT IT DOES
`r`	Show the registers, and the next instruction
`s` [*n*]	**Step**: run one instruction (or *n*); a `jsr` steps into the subroutine
`n` [*n*]	**Next**: the same, but a `jsr` runs the whole subroutine, as one step
`c`	**Continue**: run until a breakpoint, a `BRK`, the program's end, or CTRL-C
`u` *addr*	**Until**: step (as `n` does) until the program reaches *addr*
`b` [*addr*]	Set a **breakpoint** at *addr*; with none, list them
`x` [*addr*]	Clear the breakpoint at *addr*; with none, clear them all
`d` [*addr*] [*n*]	**Disassemble** *n* instructions (12) from *addr* (the program counter, or where the last `d` ended)
`m` [*addr*] [*n*]	Show *n* bytes of **memory** (64) from *addr*, in hex and as characters
`w` *addr byte ...*	**Write** bytes (in hex) into memory, from *addr*
`l` *file*	Load names from a labels file, or from `hydra.inc`
`q`	**Quit**: the program `db` started is ended; a task it stopped with `-p` runs on

Any other line shows a short list of the commands.

An **address** is a number in hex (with a `$` in front, or not), or a name from a labels file, with `+` or `-` and a distance in hex after it if you like: `0842`, `$0842`, `line`, `main+9`. A count, *n*, is in decimal.

29.2 Starting

This chapter debugs Chapter 27's `stars`. Assemble it with `-l`, so there's a labels file, and start it in `db`:

```
/ram> as -l stars.s
/ram> db ./stars
task 3
PC=0830 A=30 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830  A9 01     lda #$01
db> l /lib/as/hydra.inc
108 symbols
db> l stars.lbl
4 symbols
```

`stars` is running in task 3, stopped before its first instruction. The line of registers shows:

PART	WHAT IT IS
`PC=0830`	The program counter: the next instruction is at `$0830`
`A=30 X=FF Y=00`	The registers A, X and Y
`S=FD`	The stack pointer
`P=nv--dizc`	The flags: a capital letter for a flag that's 1, a small one for 0 (section 26.3)
`W=0 U=0`	The Hydra's W and U registers, as the task sees them
`RAM=00 ROM=00`	The task's RAM bank and ROM bank registers (`$00` and `$01`)

Under it is the next instruction: its address, its bytes, and what it is.

`l` loads names. `/lib/as/hydra.inc` has the system calls' and the call registers' names, and `stars.lbl` the program's own labels, so `db` can write `jsr PUTC` and `sta row` instead of `jsr $F950` and `sta $22`.

29.3 Looking at the program

`d` disassembles. Here is all of `stars`:

```
db> d main 19
0830  A9 01     lda #$01                ; main
0832  85 22     sta row                 ; main+2
0834  A5 22     lda row                 ; main+4
0836  20 42 08  jsr line                ; main+6
0839  E6 22     inc row                 ; main+9
083B  A5 22     lda row                 ; main+B
083D  C9 06     cmp #$06                ; main+D
083F  D0 F3     bne main+4              ; main+F
0841  60        rts                     ; main+11
0842  85 23     sta count               ; line
0844  A9 2A     lda #$2A                ; line+2
0846  20 50 F9  jsr PUTC                ; line+4
0849  C6 23     dec count               ; line+7
084B  D0 F7     bne line+2              ; line+9
084D  A9 0A     lda #$0A                ; line+B
084F  20 50 F9  jsr PUTC                ; line+D
0852  60        rts                     ; line+10
0853  07 AF     rmb0 count+8C           ; line+11
0855  85 F6     sta count+D3            ; line+13
```

After each instruction, `db` says where it is, by the nearest label before it. The last two lines aren't code: `d` doesn't know where the program ends, and the bytes after the last `rts` make nonsense instructions.

29.4 Stepping

`s` runs one instruction, and shows the registers after it:

```
db> s
PC=0832 A=01 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0832  85 22     sta row                 ; main+2
db> s
PC=0834 A=01 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0834  A5 22     lda row                 ; main+4
db> m row 2
0022  01 F6                     ..
db> s 3
0836  20 42 08  jsr line                ; main+6
0842  85 23     sta count               ; line
PC=0844 A=01 X=FF Y=00 S=FB P=nv--dizc W=0 U=0 RAM=00 ROM=00
0844  A9 2A     lda #$2A                ; line+2
```

- The first `s` ran `lda #$01`: A is 1. The second ran `sta row`.
- `m row 2` shows 2 bytes of memory from `row`: `row` is 1, as it should be. `count` is `$51`: whatever was left in that byte, since the program hasn't set it yet.
- `s 3` ran three instructions, showing each one as it went: `lda row`, the `jsr`, and, inside `line`, `sta count`. S went from `$FD` to `$FB`: the `jsr` pushed its return address, two bytes, on the stack.

29.5 Breakpoints

Stepping through a whole program would take forever. A **breakpoint** stops the program when it gets to a place:

```
db> b line
db> b
1 0842  85 23     sta count               ; line
db> c
*
breakpoint 1
PC=0842 A=02 X=FF Y=00 S=FB P=nv--dizc W=0 U=0 RAM=00 ROM=00
0842  85 23     sta count               ; line
```

`b line` set a breakpoint at `line`, and `b` alone listed it. `c` let the program run: it finished the first row, printing its star, and stopped when it called `line` again, for the second row. A is 2, the number of stars for this row. You can have 16 breakpoints, but only in RAM: `db` makes one by putting a `BRK` instruction in the program while it runs, and taking it out again when it stops, so it can't make one in ROM.

29.6 Until, and next

`u` runs (a step at a time) until the program reaches an address. `u main+9` runs to the instruction after `jsr line`, which finishes this call of `line`:

```
db> u main+9
**
PC=0839 A=0A X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0839  E6 22     inc row                 ; main+9
db> x
db> n 4
083B  A5 22     lda row                 ; main+B
083D  C9 06     cmp #$06                ; main+D
083F  D0 F3     bne main+4              ; main+F
PC=0834 A=03 X=FF Y=00 S=FD P=Nv--dizc W=0 U=0 RAM=00 ROM=00
0834  A5 22     lda row                 ; main+4
db> n 2
0836  20 42 08  jsr line                ; main+6
***
PC=0839 A=0A X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0839  E6 22     inc row                 ; main+9
```

- `u main+9` printed the second row's two stars, and stopped back in `main`. (A is `$0A`, the code of the new line `line` printed last.)
- `x` cleared the breakpoint.
- `n 4` stepped four times: `inc row`, `lda row`, `cmp #$06` and `bne main+4`, which went back to `main+4`. Look at the flags: `cmp` compared 3 with 6, and 3 - 6 is negative, so N is 1, and C is 0 (3 is less than 6).
- `n 2` stepped over `lda row` and then the whole `jsr line`, as one step: it printed the third row, and stopped after it. `s` would have gone into `line`.

`s` and `n` both run a system call (a `jsr` to HydraOS's jump table) whole: HydraOS's own code can't be stepped. And `u` steps as `n` does, so it only stops at an address in the same subroutine, or one it returns to: `u line` from `main` would run straight past, because `n` runs `line` whole.

29.7 Changing the program

`w` writes bytes into the program's memory, while it's stopped. The star is the operand of `lda #$2A`, the byte at `$0845`. Write `$23`, the code of `#`, there, and let it run on:

```
db> w 0845 23
db> c
####
#####
task 3 ended: code 0
```

The last two rows are made of `#`. The program ended, and so did `db`. The change was only in memory: the file `stars` is just as it was. So when you've found a fix this way, put it in the source, and assemble it again. (`db` can change memory, but not the registers.)

29.8 Finding a bug

Here's a real bug. Chapter 27's `show` was first written without its `pha` and `pla`. That first version is `show1.s` (section 29.12 shows how to make it). It worked, except when the file wasn't there:

```
/ram> as -l show1.s
/ram> ./show1 nothing
show: bad address
```

That's wrong: the file isn't there, so it should say `not found`. The message comes from `ERRSTR`, at `@fail`. So stop the program there, and look at what it gives `ERRSTR`:

```
/ram> db ./show1 nothing
task 3
PC=0830 A=30 X=FF Y=20 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830  B2 02     lda ($02)
db> l /lib/as/hydra.inc
108 symbols
db> l show1.lbl
4 symbols
db> d main+29 3
0859  A9 00     lda #$00                ; main+29
085B  20 C0 F8  jsr OPEN                ; main+2B
085E  B0 38     bcs main+68             ; main+2E
db> b main+68
db> c
breakpoint 1
PC=0898 A=20 X=2E Y=03 S=FD P=nv--dizC W=0 U=0 RAM=00 ROM=00
0898  A9 D1     lda #$D1                ; main+68
db> s 4
089A  85 02     sta r0                  ; main+6A
089C  A9 09     lda #$09                ; main+6C
089E  85 03     sta r0+1                ; main+6E
PC=08A0 A=09 X=2E Y=03 S=FD P=nv--dizC W=0 U=0 RAM=00 ROM=00
08A0  20 03 F8  jsr ERRSTR              ; main+70
db> q
```

- `d main+29 3` found the `OPEN`. If it fails, `bcs main+68` goes to `@fail`. (A cheap local label isn't in the labels file, so `db` calls it by how far it is from `main`.)
- `b main+68` and `c` stopped there. C is 1 (`OPEN` failed), and A is `$20`: Appendix G says that's `not found`. Right so far.
- `s 4` stepped through `LDR r0, msg`'s four instructions, and now, just before `jsr ERRSTR`, A is `$09`: `LDR`'s `lda #>msg` put the high byte of `msg`'s address in it. And `$09` is `bad address`. Found it!

The fix keeps A safe on the stack while `LDR` works: `pha` before it, and `pla` after, as `show.s` has. Most bugs are like this: something changed that you didn't expect to change. The debugger shows you exactly where.

29.9 Stopping a running program

`c` runs until something stops the program. If it never stops, CTRL-C does: it goes to `db` (the program runs in a note group of its own, so the note doesn't reach it), and `db` stops the program wherever it is. The `tick` sample prints a dot a second, for ever:

```
/> db /sd/0/sample/tick
task 3
PC=0830 A=30 X=FF Y=02 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830  9C 95 08  stz $0895
db> c
........PC=EB31 A=06 X=0E Y=03 S=FA P=nv--dIzc W=0 U=0 RAM=00 ROM=00 (in a call: its state 6)
EB31  A5 A2     lda $A2
db> q

/>
```

After eight dots, CTRL-C stopped it. `(in a call: its state 6)` means it was inside a system call, `SLEEP` (state 6 is sleeping): the PC, `$EB20`, is in HydraOS's kernel. You can't step there, but `c` runs it on, and `q` ends it.

CTRL-C stops an `n` or a `u` that's taking too long, too, and so does a breakpoint inside a subroutine that `n` is running. If the program stops in a subroutine of its own, carry on from there as you like: `s`, `n` and `c` all work. If it stops inside a system call that you were stepping over, `s` or `n` lets the call finish and stops after it, and `c` runs on.

29.10 Debugging a running task

`db -p` stops a task that's already running, to look at it. Here a `sleep` runs in the background (it's task 2, as `ps` would say):

```
/> sleep 100 &
/> db -p 2
task 2
PC=EB31 A=06 X=51 Y=02 S=F9 P=nv--dIzc W=0 U=0 RAM=00 ROM=00 (in a call: its state 6)
EB31  A5 A2     lda $A2
db> q
```

`q` lets a task `db` stopped run on, as if nothing had happened.

29.11 How db works

`db` uses the files of `/proc` (section 17.8). `/proc/3/ctl` takes the commands `stop`, `start`, `step`, `next`, `break` and `nobreak` (and `kill`, which `q` uses), and `/proc/3/mem` is task 3's memory, which `db` reads and writes like a file. While it's debugged, a task's `BRK` instructions stop it, instead of ending it, which is how breakpoints work. And HydraOS keeps a stopped task's registers where `db` can read them.

So `db` is an ordinary program, with no special powers: it only uses files that any program can. That's the Plan 9 way of doing things, which HydraOS follows (Chapter 9).

29.12 Making show1.s

`show1.s` is `show.s` (section 27.14) with three small changes, so you don't need to type it all again. Copy `show.s` to `show1.s`, and open the copy in `edit`:

1. In the first line's comment, and in the `HYX2_PROGRAM` line, change `show` to `show1`:

```
; show1 file - show.s, as it was first written (it has a mistake)
```

```
            HYX2_PROGRAM "show1", main
```

2. Near the end, find the three lines that start at `@fail`:

```
@fail:      pha                 ; The error's text (LDR changes .A: kept)
            LDR r0, msg
            pla
```

and make them one, without the `pha` and the `pla`:

```
@fail:      LDR r0, msg         ; The error's text
```

That missing pair is the bug that section 29.8 hunts down.

REFERENCE

Appendices

The appendices are for looking things up: every word, function and keyword of the three languages, the 65C02's instructions, the characters, the memory map, the error codes, the emulator's options, the parts, the words this book uses, and every system call. The index, at the very end, finds the page where a word is explained.

- Appendix A. HyForth Reference
- Appendix B. hylang Reference
- Appendix C. HyBASIC Reference
- Appendix D. The 65C02's Instructions
- Appendix E. Characters
- Appendix F. The Memory and I/O Map
- Appendix G. Error Codes
- Appendix H. The Emulator and the PC Tools
- Appendix I. Parts and Board Diagrams
- Appendix J. Glossary
- Appendix K. System Calls

APPENDIX A

HyForth Reference

This appendix lists every word in HyForth, library by library. HyForth follows the Forth 2012 standard; the words of the standard's word sets mean what the standard says, and the Hydra's own words are described in full.

A.1 How to read the entries

Each word has a **stack diagram**: what it takes, `--`, and what it leaves, with the top of the stack on the right.

ABBREVIATION	IS
n	A signed number: one cell, -32,768 to 32,767
u	An unsigned number: 0 to 65,535
x	Any cell
flag	A flag: 0 false, -1 true (any value but 0 counts as true)
char, *c*	A character's code
addr, *c-addr*	An address; the address of characters
c-addr u	A string: its address and its length
d, *ud*	A double-cell number (32 bits), signed and unsigned
xt	An **execution token**: a word, as a value (`'` gives one)
nt, *wid*	A word's name token; a word list
fileid, *fam*, *ior*	A file's id; a file access method; an I/O result (0, or an error)
"name"	A word taken from the input, after this one
`N:`	The number stack (the numbers library's)
`R:`	The return stack

(I) marks an **immediate** word, which works while a definition is being compiled; **(C)** a word that can only be used inside a definition. Words load with their library: the core is always there; `coreext`, `exception`, `file`, `tools` and (at the shell) `shell` load as HyForth starts; the rest with `lib` *name*.

A.2 The core

The stack

WORD	STACK	DOES
`dup`	(*x* -- *x x*)	Copies the top
`?dup`	(*x* -- 0 | *x x*)	Copies the top if it isn't 0

WORD	STACK	DOES
`drop`	(*x* --)	Drops the top
`swap`	(*x1 x2* -- *x2 x1*)	Swaps the top two
`over`	(*x1 x2* -- *x1 x2 x1*)	Copies the second
`rot`	(*x1 x2 x3* -- *x2 x3 x1*)	Brings the third to the top
`2dup`, `2drop`, `2swap`, `2over`		The same, for pairs of cells
`depth`	(-- *u*)	How many cells are on the stack
`>r`, `r>`, `r@` (C)	(*x* --) R: (-- *x*), (-- *x*) R: (*x* --), (-- *x*)	To, from, a copy of, the return stack

For example:

```
/> 1 2 3 .s
<3> 1 2 3
/> rot .s
<3> 2 3 1
/> over .s
<4> 2 3 1 3
/> 2drop 2drop depth .
0
/> 5 ?dup .s
<2> 5 5
/> 2drop 0 ?dup .s
<1> 0
```

Arithmetic

WORD	STACK	DOES
`+`, `-`, `*`	(*n1 n2* -- *n3*)	Add, subtract, multiply
`/`, `mod`, `/mod`	(*n1 n2* -- *n3*), (-- *rem*), (-- *rem quot*)	Divide (rounding toward zero), the remainder, both
`*/`, `*/mod`	(*n1 n2 n3* -- *n4*), (-- *rem quot*)	*n1* × *n2* ÷ *n3*, with a 32-bit product in between
`1+`, `1-`, `2*`, `2/`	(*n1* -- *n2*)	Add 1, take 1, double, halve
`negate`, `abs`	(*n1* -- *n2*)	Change the sign; the size
`min`, `max`	(*n1 n2* -- *n3*)	The smaller; the larger
`um*`, `m*`	(*u1 u2* -- *ud*), (*n1 n2* -- *d*)	A double-cell product
`um/mod`, `fm/mod`, `sm/rem`	(*ud u1* -- *u2 u3*), (*d n1* -- *n2 n3*)	Divide a double by a cell: unsigned, floored, symmetric
`s>d`	(*n* -- *d*)	A cell to a double

For example:

```
/> 7 2 /mod . .
3 1
/> 100 7 3 */ .
233
/> -5 abs . 3 9 max . 3 9 min .
5 9 3
/> 5 negate . 7 2* . 7 2/ .
-5 14 3
/> 17 5 mod . 17 5 / .
2 3
```

Logic and comparisons

WORD	STACK	DOES
`and`, `or`, `xor`	(*x1 x2 -- x3*)	Bit by bit
`invert`	(*x1 -- x2*)	Every bit flipped
`lshift`, `rshift`	(*x1 u -- x2*)	Shift the bits *u* places left, right
`=`, `<`, `>`, `u<`	(*x1 x2 -- flag*)	Equal; less, more (signed); less (unsigned)
`0=`, `0<`	(*x -- flag*)	Zero; below zero

For example:

```
/> 12 10 and . 12 10 or . 12 10 xor .
8 14 6
/> 1 4 lshift . 255 4 rshift .
16 15
/> 3 5 < . 5 5 = . -1 0< . 0 0= .
-1 -1 -1 -1
/> hex ff invert u. decimal
FF00
```

Memory

WORD	STACK	DOES
`@`, `!`	(*addr -- x*), (*x addr --*)	Fetch, store a cell
`c@`, `c!`	(*addr -- char*), (*char addr --*)	Fetch, store a byte
`2@`, `2!`	(*addr -- x1 x2*), (*x1 x2 addr --*)	Fetch, store two cells
`+!`	(*n addr --*)	Add to a cell
`move`	(*addr1 addr2 u --*)	Copy *u* bytes
`fill`	(*c-addr u char --*)	Fill with a byte
`count`	(*c-addr1 -- c-addr2 u*)	A counted string (its length first) to a string
`cells`, `cell+`, `chars`, `char+`	(*n1 -- n2*)	Sizes: a cell is 2 bytes, a character 1
`align`, `aligned`	(--), (*addr -- addr*)	(Nothing on the 65C02: any address is aligned)
`here`	(*-- addr*)	The dictionary's next free byte
`allot`	(*n --*)	Reserve *n* bytes there
`,`, `c,`	(*x --*), (*char --*)	Put a cell, a byte, there

For example:

```
/> variable n  7 n !  n @ .
7
/> 3 n +!  n @ .
10
/> create row 10 , 20 , 30 ,
/> row cell+ @ .
20
/> create buf 8 allot  buf 8 char * fill  buf 8 type
********
```

Text and numbers

WORD	STACK	DOES
`emit`	(*char* --)	Print a character
`type`	(*c-addr u* --)	Print a string
`cr`, `space`, `spaces`	(--), (--), (*n* --)	A new line; a space; *n* spaces
`.`, `u.`	(*n* --), (*u* --)	Print a number, and a space
`."` *text*`"` (I C)	(--)	Print the text
`s"` *text*`"` (I)	(-- *c-addr u*)	A string
`key`	(-- *char*)	Wait for a key
`accept`	(*c-addr n1* -- *n2*)	Read a line into a buffer
`char`, `[char]` (I)	("name" -- *char*)	The first character of the next word
`bl`	(-- *char*)	A space's code (32)
`<#`, `#`, `#s`, `hold`, `sign`, `#>`		Pictured numbers: build a number's text, digit by digit
`>number`	(*ud1 c-addr1 u1* -- *ud2 c-addr2 u2*)	Read digits into a double
`base`	(-- *addr*)	The variable holding the base
`decimal`	(--)	Base 10
`(`	("text)" --)	A comment, to `)`
`\`	("text" --)	A comment, to the end of the line
`word`, `source`, `>in`		The input: a word of it; the line; the place in it

For example:

```
/> char A . bl .
65 32
/> 65 emit space 66 emit cr
A B
/> s" Hydra" type cr
Hydra
/> : .hex ( u -- ) base @ swap hex u. base ! ;
/> 255 .hex
FF
/> : .money ( n -- ) 0 <# # # [char] . hold #s [char] $ hold #> type ;
/> 1234 .money
$12.34
```

Definitions and control

WORD	STACK	DOES
`:`	("name" --)	Begin a definition
`;` (I)	(--)	End it
`immediate`	(--)	Make the last definition immediate
`recurse` (I C)		Call the definition being made
`[`, `]`	(--)	Stop compiling; start again
`literal` (I C)	(*x* --)	Compile a number
`'`, `[']` (I)	("name" -- *xt*)	A word's execution token

WORD	STACK	DOES
`execute`	(*xt* --)	Run a word
`postpone` (I C)	("name" --)	Compile a word's compiling behaviour
`if` ... `else` ... `then` (I C)	(*flag* --)	Decisions
`begin` ... `until` (I C)	(*flag* --)	Loop until true
`begin` ... `while` ... `repeat` (I C)	(*flag* --)	Loop while true
`do` ... `loop`, `+loop` (I C)	(*limit start* --), (*n* --)	Counted loops
`i`, `j` (C)	(-- *n*)	The loop's count; the outer loop's
`leave`, `unloop` (C)	(--)	Leave the loop; drop its count (before `exit`)
`exit` (C)	(--)	Return from the definition
`create`	("name" --)	A word that gives the address of what follows
`does>` (I C)		What a `create`d word does
`>body`	(*xt* -- *addr*)	A `create`d word's data
`variable`, `constant`	("name" --), (*x* "name" --)	A variable; a constant
`state`	(-- *addr*)	True while compiling
`find`	(*c-addr* -- *c-addr* 0 | *xt* 1 | *xt* -1)	Look a word up
`evaluate`	(*c-addr u* --)	Run a string as Forth
`quit`, `abort`, `abort"`		Back to the prompt; with a message if a flag is true
`bye`	(--)	Leave HyForth
`environment?`	(*c-addr u* -- false | *x true*)	Ask about the system
`included`, `include`	(*c-addr u* --), ("file" --)	Run a file of Forth
`required`, `require`	(*c-addr u* --), ("file" --)	The same, once only

For example:

```
/> : sign ( n -- ) dup 0< if drop ." negative" exit then 0= if ." zero" else ." positive" then ;
/> -3 sign
negative
/> 0 sign
zero
/> : squares ( n -- ) 1+ 1 do i dup * . loop ;
/> 5 squares
1 4 9 16 25
/> : halve ( n -- ) begin dup . 2/ dup 0= until drop ;
/> 40 halve
40 20 10 5 2 1
/> : adder ( n "name" -- ) create , does> @ + ;
/> 10 adder ten+
/> 5 ten+ .
15
/> 7 ' dup execute * .
49
/> s" 2 3 + ." evaluate
5
```

A.3 Core Extension (`coreext`)

WORD	STACK	DOES
`nip`, `tuck`	(*x1 x2 -- x2*), (*x1 x2 -- x2 x1 x2*)	Drop the second; copy the top under the second
`pick`	(*xu … x0 u -- xu … x0 xu*)	Copy the *u*th cell to the top
`roll`	(*xu … x0 u -- xu-1 … x0 xu*)	Move the *u*th cell to the top
`<>`, `u>`, `0<>`, `0>`	(*… -- flag*)	Not equal; more (unsigned); not zero; above zero
`within`	(*n lo hi -- flag*)	*lo* ≤ *n* < *hi*
`true`, `false`	(*-- flag*)	-1, 0
`2>r`, `2r>`, `2r@` (C)		Pairs to and from the return stack
`erase`	(*addr u* --)	Fill with zeros
`pad`	(-- *c-addr*)	A scratch buffer
`unused`	(-- *u*)	The dictionary's free bytes
`hex`	(--)	Base 16
`:noname`	(-- *xt*)	A definition without a name
`compile,`	(*xt* --)	Compile a word
`[compile]` (I)	("name" --)	Compile an immediate word
`again` (I C)		`begin` … `again`: loop for ever
`?do` (I C)	(*limit start* --)	`do`, but skipped if they're equal
`case`, `of`, `endof`, `endcase` (I C)		Choose by value
`value`, `to`	(*x* "name" --), (*x* "name" --)	A value; change it
`buffer:`	(*u* "name" --)	A named buffer of *u* bytes
`defer`, `is`, `defer!`, `defer@`, `action-of`		A word whose action is set later
`marker`	("name" --)	*name* forgets everything defined after it
`parse`, `parse-name`	(*char* "text" -- *c-addr u*), ("name" -- *c-addr u*)	Take text from the input
`refill`, `source-id`, `save-input`, `restore-input`		The input source
`.(` (I)	("text)" --)	Print the text now
`.r`, `u.r`	(*n w* --), (*u w* --)	Print a number right-aligned in *w* characters
`holds`	(*c-addr u* --)	Pictured numbers: put a string in
`c"` (I C)	("text"" -- *c-addr*)	A counted string
`s\"` (I)	("text"" -- *c-addr u*)	A string with escapes: `\n`, `\t`, `\"` …

For example:

```
/> 1 2 nip .
2
/> 10 20 30 2 pick .s
<4> 10 20 30 10
/> 5 0 10 within . 15 0 10 within .
-1 0
/> 0 value lives
/> 3 to lives lives .
3
/> : medal ( n -- ) case 1 of ." gold" endof 2 of ." silver" endof ." thanks for coming" endcase ;
/> 1 medal
gold
/> 7 medal
thanks for coming
/> 42 6 .r
    42
/> defer greet
/> : hi ." hi" ;
/> ' hi is greet
/> greet
hi
```

A.4 Exception (`exception`)

WORD	STACK	DOES
`catch`	(*i*x xt* -- *j*x* 0 | *i*x n*)	Run *xt*; 0 if it worked, or the error it threw
`throw`	(*k*x n* -- *k*x* | *i*x n*)	Throw error *n* (nothing if it's 0)

For example:

```
/> : risky ( n -- ) dup 0= if 99 throw then 100 swap / . ;
/> 5 risky
20
/> : try ( n -- ) ['] risky catch ?dup if ." error " . drop then ;
/> 5 try
20
/> 0 try
error 99
```

A.5 File Access (`file`)

WORD	STACK	DOES
`r/o`, `w/o`, `r/w`	(-- *fam*)	Read only, write only, both
`bin`	(*fam1* -- *fam2*)	Bytes, as they are
`open-file`, `create-file`	(*c-addr u fam* -- *fileid ior*)	Open; make (or empty) and open
`close-file`	(*fileid* -- *ior*)	Close
`read-file`	(*c-addr u1 fileid* -- *u2 ior*)	Read bytes
`read-line`	(*c-addr u1 fileid* -- *u2 flag ior*)	Read a line (*flag* false at the end)
`write-file`, `write-line`	(*c-addr u fileid* -- *ior*)	Write bytes; a line
`file-position`, `reposition-file`	(*fileid* -- *ud ior*), (*ud fileid* -- *ior*)	The place in the file
`file-size`, `resize-file`	(*fileid* -- *ud ior*), (*ud fileid* -- *ior*)	Its length

WORD	STACK	DOES
`file-status`	(*c-addr u -- x ior*)	Is it there?
`flush-file`	(*fileid -- ior*)	Write what's waiting
`delete-file`, `rename-file`	(*c-addr u -- ior*), (*c-addr1 u1 c-addr2 u2 -- ior*)	Remove; rename
`include-file`	(*fileid* --)	Run an open file as Forth

For example, a file written and read back:

```
/> s" /ram/t.txt" w/o create-file throw constant f
/> s" first line" f write-line throw
/> f close-file throw
/> s" /ram/t.txt" r/o open-file throw constant g
/> pad 80 g read-line throw drop pad swap type cr
first line
/> g close-file throw
/> s" /ram/t.txt" delete-file throw
```

An ior from HyForth's file words is 0, or -512 less the system's error code (Appendix G), so `throw` reports it.

A.6 Programming-Tools (`tools`)

WORD	STACK	DOES
`.s`	(--)	Show the stack
`?`	(*addr* --)	Print the cell at *addr*
`dump`	(*addr u* --)	Show memory, 8 bytes a line, in hex and as text
`see`	("name" --)	Show a word's definition
`words`	(--)	List the words (the newest first), with their addresses and kinds
`libs`	(--)	List the libraries loaded
`lib`	("name" --)	Load a library (*name*`.fl`, or *name*`.fs`), or show its words again
`-lib`	("name" --)	Hide a library's words
`[if]`, `[else]`, `[then]` (I)	(*flag* --)	Choose which lines to compile
`[defined]`, `[undefined]` (I)	("name" -- *flag*)	Is a word there?
`ahead`, `cs-pick`, `cs-roll` (I C)		For building control words
`n>r`, `nr>`	(*i*x n* --) R: (-- *i*x n*)	Several cells to and from the return stack
`synonym`	("new" "old" --)	Another name for a word
`traverse-wordlist`	(*i*x xt wid* -- *j*x*)	Run *xt* for each word of a word list
`name>string`, `name>interpret`, `name>compile`	(*nt* -- ...)	A word's name, and how it runs

For example:

```
/> 1 2 3 .s
<3> 1 2 3
/> : sq dup * ;
/> see sq
: sq dup * ;
/> [defined] sq .
-1
/> synonym squared sq
/> 4 squared .
16
/> libs
forth coreext exception file tools shell
```

A.7 Facility (`facility`)

WORD	STACK	DOES
`key?`	(-- *flag*)	Is a key waiting?
`ekey`, `ekey?`	(-- *u*), (-- *flag*)	A key, special keys too; is one waiting?
`ekey>char`, `ekey>fkey`	(*u* -- *u* false | *char* true), (*u* -- *u* false | *x* true)	Is it a character? A special key?
`k-up` `k-down` `k-left` `k-right` `k-home` `k-end` `k-prior` `k-next` `k-insert` `k-delete` `k-f1` … `k-f12`	(-- *x*)	The special keys, as `ekey>fkey` gives them
`k-resize`, `k-focus`	(-- *x*)	(Not keys: the window was resized; another window of the group was chosen)
`k-shift-mask`, `k-ctrl-mask`, `k-alt-mask`	(-- *x*)	Added to a key typed with them held
`emit?`	(-- *flag*)	Can `emit` go on? (always)
`ms`	(*u* --)	Wait *u* milliseconds
`time&date`	(-- *sec min hour day month year*)	The date and time
`page`	(--)	Clear the screen
`at-xy`	(*u1 u2* --)	Cursor to column *u1*, row *u2* (from 0)
`form`	(-- *rows cols*)	The window's size
`clear-line`, `clear-below`	(--)	Clear the rest of the line, of the screen
`cursor-up`, `cursor-down`, `cursor-right`, `cursor-left`	(*n* --)	Move the cursor
`cursor-save`, `cursor-restore`, `cursor-off`, `cursor-on`	(--)	Save, restore, hide, show the cursor
`color`, `bgcolor`	(*c* --)	The text's, the background's colour (0-15)
`black` `red` `green` `yellow` `blue` `magenta` `cyan` `white`	(-- *c*)	The colours 0 to 7
`bright`	(*c1* -- *c2*)	A colour's bright version (8-15)
`plain`, `bold`, `dim`, `underline`, `blink`, `reverse`	(--)	Styles (`plain`: all off)
`sgr`	(*n* --)	Any `ESC [`*n*`m`
`beep`	(--)	Ring the bell

WORD	STACK	DOES
`begin-structure`, `end-structure`	("name" -- *addr* 0), (*addr n* --)	A structure's start and end
`+field`, `field:`, `cfield:`	(*n1 n2* "name" -- *n3*), (*n1* "name" -- *n2*)	Its fields

For example:

```
/> lib facility
/> form . .
80 24
/> time&date . . . . . .
2000 1 1 0 0 5
/> : colours ( -- ) red color ." red " green bright color ." bright green " plain cr ;
/> colours
red bright green
/> begin-structure point field: p.x field: p.y end-structure
/> create spot point allot
/> 3 spot p.x ! 4 spot p.y !
/> spot p.x @ . spot p.y @ . point .
3 4 4
```

A.8 String (`string`)

WORD	STACK	DOES
`cmove`, `cmove>`	(*c-addr1 c-addr2 u* --)	Copy bytes, from the start up; from the end down
`-trailing`	(*c-addr u1* -- *c-addr u2*)	Without the spaces at the end
`/string`	(*c-addr u n* -- *c-addr+n u-n*)	Skip *n* characters
`blank`	(*c-addr u* --)	Fill with spaces
`compare`	(*c-addr1 u1 c-addr2 u2* -- *n*)	-1, 0 or 1, as they sort
`search`	(*c-addr1 u1 c-addr2 u2* -- *c-addr3 u3 flag*)	Find the second string in the first
`sliteral` (I C)	(*c-addr u* --)	Compile a string
`replaces`, `substitute`, `unescape`		Fill in `%name%`s in a string

For example:

```
/> lib string
/> s" hello   " -trailing type char | emit cr
hello|
/> s" hello world" 6 /string type cr
world
/> s" apple" s" banana" compare .
-1
/> s" hello world" s" wor" search . type cr
-1 world
```

A.9 Search-Order (`search`)

WORD	STACK	DOES
`forth-wordlist`	(-- *wid*)	The FORTH word list

WORD	STACK	DOES
`wordlist`	(-- *wid*)	A new, empty word list
`get-current`, `set-current`	(-- *wid*), (*wid* --)	Where new words go
`definitions`	(--)	New words go in the first word list of the order
`search-wordlist`	(*c-addr u wid* -- 0 | *xt* 1 | *xt* -1)	Look a word up in a word list
`get-order`, `set-order`	(-- *widn ... wid1 n*), (*widn ... wid1 n* --)	The search order
`only`, `also`, `previous`, `forth`, `order`		Change, and show, the order
`library`	("name" -- *wid*)	Start a library of your own: a new word list, first in the order
`end-library`	(*wid* --)	End it

A.10 Double-Number (`double`)

`2constant`, `2variable`, `2value`, `2literal` (I), `d+`, `d-`, `m+`, `dnegate`, `dabs`, `d0=`, `d0<`, `d2*`, `d2/`, `d=`, `d<`, `du<`, `dmax`, `dmin`, `d>s`, `d.`, `d.r`, `m*/` and `2rot`, as the standard has them: the double-cell (32-bit) versions of the cell words.

For example:

```
/> lib double
/> 1000 1000 um* d.
1000000
/> 1000 1000 um* 2dup d+ d.
2000000
/> 123456. d.
123456
```

A.11 Memory-Allocation, Locals and Block

WORD	STACK	DOES
`allocate`	(*u* -- *addr ior*)	A block of *u* bytes, from the heap (`memory`)
`free`	(*addr* -- *ior*)	Give one back
`resize`	(*addr1 u* -- *addr2 ior*)	Change one's size
`{:` *args* `|` *vals* `--` *comment* `:}` (I C)		Local variables, from the stack (`locals`)
`locals|` *names* `|` (I C), `(local)`		The older way to make locals
`block`, `buffer`	(*u* -- *addr*)	Block *u*'s buffer, read in; a buffer for it (`block`)
`update`, `save-buffers`, `flush`, `empty-buffers`	(--)	Mark changed; write; write and free; free
`blk`, `scr`	(-- *addr*)	The block being run; the block listed
`load`, `thru`	(*u* --), (*u1 u2* --)	Run blocks
`list`	(*u* --)	Show a block
`open-blocks`	(*c-addr u* --)	Use a file of blocks

For example:

```
/> lib locals
/> : hyp2 {: a b -- c :} a a * b b * + ;
/> 3 4 hyp2 .
25
/> lib memory
/> 100 allocate throw constant room
/> room 100 erase
/> room free throw
```

A.12 The shell (`shell`)

WORD	STACK	DOES
`%`	("line" --)	The rest of the line is rc's
`cd`	("dir" --)	Change directory
`bind`, `mount`, `unmount`, `newns`	("args" --)	The namespace, as rc's (section 8.14)
`prompt`, `prompt2`	(*c-addr u* --)	The prompt's format; the second prompt's
`status`	(-- *n*)	The last command's status
`exit` (I)		At the prompt: leave HyForth; in a definition, the standard's `exit`
`exits`	(*n* --)	Leave HyForth with status *n*
`wait`	(*task* --)	Wait for a background task
`getenv`, `setenv`, `unsetenv`	(*c-addr u* -- *c-addr2 u2*), (*c-addr1 u1 c-addr2 u2* --), (*c-addr u* --)	Environment variables
`spawn`	(*c-addr u* -- *task*)	Start a command, not waiting
`sh-out`	(*c-addr u* -- *c-addr2 u2*)	A command's output
`output-of`	(*xt* -- *c-addr u*)	What a word prints
`|`	(*xt* "line" --)	What a word prints, into a command
`piped`	(*xt c-addr u* --)	The same, the command a string
`send`	("n line" --)	Type a line into window *n*

For example:

```
/> s" date" sh-out type
2000-01-01 00:00:04
/> s" colour" s" blue" setenv
/> s" colour" getenv type cr
blue
/> status .
0
/> : hello ." Hello from Forth" cr ;
/> ' hello | wc
       1       3      17
/> cd /rom
```

A.13 The Hydra (`hydra`)

WORD	STACK	DOES
`sh`	(*c-addr u -- status*)	Run a command line with rc, and wait
`run`	(*c-addr u -- status*)	Run a program and its arguments, and wait
`ctl`	(*c-addr1 u1 c-addr2 u2* --)	Write text *c-addr2 u2* to file *c-addr1 u1*, in one write
`>z`, `zcount`	(*c-addr u -- z-addr*), (*z-addr -- c-addr u*)	To and from a zero-ended string
`argc`, `arg`	(-- *n*), (*n -- c-addr u*)	A script's arguments (0: its name)
`get-dir`, `set-dir`	(*c-addr1 u1 -- c-addr2 u2*), (*c-addr u -- ior*)	The current directory
`open-dir`, `read-dir`, `close-dir`		A directory's names, one at a time
`=mkdir`	(*c-addr u mode -- ior*)	Make a directory
`bank-window`	(-- *addr*)	`$8000`, where a RAM bank is seen
`bank!`, `bank@`	(*bank* --), (-- *bank*)	Choose, and read, the bank there
`seg-bank!`	(*seg n -- ior*)	A shared segment's bank there
`code-banks`	(*flag* --)	Compile definitions into banks (true, as it starts) or the dictionary (false)
`note`, `note-group`	(*task n* --), (*group n* --)	Send a note
`on-note`	(*xt* --)	*xt* (*n -- flag*) handles notes (0: none)
`pause`	(--)	Let the other tasks have a turn
`ior>text`	(*ior -- c-addr u*)	An error's message
`sys`	(*addr a x y -- a x y p*)	Call machine code, with registers
`sys-`*name*		Each system call (Appendix K): its inputs, in order, the first deepest; its outputs, and an ior if it can fail

For example:

```
/> lib hydra
/> s" echo hi" sh .
hi
0
/> s" echo one two" run .
one two
0
/> sys-getpid .
1
/> -514 ior>text type cr
invalid argument
```

The `sys-` words are: `sys-sysinfo` `sys-errstr` `sys-kmesg` `sys-reboot` `sys-exits` `sys-wait` `sys-spawn` `sys-getpid` `sys-yield` `sys-sleep` `sys-sleep-until` `sys-ticks` `sys-preempt-off` `sys-preempt-on` `sys-taskinfo` `sys-note` `sys-modinfo` `sys-env-get` `sys-env-put` `sys-env-del` `sys-env-name` `sys-taskread` `sys-getppid` `sys-sem-new` `sys-sem-acquire` `sys-sem-try` `sys-sem-release` `sys-sem-free` `sys-exec` `sys-break` `sys-pages-alloc` `sys-pages-free` `sys-banks` `sys-banks-alloc` `sys-banks-free` `sys-seg-create` `sys-seg-attach` `sys-seg-detach` `sys-seg-map` `sys-seginfo` `sys-banks-alloc-in` `sys-seg-create-in` `sys-open` `sys-create` `sys-close` `sys-read`

`sys-write` `sys-seek` `sys-stat` `sys-fstat` `sys-wstat` `sys-fwstat` `sys-remove` `sys-dup` `sys-dup2` `sys-chdir` `sys-getcwd` `sys-pipe` `sys-fd2path` `sys-bind` `sys-mount` `sys-unmount` `sys-nsinfo` `sys-putc` `sys-puts` `sys-getc` `sys-puthex` `sys-time` `sys-time-set` and `sys-rtc`.

A.14 Numbers (`numbers`)

The number stack's words; `N:` shows it. Any number Forth's cells can't hold goes on the number stack.

WORD	STACK	DOES
`ndepth`	(-- *u*)	How many numbers
`ndrop`, `ndup`, `nover`, `nswap`, `nrot`	N:	As the cell words
`n+`, `n-`, `n*`, `n/`	N: (*x y* -- *z*)	Exactly (`n/` gives fractions)
`n/mod`	N: (*x y* -- *rem quot*)	Whole-number division
`nnegate`, `nabs`, `ngcd`, `npow`	N:	Negate; size; greatest common divisor; a power
`ncompare`	(-- -1 | 0 | 1) N: (*x y* --)	Compare
`n=`, `n<`, `n>`, `n0=`, `n0<`	(-- *flag*) N: (... --)	Tests
`int?`, `fixed?`, `rational?`, `complex?`	(-- *flag*) N: (*x* --)	Its kind
`s>n`, `d>n`	(*n* --) N: (-- *x*), (*d* --) N: (-- *x*)	From Forth's stack
`n>s`, `n>d`	(-- *n*) N: (*x* --), (-- *d*) N: (*x* --)	To Forth's stack
`truncate`, `nfloor`, `nround`	N: (*x* -- *n*)	To a whole number
`to-fixed`	(*u* --) N: (*x* -- *y*)	A fixed decimal of *u* places
`to-rational`, `rational.n`, `rational.d`	N:	A fraction; its top; its bottom
`complex`	N: (*re im* -- *z*)	A complex number
`nrandom`, `nseed`	N: (*n* -- *r*), (*u* --)	Random numbers
`nfib`	N: (*n* -- *f*)	A Fibonacci number
`nand`, `nor`, `nxor`, `ninvert`	N:	Bits
`nlshift`, `nrshift`	(*u* --) N: (*x* -- *y*)	Shift
`nbit?`	(*u* -- *flag*) N: (*x* --)	A bit
`nbytes`, `nfrom-bytes`	(-- *c-addr u*) N: (*x* --), (*c-addr u* --) N: (-- *x*)	The stored form's bytes
`n.`, `n.base`	N: (*x* --), (*c-addr u* --) N: (*x* --)	Print in the base; in a base named
`n>str`	(-- *c-addr u*) N: (*x* --)	As a string
`n.s`	(--)	Show the number stack
`>n`	(*c-addr u* -- *flag*) N: (-- *x* |)	Read a number from a string
`set-base`, `get-base`	(*c-addr u* --), (-- *c-addr u*)	The base (any of Chapter 18's)
`nformat`	(*c-addr u* --) N: (*x1 ... xn* --)	Print a format, each `{}` a number
`nsqrt` `nexp` `nlog` `nsin` `ncos` `ntan` `natan` `npi`	N:	The math functions
`digits`	(-- *addr*)	The variable holding their precision (12)

WORD	STACK	DOES
`nliteral` (I C), `nconstant`, `nvariable`, `nvalue`, `n@`, `n!`		Numbers in definitions, constants, variables

For example:

```
/> lib numbers
/> 1000000 ndup n* n.
1000000000000
/> 2/3 1 s>n n+ n.
5/3
/> 1.25 2 s>n n* n.
2.5
/> 1 s>n 3 s>n n/ 4 to-fixed n.
0.3333
/> 30 s>n nfib n.
832040
/> 2 s>n nsqrt n.
1.41421356237
/> 255 s>n s" x" n.base
FF
```

Once the library's loaded, a line at the shell's prompt can start with any of its numbers, as `1000000` and `2/3` do here (section 20.8).

A.15 Sound (`sound`)

`snd-reset`, `snd-volume` (*v* --), `snd-claim` `snd-release` `snd-claim-psg` `snd-release-psg` (*mask* --), `snd-reg` (*reg val* --), `snd-patch` (*ch p* --), `snd-note` (*ch n* --), `snd-off` (*ch* --), `snd-level` (*ch v* --), `snd-pan` (*ch pan* --), `snd-bend` `snd-glide` `snd-drum` (*ch n* --), `snd-freq` (*ch hz* --), `snd-sens` (*ch pms ams* --), `snd-noise` (*n* --), `snd-wave` (*ch w width* --), `snd-lfo` (*rate pmd amd wave* --), `snd-regs` (*c-addr* --), `snd-play` (*c-addr u times* -- *status*), `snd-mml` (*ch c-addr u* -- *status*), `snd-chord` (*ch c-addr u* -- *status*), `note-of` (*c-addr u* -- *n*), `tune` (*c-addr u ch tempo* --): section 20.6.

A.16 Bits, random numbers and the disassembler

WORD	STACK	DOES
`tbit`	(*n b* -- *n flag*)	Is bit *b* of *n* set? (`bits`)
`sbit`, `cbit`	(*n b* -- *n'*)	Set, clear bit *b*
`rand`, `rand32`	(-- *u*), (-- *ud*)	A random number (`random`)
`random`	(*u1* -- *u2*)	One from 0 to *u1* - 1
`rseed`	(*ud* --)	Start the random numbers
`disasm`	(*addr u* --)	Disassemble *u* instructions from *addr* (`disasm`: Chapter 28)

For example:

```
/> lib bits
/> 5 0 tbit . .
-1 5
/> 0 3 sbit .
8
/> lib random
/> 6 random 1+ .
6
/> 6 random 1+ .
6
```

A.17 The device libraries

These are Forth source files, in `/lib/forth`. Their words are described in Chapter 20:

LIBRARY	WORDS
gpio	gpio gpio! gpio-in gpio-out gpio-port gpio-port! gpio-ddr! gpio-ca1! gpio-ca2! gpio-wait gpio-state
i2c	i2c-read i2c-write i2c-speed i2c-reg-size i2c-devices i2c?
spi	spi spi-read spi-mode
clock	set-date rtc
disk	disk-ctl disk-cmd disk-start disk-stop cards
cons	window windows new-window new-group show-window window-label window-status window-ctl
proc	task-args task-cwd task-env task-ns task-regs task-mem task-ram
pc	pc?
video	bitmap bitmap-off border pen plot line box bar circle disc clear text home cs pu pd rt lt heading seth fd bk vpoke vpeek vram! vram@ palette! sprite! sprite-at sprite-off vsync mouse mouse-wait

A.18 Errors

HyForth's errors are the standard's **throw codes**; these are the ones you'll see most:

CODE	MESSAGE
-3, -4	Stack overflow, stack underflow
-9	Invalid memory address
-10	Division by zero
-11	Result out of range
-13	Undefined word (shown as the word and a `?`)
-14	Interpreting a compile-only word
-24	Invalid numeric argument
-28	User interrupt (CTRL-C)
-512 - *e*	A system error, *e* from Appendix G (as the file words' iors are)

APPENDIX **B**

hylang Reference

This appendix lists all of hylang: its syntax, its special forms, its built-in functions, and its library. Arguments in square brackets can be left out; `...` means as many as you like.

B.1 Syntax

WRITTEN	IS
`(f a b ...)`	An **S-expression**: a call of *f* with the values of *a*, *b* ...
`{a b ...}`	A **Q-expression**: a list, not worked out (code to run later, or names)
`[a b ...]`	`(list a b ...)`: a list of the values
`42`, `-3`, `2/3`, `1.25`, `2i`, `1+2i`, `1_000`, `#xFF` ...	Numbers, in any of Chapter 18's forms; read in the current base
`"text"`	A string; escapes `\n \t \r \0 \a \b \f \v \e \\ \"` and `\x`*HH*
`"""text"""`	A here string: the text as it is (as many quotes close it as opened it)
`\a`, `\space`, `\newline`, `\tab`	Characters: one, or one by its name
`:name`	An atom
`name`	A symbol (in small letters); `T` and `NIL` are true and false
`$name`	`(env "name")`: an environment variable
`; text`	A comment, to the end of the line
`:(...)` `=(...)` `?(...)` `@(...)` `#(...)` `.(...)` `~(...)`	`(def ...)`, `(set ...)`, `(if ...)`, `(fn ...)`, `(hash-create ...)`, `(unpack ...)`, `(format ...)`
`&_`	In a function: its arguments beyond its formals, as a list
`&1`, `&2` ...	In a function made with no formals: its arguments, by position
`&0`	In a method: the hash

For example:

```
hylang> (+ 1 2)
=> 3
hylang> {+ 1 2}
=> {+ 1 2}
hylang> [1 (+ 1 1) "three"]
=> {1 2 "three"}
hylang> (len "Hydra")
=> 5
hylang> \a
=> \a
hylang> :(lives 3)
=> NIL
hylang> ?((> lives 0) "playing" "game over")
=> "playing"
hylang> ((fn {} {+ &1 &2}) 3 4)
=> 7
```

A prefix before `{` instead of `(` makes a Q-expression of the call: `?{a b c}` is `{if a b c}`.

B.2 Special forms

These get their arguments as they're written, and decide themselves what to work out.

FORM	WHAT IT DOES
`(def {names} values ...)`	Give names values, globally; `NIL`
`(set {names} values ...)`	The same, in this scope
`(set! {names} values ...)`	The same, where each name was made
`(let {{name value} ...} body ...)`	The body in a scope of its own, with the names given values
`(fn {formals} {body})`	A function
`(fun {name formals ...} {body})`	Define a named function
`(fexpr {formals} {body})`	A function that gets its arguments as they're written
`(if test then [else])`	*then* if *test* isn't `NIL`, else *else*
`(do x ...)`	Each in turn; the last one's value
`(and x ...)`, `(or x ...)`	The first `NIL`, or the last value; the first value that isn't `NIL`
`(while test body ...)`	Repeat the body while *test* isn't `NIL`
`(each {x list} body ...)`, `(each f list)`	The body with *x* each item (or character); or *f* called with each
`(dotimes {i n} body ...)`, `(dotimes n f)`	The body *n* times, *i* 0 to *n* - 1
`(<=> n {test value} ...)`	The value of the first case whose test (= <> < > <= >=) is true of *n*'s sign
`(try x [handler])`	*x*'s value; if that's an error, the handler's (with `&err` and `&code`), or `NIL`
`(eval x)`	Run a Q-expression as code
`(output-of x ...)`	What the expressions print, as a string
`(defined? x)`, `(expr? x)`	Is *x* a name in use here? Is it an expression?
`(hold body ...)`	The body, with no other task given a turn meanwhile
`(load path ...)`	Run each file's expressions
`(newns)`	Build the namespace again

FORM	WHAT IT DOES
`(exit [n])`	Leave hylang, with status *n*

For example:

```
hylang> (def {x y} 3 4)
=> NIL
hylang> (let {{a 10} {b 20}} (* a b))
=> 200
hylang> (fun {hyp2 a b} {+ (* a a) (* b b)})
=> NIL
hylang> (hyp2 x y)
=> 25
hylang> (if (> x y) "x is bigger" "y is bigger")
=> "y is bigger"
hylang> (and (> x 0) (> y 0))
=> T
hylang> (do (print "one") (print "two") 3)
one
two
=> 3
hylang> (try (/ 1 0) (+ "caught: " &err))
=> "caught: Division by zero."
hylang> (eval {+ 1 2})
=> 3
hylang> (output-of (print "hello"))
=> "hello\n"
hylang> (defined? {x})
=> T
```

B.3 Lists

FUNCTION	GIVES
`(list x ...)`	A list of the values
`(len l)`	Its length (a string's too)
`(head l)`, `(tail l)`	A list of the first item; all but the first
`(init l)`, `(end l)`	All but the last; a list of the last
`(fst l)` `(snd l)` `(thd l)` `(last l)`, `(nth n l)`	An item (from 0)
`(item-at l i)`	Item *i*, as it's written
`(subset l i [n])`	*n* items from *i*
`(cons x l)`	*x* in front of *l*
`(join l ...)`	The lists joined
`(take n l)`, `(drop n l)`	The first *n*; all but them
`(reverse l)`	Backwards (a string's too)
`(range to)`, `(range from to [step])`	Numbers, up to (not including) *to*
`(sort l [less])`	In order (by `cmp`, or by *less*)
`(elem? x l)`, `(in? x l)`	Is *x* in *l*?
`(map f l)`	*f* of each item
`(filter f l)`	The items *f* is true of

FUNCTION	GIVES
`(foldl f init l)`, `(foldr f init l)`	Combine the items, from the left, from the right
`(any? f l)`, `(all? f l)`	Is *f* true of any item? Of every one?
`(find f l)`, `(count f l)`	The first item *f* is true of; how many
`(sum l)`, `(product l)`	All the items added; multiplied
`(min x ...)`, `(max x ...)`	The smallest; the largest

For example:

```
hylang> (def {l} {5 3 8 1})
=> NIL
hylang> (len l)
=> 4
hylang> (fst l)
=> 5
hylang> (nth 2 l)
=> 8
hylang> (tail l)
=> {3 8 1}
hylang> (cons 9 l)
=> {9 5 3 8 1}
hylang> (join l {7 7})
=> {5 3 8 1 7 7}
hylang> (take 2 l)
=> {5 3}
hylang> (reverse l)
=> {1 8 3 5}
hylang> (sort l)
=> {1 3 5 8}
hylang> (range 2 11 3)
=> {2 5 8}
hylang> (map (fn {n} {* n n}) l)
=> {25 9 64 1}
hylang> (filter (fn {n} {> n 2}) l)
=> {5 3 8}
hylang> (foldl + 0 l)
=> 17
hylang> (any? (fn {n} {> n 7}) l)
=> T
hylang> (count (fn {n} {> n 2}) l)
=> 3
hylang> (sum l)
=> 17
hylang> (max 5 3 8 1)
=> 8
hylang> (elem? 8 l)
=> T
```

B.4 Numbers

FUNCTION	GIVES
`(+ x ...)`, `(- x ...)`, `(* x ...)`, `(/ x ...)`	Sum, difference, product, quotient (exact)
`(1+ x)`, `(1- x)`, `(abs x)`	Plus one; minus one; its size
`(truncate x)`	The whole part (toward zero)
`(to-fixed x [places])`, `(to-rational x)`	As a fixed decimal; as a fraction

FUNCTION	GIVES
`(rational.n x)`, `(rational.d x)`	Its top; its bottom
`(complex re im)`	A complex number
`(val s)`	The number a string holds
`(fib n)`	The *n*th Fibonacci number
`(random n)`	A random whole number from 0 to *n* - 1
`(base)`, `(base b)`	The base; set it (giving the one before)
`(digits)`, `(digits n)`	The math functions' precision; set it
`(sqrt x)` `(exp x)` `(log x)` `(sin x)` `(cos x)` `(tan x)` `(atan x)` `(pi)` `(pow x y)`	The math functions
`(number-bytes n)`, `(bytes-number b)`	A number as bytes, and back
`(== x y)`, `(eq x y)`, `(neq x y)`	Equal; not equal
`(< x y)` `(> x y)` `(<= x y)` `(>= x y)`, `(cmp x y)`	Comparisons; -1, 0 or 1
`(neg? x)` `(pos? x)` `(zero? x)` `(one? x)` `(not x)`	Tests
`(bit-and n ...)` `(bit-or n ...)` `(bit-xor n ...)` `(bit-not n)`	Bits
`(shl n k)`, `(shr n k)`, `(bit? n k)`	Shifts; a bit
`(hex n [width])`, `(bin n [width])`	Its digits, in hex or binary
`(lo n)`, `(hi n)`, `(word lo hi)`	Its low byte, its next; two bytes made a word
`(bytes s)`, `(from-bytes l)`	A string's bytes; a string from bytes

For example:

```
hylang> (/ 22 7)
=> 22/7
hylang> (to-fixed (/ 22 7) 4)
=> 3.1428
hylang> (truncate 22/7)
=> 3
hylang> (* 1.5 4)
=> 6
hylang> (pow 2 100)
=> 1267650600228229401496703205376
hylang> (sqrt -4)
=> 2i
hylang> (fib 40)
=> 102334155
hylang> (cmp 3 7)
=> -1
hylang> (bit-and 12 10)
=> 8
hylang> (shl 1 10)
=> 1024
hylang> (hex 255)
=> "FF"
hylang> (bin 10 8)
=> "00001010"
hylang> (word 52 18)
=> 4660
hylang> (val "2/3")
=> 2/3
```

B.5 Strings and characters

FUNCTION	GIVES
`(+ s x ...)`	The values joined, as `print` shows them
`(substring s i [n])`, `(char-at s i)`	Part of a string; a character
`(index-of s sub)`, `(last-index-of s sub)`	Where *sub* first, last, is (or `NIL`)
`(str-split s [sep])`, `(str-join l [sep])`, `(str-chars s)`	Split; join; its characters
`(str-upper s)`, `(str-lower s)`, `(str-trim s)`	Capitals; small letters; trimmed
`(str-replace s old new)`, `(str-repeat s n)`	Replaced; repeated
`(str-pad-left s n [c])`, `(str-pad-right s n [c])`	Padded to *n* characters
`(format text x ...)`	The text, each `{}` the next value
`(char-code c)`, `(code-char n)`	A character's code; the character for a code
`(alpha? c)` `(digit? c)` `(space? c)` `(upper? c)` `(lower? c)`	Its kind
`(to-str x [base])`, `(repr x)`	As `print` shows it; as the prompt shows it
`(to-sym x)`, `(to-atom x)`	A symbol; an atom
`(read text)`	The expressions in the text, not worked out
`(gensym [prefix])`	A new symbol, unlike any other

For example:

```
hylang> (+ "Hydra" "-" 16)
=> "Hydra-16"
hylang> (substring "Hydra-16" 0 5)
=> "Hydra"
hylang> (index-of "Hydra-16" "-")
=> 5
hylang> (str-split "one two three" " ")
=> {"one" "two" "three"}
hylang> (str-join {"a" "b" "c"} "-")
=> "a-b-c"
hylang> (str-upper "Hydra")
=> "HYDRA"
hylang> (str-replace "a-b-c" "-" "+")
=> "a+b+c"
hylang> (str-repeat "ab" 3)
=> "ababab"
hylang> (str-pad-left "7" 3 "0")
=> "007"
hylang> (format "{} is {} years old" "Sam" 12)
=> "Sam is 12 years old"
hylang> (char-code \A)
=> 65
hylang> (code-char 66)
=> \B
hylang> (to-str 255 "x")
=> "FF"
hylang> (read "(+ 1 2)")
=> {(+ 1 2)}
```

B.6 Types

`(type-of x)` gives `:number`, `:string`, `:char`, `:atom`, `:symbol`, `:function`, `:error`, `:t`, `:list`, `:sexpr` or `:hash`. And each kind has a test: `t?`, `nil?`, `num?`, `int?`, `fixed?`, `rational?`, `complex?`, `atom?`, `symbol?`, `string?`, `char?`, `function?`, `error?`, `qexpr?`, `sexpr?`.

For example:

```
hylang> (type-of 42)
=> :number
hylang> (type-of "42")
=> :string
hylang> (type-of {4 2})
=> :list
hylang> (num? 42)
=> T
hylang> (rational? 2/3)
=> T
hylang> (string? 42)
=> NIL
hylang> (nil? {})
=> T
```

B.7 Hashes

FUNCTION	WHAT IT DOES
`(hash-create [entries])`, `(to# entries)`	A hash, from a list of `{key value}` entries
`(from# h)`	A hash as a list of entries
`(h key)`, `(hash-get h key)`	A key's value (`NIL` if none)
`(hash-put h {key value} ...)`	Add or change entries
`(hash-key? h key)`, `(hash-keys h)`, `(hash-values h)`	Is the key there? The keys; the values
`(hash-remove h key)`	Take an entry out
`(hash-call h key x ...)`, `(h key x ...)`	Call a method (`&0` the hash)
`(hash-clone h [override ...])`	A copy
`(hash-add-tag h tag ...)`, `(hash-tag? h tag)`	Tags, on the hash or an entry
`(hash-lock h [key ...])`, `(hash-make-const ...)`, `(hash-make-private ...)`, `(hash-make-not-nil ...)`	Protect the hash, or entries
`(hash-locked? h [key])`, `(hash-private? ...)`, `(hash-const? ...)`	Are they?

For example:

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (pet :name)
=> "Rex"
hylang> (hash-put pet {:legs 3} {:colour "brown"})
=> {4 NIL}
hylang> (hash-keys pet)
=> {:name :legs :colour}
hylang> (hash-key? pet :wings)
=> NIL
hylang> (hash-remove pet :colour)
=> "brown"
hylang> (from# pet)
=> {{:name "Rex"} {:legs 3}}
hylang> (def {twin} (hash-clone pet))
=> NIL
hylang> (hash-put twin {:name "Max"})
=> "Rex"
hylang> (twin :name)
=> "Max"
hylang> (pet :name)
=> "Rex"
```

B.8 Errors, output and input

FUNCTION	WHAT IT DOES
`(error message [code])`	An error
`(error-message e)`, `(error-code e)`	Its message; its code
`(print x ...)`	Print the values, spaces between, and a new line
`(write x ...)`	Print them, without the new line
`(save x [path mode ...])`	Print *x* as it reads back, or write it to a file
`(read-line [s])`, `(read-byte [s])`, `(read-all [s])`	Read a line, a byte, the rest (from the keyboard, or a stream)
`(key)`, `(key?)`	The next key, as it's pressed; is one waiting?

For example:

```
hylang> (def {e} (error "out of cheese" :cheese))
=> NIL
hylang> (error-message e)
=> "out of cheese"
hylang> (error-code e)
=> :cheese
hylang> (print "one" 2 {3})
one 2 {3}
=> NIL
hylang> (write "no new line")
no new line=> NIL
hylang> (save "a string")
"a string"
=> NIL
```

B.9 Files and streams

FUNCTION	WHAT IT DOES
`(open path [mode])`	A stream on a file: `:read`, `:write`, `:append` or `:update`
`(close s)`	Close it
`(print-to s x ...)`, `(write-to s x ...)`	Write to it
`(read-bytes s n)`, `(write-bytes s bytes [i [n]])`	Bytes
`(seek s n)`, `(tell s)`	Its place
`(read-file p)`, `(read-lines p)`	A whole file; its lines
`(write-file p x ...)`, `(append-file p x ...)`	Write a file; add to it
`(ls [p])`, `(dir [p])`, `(stat p)`, `(glob pattern)`	Names; their details; one's; those matching
`(exists? p)`, `(dir? p)`, `(file? p)`	Tests
`(mkdir p)`, `(remove p)`, `(rename p q)`, `(copy-file p q)`	Directories and files
`(cd [p])`, `(cwd)`	The current directory

For example:

```
hylang> (write-file "/ram/pets.txt" "Rex\nMax")
=> NIL
hylang> (append-file "/ram/pets.txt" "\nTiddles")
=> NIL
hylang> (read-lines "/ram/pets.txt")
=> {"Rex" "Max" "Tiddles"}
hylang> (exists? "/ram/pets.txt")
=> T
hylang> ((stat "/ram/pets.txt") :length)
=> 15
hylang> (def {s} (open "/ram/pets.txt"))
=> NIL
hylang> (read-line s)
=> "Rex"
hylang> (tell s)
=> 4
hylang> (close s)
=> NIL
hylang> (rename "/ram/pets.txt" "/ram/cats.txt")
=> NIL
hylang> (ls "/ram")
=> {"bin" "cats.txt" "lib"}
hylang> (remove "/ram/cats.txt")
=> NIL
```

B.10 Buffers

A **buffer** is a block of bytes, which can be called with an index for a byte: `(b 3)`.

FUNCTION	WHAT IT DOES
`(buffer n [fill])`, `(buffer s)`, `(buffer l)`	A buffer of *n* bytes; of a string's; of a list's
`(buffer? x)`	Is it one?
`(buffer-get b i)`, `(buffer-put b i byte)`	A byte; set one

FUNCTION	WHAT IT DOES
`(buffer-fill b byte [i [n]])`, `(buffer-copy to at from [i [n]])`	Fill; copy
`(read-buffer s b [at [n]])`	Read a stream into it
`(buffer-cmp b at x [i [n]])`	Compare with another buffer or string

For example:

```
hylang> (def {b} (buffer 4 0))
=> NIL
hylang> (buffer-put b 0 65)
=> 0
hylang> (buffer-put b 1 66)
=> 0
hylang> (b 0)
=> 65
hylang> (buffer-get b 1)
=> 66
hylang> (buffer? b)
=> T
```

B.11 Programs, time and the system

FUNCTION	WHAT IT DOES
`(run prog x ...)`, `(spawn prog x ...)`, `(wait task)`, `(kill task)`	Programs
`(sh line [input])`, `(sh-out line [input])`	Command lines (their status; their output)
`(pid)`	This task
`(env [name])`, `(setenv name value)`, `(unsetenv name)`	The environment
`(time)`, `(date [t])`, `(date-parts [t])`, `(seconds-of y m d [h mi s])`	The clock
`(ticks)`, `(tick-rate)`, `(clock)`	Ticks (0-32,767), 200 a second; seconds since hylang started
`(sleep s)`, `(sleep-until tick)`, `(yield)`	Wait
`(platform)`, `(hydra?)`	`:hydra`; `T`
`(sysinfo)`, `(mods)`, `(ps)`, `(task-info task)`, `(errstr code)`, `(free)`	The system
`(peek a)` `(poke a b)` `(peek-word a)` `(poke-word a n)`	Memory
`(banks)` `(bank-alloc n)` `(bank-free b n)` `(bank-read b off n)` `(bank-write b off bytes)`	RAM banks
`(seg-create n)` `(seg-attach s)` `(seg-detach s)` `(seg-read s b off n)` `(seg-write s b off bytes)`	Shared segments
`(bind new old ...)`, `(mount dev old ...)`, `(unmount old [new])`, `(ns)`	The namespace (`:before`, `:after`, `:create`)
`(note task n)`, `(note-group g n)`, `(on-note f)`	Notes
`(sys name x ...)`, `(sys-`*name* `x ...)`	A system call (Appendix K)

For example:

```
hylang> (run "echo" "hello" "from" "hylang")
hello from hylang
=> 0
hylang> (sh-out "echo hi")
=> "hi\n"
hylang> (sh "ls /nowhere")
ls: /nowhere: not found
=> 1
hylang> (setenv "colour" "blue")
=> NIL
hylang> (env "colour")
=> "blue"
hylang> (date)
=> "2000-01-01 00:00:09"
hylang> (tick-rate)
=> 200
hylang> (platform)
=> :hydra
hylang> (hex (peek-word 65532) 4)
=> "E000"
hylang> (sys-getpid)
=> 2
```

B.12 The library

hylang's library (`globals`, loaded as it starts) adds:

FUNCTION	WHAT IT DOES
`math.pi` `math.e` `math.phi` `math.gamma`	Constants, to many places
`(square n)`, `(cube n)`, `(recip n)` (`inverse`)	n^2, n^3, $1/n$
`(floor a)` `(ceil a)` `(round a)` `(trunc a)`	Whole numbers
`(div a b)`, `(mod a b)`, `(divmod a b)`, `(fdiv a b)`	Division, rounding down
`(odd? n)` `(even? n)` `(positive? a)` `(negative? a)`	Tests
`(xor a b)`, `(<> a b)` (`!=`)	Logic; not equal
`(=0 n)` `(=1 n)` `(>0 n)` `(<0 n)` `(>=0 n)` `(<=0 n)` `(!=0 n)` `(!=1 n)`	Comparisons with 0 and 1
`first` … `tenth`	Items 1 to 10 of a list
`(split n l)`, `(zip a b)`, `(flatten l)`, `(distinct l)`, `(avg l)`	Lists
`(minimum ...)`, `(maximum ...)`, `(sort-desc l)`, `(best-of n l)`, `(worst-of n l)`	Order
`(remove-fst f l)`, `(remove-all f l)`, `(remove-min ...)`, `(remove-max ...)`	Removing items
`(between? lo hi x)` (`><?`), `(not-between? ...)` and others	Is *x* between *lo* and *hi*?
`(starts-with? s sub)`, `(ends-with? s sub)`, `(contains? s sub)`	Strings
`(repeat n d)` (`times`)	A list of *d*'s value *n* times
`(cond {test value} ...)`, `(case x {value result} ...)`	Choices
`(flip f a b)`, `(comp f g x)`, `(unpack f l)` (`apply`, `curry`), `(pack f ...)` (`uncurry`)	Functions on functions
`(?? a ...)`	The first argument that isn't `NIL`
`(use "name")`	Load a library once
`(buffer-cmp ...)`	(B.10)

The device libraries (`gpio`, `i2c`, `spi`, `clock`, `disk`, `cons`, `proc`, `pc`, `screen`, `snd` and `video`) and `dice` are loaded with `(use "name")`; their functions are in sections 22.5 to 22.7.

For example:

```
hylang> (square 12)
=> 144
hylang> (round 7/2)
=> 4
hylang> (mod 17 5)
=> 2
hylang> (even? 10)
=> T
hylang> (third {10 20 30})
=> 30
hylang> (zip {1 2 3} {"a" "b" "c"})
=> {{1 "a"} {2 "b"} {3 "c"}}
hylang> (distinct {1 2 2 3 1})
=> {1 2 3}
hylang> (avg {1 2 3 4})
=> 5/2
hylang> (contains? "Hydra-16" "dra")
=> T
hylang> (cond {(> 1 2) "no"} {T "yes"})
=> "yes"
hylang> (case 2 {1 "one"} {2 "two"})
=> "two"
hylang> (?? NIL NIL 7)
=> 7
```

And the libraries loaded with `use` (Chapter 22): `gpio`, `i2c`, `spi`, `clock`, `disk`, `cons`, `proc`, `pc`, `snd`, `video`, `screen`, `dev`; and `dice` (with `load`): `(d6)`, `(d20)`, `(dice n sides)`, and more for games.

APPENDIX C

HyBASIC Reference

This appendix lists HyBASIC's keywords, in alphabetical order. In the syntax, words in `CAPITALS` are typed as they are, *italic* words stand for something you choose, `[ ]` marks something you can leave out, and `|` separates choices. *n* and *x* are numbers, *s$* a string, *v* a variable, *f* a file number.

C.1 The rules

- **Lines.** A line is a statement, or several separated by `:`. With a number first, it's part of the program; without, it's done at once. A label (a name and a `:`) can mark a line instead of a number, for `GOTO` and `GOSUB`.
- **Names.** Letters, digits, _ and `.`, a letter first; capitals and small letters are the same. A name ending in `$` is a string's; any other a number's (the endings `%`, `&`, `!` and `#` are allowed, and mean a number).
- **Numbers.** Chapter 18's, in any of its forms, and `&H`, `&O`, `&B` and `E` notation. Every number variable can hold any kind of number: `INTEGER`, `LONG`, `SINGLE`, `DOUBLE` and `NUMBER` all mean "a number".
- **Strings.** In double quotes. `+` joins them.
- **Comments.** `REM`, or `'`, to the end of the line.
- **Truth.** A comparison gives -1 (true) or 0 (false); any number but 0 counts as true.
- **Operators**, from first to last: `^`; – (negation); `* /`; `\`; `MOD`; `+ –`; `= <> < > <= >=`; `NOT`; `AND`; `OR`; `XOR`; `EQV`; `IMP`.

C.2 The keywords

KEYWORD	SYNTAX	WHAT IT DOES
`ABS`	`ABS(x)`	The size of *x*
`ABSOLUTE`	`CALL ABSOLUTE (addr)`	Call machine code
`AND`	*a* `AND` *b*	True if both are; bit by bit on numbers
`APPEND`	`OPEN ... FOR APPEND`	Open a file to write on to its end
`ARG$`	`ARG$(n)`	A program's argument *n* (0: its file's name)
`AS`		In `DIM`, `OPEN`, `NAME` and `TYPE`
`ASC`	`ASC(s$)`	The code of the first character
`ASM`	`ASM` ... `END ASM`	Assembly language in the program (Chapter 27); `CALL ASM` *label* calls it
`ATN`	`ATN(x)`	The arctangent
`BANK`	`BANK n`, `BANK()`	Which RAM bank `PEEK` and `POKE` see at `$8000`
`BASE`	`BASE b$`; `OPTION BASE 0 | 1`	The base numbers are read and shown in; arrays' lowest index

KEYWORD	SYNTAX	WHAT IT DOES
BASE$	BASE$	The base's string
BEEP	BEEP	Ring the bell
BINARY	OPEN ... FOR BINARY	Open a file for bytes
BIT	BIT(x, n)	Bit *n* of *x*: 0 or 1
BYE	BYE [n]	Leave HyBASIC (status *n*)
CALL	CALL *name*[(*args*)]	Call a SUB (the CALL can be left out)
CASE	SELECT CASE *x* / CASE *items* / CASE ELSE / END SELECT	Choose by value: items are values, *a* TO *b*, or IS *op value*
CDBL, CSNG	CDBL(x)	*x* (no change: there's one kind of number)
CHDIR	CHDIR d$	Change directory
CHR$	CHR$(n)	The character with code *n*
CINT, CLNG	CINT(x)	The nearest whole number (a half to the even one)
CIRCLE	CIRCLE [STEP](x, y), r [, c [, start [, end [, aspect [, F]]]]]	A circle (F: filled)
CLEAR	CLEAR	(At the prompt) forget the variables
CLOSE	CLOSE [[#]f, ...]	Close files (all, if none named)
CLS	CLS	Clear the screen (and a bitmap)
COLOR	COLOR [fg] [, bg]	Colours (with a bitmap: the pen)
COMMAND$	COMMAND$	A program's arguments
COMPLEX	COMPLEX(re, im)	A complex number
CONST	CONST *name* = *value*, ...	Constants
CONT	CONT	Carry on after STOP or CTRL-C
COS	COS(x)	The cosine
CSRLIN	CSRLIN	The cursor's row
CVN	CVN(s$)	The number whose bytes *s$* is (MKN$'s opposite)
DATA	DATA *items*	Values for READ
DATE$	DATE$	The date: yyyy-mm-dd
DECLARE	DECLARE SUB | FUNCTION *name* (*params*)	(Allowed; not needed)
DEFINT ... DEFSTR	DEFSTR *a-z*, ...	Names starting with these letters are strings' (DEFSTR) or numbers' (the rest)
DELETE	DELETE *from-to*	Remove program lines
DENOMINATOR	DENOMINATOR(x)	A fraction's bottom
DIGITS	DIGITS n, DIGITS()	The math functions' precision
DIM	DIM [SHARED] *name*[(*bounds*)] [AS *type*], ...	Arrays, and variables of a type
DIR$	DIR$(path$), DIR$	A directory's first name; the next
DO	DO [WHILE | UNTIL *c*] ... LOOP [WHILE | UNTIL *c*]	A loop
DRAW	DRAW s$	Draw with moves (section 24.6)

KEYWORD	SYNTAX	WHAT IT DOES
EDIT	`EDIT [label]`	Edit the program in the screen editor
ELSE, ELSEIF		In `IF`
END	`END`; `END IF` | `SELECT` | `SUB` | `FUNCTION` | `TYPE`	End the program; end a block
ENV$, ENVIRON$	`ENV$(name$)`	An environment variable
ENVIRON	`ENVIRON "name=value"`	Set one (no value: remove it)
EOF	`EOF(f)`	-1 at the file's end
EQV	*a* `EQV` *b*	`NOT (a XOR b)`
ERASE	`ERASE` *array*, ...	Drop arrays
ERL, ERR, ERR$		The last error's line, number and message
ERROR	`ERROR n`	Make error *n*
EXIT	`EXIT FOR` | `DO` | `SUB` | `FUNCTION`	Leave a loop or procedure at once
EXP	`EXP(x)`	*e* to the power *x*
FIB	`FIB(n)`	The *n*th Fibonacci number
FILES	`FILES [path$]`	List a directory
FIX	`FIX(x)`	The whole part (toward zero)
FIXED	`FIXED(x, n)`	*x* as a fixed decimal of *n* places
FOR	`FOR` *v* = *a* `TO` *b* `[STEP` *s*`]` ... `NEXT [`*v*`]`	A counting loop
FRE	`FRE(0)`	Free memory, in bytes
FREEFILE	`FREEFILE`	A file number not in use
FUNCTION	`FUNCTION` *name*`(`*params*`)` ... `END FUNCTION`	A function: give its name the value
GCD	`GCD(a, b)`	The greatest common divisor
GET	`GET [#]f, [pos], v`	Read from a binary file
GOSUB	`GOSUB` *line*	Jump to a line, and come back at `RETURN`
GOTO	`GOTO` *line*	Jump to a line
GPRINT	`GPRINT [STEP](x, y), text$`	Text on the bitmap
HEX$	`HEX$(x)`	Its digits in hex
IF	`IF` *c* `THEN` ... `[ELSE` ...`]`; or a block to `END IF`	A decision
IMAG	`IMAG(z)`	A complex number's imaginary part
IMP	*a* `IMP` *b*	`(NOT a) OR b`
INCLUDE	`INCLUDE "file"`	Another file's lines, here
INKEY$	`INKEY$`	The key pressed, or `""`
INPUT	`INPUT [;] ["prompt" ; | ,] v, ...`; `INPUT #f, v, ...`	Read values
INPUT$	`INPUT$(n [, #f])`	*n* keys, or bytes of a file
INSTR	`INSTR([i,] s$, t$)`	Where *t$* is in *s$* (0: not there)
INT	`INT(x)`	Round down

KEYWORD	SYNTAX	WHAT IT DOES
IS	`CASE IS > 5`	In `CASE`
KILL	`KILL f$`	Remove a file
LBOUND, UBOUND	`LBOUND(a [, d])`	An array's lowest and highest index
LCASE$, UCASE$	`LCASE$(s$)`	Small letters; capitals
LEFT$, RIGHT$	`LEFT$(s$, n)`	The first, last, *n* characters
LEN	`LEN(s$)`	Its length
LET	`[LET]` *v* = *x*	Give a variable a value
LINE	`LINE [[STEP](x1, y1)]-[STEP](x2, y2) [, c [, B | BF]]`	A line, box or filled box
LINE INPUT	`LINE INPUT ["prompt";] v$; LINE INPUT #f, v$`	Read a whole line
LIST	`LIST [from][-[to]]`	Show the program
LOAD	`LOAD "file"`	Load a program
LOC, LOF	`LOC(f), LOF(f)`	The place in a file; its length
LOCATE	`LOCATE [row] [, [col] [, cursor]]`	Move the cursor (from 1)
LOG	`LOG(x)`	The natural logarithm
LOOP		In `DO`
LTRIM$, RTRIM$	`LTRIM$(s$)`	Without spaces at the start; at the end
MID$	`MID$(s$, i [, n])`	*n* characters from the *i*th
MKDIR, RMDIR	`MKDIR d$`	Make, remove a directory
MKN$	`MKN$(x)`	A number's bytes, as a string
MOD	*a* `MOD` *b*	The remainder
NAME	`NAME old$ AS new$`	Rename a file
NEW	`NEW`	Throw the program away
NEXT		In `FOR`
NOT	`NOT` *a*	The opposite
NUMERATOR	`NUMERATOR(x)`	A fraction's top
OCT$	`OCT$(x)`	Its digits in octal
ON	`ON n GOTO | GOSUB` *lines*; `ON ERROR GOTO` *line*	Jump to the *n*th line; catch errors (`GOTO 0`: stop catching)
OPEN	`OPEN path$ FOR INPUT | OUTPUT | APPEND | BINARY AS [#]f [LEN = n]`	Open a file
OPTION	`OPTION BASE 0 | 1`	Arrays' lowest index
OR	*a* `OR` *b*	True if either is
OUTPUT	`OPEN ... FOR OUTPUT`	Open a file to write
PAINT	`PAINT [STEP](x, y) [, c [, border]]`	Fill an area
PALETTE	`PALETTE n, rgb`	Change a colour
PEEK, POKE	`PEEK(addr), POKE addr, b`	Memory

KEYWORD	SYNTAX	WHAT IT DOES
PI	`PI`	π, to the precision
PLAY	`PLAY [ch,] mml$`; `PLAY file$`	Music
POINT	`POINT(x, y)`	A dot's colour (-1: off the bitmap)
POS	`POS(0)`	The cursor's column
PRESERVE	`REDIM PRESERVE`	Keep an array's contents
PRESET, PSET	`PSET [STEP](x, y) [, c]`	Clear, set a dot
PRINT	`PRINT [#f,] [USING fmt$;] items`	Print (`;` together, `,` in zones)
PUT	`PUT [#]f, [pos], x`	Write to a binary file
RANDOMIZE	`RANDOMIZE [n]`	Start the random numbers (none: from the clock)
RATIONAL	`RATIONAL(x)`	As a fraction
READ	`READ v, ...`	The next `DATA` values
REAL	`REAL(z)`	A complex number's real part
REDIM	`REDIM [PRESERVE]` *arrays*	Make arrays again
REM	`REM` *text*	A comment
RESTORE	`RESTORE [line]`	`READ` from the first `DATA` again
RESUME	`RESUME [0 | NEXT | line]`	After an error: again; the next statement; a line
RETURN	`RETURN [line]`; `RETURN` *value*	Back from `GOSUB`; a FUNCTION's value
RND	`RND`, `RND(x)`	A random number from 0 to 1
RREG	`RREG [a] [, x] [, y] [, p]`	The registers after the last `SYS`
RUN	`RUN`	Run the program
SAVE	`SAVE "file"`	Keep the program
SCREEN	`SCREEN n`	A graphics mode: 0, 1, 2, 7, 12 or 13 (section 24.6)
SEEK	`SEEK [#]f, pos`; `SEEK(f)`	The place in a file
SELECT	`SELECT CASE` *x*	(`CASE`)
SGN	`SGN(x)`	-1, 0 or 1
SHARED	`SHARED` *names*	In a procedure: use the main program's variables
SHELL	`SHELL [line$]`	Run a command line
SHELL$	`SHELL$(line$)`	A command line's output
SHL, SHR	`SHL(x, n)`	Shift bits left, right
SIN	`SIN(x)`	The sine
SLEEP	`SLEEP [s]`	Wait (none: until a key)
SOUND	`SOUND ch [, note [, patch [, vol]]]`; `SOUND "text"`	A note; a sound command
SPACE$	`SPACE$(n)`	*n* spaces
SPC	`PRINT SPC(n)`	*n* spaces, in `PRINT`
SPRITE	`SPRITE n, x, y [, image$]`; `SPRITE n OFF`	Sprites

KEYWORD	SYNTAX	WHAT IT DOES
SQR	`SQR(x)`	The square root
STATIC	`STATIC` *names*	In a procedure: variables kept between calls
STATUS	`STATUS`	The last `SHELL`'s status
STEP		In `FOR`, and graphics' points
STOP	`STOP`	Stop the program (`CONT` carries on)
STR$	`STR$(x [, base$])`	A number's text
STRING, STRING$	`AS STRING [* n]`; `STRING$(n, c)`	A string type; *n* of a character
SUB	`SUB` *name* (*params*) … `END SUB`	A procedure
SWAP	`SWAP a, b`	Exchange two variables' values
SYS	`SYS addr | "NAME" [, a [, x [, y]]]`	Machine code; a system call
SYSTEM	`SYSTEM [n]`	Leave HyBASIC
TAB	`PRINT TAB(n)`	To column *n*
TAN	`TAN(x)`	The tangent
THEN		In `IF`
TIME$, TIMER		The time; seconds since midnight
TO		In `FOR` and `CASE`
TYPE	`TYPE` *name* / *field* `AS` *type* / `END TYPE`	A record type
UNTIL		In `DO`
USING	`PRINT USING fmt$;`	A number's form
VAL	`VAL(s$)`	The number at the start of *s$* (0 if none)
VIEW	`VIEW [[SCREEN] (x1, y1)-(x2, y2)]`	Draw in only part of the bitmap
WHILE, WEND	`WHILE c` … `WEND`	A loop
WIDTH	`WIDTH n`	The screen's width
WINDOW	`WINDOW [[SCREEN] (x1, y1)-(x2, y2)]`	Coordinates of your own
WRITE	`WRITE [#f,] items`	Items, strings in quotes, commas between
XOR	*a* `XOR` *b*	True if one is, not both

C.3 Examples

Each of these was typed into HyBASIC, as it is here. The ones with line numbers are programs, run with `RUN`.

Numbers and printing

```
> PRINT 7 / 2; 7 \ 2; 7 MOD 2
 7/2  3  1
> PRINT 2 ^ 100
 1267650600228229401496703205376
> PRINT FIXED(22 / 7, 4), RATIONAL(0.75)
 3.1428        3/4
> PRINT INT(-2.5); FIX(-2.5); CINT(2.5); SGN(-9); ABS(-9)
-3 -2  2 -1  9
> PRINT SQR(16); SQR(-16); GCD(12, 18); FIB(30)
 4  4i  6  832040
> PRINT HEX$(255); " "; OCT$(8); " "; STR$(255, "b")
FF 10 11111111
> PRINT USING "###.##"; 3.14159
  3.14
> PRINT "A"; TAB(10); "B"; SPC(3); "C"
A         B   C
> WRITE "Sam", 12
"Sam",12
```

Strings

```
> A$ = "Hydra-16"
> PRINT LEN(A$); LEFT$(A$, 5); " "; RIGHT$(A$, 2); " "; MID$(A$, 2, 3)
 8 Hydra 16 ydr
> PRINT INSTR(A$, "-"); UCASE$(A$); " "; LCASE$(A$)
 6 HYDRA-16 hydra-16
> PRINT ASC("A"); CHR$(66); VAL("42 apples") + 1; STR$(7) + "!"
 65 B 43  7!
> PRINT STRING$(5, "*"); SPACE$(2); LTRIM$("   x")
*****  x
```

Decisions and loops

```
> 10 FOR I = 10 TO 1 STEP -3 : PRINT I; : NEXT : PRINT
> 20 N = 1 : DO WHILE N < 50 : N = N * 2 : LOOP : PRINT N
> 30 SELECT CASE N
> 40   CASE 1 TO 9 : PRINT "small"
> 50   CASE IS > 60 : PRINT "big"
> 60   CASE ELSE : PRINT "middling"
> 70 END SELECT
> 80 ON 2 GOSUB 100, 200 : END
> 100 PRINT "first" : RETURN
> 200 PRINT "second" : RETURN
> RUN
 10  7  4  1
 64
big
second
```

Arrays, data and records

```
> 10 DIM A(4)
> 20 FOR I = 0 TO 4 : READ A(I) : NEXT
> 30 DATA 5, 3, 8, 1, 9
> 40 IF A(0) > A(1) THEN SWAP A(0), A(1)
> 50 PRINT A(0); A(1); LBOUND(A); UBOUND(A)
> 60 TYPE PET
> 70   KIND AS STRING
> 80   LEGS AS INTEGER
> 90 END TYPE
> 100 DIM P AS PET
> 110 P.KIND = "dog" : P.LEGS = 4
> 120 PRINT "A "; P.KIND; " has"; P.LEGS; "legs"
> RUN
 3  5  0  4
A dog has 4 legs
```

Procedures

```
> 10 PRINT TWICE(21)
> 20 SHOUT "hello"
> 30 END
> 40 FUNCTION TWICE(N)
> 50   TWICE = N * 2
> 60 END FUNCTION
> 70 SUB SHOUT(S$)
> 80   PRINT UCASE$(S$); "!"
> 90 END SUB
> RUN
 42
HELLO!
```

Files

```
> OPEN "/ram/pets.txt" FOR OUTPUT AS #1
> PRINT #1, "Rex" : PRINT #1, "Max"
> CLOSE #1
> OPEN "/ram/pets.txt" FOR INPUT AS #1
> LINE INPUT #1, A$ : PRINT A$; EOF(1)
Rex 0
> LINE INPUT #1, A$ : PRINT A$; EOF(1)
Max-1
> CLOSE
> FILES "/ram"
bin/            lib/            pets.txt
> KILL "/ram/pets.txt"
```

Catching errors

```
> 10 ON ERROR GOTO 100
> 20 PRINT 1 / 0
> 30 PRINT "carried on"
> 40 END
> 100 PRINT "error"; ERR; "in line"; ERL; ": "; ERR$
> 110 RESUME NEXT
> RUN
error 11 in line 20 : division by zero
carried on
```

The system, and bits

```
> PRINT DATE$; " "; TIME$
2000-01-01 00:00:05
> SHELL "echo hello from the shell"
hello from the shell
> PRINT STATUS
 0
> A$ = SHELL$("date") : PRINT LEFT$(A$, 10)
2000-01-01
> ENVIRON "colour=blue" : PRINT ENV$("colour")
blue
> PRINT HEX$(PEEK(65533)); FRE(0) > 0
E0-1
> PRINT BIT(5, 0); SHL(1, 4); 12 AND 10; 12 OR 10; 12 XOR 10
 1  16  8  14  6
```

C.4 Errors

HyBASIC's error messages name what went wrong; `ERR` gives its number, `ERR$` its message, and `ERL` its line. A system error's number is 256 plus its code from Appendix G.

APPENDIX D

The 65C02's Instructions

This appendix lists every instruction of the W65C02S, as the assembler `as` writes it: what it does, the flags it changes, and its opcode and cycles in each of its addressing modes. Chapter 26 explains them; WDC's data sheet for the W65C02S has the chip's own tables.

D.1 The addressing modes

The tables write each mode short, like this:

SHORT	MODE	WRITTEN	BYTES
	Implied	`inx`	1
`a`	Accumulator	`asl a`	1
`#`	Immediate	`lda #$41`	2
`zp`	Zero page	`lda $22`	2
`zp,x`	Zero page, X	`lda $22,x`	2
`zp,y`	Zero page, Y	`ldx $22,y`	2
`abs`	Absolute	`lda $1234`	3
`abs,x`	Absolute, X	`lda $1234,x`	3
`abs,y`	Absolute, Y	`lda $1234,y`	3
`(zp)`	Zero page indirect	`lda ($22)`	2
`(zp),y`	Indirect, Y	`lda ($22),y`	2
`(zp,x)`	Indexed indirect	`lda ($22,x)`	2
`(abs)`	Absolute indirect	`jmp ($1234)`	3
`(abs,x)`	Absolute indexed indirect	`jmp ($1234,x)`	3
`rel`	Relative	`bne label`	2
`zp,rel`	Zero page, relative	`bbr3 $22,label`	3

D.2 The flags and the cycles

The **Flags** column lists the flags an instruction changes (section 26.3): **N** negative, **V** overflow, **B** break, **D** decimal, **I** interrupt disable, **Z** zero, **C** carry. A – means it changes none.

The number in brackets after each opcode is how many **cycles** it takes (at 3.58 MHz, a cycle is about 0.28 millionths of a second). Some take more:

- `+` after the number: one more if the indexing (X or Y) carries the address into another page.
- A branch (`BCC` ... `BVS`, `BBR`, `BBS`): one more if it's taken, and one more again if it goes to another page. `BRA` is always taken: 3 cycles (4 to another page).
- `ADC` and `SBC`: one more in decimal mode.

D.3 The instructions

In the table, *M* is the byte the instruction works on (its operand: section 26.6).

INSTRUCTION	WHAT IT DOES	FLAGS	OPCODES
`ADC`	Add with carry: A = A + M + C	N V Z C	(zp,x) `$61` (6), zp `$65` (3), # `$69` (2), abs `$6D` (4), (zp),y `$71` (5+), (zp) `$72` (5), zp,x `$75` (4), abs,y `$79` (4+), abs,x `$7D` (4+)
`AND`	A = A AND M	N Z	(zp,x) `$21` (6), zp `$25` (3), # `$29` (2), abs `$2D` (4), (zp),y `$31` (5+), (zp) `$32` (5), zp,x `$35` (4), abs,y `$39` (4+), abs,x `$3D` (4+)
`ASL`	Shift left: C gets bit 7, bit 0 gets 0	N Z C	zp `$06` (5), a `$0A` (2), abs `$0E` (6), zp,x `$16` (6), abs,x `$1E` (6+)
`BBR`*n*	Branch if bit *n* of a zero page byte is 0	-	BBR0 `$0F` (5), BBR1 `$1F` (5), BBR2 `$2F` (5), BBR3 `$3F` (5), BBR4 `$4F` (5), BBR5 `$5F` (5), BBR6 `$6F` (5), BBR7 `$7F` (5)
`BBS`*n*	Branch if bit *n* of a zero page byte is 1	-	BBS0 `$8F` (5), BBS1 `$9F` (5), BBS2 `$AF` (5), BBS3 `$BF` (5), BBS4 `$CF` (5), BBS5 `$DF` (5), BBS6 `$EF` (5), BBS7 `$FF` (5)
`BCC`	Branch if C = 0	-	rel `$90` (2)
`BCS`	Branch if C = 1	-	rel `$B0` (2)
`BEQ`	Branch if Z = 1 (equal)	-	rel `$F0` (2)
`BIT`	Test bits: Z = 1 if (A AND M) is 0; N and V get bits 7 and 6 of M (but not with #)	N V Z	zp `$24` (3), abs `$2C` (4), zp,x `$34` (4), abs,x `$3C` (4+), # `$89` (2)
`BMI`	Branch if N = 1 (minus)	-	rel `$30` (2)
`BNE`	Branch if Z = 0 (not equal)	-	rel `$D0` (2)
`BPL`	Branch if N = 0 (plus)	-	rel `$10` (2)
`BRA`	Branch always	-	rel `$80` (2)
`BRK`	Break: a software interrupt	B D I	`$00` (7)
`BVC`	Branch if V = 0	-	rel `$50` (2)
`BVS`	Branch if V = 1	-	rel `$70` (2)
`CLC`	C = 0	C	`$18` (2)
`CLD`	D = 0: binary arithmetic	D	`$D8` (2)
`CLI`	I = 0: interrupts allowed	I	`$58` (2)
`CLV`	V = 0	V	`$B8` (2)
`CMP`	Compare A with M (A - M, kept only in the flags)	N Z C	(zp,x) `$C1` (6), zp `$C5` (3), # `$C9` (2), abs `$CD` (4), (zp),y `$D1` (5+), (zp) `$D2` (5), zp,x `$D5` (4), abs,y `$D9` (4+), abs,x `$DD` (4+)
`CPX`	Compare X with M	N Z C	# `$E0` (2), zp `$E4` (3), abs `$EC` (4)
`CPY`	Compare Y with M	N Z C	# `$C0` (2), zp `$C4` (3), abs `$CC` (4)

INSTRUCTION	WHAT IT DOES	FLAGS	OPCODES
DEC	M = M - 1 (or A = A - 1)	N Z	a $3A (2), zp $C6 (5), abs $CE (6), zp,x $D6 (6), abs,x $DE (7)
DEX	X = X - 1	N Z	$CA (2)
DEY	Y = Y - 1	N Z	$88 (2)
EOR	A = A XOR M	N Z	(zp,x) $41 (6), zp $45 (3), # $49 (2), abs $4D (4), (zp),y $51 (5+), (zp) $52 (5), zp,x $55 (4), abs,y $59 (4+), abs,x $5D (4+)
INC	M = M + 1 (or A = A + 1)	N Z	a $1A (2), zp $E6 (5), abs $EE (6), zp,x $F6 (6), abs,x $FE (7)
INX	X = X + 1	N Z	$E8 (2)
INY	Y = Y + 1	N Z	$C8 (2)
JMP	Jump	-	abs $4C (3), (abs) $6C (6), (abs,x) $7C (6)
JSR	Jump to a subroutine (pushing the return address)	-	abs $20 (6)
LDA	A = M	N Z	(zp,x) $A1 (6), zp $A5 (3), # $A9 (2), abs $AD (4), (zp),y $B1 (5+), (zp) $B2 (5), zp,x $B5 (4), abs,y $B9 (4+), abs,x $BD (4+)
LDX	X = M	N Z	# $A2 (2), zp $A6 (3), abs $AE (4), zp,y $B6 (4), abs,y $BE (4+)
LDY	Y = M	N Z	# $A0 (2), zp $A4 (3), abs $AC (4), zp,x $B4 (4), abs,x $BC (4+)
LSR	Shift right: C gets bit 0, bit 7 gets 0	N Z C	zp $46 (5), a $4A (2), abs $4E (6), zp,x $56 (6), abs,x $5E (6+)
NOP	Nothing	-	$EA (2)
ORA	A = A OR M	N Z	(zp,x) $01 (6), zp $05 (3), # $09 (2), abs $0D (4), (zp),y $11 (5+), (zp) $12 (5), zp,x $15 (4), abs,y $19 (4+), abs,x $1D (4+)
PHA	Push A	-	$48 (3)
PHP	Push P	-	$08 (3)
PHX	Push X	-	$DA (3)
PHY	Push Y	-	$5A (3)
PLA	Pull A	N Z	$68 (4)
PLP	Pull P	N V D I Z C	$28 (4)
PLX	Pull X	N Z	$FA (4)
PLY	Pull Y	N Z	$7A (4)
RMB*n*	Clear bit *n* of a zero page byte	-	RMB0 $07 (5), RMB1 $17 (5), RMB2 $27 (5), RMB3 $37 (5), RMB4 $47 (5), RMB5 $57 (5), RMB6 $67 (5), RMB7 $77 (5)
ROL	Rotate left: C gets bit 7, bit 0 gets C	N Z C	zp $26 (5), a $2A (2), abs $2E (6), zp,x $36 (6), abs,x $3E (6+)
ROR	Rotate right: C gets bit 0, bit 7 gets C	N Z C	zp $66 (5), a $6A (2), abs $6E (6), zp,x $76 (6), abs,x $7E (6+)
RTI	Return from an interrupt (pulling P, then the address)	N V D I Z C	$40 (6)
RTS	Return from a subroutine	-	$60 (6)

INSTRUCTION	WHAT IT DOES	FLAGS	OPCODES
`SBC`	Subtract with borrow: A = A - M - (1 - C)	N V Z C	(zp,x) `$E1` (6), zp `$E5` (3), # `$E9` (2), abs `$ED` (4), (zp),y `$F1` (5+), (zp) `$F2` (5), zp,x `$F5` (4), abs,y `$F9` (4+), abs,x `$FD` (4+)
`SEC`	C = 1	C	`$38` (2)
`SED`	D = 1: decimal (BCD) arithmetic	D	`$F8` (2)
`SEI`	I = 1: interrupts held off	I	`$78` (2)
`SMB`*n*	Set bit *n* of a zero page byte	-	SMB0 `$87` (5), SMB1 `$97` (5), SMB2 `$A7` (5), SMB3 `$B7` (5), SMB4 `$C7` (5), SMB5 `$D7` (5), SMB6 `$E7` (5), SMB7 `$F7` (5)
`STA`	M = A	-	(zp,x) `$81` (6), zp `$85` (3), abs `$8D` (4), (zp),y `$91` (6), (zp) `$92` (5), zp,x `$95` (4), abs,y `$99` (5), abs,x `$9D` (5)
`STP`	Stop the processor, until a reset	-	`$DB` (3)
`STX`	M = X	-	zp `$86` (3), abs `$8E` (4), zp,y `$96` (4)
`STY`	M = Y	-	zp `$84` (3), abs `$8C` (4), zp,x `$94` (4)
`STZ`	M = 0	-	zp `$64` (3), zp,x `$74` (4), abs `$9C` (4), abs,x `$9E` (5)
`TAX`	X = A	N Z	`$AA` (2)
`TAY`	Y = A	N Z	`$A8` (2)
`TRB`	Test and reset bits: Z as BIT's; then M = M AND NOT A	Z	zp `$14` (5), abs `$1C` (6)
`TSB`	Test and set bits: Z as BIT's; then M = M OR A	Z	zp `$04` (5), abs `$0C` (6)
`TSX`	X = S	N Z	`$BA` (2)
`TXA`	A = X	N Z	`$8A` (2)
`TXS`	S = X	-	`$9A` (2)
`TYA`	A = Y	N Z	`$98` (2)
`WAI`	Wait for an interrupt	-	`$CB` (3)

D.4 The opcodes in order

Each square is an opcode: its row is the opcode's first hex digit, and its column the second. So `$A9`, row `Ax` and column `x9`, is `LDA #`, taking 2 cycles. An empty square is an opcode the W65C02S doesn't use: it does nothing, as a `NOP` does.

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	BRK 7	ORA (zp,x) 6			TSB zp 5	ORA zp 3	ASL zp 5	RMB0 zp 5	PHP 3	ORA # 2	ASL a 2		TSB abs 6	ORA abs 4	ASL abs 6	BBR0 zp,rel 5
1x	BPL rel 2	ORA (zp),y 5+	ORA (zp) 5		TRB zp 5	ORA zp,x 4	ASL zp,x 6	RMB1 zp 5	CLC 2	ORA abs,y 4+	INC a 2		TRB abs 6	ORA abs,x 4+	ASL abs,x 6+	BBR1 zp,rel 5
2x	JSR abs 6	AND (zp,x) 6			BIT zp 3	AND zp 3	ROL zp 5	RMB2 zp 5	PLP 4	AND # 2	ROL a 2		BIT abs 4	AND abs 4	ROL abs 6	BBR2 zp,rel 5
3x	BMI rel 2	AND (zp),y 5+	AND (zp) 5		BIT zp,x 4	AND zp,x 4	ROL zp,x 6	RMB3 zp 5	SEC 2	AND abs,y 4+	DEC a 2		BIT abs,x 4+	AND abs,x 4+	ROL abs,x 6+	BBR3 zp,rel 5
4x	RTI 6	EOR (zp,x) 6				EOR zp 3	LSR zp 5	RMB4 zp 5	PHA 3	EOR # 2	LSR a 2		JMP abs 3	EOR abs 4	LSR abs 6	BBR4 zp,rel 5

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
5x	BVC rel 2	EOR (zp),y 5+	EOR (zp) 5			EOR zp,x 4	LSR zp,x 6	RMB5 zp 5	CLI 2	EOR abs,y 4+	PHY 3			EOR abs,x 4+	LSR abs,x 6+	BBR5 zp,rel 5
6x	RTS 6	ADC (zp,x) 6			STZ zp 3	ADC zp 3	ROR zp 5	RMB6 zp 5	PLA 4	ADC # 2	ROR a 2		JMP (abs) 6	ADC abs 4	ROR abs 6	BBR6 zp,rel 5
7x	BVS rel 2	ADC (zp),y 5+	ADC (zp) 5		STZ zp,x 4	ADC zp,x 4	ROR zp,x 6	RMB7 zp 5	SEI 2	ADC abs,y 4+	PLY 4		JMP (abs,x) 6	ADC abs,x 4+	ROR abs,x 6+	BBR7 zp,rel 5
8x	BRA rel 2	STA (zp,x) 6			STY zp 3	STA zp 3	STX zp 3	SMB0 zp 5	DEY 2	BIT # 2	TXA 2		STY abs 4	STA abs 4	STX abs 4	BBS0 zp,rel 5
9x	BCC rel 2	STA (zp),y 6	STA (zp) 5		STY zp,x 4	STA zp,x 4	STX zp,y 4	SMB1 zp 5	TYA 2	STA abs,y 5	TXS 2		STZ abs 4	STA abs,x 5	STZ abs,x 5	BBS1 zp,rel 5
Ax	LDY # 2	LDA (zp,x) 6	LDX # 2		LDY zp 3	LDA zp 3	LDX zp 3	SMB2 zp 5	TAY 2	LDA # 2	TAX 2		LDY abs 4	LDA abs 4	LDX abs 4	BBS2 zp,rel 5
Bx	BCS rel 2	LDA (zp),y 5+	LDA (zp) 5		LDY zp,x 4	LDA zp,x 4	LDX zp,y 4	SMB3 zp 5	CLV 2	LDA abs,y 4+	TSX 2		LDY abs,x 4+	LDA abs,x 4+	LDX abs,y 4+	BBS3 zp,rel 5
Cx	CPY # 2	CMP (zp,x) 6			CPY zp 3	CMP zp 3	DEC zp 5	SMB4 zp 5	INY 2	CMP # 2	DEX 2	WAI 3	CPY abs 4	CMP abs 4	DEC abs 6	BBS4 zp,rel 5
Dx	BNE rel 2	CMP (zp),y 5+	CMP (zp) 5			CMP zp,x 4	DEC zp,x 6	SMB5 zp 5	CLD 2	CMP abs,y 4+	PHX 3	STP 3		CMP abs,x 4+	DEC abs,x 7	BBS5 zp,rel 5
Ex	CPX # 2	SBC (zp,x) 6			CPX zp 3	SBC zp 3	INC zp 5	SMB6 zp 5	INX 2	SBC # 2	NOP 2		CPX abs 4	SBC abs 4	INC abs 6	BBS6 zp,rel 5
Fx	BEQ rel 2	SBC (zp),y 5+	SBC (zp) 5			SBC zp,x 4	INC zp,x 6	SMB7 zp 5	SED 2	SBC abs,y 4+	PLX 4			SBC abs,x 4+	INC abs,x 7	BBS7 zp,rel 5

APPENDIX E

Characters

Every character is stored as a number, its **code**: one byte, from 0 to 255. This appendix lists them. The first 128 are **ASCII** (the American Standard Code for Information Interchange, from 1963), which nearly every computer uses. The Hydra uses **ISO-8859-15** for the codes from 160 to 255: the letters of the Western European languages, and signs like €, £ and ©.

In HyForth, `char A .` prints 65 and `65 emit` prints `A`; in hylang, `(char-code "A")` and `(code-char 65)`; in HyBASIC, `ASC("A")` and `CHR$(65)`.

E.1 The control characters

The codes from 0 to 31, and 127, aren't printed: they're **control characters**, which tell a terminal to do something. Each can be typed by holding CTRL and pressing the key in the **Key** column (so CTRL-C is code 3). The last column says what the Hydra's console does with each when a program prints it (Chapter 13), or, for the keys, what it does when you type it.

DEC	HEX	NAME	KEY	WHAT THE CONSOLE DOES
0	`$00`	NUL	CTRL-@	Nothing
1	`$01`	SOH	CTRL-A	Typed: to the start of the line
2	`$02`	STX	CTRL-B	
3	`$03`	ETX	CTRL-C	Typed: the `interrupt` note, to the programs in the window
4	`$04`	EOT	CTRL-D	Typed at the start of a line: the end of the input
5	`$05`	ENQ	CTRL-E	Typed: to the end of the line
6	`$06`	ACK	CTRL-F	
7	`$07`	BEL	CTRL-G	Rings the bell
8	`$08`	BS	CTRL-H, BACKSPACE	Moves the cursor left; typed: rubs out the character before the cursor
9	`$09`	HT	CTRL-I, TAB	Moves to the next tab stop
10	`$0A`	LF	CTRL-J	A new line
11	`$0B`	VT	CTRL-K	As LF
12	`$0C`	FF	CTRL-L	As LF
13	`$0D`	CR	CTRL-M, RETURN	Back to the first column; typed: ends a line
14	`$0E`	SO	CTRL-N	Uses character set G1
15	`$0F`	SI	CTRL-O	Uses character set G0
16	`$10`	DLE	CTRL-P	
17	`$11`	DC1	CTRL-Q	
18	`$12`	DC2	CTRL-R	

DEC	HEX	NAME	KEY	WHAT THE CONSOLE DOES
19	$13	DC3	CTRL-S	
20	$14	DC4	CTRL-T	
21	$15	NAK	CTRL-U	Typed: rubs out the whole line
22	$16	SYN	CTRL-V	
23	$17	ETB	CTRL-W	
24	$18	CAN	CTRL-X	Cancels an escape sequence
25	$19	EM	CTRL-Y	
26	$1A	SUB	CTRL-Z	Cancels an escape sequence
27	$1B	ESC	CTRL-[, ESC	Starts an escape sequence (section 13.7)
28	$1C	FS	CTRL-\	
29	$1D	GS	CTRL-]	
30	$1E	RS	CTRL-^	
31	$1F	US	CTRL-_	
127	$7F	DEL	BACKSPACE (on some keyboards)	Typed: rubs out the character before the cursor

Programs that read keys one at a time (in raw mode, section 13.3) get every code as it's typed, and decide for themselves what to do with it: `edit`, for one, uses many of them.

E.2 ASCII

The printable ASCII characters, from 32 (the space) to 126:

DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR
32	$20	(space)	56	$38	8	80	$50	P	104	$68	h
33	$21	!	57	$39	9	81	$51	Q	105	$69	i
34	$22	"	58	$3A	:	82	$52	R	106	$6A	j
35	$23	#	59	$3B	;	83	$53	S	107	$6B	k
36	$24	$	60	$3C	<	84	$54	T	108	$6C	l
37	$25	%	61	$3D	=	85	$55	U	109	$6D	m
38	$26	&	62	$3E	>	86	$56	V	110	$6E	n
39	$27	'	63	$3F	?	87	$57	W	111	$6F	o
40	$28	(	64	$40	@	88	$58	X	112	$70	p
41	$29	)	65	$41	A	89	$59	Y	113	$71	q
42	$2A	*	66	$42	B	90	$5A	Z	114	$72	r
43	$2B	+	67	$43	C	91	$5B	[	115	$73	s
44	$2C	,	68	$44	D	92	$5C	\	116	$74	t
45	$2D	-	69	$45	E	93	$5D	]	117	$75	u
46	$2E	.	70	$46	F	94	$5E	^	118	$76	v

DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR
47	$2F	/	71	$47	G	95	$5F	_	119	$77	w
48	$30	0	72	$48	H	96	$60	`	120	$78	x
49	$31	1	73	$49	I	97	$61	a	121	$79	y
50	$32	2	74	$4A	J	98	$62	b	122	$7A	z
51	$33	3	75	$4B	K	99	$63	c	123	$7B	{
52	$34	4	76	$4C	L	100	$64	d	124	$7C	|
53	$35	5	77	$4D	M	101	$65	e	125	$7D	}
54	$36	6	78	$4E	N	102	$66	f	126	$7E	~
55	$37	7	79	$4F	O	103	$67	g	127	$7F	(DEL)

E.3 ISO-8859-15

The codes from 128 to 159 are more control characters, which the Hydra doesn't use. From 160 to 255 are ISO-8859-15's characters:

DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR
160	$A0	(no-break space)	184	$B8	ž	208	$D0	Ð	232	$E8	è
161	$A1	¡	185	$B9	¹	209	$D1	Ñ	233	$E9	é
162	$A2	¢	186	$BA	º	210	$D2	Ò	234	$EA	ê
163	$A3	£	187	$BB	»	211	$D3	Ó	235	$EB	ë
164	$A4	€	188	$BC	Œ	212	$D4	Ô	236	$EC	ì
165	$A5	¥	189	$BD	œ	213	$D5	Õ	237	$ED	í
166	$A6	Š	190	$BE	Ÿ	214	$D6	Ö	238	$EE	î
167	$A7	§	191	$BF	¿	215	$D7	×	239	$EF	ï
168	$A8	š	192	$C0	À	216	$D8	Ø	240	$F0	ð
169	$A9	©	193	$C1	Á	217	$D9	Ù	241	$F1	ñ
170	$AA	ª	194	$C2	Â	218	$DA	Ú	242	$F2	ò
171	$AB	«	195	$C3	Ã	219	$DB	Û	243	$F3	ó
172	$AC	¬	196	$C4	Ä	220	$DC	Ü	244	$F4	ô
173	$AD	(soft hyphen)	197	$C5	Å	221	$DD	Ý	245	$F5	õ
174	$AE	®	198	$C6	Æ	222	$DE	Þ	246	$F6	ö
175	$AF	¯	199	$C7	Ç	223	$DF	ß	247	$F7	÷
176	$B0	º	200	$C8	È	224	$E0	à	248	$F8	ø
177	$B1	±	201	$C9	É	225	$E1	á	249	$F9	ù
178	$B2	²	202	$CA	Ê	226	$E2	â	250	$FA	ú
179	$B3	³	203	$CB	Ë	227	$E3	ã	251	$FB	û
180	$B4	Ž	204	$CC	Ì	228	$E4	ä	252	$FC	ü

DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR	DEC	HEX	CHAR
181	$B5	µ	205	$CD	Í	229	$E5	å	253	$FD	ý
182	$B6	¶	206	$CE	Î	230	$E6	æ	254	$FE	þ
183	$B7	·	207	$CF	Ï	231	$E7	ç	255	$FF	ÿ

Over the serial port, the Hydra sends a character as its code. A terminal program on a PC shows the codes from 160 up correctly when it's set to ISO-8859-15 (it's sometimes called **Latin-9**); most terminal programs have a setting for it. Codes 164, 166, 168, 180, 184, 188, 189 and 190 are the ones where ISO-8859-15 differs from the older ISO-8859-1 (Latin-1): €, Š, š, Ž, ž, Œ, œ and Ÿ.

E.4 The DEC Special Graphics

`ESC (0` switches the console to the **DEC Special Graphics** (section 13.7), and `ESC (B` back to ASCII. In it, the codes from 95 to 126 draw these instead:

CODE	TYPED	DRAWS	CODE	TYPED	DRAWS	CODE	TYPED	DRAWS	CODE	TYPED	DRAWS
95	_	(blank)	103	g	±	111	o	⎺ (scan line 1)	119	w	┬
96	`	◆	104	h	NL	112	p	⎻ (scan line 3)	120	x	│
97	a	▒	105	i	VT	113	q	─	121	y	≤
98	b	HT	106	j	┘	114	r	⎼ (scan line 7)	122	z	≥
99	c	FF	107	k	┐	115	s	⎽ (scan line 9)	123	{	π
100	d	CR	108	l	┌	116	t	├	124	|	≠
101	e	LF	109	m	└	117	u	┤	125	}	£
102	f	°	110	n	┼	118	v	┴	126	~	·

(HT, FF, CR, LF, NL and VT are tiny pairs of letters, as the old terminals showed them.)

E.5 The screen's fonts

The Vera X draws each character from a **font**: a picture of each code, 8 dots by 8. The console's font has ISO-8859-15's characters, with the DEC Special Graphics in the places of the control codes, and reversed letters from 128 to 159:



Figure E-1. The console's font, ISO-8859-15. The row is a code's first hex digit, the column its second.

The ROM disk has a second font, the PC's (**CP437**, the IBM PC's characters), with more lines and blocks for drawing, Greek letters and math signs. `cat /lib/font/cp437 >/dev/vid/font` loads it (section 16.5), and `cat /lib/font/iso8859-15 >/dev/vid/font` puts the console's back.



Figure E-2. The PC's font, CP437.

A font is 2048 bytes: 8 for each code, the top row of dots first, and in each byte the leftmost dot is bit 7. So you can make your own, with any program that writes bytes to a file.

APPENDIX F

The Memory and I/O Map

This appendix collects the Hydra's addresses in one place: what the 65C02 sees, what's behind its windows, and the registers of each chip. Chapter 7 explains how it all fits together.

F.1 The 64K the 65C02 sees

ADDRESSES	SIZE	WHAT'S THERE	CHOSEN BY
`$0000-$7FFF`	32K	The running task's own RAM (U7)	T (`$FFF0`)
`$8000-$9FFF`	8K	A RAM bank: the task's own, or a shared one	The RAM bank register (`$00`), and U
`$A000-$DFFF`	16K	A paged ROM bank	The ROM bank register (`$01`)
`$E000-$FEFF`	8K (less 256 bytes)	A kernel ROM page (U6)	W (`$FFF3`)
`$FF00-$FFEF`	240	The I/O ports: 15 ports of 16 bytes (F.5)	(fixed)
`$FFF0-$FFF3`	4	The registers T, U, V and W	(fixed)
`$FFFA-$FFFD`	4	The NMI and reset vectors, from the kernel ROM page	W
`$FFFE-$FFFF`	2	The interrupt vector: the vector RAM (F.6)	The interrupt line, or V

F.2 A task's 32K

ADDRESSES	WHAT'S THERE
`$00`	The RAM bank register (written: chooses the bank at `$8000`; read: the bank chosen)
`$01`	The ROM bank register (the same, for `$A000`)
`$02-$21`	r0 to r15: the call registers, 2 bytes each, low byte first
`$22-$7F`	The program's zero page (94 bytes)
`$80-$FF`	HydraOS's zero page for the task
`$0100-$01FF`	The stack (a program starts with S = `$FD`)
`$0200-$03FF`	HydraOS's area for the task: its open files, its name, its arguments ...
`$0400-$7FFF`	The program's memory. A program from a file is loaded at `$0800`: its header (48 bytes), code, data, BSS, then free memory up to its break. A program in the paged ROM runs there, in place, and has its data and BSS here from `$0400`
`$7FF8-$7FFF`	In task F (15) only, with a DS1747 for U7: the clock's registers

F.3 The RAM banks

A task's RAM bank register takes a bank number from `$00` to `$FF`:

BANKS	WHAT THEY ARE
$m0-$mF	The task's own 16 banks on RAM module *m* (0 to 14) of the memory cards: 128K for each task on each module
`$F0-$FF`	16 of the shared RAM's 256 banks (2 MB), chosen by U: bank $Fn is shared bank U × 16 + *n*

So with three RAM modules (modules 0, 1 and 2), a task has banks `$00-$2F`: 48 banks, 384K. Programs ask HydraOS for banks with `BANKS_ALLOC` (and for shared ones, through **segments**, with `SEG_CREATE`), rather than taking them.

F.4 The paged ROM

The ROM bank register chooses one of 256 banks of 16K: 4 MB in eight sockets, U30 to U37, of 512K (32 banks) each. HydraOS's three chips (in U31, U32 and U34, section 4.4) hold its modules (the drivers, the shells, the languages, the libraries) and then the ROM disk. A big module has banks of its own; a small one shares a bank with others, each in its own part of it. `mods` lists the modules and their banks (Chapter 11).

F.5 The I/O ports

PORT	ADDRESSES	IRQ LINE	CHIP
0	`$FF00-$FF0F`	0	65C22 VIA (U2): the tick, the SPI bus, the I/O header J27
1	`$FF10-$FF1F`	1	65C51 ACIA (U3): the serial port
2, 3	`$FF20-$FF3F`	2, 3	Slot 0: the Vera X
4	`$FF40-$FF4F`	4	YM2151 (U38): FM sound
5-9	`$FF50-$FF9F`	5-9	Slots 1 to 5: select A
10-14	`$FFA0-$FFEF`	10-14	Slots 1 to 5: select B

The VIA (65C22, port 0):

ADDRESS	REGISTER	ON THE HYDRA
`$FF00`	Port B	The SPI bus: PB0 the clock, PB1 the select enable, PB2 MOSI, PB3-PB6 the device, PB7 MISO
`$FF01`	Port A	J27's pins: PA0 SCL and PA1 SDA (the I²C bus), PA2-PA7 general purpose
`$FF02`	DDRB	Port B's directions (1: an output)
`$FF03`	DDRA	Port A's directions
`$FF04`, `$FF05`	T1C-L, T1C-H	Timer 1's counter: HydraOS's tick, 200 a second
`$FF06`, `$FF07`	T1L-L, T1L-H	Timer 1's latch
`$FF08`, `$FF09`	T2C-L, T2C-H	Timer 2 (the console's pacing)
`$FF0A`	SR	The shift register
`$FF0B`	ACR	The auxiliary control register
`$FF0C`	PCR	The peripheral control register (CA1: J27 pin 11, CA2: pin 12)
`$FF0D`	IFR	The interrupt flags
`$FF0E`	IER	The interrupt enables
`$FF0F`	Port A (no handshake)	Port A, without CA1 and CA2's handshake

The ACIA (65C51, port 1):

ADDRESS	REGISTER
$FF10	Data: the byte received (read), or to send (write)
$FF11	Status (read); writing it resets the chip
$FF12	Command
$FF13	Control: the speed, and the bits of each byte

The ACIA's clock on this board is 1.790 MHz, so every speed is 2.9% slow (section 13.6).

The YM2151 (port 4):

ADDRESS	REGISTER
$FF40	Write: chooses one of the chip's registers
$FF41	Write: the chosen register; read: the status (bit 7: busy; bits 0 and 1: the timers)

The Vera X (slot 0, ports 2 and 3): the VERA's 32 registers, $FF20 + 0 to $FF20 + $1F, in the same order as on the Commander X16 (where they're at $9F20), so the X16's documentation describes them:

ADDRESS	REGISTER	ADDRESS	REGISTER
$FF20-$FF22	ADDRx_L, ADDRx_M, ADDRx_H: a VRAM address	$FF2D-$FF33	Layer 0: CONFIG, MAPBASE, TILEBASE, scrolls
$FF23, $FF24	DATA0, DATA1: VRAM at the addresses	$FF34-$FF3A	Layer 1: the same
$FF25	CTRL	$FF3B	AUDIO_CTRL: recorded sound
$FF26, $FF27	IEN, ISR: interrupts	$FF3C	AUDIO_RATE
$FF28	IRQLINE_L, SCANLINE_L	$FF3D	AUDIO_DATA
$FF29-$FF2C	DC_VIDEO ... (chosen by CTRL's DCSEL)	$FF3E, $FF3F	SPI_DATA, SPI_CTRL: the SD card

The Vera X's keyboard and mouse controller isn't on these ports: it's on the I²C bus, at address $42 (section 16.9).

The Hydra's registers:

ADDRESS	REGISTER	WHAT IT DOES
$FFF0	T	The task: which 32K of U7 is at $0000-$7FFF, and whose bank registers $00 and $01 are
$FFF1	U	Which 16 of the shared RAM's 256 banks are banks $F0-$FF
$FFF2	V	The vector RAM's entry when no interrupt line is active (the BRK's)
$FFF3	W	The kernel ROM page at $E000-$FEFF

They can be written and read back. Only HydraOS writes them.

F.6 The interrupt lines

The lines go to the priority encoder (U17, U18): when several are active, the lowest number wins. The vector RAM (IC5 to IC8) holds a handler's address for each line, which the 65C02 reads at $FFFE.

LINE	ITS INTERRUPT	LINE	ITS INTERRUPT
0	The VIA: its timers (the tick), CA1	8	Slot 4, select A
1	The ACIA: the serial port	9	Slot 5, select A
2	Slot 0: the Vera X's VERA	10	Slot 1, select B
3	Slot 0's second line	11	Slot 2, select B
4	The YM2151's timers	12	Slot 3, select B
5	Slot 1, select A	13	Slot 4, select B
6	Slot 2, select A	14	Slot 5, select B
7	Slot 3, select A	15	(None: the `BRK`'s entry)

F.7 HydraOS's addresses

The kernel's ROM, U6, is 16 pages of 8K, and W chooses which is at `$E000`. Every page has the same few things in the same places, so the kernel can switch pages and carry on:

ADDRESS	WHAT'S THERE
`$E000`	The reset stub, on every page: it switches to page 0 (W isn't cleared by a reset)
`$F800-$FA18`	On page 0: the system calls' jump table (Appendix K), 3 bytes each, a `jmp` to the call's code
`$FD00-$FDFE`	The COMMON block, the same on every page: the interrupts' entry and exit, and the kernel's calls between pages
`$FDFF`	The page's number (the power-on test reads it to check the W lines)
`$FFFA-$FFFD`	The NMI and reset vectors, on every page

Page 0 has the kernel's core, page 1 its tasks and memory, page 2 its files, page 3 its namespaces, and page 4 the power-on test. In the paged ROM, a module's (or library's) header is its first 48 bytes: at `$A000` in its first bank, or, for a small module that shares a bank, at `$A000` plus 256 times its **page** (`mods` shows bank 4, page `$33`, as `4+33`: its header is at `$D300`).

APPENDIX G

Error Codes

When a system call fails, HydraOS gives an **error code**: a number that says what went wrong. Programs show it as its **message**, usually after the name of the file or the thing it was about:

```
/> cat nothing
cat: nothing: not found
```

G.1 HydraOS's errors

CODE	HEX	NAME	MESSAGE	WHAT IT USUALLY MEANS
1	`$01`	`E_PERM`	not allowed	Not allowed: something only HydraOS or a driver may do, or a task that can't be debugged
2	`$02`	`E_INVAL`	invalid argument	A call was given something it can't use: a bad number, a bad name, a wrong command in a ctl file
3	`$03`	`E_NOSYS`	no such call	There is no such system call, or a device doesn't do that
4	`$04`	`E_AGAIN`	not yet: try again	Not ready yet: nothing to read, with waiting turned off. Try again later
5	`$05`	`E_INTR`	interrupted	A note (CTRL-C, say) stopped a call while it waited
6	`$06`	`E_NOMEM`	out of memory	No memory left: no free RAM bank, no room in a program's memory
7	`$07`	`E_BUSY`	busy	Something else is using it: a device opened by another program
8	`$08`	`E_RANGE`	out of range	A number outside the range allowed
9	`$09`	`E_FAULT`	bad address	An address a call can't use
10	`$0A`	`E_NAMETOOLONG`	name too long	A path longer than 63 characters, or a name longer than 31
11	`$0B`	`E_TOOBIG`	too big	Too big: a program's arguments longer than 176 bytes in all, or a program too big for its task's memory
32	`$20`	`E_NOENT`	not found	There's no file (or directory, or program, or variable) of that name: check the spelling, and `ls`
33	`$21`	`E_EXIST`	already exists	Something of that name is there already
34	`$22`	`E_NOTDIR`	not a directory	A part of the path isn't a directory
35	`$23`	`E_ISDIR`	is a directory	It's a directory, and the call wanted a file
36	`$24`	`E_NOTEMPTY`	directory not empty	A directory to be removed still has files in it
37	`$25`	`E_BADF`	bad file descriptor	A file descriptor that isn't open (or isn't open for that: writing a file opened to read)

CODE	HEX	NAME	MESSAGE	WHAT IT USUALLY MEANS
38	`$26`	`E_MFILE`	no free file descriptor	The program has 16 files open already
39	`$27`	`E_NFILE`	too many open files	Too many files are open in the whole Hydra (or in one device): close some
40	`$28`	`E_NOSPC`	disk full	The disk is full
41	`$29`	`E_ROFS`	read-only	The disk can only be read: the ROM disk
42	`$2A`	`E_IO`	i/o error	The hardware failed: an SD card that didn't answer, a bad read
43	`$2B`	`E_NODEV`	no such device	There's no such device, or its hardware isn't there (a Vera X, a card)
44	`$2C`	`E_PIPE`	broken pipe	Writing to a pipe whose reader has gone
45	`$2D`	`E_NOEXEC`	not a program	The file isn't a program the Hydra can run
46	`$2E`	`E_EOF`	end of file	The end of a file came too soon
47	`$2F`	`E_NOTFS`	no file system	The disk has no HydraFS file system on it: `mkfs` makes one
48	`$30`	`E_MEDIA`	medium error	The disk (the card) has a fault, or was changed
64	`$40`	`E_NOTASK`	no free task	All 16 tasks are in use: end a program or two
65	`$41`	`E_SRCH`	no such task	There's no task of that number
66	`$42`	`E_CHILD`	no such child	Waiting for a task that wasn't started by this one
80	`$50`	`E_NSFULL`	namespace full	The namespace has too many binds and mounts
81	`$51`	`E_NSLOOP`	too many binds	Binds that lead round in a circle

G.2 In each language

Each language shows these errors in its own way:

WHERE	HOW AN ERROR SHOWS
rc	The program's message; `$status` holds it (section 8.9)
Assembly language	C = 1 after the call, and the code in A; `ERRSTR` gives the message (section 27.13)
HyForth	An **ior** of -512 less the code (so `not found`, `$20`, is -544), which `throw` reports with its message; `ior>text` gives the message
hylang	An error whose message is "*name*: *message*", and whose code (`&code`, `(error-code e)`) is the name in small letters without its `E_`: `:noent`
HyBASIC	Error number 256 plus the code (so `not found` is 288), with its message (`ERR`, `ERR$`, `ERL`)
C	`errno` set to the nearest C error, and `_oserror` to the code

The languages have errors of their own too: HyForth's `throw` codes (Appendix A), HyBASIC's errors (Appendix C), and hylang's (Appendix B).

G.3 The numbers library's errors

The numbers library (Chapter 18) has its own codes, which the languages show as their own errors:

CODE	WHAT IT MEANS
1	Too big: an integer of more than 255 bytes, or a fixed decimal of more than 65,535 places
2	Division by zero
3	Not a number
4	No room for the result
5	Outside the function's domain (like the square root of a negative number, for a real result)
6	Not a base: a base string that names none (section 18.3)
7	An integer is needed
8	A real number is needed, not a complex one
9	The library wasn't set up (in assembly language: `NUM_INIT` first)
10	A format's placeholders and its values don't match

APPENDIX H

The Emulator and the PC Tools

Chapter 2 shows how to use the emulator. This appendix lists everything it can do, and the other tools for the PC that come with HydraOS, in `reborn/sim/tools`. Each is a Node.js program: run it with `node`, from the HydraOS folder.

H.1 The terminal emulator's options

```
node reborn/sim/run.js [options]
```

Running it:

OPTION	WHAT IT DOES
`-i`, `--interactive`	The terminal is the Hydra's serial console, in real time. Without it, the emulator runs for a while, then prints a report (section 2.4)
`--speed N`	With `-i`: run N times faster than real time (0: as fast as it can)
`--cycles N`	Stop at cycle N (without `-i`: 30,000,000, about 8.4 seconds of the Hydra's time)
`--input TEXT`	Keys to type: `\r` or `\n` is RETURN, `\w` waits 2 million cycles. One key every 20,000 cycles, from cycle 200,000. Put a `\w` or two before the first key, so that HydraOS's console is running. The emulator's terminal obeys the Hydra's XON and XOFF, so a long input arrives whole. `manual/tools/capture.js` types a script a line at a time, waiting for each prompt
`--paste`	Type `--input`'s keys as fast as the serial line takes them (a key that arrives before the last is read is lost, as on a real Hydra)
`--flow none`	The emulator's terminal ignores the Hydra's XON and XOFF (section 13.6). Without this it obeys them, as a terminal set to XON/XOFF does
`--win32-input`	With `-i` in Windows Terminal: lets CTRL-TAB and CTRL-SHIFT-TAB reach the Hydra (after you remove Windows Terminal's own use of them from its settings)

The hardware:

OPTION	WHAT IT DOES
`--modules N`	RAM modules on the memory cards (2 if it isn't given)
`--clock MHz`	The processor's clock: 3.579545 (the normal one), or 7.15909 (jumper J8, for a board with faster parts than Appendix I's; HydraOS must be built with `--clock 2` too)
`--acia wdc`	A WDC W65C51N chip for the serial port, instead of a Rockwell one (HydraOS must be built with `--acia wdc` too)
`--seed N`	The random values that RAM and the registers have at power-up: the same N gives the same values every time
`--bios FILE`, `--prom FILE`	Other ROM images than HydraOS's own (`--prom`: the whole paged ROM, its chips' images one after another)

OPTION	WHAT IT DOES
`--sd FILE`	An SD card image, in SD device 0 (then 1, and so on, for more). The Hydra reads and writes the file itself
`--no-sd`	No SD card. (Without `--sd` or `--no-sd`, the emulator puts in HydraOS's sample card, `bin/sdcard.img`, keeping its changes only in memory)

The Vera X:

OPTION	WHAT IT DOES
`--vera [V]`	A Vera X card in slot 0; V is its gateware's version (47.0.2 if it isn't given; 0.9 for the oldest, without the FX registers)
`--vera-config MS`	How long the Vera X's FPGA takes to start up, after power-up or a reset (100 milliseconds)
`--vera-sd FILE`	A card image in the Vera X's own SD card slot (`/sd/v`)
`--smc`	The Vera X's keyboard and mouse controller
`--kbd TEXT`	Keys typed on the Vera X's keyboard, as `--input`'s are (`\e` is ESC, `\b` BACKSPACE)
`--view [PORT]`	With `-i`: the Vera X's screen in a web browser, at http://localhost:8016 (or PORT). With `--smc`, the page's keys and mouse go to the Hydra
`--sound [PORT]`	With `-i`: the Hydra's sound in a web browser (the same page as `--view`, if there is one): press its **Sound** button. It plays about 0.15 seconds late
`--wav FILE`	Record the sound into a WAV file, as the Hydra's time passes
`--screen`	After the report: the Vera X's text, as text
`--frame-png FILE`	At the end, a picture of the Vera X's screen (640 by 480); with `-i`, the file CTRL-A P saves to

/pc (section 2.5):

OPTION	WHAT IT DOES
`--pc-dir FOLDER`	The Hydra sees FOLDER, on your PC, as `/pc`. FOLDER can be an SD card image file instead: then `/pc` is the card in it
`--pc-read-only`	The Hydra can read `/pc`, but not change it
`--pc-log`	List what the Hydra does with `/pc` as it happens
`--pc-damage F,...`	Damage some of `/pc`'s messages on the serial line, to test that they're sent again: `qN` the Nth message from the Hydra, `rN` the Nth reply

Looking inside (for finding bugs in HydraOS itself):

OPTION	WHAT IT DOES
`--trace N`	The last N instructions run, in the report (25)
`--trace-calls [LIST]`	Log every system call a program makes: its name, its registers, and what it gave back. LIST limits it: names (`OPEN,READ`), or `t3` for task 3's
`--break SPEC`	Stop before the instruction at SPEC (below); with `-i`, open the monitor there
`--watch-pc ADDR`	Log each time the processor reaches ADDR (on the kernel's page 0)
`--watch ADDR[:T]`	Log each write to ADDR (`$0000-$7FFF`), by any task, or by task T only
`--watch-read ADDR[:T]`	Log each read of ADDR

OPTION	WHAT IT DOES
`--log FILE`	Write the log (the watches, the calls) into FILE instead of the terminal

A SPEC, for `--break` and the monitor's `b`, is a kernel label (`K_OPEN`), a module's label while its task has its bank (`rc:main`), a kernel ROM page and an address (`0:E000`), or just an address (on any page or bank).

H.2 The emulator's keys

With `-i`, CTRL-A is the emulator's own key; press it, and then:

THEN	WHAT IT DOES
X	Quit
R	Reset the Hydra (the RESET button)
S	Show the state: the tasks, and more
B	Stop, and open the monitor
V	Show the Vera X's screen as text
P	Save the Vera X's screen as a PNG picture
H	Help
CTRL-A	Send a CTRL-A to the Hydra

H.3 The monitor

The monitor (CTRL-A B, or a `--break`) stops the whole Hydra, kernel and all, and takes commands:

COMMAND	WHAT IT DOES
`c`	Continue
`n` [N]	Run the next N instructions (1), showing each
`r`	The registers
`t`	The tasks
`m` ADDR [N] [T]	N bytes (64) at ADDR, as task T sees memory (the running task's, without T)
`b` [SPEC]	Stop at SPEC; with none, list the stops
`d` N	Take away stop number N
`w` ADDR[:T]	Stop when ADDR is written (by task T)
`x`	Quit the emulator

The debugger, `db` (Chapter 29), is for debugging a program; the monitor is for when it's HydraOS itself you need to watch.

H.4 The web emulator

The web emulator (section 2.1) has its choices on its page: the buttons **Power cycle**, **Reset**, **Pause** and **Sound**; the speed (**Real time**, **2×**, **4×** or **As fast as it goes**); and the **Setup…** panel, which chooses a Vera X, its keyboard and mouse, the clock chip, one to three RAM modules, and the SD cards.

H.5 The PC tools

hydrafs.js makes and changes SD card images (section 2.6):

```
node reborn/sim/tools/hydrafs.js COMMAND IMAGE [ARGS]
```

COMMAND	WHAT IT DOES
`mkfs IMAGE MB [LABEL] [-q] [-p FATMB]`	A new, empty image of MB megabytes. `-q`: as the Hydra's quick format makes one. `-p`: in a partition, after a FAT partition of FATMB megabytes
`info IMAGE`	The card's label, size and free space
`ls IMAGE [-l] [PATH]`	List a directory (`-l`: with the dates)
`put IMAGE FILE PATH`	Copy a file from the PC onto the card
`get IMAGE PATH FILE`	Copy a file from the card to the PC
`mkdir IMAGE PATH`	Make a directory
`rm IMAGE PATH`	Remove a file, or an empty directory
`import IMAGE FOLDER [PATH]`	Copy a whole folder from the PC onto the card
`check IMAGE`	Check the card for lost or doubly used space

Paths on the card start at its root: `/games/star.fs` is `/sd/0/games/star.fs` on the Hydra. (In Git Bash, leave out the first `/`.)

sdwrite.js writes an image onto a real SD card in your PC's card reader (section 2.6):

```
node reborn/sim/tools/sdwrite.js --list
node reborn/sim/tools/sdwrite.js IMAGE DISK [--yes] [--force] [--no-verify] [--dry-run]
```

OPTION	WHAT IT DOES
`--list`	List the PC's disks; the removable ones (an SD card is one) are marked
`--yes`	Don't ask before writing (normally it asks you to type the disk again)
`--force`	Allow a disk that isn't removable (an external drive; never the system's own)
`--no-verify`	Don't read the card back afterwards to check it
`--dry-run`	Make the checks, and say what it would do, but write nothing

It needs an administrator's rights (on Linux and macOS, `sudo`), and refuses the disk your system is on, or one smaller than the image.

hydrapc.js is the PC tool: a terminal for a real Hydra's serial port, which also serves a folder on the PC as `/pc` (Chapter 5):

```
node reborn/sim/tools/hydrapc.js PORT FOLDER [options]
node reborn/sim/tools/hydrapc.js PORT IMAGE [options]
```

With an IMAGE (an SD card image file, as `hydrafs.js mkfs` makes and `sdwrite.js` writes to a card), `/pc` is the card in the image: its files, in the card's own order, with the card's own dates. What the Hydra writes goes into the image, which is complete on the PC after every change. An image file that can't be written is served read-only. Don't give the same image to an emulator (`--sd`) while the PC tool has it.

OPTION	WHAT IT DOES
`--list`	List the PC's serial ports

OPTION	WHAT IT DOES
`--read-only`	The Hydra can read the folder (or the image), but not change it
`--baud N`	The serial line's speed (9600, the Hydra's when it starts)
`--log`	List the Hydra's requests for `/pc` as they're served
`--no-size`	Don't tell the Hydra the window's size
`--win32-input`	As the emulator's (H.1)

Its keys: CTRL-A X quits, CTRL-A L turns the log on or off, CTRL-A S says `/pc`'s counts so far (requests, and frames that came damaged or replies that had to be sent again: a bad line shows here), and CTRL-A CTRL-A sends a CTRL-A. It needs the `serialport` package: run `npm install` in `reborn/sim` once.

hysong.js turns a score (section 15.6) into a ZSM song on the PC, as `play -o` does on the Hydra, and can make a VGM file too, to hear it on the PC:

```
node reborn/sim/tools/hysong.js SCORE.mml SONG.zsm [--vgm SONG.vgm] [--quiet]
```

APPENDIX I

Parts and Board Diagrams

This appendix lists every part of the Hydra-16 (board revision 1.8D) and its memory cards, in the order Chapter 4 fits them, with a box to tick as each one goes in. Then it says what each chip does, and shows where every part is.

I.1 The main board's parts

Resistors (Chapter 4, step 2):

☐	PARTS	VALUE	CODE
☐	R23, R24, R25, R26, R27	22 Ω	220
☐	R30, R31	33 Ω	330
☐	R3	100 Ω	101
☐	R22	330 Ω	331
☐	R2	470 Ω	471
☐	R4, R5, R6, R7, R13, R19, R20, R21, R28, R32, R33	10 kΩ	103
☐	R1	1 MΩ	105
☐	R29	0 Ω (a link)	0
☐	RN1, RN2, RN3 (networks of 8, 9 pins)	3.3 kΩ	332
☐	RN4 (a network of 8, 9 pins)	10 kΩ	103

Ceramic capacitors (step 3):

☐	PARTS	VALUE	CODE
☐	C47, C48	33 pF	330
☐	C50, C52	3.3 nF	332
☐	C1, C2, C3, C5, C6, C12 to C46, C57 to C64, C66 to C85 (68)	100 nF	104
☐	C4, C7, C8, C9, C10	1 µF	105
☐	C49	4.7 µF	475

Sockets (step 4), and the chips that go in them (section 4.5):

☐	PLACE	SOCKET	CHIP
☐	U1	40 pins, wide	W65C02S
☐	U2	40 pins, wide	R65C22 (or a W65C22N)
☐	U3	28 pins, wide	R65C51 (or a W65C51N: section 4.5)

☐	PLACE	SOCKET	CHIP
☐	U4, U20, U24	16 pins	74F138
☐	U5	16 pins	MAX232
☐	U6	PLCC-32	SST39SF010A
☐	U7	32 pins, wide	HM628512LP-5 (or a DS1747)
☐	U8, U9	14 pins	74LS260
☐	U10	14 pins	74F20
☐	U11	14 pins	74ACT08
☐	U12	14 pins	74F86
☐	U13, U16, U56	14 pins	74F32
☐	U14	14 pins	74F30
☐	U15, U44	14 pins	74F00
☐	U17, U18	16 pins	74LS148
☐	U19, U23	24 pins, wide	74LS154
☐	U21, U49, U51, U53, U55, U58, U60	20 pins	74F541
☐	U22	20 pins	74ACT240
☐	U26	16 pins	74F139
☐	U30 to U37	32 pins, wide	SST39SF040 (U31, U32 and U34 programmed; the rest empty)
☐	U38	24 pins, wide	YM2151
☐	U39	14 pins	74ACT14
☐	U40	16 pins	74F191
☐	U41	16 pins	YM3012
☐	U42	14 pins	TL074
☐	U45	16 pins	74F157
☐	U48, U50, U52, U54	20 pins	74F573
☐	U57	14 pins	74F02
☐	U59	20 pins	74ACT245
☐	U61	14 pins	74F125
☐	IC1 to IC8	16 pins	74LS219
☐	IC9	8 pins	NJM4556AD
☐	J28	16 pins	SP720

Soldered straight to the board (steps 1, 5, 6 and 7):

☐	PARTS	WHAT
☐	U25, U27, U28, U29	HM628512 RAM chips, surface-mount (SOP-32)
☐	Y1	14.31818 MHz crystal
☐	U43	DS1813-10 reset chip (TO-92)

☐	PARTS	WHAT
☐	J5, J6, J7, J8	2-pin headers: the clock jumpers (0.89, 1.79, 3.58 and 7.16 MHz)
☐	J9, J29	2-pin headers: the sound chip's CT1/CT2, and Audio In
☐	J30	3-pin header: the serial port's CTS
☐	J10	2 × 6 header: the front panel
☐	J27	2 × 6 header: the I/O header (the VIA's port A)
☐	J18 to J25	6-hole pin sockets: SPI devices 0 to 7
☐	J1, J2	32-hole and 24-hole pin sockets: the memory card
☐	C90, C91	10 µF electrolytic (106)
☐	C86, C87, C88, C89	47 µF electrolytic (476)
☐	C51, C53, C54, C55, C56	100 µF electrolytic (107)
☐	C92, C93	220 µF non-polarized electrolytic (227 NP)
☐	J3	9-pin D socket (female): the serial port
☐	J26	3.5 mm stereo audio jack
☐	J12 to J17	62-pin card edge connectors: slots 0 to 5
☐	J11	24-pin ATX power connector

Jumpers (section 4.6): one on J7 (the clock), one on J30 (pins 1 and 2), and one for each RAM module (section 4.7). J9 and J29 get none.

I.2 What each chip does

CHIPS	PART	ITS JOB
U1	W65C02S	The processor (Chapter 26)
U2	65C22 VIA	The tick (its timer), the SPI bus, and the I/O header J27
U3	65C51 ACIA	The serial port
U5	MAX232	Turns the ACIA's 5-volt signals into the serial cable's RS-232 ones, and back
U6	SST39SF010A	The kernel ROM: 16 pages of 8K
U7	HM628512	The task RAM: 512K, 32K for each of the 16 tasks
U25, U27, U28, U29	HM628512	The shared RAM: 2 MB
U26	74F139	Chooses one of the shared RAM chips
U30 to U37	SST39SF040	The paged ROM: up to 4 MB
U4, U8 to U16, U19 to U22	74-series logic	The **address decoder**: works out, from each address, which chip answers
U48, U50, U52, U54	74F573	The registers T, U, V and W
U49, U51, U53, U55	74F541	Read T, U, V and W back
U56, U57	74F32, 74F02	Gates that tell the registers when they're written and read
IC1 to IC4, U23, U24	74LS219, 74LS154, 74F138	The bank registers: a RAM and a ROM bank register for every task

CHIPS	PART	ITS JOB
U17, U18, U44, U45	74LS148, 74F00, 74F157	The interrupts' priority encoder
IC5 to IC8	74LS219	The vector RAM: an interrupt handler's address for each line
U58, U59, U60, U61	74F541, 74ACT245, 74F125	The slots' buffers: they drive the address, data and control lines to the six slots
Y1, U39, U40	Crystal, 74ACT14, 74F191	The clock: the crystal's 14.31818 MHz, divided by 4 (or 16, 8 or 2) for the processor
U43	DS1813	The reset: holds the processor in reset at power-up, and when RESET is pressed
U38	YM2151	The FM sound chip
U41	YM3012	Turns the YM2151's sound into a voltage
U42, IC9	TL074, NJM4556	The sound's mixer and headphone amplifier
J28	SP720	Protects the I/O header's pins from static

I.3 The memory cards

The through-hole card (Figure 4-8):

☐	PARTS	WHAT
☐	C1 to C5	100 nF ceramic capacitors (104)
☐	U1	74LS139 (a 16-pin socket)
☐	IC1 to IC4	HM628512LP-7 RAM chips, in 32-pin wide sockets: one RAM module
☐	J1, J2	32-pin and 24-pin headers, under the card: they plug into the main board
☐	J3	2 × 15 header: the module number's jumper (section 4.7)

The surface-mount card (Figure 4-10):

☐	PARTS	WHAT
☐	C1 to C5	100 nF ceramic capacitors (104)
☐	U1	74LS139
☐	IC1 to IC8	HM628512LFP-7 RAM chips, surface-mount: two RAM modules
☐	J1, J2	32-pin and 24-pin headers, under the card
☐	J3	2 × 15 header: the first module's number (IC1 to IC4)
☐	J4	15-pin header: with J3, the second module's number (IC5 to IC8)

I.4 The board

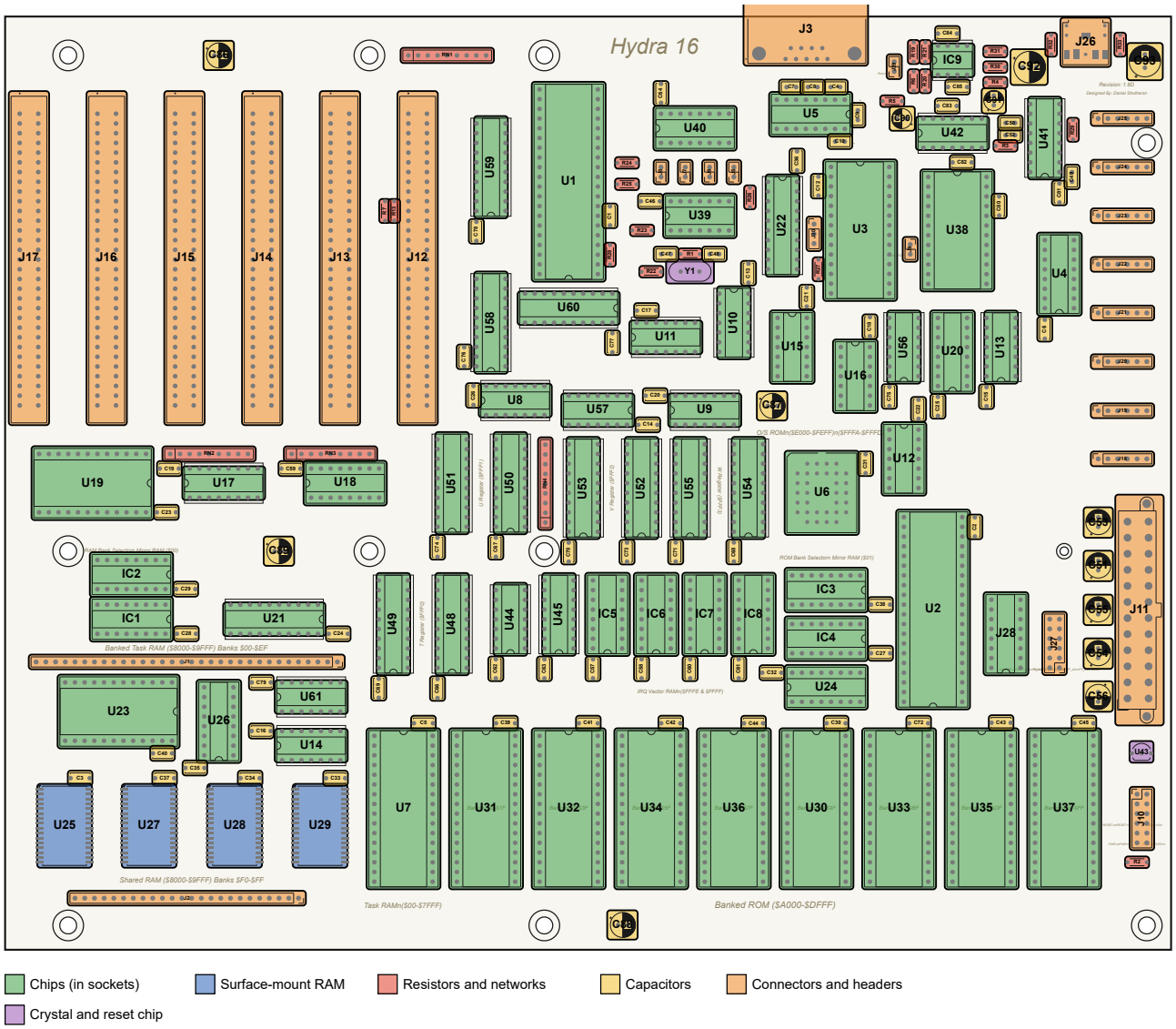


Figure I-1. Every part of the main board, by kind.

Figure 1-1 shows the board by what its parts do, and Figures 4-1 to 4-7 each step of building it.

APPENDIX J

Glossary

The words this book uses, and what they mean. The number in brackets after a word is the section where it's explained in full.

Accumulator. The 65C02's A register, where its arithmetic and logic are done (26.3).

ACIA. Asynchronous Communications Interface Adapter: the 65C51 chip, U3, that runs the serial port (7.2).

Address. The number of a byte in memory. The 65C02's addresses go from `$0000` to `$FFFF` (7.2).

Addressing mode. The way an instruction says which byte it works on: `#$41`, `$22,x`, `($22),y` ... (26.6).

Argument. A word given to a command after its name: in `ls -l /rom`, `-l` and `/rom` are arguments (8.2).

Array. A numbered row of values under one name: `SCORES(3)` is the fourth of `SCORES` (23.10).

ASCII. The standard code for characters, from 0 to 127: `A` is 65 (Appendix E).

Assembler. A program that turns assembly language into machine code. The Hydra's is `as` (Chapter 27).

Assembly language. A program written with the processor's instruction names, like `LDA`, instead of numbers (26.1).

Bank. A piece of a bigger memory that can be shown through a window in the 65C02's 64K: an 8K RAM bank at `$8000`, a 16K ROM bank at `$A000` (7.2).

Base. How many digits a way of writing numbers uses: 10 for decimal, 2 for binary, 16 for hex (1.6, 18.3).

Base string. The text that names a base in HydraOS's languages: `x` for hex, `b` for binary, `7r` for base 7 (18.3).

Baud. A serial line's speed, in bits a second. At 9600 baud, about 960 characters go by each second (5.2, 5.5).

Binary. Numbers written in base 2, with only the digits 0 and 1 (1.6).

Bind. To make a file or directory appear at another name in the namespace (9.5).

Bit. A binary digit, 0 or 1: the smallest piece of information (1.6).

Bitmap. A picture made of dots, each dot's colour kept in memory (16.4).

Block. A piece of a disk: 512 bytes on HydraFS (14.5). In HyForth, a numbered 1K piece of a file of blocks (20.10).

Boot. Starting the computer: everything from the reset to the first prompt (Chapter 7).

Breakpoint. A place in a program where the debugger stops it (29.5).

BSS. The part of a program for variables that start as zero. It isn't stored in the program's file (27.8).

Bug. A mistake in a program. Finding bugs is **debugging** (Chapter 29).

Bus. The wires that carry addresses, data and control signals between the processor and the other chips (1.2).

Byte. Eight bits: a number from 0 to 255 (1.6).

Call registers. r0 to r15: sixteen 2-byte places on the zero page, for passing values to and from system calls (7.3).

Capacitor. A part that stores a little electric charge. Most of the Hydra's smooth out the power beside each chip (3.4).

Card. A board that plugs into one of the Hydra's slots, like the Vera X; or an SD card.

Carry. The C flag: the bit carried out of an addition, or shifted out of a byte (26.8).

Cell. In HyForth, one item on the stack: 16 bits (19.2).

Channel. One voice of the sound chips. The Hydra has 24: 8 on the YM2151 and 16 on the Vera X (Chapter 15).

Chip. An integrated circuit: a tiny circuit in a plastic package with metal legs (3.4).

Clock. The signal whose ticks pace the processor: 3.58 million a second (7.1). Also the clock chip, which keeps the date and time (17.5).

Cluster. The piece of a disk that HydraFS hands out to files: a few blocks together (14.5).

CMOS. A way of making chips that uses very little power, but that static electricity can damage (3.3).

Command. A line typed for the shell to do; or a program it runs (8.2).

Comment. Text in a program for people to read, which the computer skips.

Console. The keyboard and screen you use the Hydra through: the serial terminal, or the Vera X's screen (Chapter 13).

Constant. A name for a value that doesn't change.

Control character. A code from 0 to 31 that tells a terminal to do something, instead of printing (Appendix E).

Control file. A device's file, usually called `ctl`, that takes commands written to it as text (12.4).

Cooked mode. The console's usual mode, in which a program gets a whole line at a time (13.2).

Crystal. A sliver of quartz that vibrates at an exact rate. The Hydra's, Y1, gives the beat for its clocks (1.4).

Current directory. The directory a program is "in": a file name that doesn't start with `/` is found there (9.1).

Cursor. The mark on the screen that shows where the next character you type will go (6.3).

Cycle. One tick of the processor's clock (26.12).

Debugger. A program that runs another one step by step, to find its bugs: `db` (Chapter 29).

Decimal. Base 10, the usual way of writing numbers (1.6).

Device. A part of the Hydra that does input or output: the serial port, the sound chips, the SD card (Chapter 12).

Dictionary. In HyForth, the list of all the words it knows (19.5).

DIP. Dual in-line package: a chip with two rows of legs, the kind that fits a socket (3.4).

Directive. An instruction to the assembler itself, starting with a dot: `.byte` (27.4).

Directory. A file that holds other files: a folder (9.1).

Disassembler. A program that turns machine code back into assembly language: `dis` (Chapter 28).

Driver. A program that looks after a device, and gives other programs files to use it with (Chapter 12).

Editor. A program for typing and changing text. The Hydra has two: `edit`, which uses the whole screen (6.6), and `ed`, which works a line at a time.

Emulator. A program that imitates a computer. The Hydra emulator runs HydraOS on a PC (Chapter 2).

Environment. A task's variables, each a name and a value, which programs can read: `$path`, `$window` (8.4).

Error code. A number that says what went wrong in a system call (Appendix G).

Escape sequence. A few characters starting with ESC that tell a terminal to move the cursor, change colour, and so on (13.7).

Expression. A calculation, like `ROWS + 1`.

File. A named piece of data on a disk. In HydraOS, devices are files too (Chapter 9).

File descriptor (fd). A small number that stands for a file a program has open (27.13).

Flag. A bit that says yes or no, like the 65C02's N, V, D, I, Z and C flags (26.3). In Forth, a true or false value (19.7). And an option, like `-l`.

Flash ROM. ROM that can be erased and programmed again electrically (4.4).

Flow control. A way for a receiver to tell a sender to wait: the Hydra's is XON/XOFF (13.6).

Flux. The cleaner inside solder (and in flux pens) that lets melted solder stick to metal (3.5).

FM synthesis. Making sounds by changing one wave's frequency with another wave: the YM2151's way (15.1).

Font. The pictures of the characters, 8 dots by 8 on the Vera X (Appendix E).

FPGA. Field-programmable gate array: a chip that becomes whatever circuit is loaded into it. The Vera X is one (16.1).

Function. A named piece of a program that can be given values (its arguments) and gives one back.

Gateware. The configuration that makes an FPGA chip, like the Vera X's, into a particular circuit.

Hash. In hylang, a table that finds a value by its name (its key) (21.12).

Header. A row of metal pins on a board (3.4). Also the first 48 bytes of a program file, which describe it (27.8).

Hexadecimal (hex). Base 16: the digits 0 to 9 and A to F. `$FF` is 255 (1.6).

HydraFS. The Hydra's file system: the way it keeps files on its disks (14.5).

HydraOS. The Hydra's operating system (1.5).

I/O. Input and output (1.2).

I²C. A bus of two wires for talking to small chips, on the I/O header (17.3).

Index register. X or Y: registers for counting, and for stepping through tables (26.3).

Instruction. One step the processor can do (26.1).

Integer. A whole number (18.1).

Interrupt. A signal from a chip that makes the processor stop what it's doing to deal with it (26.11).

Jumper. A small cap that connects two pins of a header (4.6).

Kernel. The heart of HydraOS, which runs the tasks, the files and the namespaces (7.1).

Label. A name for a place in an assembly language program (27.5).

LED. Light-emitting diode: a small coloured light. It lights only one way round (4.8, 17.1).

Library. A collection of words, functions or routines that programs can use.

List. In hylang, values in order, written in braces or parentheses: the language's main way to keep things together (21.6).

Little-endian. Keeping a 16-bit number with its low byte first, as the 65C02 does (26.2).

Login shell. The shell each window starts with (8.16).

Loop. A part of a program that's done again and again: until a count runs out, or while something stays true (19.9, 21.9, 23.8).

Machine code. A program as the processor reads it: bytes (26.1).

Macro. A named piece of source that is put in wherever its name is used (27.11).

Memory card. The board that holds the Hydra's extra RAM, plugged into J1 and J2 (4.7).

MIDI. Musical Instrument Digital Interface: the standard that electronic instruments use. The Hydra borrows its note numbers (60 is middle C) and its lists of instruments and drums (15.2, 15.4).

MML. Music Macro Language: a tune written as text, like `cdefg. play` and all three languages can play it (15.6).

Mnemonic. An instruction's short name, like `LDA` (26.1).

Module. A program or driver kept in the paged ROM (7.4). Also a **RAM module**: 2 MB of RAM on a memory card (4.7).

Monitor. The emulator's own command line, for looking inside the processor (2.4).

Mount. To attach a device's files to the namespace (9.5).

Multitasking. Running several programs at once, by giving each a short turn at the processor, many times a second (7.3).

Namespace. The tree of names a task sees, made by binds and mounts (9.5).

Note. A message to a task, like `interrupt` (CTRL-C). A **note group** is the tasks that get a note together (27.16).

Octal. Base 8: the digits 0 to 7 (18.3).

Opcode. The number of an instruction (26.1).

Operand. What an instruction works on: a number, or an address (26.1).

Operating system. The program that runs a computer, and the other programs on it (1.5).

Option. A word starting with – that changes what a command does: the `-l` in `ls -l` (8.2).

Page. 256 bytes of memory, from $xx00 to $xxFF (26.5). Also one 8K page of the kernel's ROM (7.2).

Paged ROM. The Hydra's ROM of up to 4 MB, seen 16K at a time at `$A000` (7.2).

Palette. The colours a screen can show (16.5).

Parameter. A name for a value that a function or macro is given.

Partition. A part of a disk that's used as if it were a disk of its own (14.2).

Patch. The settings that make the sound chip sound like an instrument (15.4).

Path. A file's name with its directories: `/sd/0/sample/hi` (9.1).

PCM. Pulse-code modulation: recorded sound, kept as a long row of numbers. The Vera X plays it (15.8).

Pipe. A connection that carries one program's output into another's input: `|` (8.7).

PLCC. Plastic leaded chip carrier: a square chip with legs on all four sides, like the kernel ROM, U6 (3.4).

Pointer. An address kept in memory, which points at something else (26.6).

Port. A group of 16 addresses for a chip (7.2); or a connector.

POST. Power-on self test: the checks of its memory that the Hydra makes each time it starts (7.4).

Processor. The chip that runs programs: the W65C02S (Chapter 26).

Program. A list of instructions for a computer (1.1). On the Hydra, a file you can run by typing its name (6.8).

Prompt. What a program prints when it's waiting for you to type, like `/>`.

PSG. The Vera X's programmable sound generator, with 16 simple voices (15.1).

RAM. Random-access memory: memory that can be read and written, and forgets everything when the power goes off.

RAM disk. A disk made of RAM: `/ram` (14.3).

Raw mode. The console's mode in which a program gets each key as it's pressed (13.3).

Recursion. A function (or word) that calls itself (19.14).

Register. A small memory inside the processor (A, X, Y ...: 26.3), or a chip's control location, or one of the Hydra's own T, U, V and W (7.2).

Reset. Starting the processor again from the beginning: the RESET button (7.1).

Resistor. A part that limits the flow of electric current (3.4).

ROM. Read-only memory, which keeps what's in it without power.

ROM disk. The files kept in the paged ROM: `/rom` (14.3).

Root. The directory at the top of the tree of files, written `/` (6.5, 9.1).

RS-232. The standard for serial ports like the Hydra's J3: its plug, and its voltages (5.1).

Scheduler. The part of the kernel that decides which task runs next (7.3).

Script. A file of commands for a shell or a language to run (8.13).

SD card. The small memory card the Hydra uses as a disk (14.2).

Segment. A part of an assembly language program: `CODE`, `DATA`, `BSS` ... (27.8). Also a shared memory segment (17.9).

Serial port. The connector J3, which sends and receives data one bit at a time: the Hydra's console (13.6).

Server. A driver with files of its own (12.2).

Shell. The program that reads your commands and runs them: HyForth, rc, hylang or HyBASIC (Chapter 8).

Slot. One of the six connectors for expansion cards, J12 to J17 (1.4).

Socket. A holder soldered to the board, which a chip plugs into (3.4).

Solder. A metal that melts at a low temperature, used to join parts to the board (3.5).

Source. A program as it's written, as text.

SPI. A simple serial bus for chips and cards. The Hydra has eight SPI device headers (17.4).

Sprite. A small picture that can move over the screen without changing what's under it (16.5).

Stack. A pile of values where the last one put on is the first taken off. The 65C02 has one (26.10), and so does Forth (19.2).

Standard input, output and error. A program's fds 0, 1 and 2: usually the keyboard and the screen (27.13).

Status. What a program said when it ended: nothing for success, or a message (8.9).

String. Text, as a value in a program.

Subroutine. A piece of code that's called with `JSR` and returns with `RTS` (26.10).

Surface-mount. A part soldered onto pads on the board's surface, with no holes. The Hydra's shared RAM chips are (3.7).

System call. A request from a program to HydraOS: open a file, start a program, wait (27.13).

Task. One of the sixteen programs the Hydra can run at once, each with its own 32K of memory (7.3).

Terminal. A screen and keyboard for a computer; or a program that acts as one (5.2).

Through-hole. A part whose legs go through holes in the board and are soldered on the other side. Nearly all the Hydra's parts are (3.5).

Tick. One of HydraOS's timer interrupts: 200 a second (7.3).

Two's complement. The way a byte holds a negative number: `$FF` is -1 (26.2).

Union directory. A directory made of several, which are looked in in turn (9.7).

Usage. A line that shows how to type a command: its options, and its arguments, with the ones you can leave out in square brackets (6.10).

Variable. A name for a value that can change.

Vector. An address, kept at a known place in memory, that tells the processor where to go: when it's reset (7.1), or for an interrupt (26.11).

VERA. The video chip on the Vera X card (16.1).

VIA. Versatile Interface Adapter: the 65C22 chip, U2, with the Hydra's timer and its I/O pins (7.2).

Window. A part of the 65C02's 64K through which a bigger memory is seen (7.2). Also one of the console's text windows (Chapter 10).

Word. In Forth, a named command (19.1). Also a 16-bit value.

XON and XOFF. The two bytes of the serial port's flow control: XOFF (`$13`) asks the terminal to stop sending, and XON (`$11`) to go on (13.6).

YM2151. The Hydra's sound chip: a Yamaha FM synthesizer with eight voices (15.1).

Zero page. The first 256 bytes of memory, `$0000` to `$00FF`, which the 65C02 reaches with shorter, quicker instructions (26.5).

ZSM. The kind of song file that `play` plays: the sound chips' settings, moment by moment (15.7).

APPENDIX K

System Calls

A **system call** asks HydraOS to do something for a program. This appendix lists all of them, group by group.

From assembly language (section 27.13), a call is a `jsr` to its address. Its inputs go in A, X, Y and the call registers r0 to r15; its answers come back in the same places, a 16-bit one in A (its low byte) and X (its high byte). C = 0 means it worked; C = 1 means it failed, with the error code (Appendix G) in A. A call may change A, X, Y, r0 to r15 and the flags, but never the program's zero page, `$22` to `$7F`. `hydra.inc` (in `/lib/as`) names every call, constant and error code used here.

From the languages, the calls a program makes (all but those marked *Not in the languages*) have the call's name in small letters, with - for _:

LANGUAGE	OPEN IS	SECTION
HyForth (`lib hydra`)	`sys-open` (*inputs -- outputs ior*)	20.9
hylang	`(sys-open ...)`, or `(sys :open ...)`	22.4
HyBASIC	`SYS "OPEN"`, with `RREG` for the registers after	24.8

It can wait means the call can make the program wait: for a key, a disk, another task, or time to pass.

K.1 The system

SYSINFO (`$F800`). What the Hydra has: the version of the system calls, the RAM modules, and the free tasks. *Out:* A = the ABI version (1); X = how many RAM modules are installed; r0 = the free tasks (bit = task).

ERRSTR (`$F803`). An error code's message, as text (Appendix G). *In:* A = an error code; r0 = a buffer (32 bytes). *Out:* the error's text in the buffer, zero-terminated ("error $xx" for an unknown code).

KMESG (`$F806`). The kernel's messages: what it printed as the Hydra started (also `/dev/kmesg`). *In:* r0 = a buffer; r1 = its size, the most it copies; r2 = an offset in the messages held, 0 the oldest. *Out:* A/X = the bytes held, KMESG_SIZE at most; the buffer has those from r2, r1 at most.

REBOOT (`$F809`). Start the Hydra again, as the RESET button does; or start the hardware test. *In:* A = 0, or REBOOT_HWTEST. *Out:* (doesn't return).

XCALL (`$F80C`). Call a routine in another bank of the paged ROM: a library's. Every register goes to it and comes back. *In:* r15 = a routine's address (`$A000`-`$DFFF`); r14 (its low byte) = the paged ROM bank it's in; A, X, Y, the flags and r0-r13: the routine's. *Out:* A, X, Y, the flags and r0-r15 as the routine leaves them. *(Not in the languages.)*

K.2 Tasks

EXITS (`$F830`). End this program, with an exit code and a message (its status). Returning from `main` is `EXITS` with code 0. *In:* A = the exit code (0: success); r0 = a message (zero-terminated, 31 characters at most), or 0 for none. *Out:* (doesn't return).

WAIT (`$F833`). Wait for a program this one started to end, and get its exit code and message. *In:* A = a task this one started, or `$FF` for any; r0 = a buffer for the message (32 bytes), or 0. *Out:* A = the task that ended; X = its exit code; its message in r0's buffer ("" for none). *Errors:* E_CHILD, E_INTR. *It can wait.*

SPAWN (`$F836`). Start a program, in a task of its own. *In:* r0 = the program's path (through the namespace: /bin/NAME; or #m/NAME, a module in the paged ROM); r1 = its arguments (zero-terminated strings, an empty one after the last: 176 bytes at most), or 0; A = flags (SPAWN_NEWGROUP, SPAWN_NEWNS, SPAWN_FDMAP, SPAWN_NOENV, SPAWN_STOPPED); r2 = an fd map (with SPAWN_FDMAP). *Out:* A = the new task. *Errors:* E_NOENT, E_NOEXEC, E_NOTASK, E_TOOBIG, E_INVAL, E_MFILE, E_INTR. *It can wait.*

GETPID (`$F839`). This task's number (0 to 15). *Out:* A = this task's number.

YIELD (`$F83C`). Let the other tasks have a turn.

SLEEP (`$F83F`). Wait for a number of ticks (200 a second). *In:* A/X = ticks (TICK_HZ a second; 0-32767). *Errors:* E_INTR. *It can wait.*

SLEEP_UNTIL (`$F842`). Wait until the tick count reaches a number: for keeping steady time. *In:* A/X = a tick count (at most 32767 ticks ahead). *Errors:* E_INTR. *It can wait.*

TICKS (`$F845`). The tick count: 200 a second since the Hydra started. *Out:* A/X = the tick count (TICK_HZ a second; it wraps after about 5.5 minutes).

PREEMPT_OFF (`$F848`). Keep the processor: no other task runs until `PREEMPT_ON`. Use it only for a moment.

PREEMPT_ON (`$F84B`). Let the other tasks run again, after `PREEMPT_OFF`.

TASKINFO (`$F84E`). What the kernel knows about a task: its state, parent, time used, name (as `ps` shows). *In:* A = a task (0-15); r0 = a buffer (TI_SIZE bytes). *Out:* the buffer: TI_STATE, TI_FLAGS, TI_PARENT, TI_CPU (3 bytes: ticks), TI_BANK, TI_TYPE, TI_NAME (16). *Errors:* E_SRCH.

NOTIFY (`$F851`). Set this task's note handler (section 27.16), or take it away. *In:* r0 = the note handler, or 0 for none. *(Not in the languages.)*

NOTE (`$F854`). Send a note to a task, or to a note group. *In:* A = a task, or NOTE_GROUP | a note group; X = the note (1-31: NOTE_). **Errors:* E_INVAL, E_SRCH, E_PERM.

MODINFO (`$F857`). An entry of the paged ROM's module directory: a module's banks, its page in its first bank (where its header is, for a small module that shares the bank), its type and its name. *In:* A = an entry of the module directory (0 on); r0 = a buffer (ME_SIZE bytes). *Out:* the buffer: ME_BANK, ME_BANKS, ME_TYPE (HT_), ME_FLAGS (HF_), ME_NAME (11, zero-padded), ME_PAGE. *Errors:* E_NOENT.

ENV_GET (`$F85A`). Read an environment variable, of this task or another. *In:* A = a task (`$FF`: this one); r0 = a variable's name; r1 = a buffer; r2 = its size; r3 = an offset in the value. *Out:* A/X = the value's length from the offset (0 at its end or past it); as much of it as fits, in the buffer. *Errors:* E_NOENT, E_SRCH, E_INVAL, E_NAMETOOLONG.

ENV_PUT (`$F85D`). Set an environment variable (or part of its value). *In:* A = a task (`$FF`: this one); r0 = a variable's name; r1 = bytes; r2 = their count; r3 = where in its value they go (the value cut there first: 0 for a new value; its length at most). *Errors:* E_NOMEM, E_INVAL, E_SRCH, E_NOENT, E_NAMETOOLONG.

ENV_DEL (`$F860`). Remove an environment variable. *In:* A = a task (`$FF`: this one); r0 = a variable's name. *Errors:* E_NOENT, E_SRCH, E_INVAL, E_NAMETOOLONG.

ENV_NAME (`$F863`). The name of an environment's variable number X: for listing them all. *In:* A = a task (`$FF`: this one); X = which variable (0 on); r0 = a buffer (ENV_NAME_MAX + 1 bytes). *Out:* its name in the buffer, zero-terminated; A/X = its value's length. *Errors:* E_NOENT, E_SRCH.

TASKREAD (`$F866`). Read another task's arguments, current directory, environment, registers or open files (as `/proc` does). *In:* A = a task (`$FF`: this one); X = what: TR_ARGS (its arguments: ARGS_MAX bytes), TR_CWD (its current directory: PATH_MAX + 1 bytes), TR_ENV (its environment's first variables, whole, and a 0: ENV_SIZE bytes at most), TR_FRAME (its state and the frame it left as it was switched out: TF_SIZE bytes, TF_*) or TR_ENVAT (part of its environment), or TR_FD (its fd r2: FI_SIZE bytes, FI_*); r0 = a buffer that big; r1 = TR_ENVAT's count (the most it copies); r2 = TR_ENVAT's offset, or TR_FD's fd. *Out:* the bytes in the buffer; A/X = TR_ENVAT's: the environment's bytes in use (its 0 too: it copied as many as there are from r2, r1 at most). *Errors:* E_SRCH, E_INVAL, E_BADF.

GETPPID (`$F869`). The task that started this one. *Out:* A = this task's parent (`$FF`: none).

SEM_NEW (`$F86C`). Make a **semaphore**: a counter that tasks take and give back, to share something without getting in each other's way. *In:* A = its count: how many can take it before a taker waits (0-255); X = 0, or SEM_MUTEX: a mutex (its count 1, and it has a holder: only the task that took it gives it back). *Out:* A = the semaphore (0 to SEM_MAX - 1). *Errors:* E_NOMEM.

SEM_ACQUIRE (`$F86F`). Take one from a semaphore; if it has none, wait until another task gives one back. *In:* A = a semaphore. *Errors:* E_INVAL, E_BUSY, E_INTR. *It can wait.*

SEM_TRY (`$F872`). Take one from a semaphore, but don't wait if it has none. *In:* A = a semaphore. *Errors:* E_INVAL, E_BUSY, E_AGAIN.

SEM_RELEASE (`$F875`). Give one back to a semaphore, waking a task that's waiting for it. *In:* A = a semaphore. *Errors:* E_INVAL, E_PERM, E_RANGE.

SEM_FREE (`$F878`). Free a semaphore. *In:* A = a semaphore. *Errors:* E_INVAL.

EXEC (`$F87B`). Run another program in this task, in this one's place. *In:* r0 = the program's path (as SPAWN's); r1 = its arguments (as SPAWN's: 176 bytes at most), or 0. *Out:* (doesn't return, once the program is found: it runs in this task's place). *Errors:* E_NOENT, E_NOEXEC, E_TOOBIG, E_PERM, E_INTR. *It can wait.*

K.3 Memory

BREAK (`$F890`). Move the end of the program's memory (its **break**), or ask where it is. *In:* r0 = the new end of the program's data (the first byte past it), or 0 to ask. *Out:* r0 = the end. *Errors:* E_INVAL, E_NOMEM.

PAGES_ALLOC (`$F893`). Take some pages (256 bytes each) of the task's memory, above the break. *In:* A = pages (256 bytes each). *Out:* r0 = the first one's address; A = its page. *Errors:* E_INVAL, E_NOMEM.

PAGES_FREE (`$F896`). Give pages back. *In:* A = pages; r0 = the first one's address. *Errors:* E_INVAL.

BANKS (`$F899`). How many RAM banks the task has, and on which modules. *Out:* A = the banks the task has (16 for each good RAM module); r0 = the good modules (bit = module).

BANKS_ALLOC (`$F89C`). Take some of the task's RAM banks: A gives the first bank's number, to write to `$00`. *In:* A = banks. *Out:* A = the first (for `$00`). *Errors:* E_INVAL, E_NOMEM.

BANKS_FREE (`$F89F`). Give banks back. *In:* A = the first bank; X = how many. *Errors:* E_INVAL.

SEG_CREATE (`$F8A2`). Make a shared **segment**: shared RAM banks that other tasks can attach to. *In:* A = banks (1-128, 8K each). *Out:* A = the segment (0-15). *Errors:* E_INVAL, E_NOMEM.

SEG_ATTACH (`$F8A5`). Attach to a segment. *In:* A = a segment. *Errors:* E_INVAL.

SEG_DETACH (`$F8A8`). Detach from a segment (the last task to leave frees it). *In:* A = a segment. *Errors:* E_INVAL.

SEG_MAP (`$F8AB`). How to reach a bank of a segment: the values to write to U and to `$00`. *In:* A = a segment; X = one of its banks (0 on). *Out:* A = the value for U; X = the value for `$00`. *Errors:* E_INVAL, E_RANGE.

SEGINFO (`$F8AE`). The shared RAM: how much there is, and which segments are in use. *Out:* A = the shared banks segments can have (8K each: the good chips'); X = those segments have now; r0 = the segments in use (bit = segment).

BANKS_ALLOC_IN (`$F8B1`). As `BANKS_ALLOC`, from a range of bank numbers only. *In:* A = how many; X = the lowest bank it may take; Y = the highest (module m's are $m0-$mF). *Out:* A = the first. *Errors:* E_INVAL, E_NOMEM.

SEG_CREATE_IN (`$F8B4`). As `SEG_CREATE`, from a range of the shared RAM only. *In:* A = how many banks (1-128); X = the lowest shared bank ID it may take; Y = the highest (U << 4 | the bank's low nibble; segments have `$80-$FF`). *Out:* A = the segment. *Errors:* E_INVAL, E_NOMEM.

K.4 Files

OPEN (`$F8C0`). Open a file, for reading or writing or both. *In:* r0 = a path; A = the mode (O_READ, O_WRITE, O_RDWR; | O_TRUNC, O_NONBLOCK). *Out:* A = an fd. *Errors:* E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE. *It can wait.*

CREATE (`$F8C3`). Make a new file, or a directory, and open it. *In:* r0 = a path; A = the mode; X = the new file's mode, high byte (DM_DIR: a directory). *Out:* A = an fd. *Errors:* E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE, E_EXIST. *It can wait.*

CLOSE (`$F8C6`). Close a file. *In:* A = an fd. *Errors:* E_BADF. *It can wait.*

READ (`$F8C9`). Read bytes from a file. *In:* A = an fd; r0 = a buffer; r1 = a count. *Out:* A/X = the count read (0: the end of the file). *Errors:* E_BADF, E_AGAIN, E_INTR. *It can wait.*

WRITE (`$F8CC`). Write bytes to a file. *In:* A = an fd; r0 = the data; r1 = its count. *Out:* A/X = the count written. *Errors:* E_BADF, E_AGAIN, E_INTR. *It can wait.*

SEEK (`$F8CF`). Move the place in a file where the next read or write happens. *In:* A = an fd; r0, r1 = an offset (low word, high word: signed); X = from: 0 the start, 1 the offset, 2 the end. *Out:* r0, r1 = the new offset. *Errors:* E_BADF, E_INVAL. *It can wait.*

STAT (`$F8D2`). A file's **stat record**: its name, type, length, time and more. *In:* r0 = a path; r1 = a buffer (SR_SIZE bytes). *Out:* the file's stat record in the buffer (SR_). **Errors:* E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE. *It can wait.*

FSTAT (`$F8D5`). An open file's stat record. *In:* A = an fd; r0 = a buffer (SR_SIZE bytes). *Out:* the stat record in the buffer. *Errors:* E_BADF. *It can wait.*

WSTAT (`$F8D8`). Change a file: its name, its mode, or its length. *In:* r0 = a path; r1 = a stat record. *Errors:* E_NOENT, E_NODEV, E_NAMETOOLONG, E_MFILE, E_NFILE, E_NOSYS. *It can wait.*

FWSTAT (`$F8DB`). Change an open file, as `WSTAT` does. *In:* A = an fd; r0 = a stat record. *Errors:* E_BADF, E_NOSYS. *It can wait.*

REMOVE (`$F8DE`). Remove a file, or an empty directory. *In:* r0 = a path. *Errors:* E_NOENT, E_NODEV, E_NAMETOOLONG, E_NOTEMPTY, E_NOSYS. *It can wait.*

DUP (`$F8E1`). Another fd for an open file. *In:* A = an fd. *Out:* A = a new fd (the lowest free), on the same channel (the same offset). *Errors:* E_BADF, E_MFILE.

DUP2 (`$F8E4`). Make fd X the same as fd A (closing what X was first): how the shell does > and |. *In:* A = an fd; X = another (0-15). *Errors:* E_BADF. *It can wait.*

CHDIR (`$F8E7`). Change the current directory. *In:* r0 = a directory's path. *Errors:* E_NOTDIR, E_NOENT, E_NODEV, E_NAMETOOLONG. *It can wait.*

GETCWD (`$F8EA`). The current directory. *In:* r0 = a buffer (PATH_MAX + 1 bytes). *Out:* A = the current directory's length; the directory in the buffer, zero-terminated.

PIPE (`$F8ED`). Make a **pipe**: two fds, and what's written to one is read from the other. *Out:* A = an fd to read from; X = an fd to write to. *Errors:* E_NODEV, E_MFILE, E_NFILE, E_NOMEM. *It can wait.*

FD2PATH (`$F8F0`). The name an open file was opened by. *In:* A = an fd; r0 = a buffer (PATH_MAX + 1 bytes). *Out:* the name in the buffer, zero-terminated. *Errors:* E_BADF.

K.5 Namespaces

BIND (`$F920`). Bind a name to another place in the namespace (section 9.5). *In:* r0 = new (a path); r1 = old (a path: absolute); A = MREPL, MBEFORE or MAFTER, and MCREATE. *Errors:* E_NOENT, E_INVAL, E_NSFULL, E_NODEV, E_NAMETOOLONG. *It can wait.*

MOUNT (`$F923`). Mount a device's files in the namespace. *In:* X = a device letter; r0 = its spec (a string, 8 characters at most), or 0; r1 = old; A = BIND's flags. *Errors:* E_NODEV, E_NOENT, E_INVAL, E_NSFULL, E_NAMETOOLONG. *It can wait.*

UNMOUNT (`$F926`). Undo a bind or a mount; or, with no old, all of them: an empty namespace of the task's own. *In:* r0 = new, or 0; r1 = old, or 0. *Errors:* E_NOENT, E_INVAL, E_NSFULL, E_NAMETOOLONG. *It can wait.*

NSINFO (`$F929`). An entry of a task's namespace: what's bound or mounted where (as `ns` shows). *In:* A = a task (`$FF`: this one); X = which entry of its namespace (0 on); r0 = a buffer (NI_SIZE bytes). *Out:* the entry in the buffer (NI_*): its mount point, its device letter and spec, the path in that device, its flags (MCREATE) and its place among the union's members.* **Errors:* E_RANGE, E_SRCH.

K.6 The console

PUTC (`$F950`). Write a byte (a character) to the standard output. *In:* A = a byte. *It can wait.*

PUTS (`$F953`). Write a string to the standard output. *In:* r0 = a string, zero-terminated. *Errors:* E_INTR. *It can wait.*

GETC (`$F956`). Read a byte from the standard input, waiting for one. *Out:* A = a byte. *Errors:* E_EOF, E_INTR. *It can wait.*

PUTHEX (`$F959`). Write a byte to the standard output as two hex digits. *In:* A = a byte. *It can wait.*

K.7 The clock

TIME (`$F980`). The date and time: seconds since the start of 2000. *Out:* r0/r1 = the clock: seconds since 2000-01-01 00:00:00.

TIME_SET (`$F983`). Set the date and time. *In:* r0/r1 = seconds since 2000-01-01 00:00:00.

RTC (`$F986`). Read or write the DS1747 clock chip's registers. *In:* A = 0, read, or 1, write; r0 = 8 bytes: the DS1747's registers (RTC_REGS, hw.inc), BCD. *Errors:* E_INVAL.

K.8 Drivers

IRQ_OWN (`$F9B0`). A driver: take an interrupt line, so its interrupts run the driver's handler. *In:* A = an IRQ line (0-14), or LINE_VIA_T2 (VIA timer 2), or LINE_VIA_CA1 (VIA CA1). *Errors:* E_BUSY, E_RANGE, E_INVAL. *(Not in the languages.)*

IRQ_RELEASE (`$F9B3`). A driver: give an interrupt line back. *In:* A = an IRQ line. *Errors:* E_PERM, E_RANGE. *(Not in the languages.)*

WAKE (`$F9B6`). Wake a task that's waiting in `PAUSE`. *In:* A = a task. *(Not in the languages.)*

PAUSE (`$F9B9`). Wait until another task (or an interrupt handler) wakes this one. *It can wait. (Not in the languages.)*

NOTE_POST (`$F9BC`). A driver: send a note. *In:* A = a task, or NOTE_GROUP | a note group; X = the note. *Errors:* E_INVAL, E_SRCH, E_PERM. *(Not in the languages.)*

SRV_REGISTER (`$F9BF`). A driver: serve a device letter, so the requests for its files come to this task. *In:* A = a device letter (printable; not #). *Errors:* E_INVAL, E_EXIST, E_NOMEM. *(Not in the languages.)*

SRV_TAKE (`$F9C2`). A driver: take the request being served. *Out:* A = the request (R_*); Y = the client; the request block in TASK_INBOX, and (R_OPEN, R_CREATE, R_REMOVE) the name in TASK_PATH. *(Not in the languages.)*

SRV_REPLY (`$F9C5`). A driver: answer the request being served. *In:* TASK_INBOX's RQ_FID, RQ_DONE, RQ_PERM, RQ_EVENT: what the request returns; A, C: the answer. *Out:* A and C kept. *(Not in the languages.)*

CLIENT_READ (`$F9C8`). A driver: copy bytes from the program whose request it's serving. *In:* r0 = here; r1 = there, in the client's view; r2 = a count. *(Not in the languages.)*

CLIENT_WRITE (`$F9CB`). A driver: copy bytes to the program whose request it's serving. *In:* r0 = here (the data); r1 = there, in the client's view; r2 = a count. *(Not in the languages.)*

NOTE_QUEUE (`$F9CE`). An interrupt handler: send a note to a note group (as the console does for CTRL-C). *In:* IRQs off; X = a note group (0-15); A = the note's bit: 1 << (note - 1), for a note 1-8. *Out:* X kept. *(Not in the languages.)*

ROMREAD (`$F9D1`). Copy bytes from any bank of the paged ROM. *In:* A = a paged ROM bank; r0 = where in it (`$A000-$DFFF`); r1 = a buffer (not at `$A000-$DFFF`); r2 = a count. *Out:* the bytes in the buffer. *Errors:* E_PERM. *(Not in the languages.)*

TASKMEM (`$F9D4`). A driver: read or write another task's memory (as `/proc`'s `mem` does). *In:* A = a task; X = TM_READ or TM_WRITE, | TM_BANK; r0 = a buffer; r1 = an address in the task's view, `$0000-$DFFF` (with TM_BANK: `$8000-$9FFF`, in its RAM bank r3); r2 = a count (the range ends at `$E000`, or with TM_BANK at `$A000`, at most). *Errors:* E_PERM, E_SRCH, E_INVAL. *(Not in the languages.)*

TASKSTOP (`$F9D7`). A driver: stop a task where it is, or start it again. *In:* A = a task; X = 1 to stop it, 0 to start it again. *Errors:* E_PERM, E_SRCH. *(Not in the languages.)*

TASKSTEP (`$F9DA`). A driver: the debugger's step. *In:* A = a task; X = TS_STEP, TS_NEXT, TS_BREAKS or TS_NOBREAKS. *Errors:* E_PERM, E_SRCH, E_INVAL, E_BUSY. *(Not in the languages.)*

SRV_NAP (`$F9DD`). A driver with work of its own between its requests: wait a while (its requests are served meanwhile), then go on. *In:* A/X = ticks (1-32767); or 0, and Y = the driver's event count (TASK_EVENT) as it last saw it. *It can wait. (Not in the languages.)*

INIT_RUN (`$F9E0`). Start init again in a task of its own, with a number as its argument (the console: a new window's shell). *In:* A = a number, 0-15. *(Not in the languages.)*

K.9 Debugging HydraOS

DBG_SCALL (`$FA10`). For testing: call a driver directly. *In:* Y = a driver task; A, X = what its serve entry gets. *Out:* A, X and C from its serve entry. *Errors:* E_SRCH, E_NODEV. *It can wait. (Not in the languages.)*

DBG_KCOPY (`$FA13`). For testing: copy bytes between this task and another. *In:* A = another task; r0 = an address in this task; r1 = an address in that task; r2 = a byte count; C = 0 to copy from this task to that one, C = 1 the other way. *Errors:* E_SRCH. *(Not in the languages.)*

DBG_PS (`$FA16`). For testing: print a line for each task. *It can wait. (Not in the languages.)*

Index

#

&0 *hylang* 292
=0 *hylang* 302
>0 *hylang* 302
<0 *hylang* 302
>=0 *hylang* 302
<=0 *hylang* 302
!=0 *hylang* 302
0< *HyForth* 278
0<> *HyForth* 281
0= *HyForth* 278
0> *HyForth* 281
&1 *hylang* 292
=1 *hylang* 302
!=1 *hylang* 302
1+ *HyForth* 277
1+ *hylang* 295
1+2i *hylang* 292
1- *HyForth* 277
1- *hylang* 295
1.25 *hylang* 292
1_000 *hylang* 292
&2 *hylang* 292
2! *HyForth* 278
2* *HyForth* 277
2/ *HyForth* 277
2/3 *hylang* 292
2>r *HyForth* 281
2@ *HyForth* 278
2drop *HyForth* 277
2dup *HyForth* 277
2i *hylang* 292
2over *HyForth* 277
2r> *HyForth* 281
2r@ *HyForth* 281
2swap *HyForth* 277
-3 *hylang* 292
42 *hylang* 292

A

\a *hylang* 292
abort *HyForth* 280
abort" *HyForth* 280
ABS *HyBASIC* 304
abs *HyForth* 277
abs *hylang* 295
ABSOLUTE *HyBASIC* 304
accept *HyForth* 279
ACIA 325
action-of *HyForth* 281
address 16, 22, 76, 164, 182, 269
address decoder 62
addressing modes 237
again *HyForth* 281
ahead *HyForth* 283
align *HyForth* 278
aligned *HyForth* 278
all? *hylang* 295
allocate *HyForth* 286
allot *HyForth* 278
alpha? *hylang* 297
also *HyForth* 286
AND *HyBASIC* 304
and *HyForth* 278
and *hylang* 293
anti-static foam or tubes 34
anti-static wrist strap 34
any? *hylang* 295
APPEND *HyBASIC* 304
append-file *hylang* 300
apply *hylang* 302
arg *HyForth* 288
ARG$ *HyBASIC* 304
argc *HyForth* 288
arguments 65, 84, 197
arithmetic 277
arithmetic, in HyBASIC 216
arithmetic, in HyForth 177
arithmetic, in hylang 197
array 218
arrays, in HyBASIC 218
as command 108
AS *HyBASIC* 304
ASC *HyBASIC* 304
ASCII 318
ASM *HyBASIC* 304
asm library 258
assembler 19, 234, 243
assembly language 108, 234
at-xy *HyForth* 284
atan *hylang* 296
ATN *HyBASIC* 304
atom 198
ATX power supply 23, 55
Audio In 148, 155
avg *hylang* 302

B

backing up your work 71, 147
balanced 173
bands 35
BANK *HyBASIC* 304
bank! *HyForth* 288
bank-alloc *hylang* 301
bank-free *hylang* 301
bank-read *hylang* 301
bank-window *HyForth* 288
bank-write *hylang* 301
bank@ *HyForth* 288
BANKS 349
banks *hylang* 301
BANKS_ALLOC 349
BANKS_ALLOC_IN 350
BANKS_FREE 349
bar 105
base 170
base 10 20
base 16 21
base 2 20
Base 3 171
Base 36 171
Base 8 171
BASE *HyBASIC* 304
base *HyForth* 279
base *hylang* 296
base setting 175
base string 172
BASE$ *HyBASIC* 305
BASIC 213
basic command 108
becomes the shell 82
BEEP *HyBASIC* 305
beep *HyForth* 284
begin *HyForth* 280
begin-structure *HyForth* 285
best-of *hylang* 302
between? *hylang* 302
bgcolor *HyForth* 284
bin *HyForth* 282
bin *hylang* 296
binary 20
BINARY *HyBASIC* 305
binary-coded decimal 239
bind 96, 351
bind *HyForth* 287
bind *hylang* 301
bit 20, 176
BIT *HyBASIC* 305
bit-and *hylang* 296
bit-not *hylang* 296
bit-or *hylang* 296
bit-xor *hylang* 296
bit? *hylang* 296
bitmap 156
bl *HyForth* 279
black *HyForth* 284
blank *HyForth* 285
blink *HyForth* 284
blk *HyForth* 286
block 145, 146, 196, 216
block *HyForth* 286
blue *HyForth* 284
body 199
>body *HyForth* 280
bold *HyForth* 284
boot drivers 81
break 237, 255, 349
breakpoint 270
bright *HyForth* 284
bring-up console 79
buffer 300
buffer *HyForth* 286
buffer *hylang* 300

buffer-cmp *hylang* 301, 302
buffer-copy *hylang* 301
buffer-fill *hylang* 301
buffer-get *hylang* 300
buffer-put *hylang* 300
buffer: *HyForth* 281
buffer? *hylang* 300
bug 268
burns 33
bus 16
buying the parts 40
BYE *HyBASIC* 305
bye *HyForth* 280
byte 16, 20, 234
bytes *hylang* 296
bytes-number *hylang* 296

C

C = 0 252
C = 1 252
c! *HyForth* 278
c" *HyForth* 281
c, *HyForth* 278
C, programs in 255
c@ *HyForth* 278
ca65 243
calc command 109
call 197
CALL *HyBASIC* 305
capacitor 36
capital letters in names 67, 93
card-edge sockets 37
CASE *HyBASIC* 305
case *HyForth* 281
case *hylang* 302
cat 109
cat command 109
catch *HyForth* 282
cbit *HyForth* 290
cc65 31
cc65.github.io 31
cd *HyForth* 287
cd *hylang* 300
CDBL *HyBASIC* 305
ceil *hylang* 302
cell 178
cell+ *HyForth* 278
cells *HyForth* 278
ceramic capacitors 336
cfield: *HyForth* 285
channels 148
char *HyForth* 279
char+ *HyForth* 278
char-at *hylang* 297
char-code *hylang* 297
chars *HyForth* 278
CHDIR 350
CHDIR *HyBASIC* 305
chord 149
CHR$ *HyBASIC* 305
chrome 105
CINT *HyBASIC* 305
CIRCLE *HyBASIC* 305
circuit boards, having them made 40
claim 133
CLEAR *HyBASIC* 305
clear-below *HyForth* 284
clear-line *HyForth* 284
CLIENT_READ 352
CLIENT_WRITE 352
CLNG *HyBASIC* 305
clock 16
clock chip 27
clock *HyForth* 291
clock *hylang* 301
clock jumper 62
CLOSE 350
CLOSE *HyBASIC* 305
close *hylang* 300
close-dir *HyForth* 288
close-file *HyForth* 282
CLS *HyBASIC* 305
cluster 146
CMOS 34
cmove *HyForth* 285
cmove> *HyForth* 285
cmp command 110
cmp *hylang* 296
code word 266
code-banks *HyForth* 288
code-char *hylang* 297
COLOR *HyBASIC* 305
color *HyForth* 284
command 65, 84
COMMAND$ *HyBASIC* 305
comment 179, 215, 244, 247
comp *hylang* 302
compare *HyForth* 285
comparison 180
compile, *HyForth* 281
compiling 179
COMPLEX *HyBASIC* 305
complex *HyForth* 289
complex *hylang* 296
complex numbers 171
complex? *HyForth* 289
cond *hylang* 302
conditional assembly 251
cons *HyForth* 291
cons *hylang* 294
console 81, 134
CONST *HyBASIC* 305
constant 182, 244, 249
constant *HyForth* 280
CONT *HyBASIC* 305
contains? *hylang* 302
control characters 138, 318
control file 132
cooked mode 134
copy-file *hylang* 300
copying a file 69
COS *HyBASIC* 305
cos *hylang* 296
count *HyForth* 278
count *hylang* 295
cp command 110
CP437 322
CPU 15
cr *HyForth* 279
CREATE 350
create *HyForth* 280
create-file *HyForth* 282
crystal 18
cs-pick *HyForth* 283
cs-roll *HyForth* 283
CSI 138
CSNG *HyBASIC* 305
CSRLIN *HyBASIC* 305
ctl *HyForth* 288
CTRL-C (stopping a program) 66, 67
CTRL-D (the end of your typing) 67
cube *hylang* 302
current directory 68, 93
curry *hylang* 302
cursor-down *HyForth* 284
cursor-left *HyForth* 284
cursor-off *HyForth* 284
cursor-on *HyForth* 284
cursor-restore *HyForth* 284
cursor-right *HyForth* 284
cursor-save *HyForth* 284
cursor-up *HyForth* 284
CVN *HyBASIC* 305
cwd *hylang* 300
cyan *HyForth* 284
cycles 241, 313

D

d>n *HyForth* 289
danlang 197
data area 146
DATA *HyBASIC* 305
date and time, setting 165
date command 110
date *hylang* 301
DATE$ *HyBASIC* 305
date-parts *hylang* 301
db command 111
DBG_KCOPY 352
DBG_PS 352
DBG_SCALL 352
debugger 19, 268
debugging 268, 341
DEC Special Graphics 141, 321
decimal 20, 171
decimal *HyForth* 279
decisions, in HyBASIC 216
decisions, in HyForth 181
decisions, in hylang 201
decisions, in rc 88
DECLARE *HyBASIC* 305
decoupling capacitors 36
def *hylang* 293
defer *HyForth* 281
defer! *HyForth* 281
defer@ *HyForth* 281
defined? *hylang* 293
definition 179
definitions *HyForth* 286
DEFINT *HyBASIC* 305
DEFSTR *HyBASIC* 305
DELETE *HyBASIC* 305

delete-file *HyForth* 283
deleting a file 69
DENOMINATOR *HyBASIC* 305
depth *HyForth* 277
desoldering braid 39
/dev/bell 153
/dev/psg 152
/dev/snd 152
device name 95, 131
df command 111
dictionary 82
digit? *hylang* 297
DIGITS *HyBASIC* 305
digits *HyForth* 289
digits *hylang* 296
digits of your own 173, 174
DIM *HyBASIC* 305
dim *HyForth* 284
DIP 37
dir *hylang* 300
DIR$ *HyBASIC* 305
dir? *hylang* 300
direct mode 213
directive 247
directories, making 69
directory 66, 93, 146
dis command 111
disasm *HyForth* 290
disassembler 19, 261
disk *HyForth* 291
disks 94
distinct *hylang* 302
div *hylang* 302
divmod *hylang* 302
DO *HyBASIC* 305
do *HyForth* 280
?do *HyForth* 281
do *hylang* 293
does> *HyForth* 280
dot 36, 37
dotimes *hylang* 293
drag-solder 39
DRAW *HyBASIC* 305
driver 78, 81, 130
drop *HyForth* 277
drop *hylang* 294
DS1747 51, 165
du command 111
dump *HyForth* 283
DUP 350
dup *HyForth* 276
?dup *HyForth* 276
DUP2 350

E

each *hylang* 293
echo command 112
ed command 112
edit command 113
EDIT *HyBASIC* 306
editing a file 69
editing a line 66
ekey *HyForth* 284
ekey>char *HyForth* 284
ekey>fkey *HyForth* 284
ekey? *HyForth* 284
electrolytic capacitors 36
elem? *hylang* 294
ELSE *HyBASIC* 306
else *HyForth* 280
ELSEIF *HyBASIC* 306
emit *HyForth* 279
emit? *HyForth* 284
empty-buffers *HyForth* 286
END *HyBASIC* 306
end *hylang* 294
end of the input 134
end-library *HyForth* 286
end-structure *HyForth* 285
endcase *HyForth* 281
endof *HyForth* 281
ends-with? *hylang* 302
env *hylang* 301
ENV$ *HyBASIC* 306
ENV_DEL 348
ENV_GET 348
ENV_NAME 348
ENV_PUT 348
envelope 154
ENVIRON *HyBASIC* 306
ENVIRON$ *HyBASIC* 306
environment 86, 166
environment? *HyForth* 280
EOF *HyBASIC* 306
eq *hylang* 296
EQV *HyBASIC* 306
ERASE *HyBASIC* 306
erase *HyForth* 281
ERL *HyBASIC* 306
ERR *HyBASIC* 306
ERR$ *HyBASIC* 306
error code 132, 252, 328
ERROR *HyBASIC* 306
error *hylang* 299
error-code *hylang* 299
error-message *hylang* 299
errors, when you type 67
ERRSTR 347
errstr *hylang* 301
escape sequences 134, 138
eval *hylang* 293
evaluate *HyForth* 280
even? *hylang* 302
EXEC 349
execute *HyForth* 280
execution token 184
exists? *hylang* 300
EXIT *HyBASIC* 306
exit *HyForth* 280, 287
exit *hylang* 294
EXITS 347
exits *HyForth* 287
EXP *HyBASIC* 306
exp *hylang* 296
expr? *hylang* 293
expression 109, 197, 248
extension cable 58
extent blocks 146
extents 146

F

f *hylang* 292
factorial 184
false *HyForth* 281
fault finding 62
FD2PATH 351
fdiv *hylang* 302
fexpr *hylang* 293
FIB *HyBASIC* 306
fib *hylang* 296
Fibonacci number 170
+field *HyForth* 285
field: *HyForth* 285
fields 224
FIFO 160
file 93
file access method 189
file id 189
file number 222
file server 130
file-position *HyForth* 282
file-size *HyForth* 282
file-status *HyForth* 283
file? *hylang* 300
FILES *HyBASIC* 306
files, in HyBASIC 222
files, in HyForth 189
files, in hylang 207
files, making 69
fill *HyForth* 278
filter *hylang* 294
find *HyForth* 280
find *hylang* 295
first *hylang* 302
FIX *HyBASIC* 306
fixed decimals 170
FIXED *HyBASIC* 306
fixed? *HyForth* 289
flag 180, 235, 313
flash programmer 49
flat side 46
flatten *hylang* 302
flip *hylang* 302
floor *hylang* 302
Flow control 59, 137
flow control (XON/XOFF) 58, 137
flush *HyForth* 286
flush-file *HyForth* 283
flux 37
FM synthesis 148
fm/mod *HyForth* 277
fn *hylang* 293
foldl *hylang* 295
foldr *hylang* 295
font 321
footer 105
FOR *HyBASIC* 306
form *HyForth* 284
formals 199
format 91, 106
format *hylang* 297

Forth 177
`forth` command 115
`forth` *HyForth* 286
Forth word 84
`forth-wordlist` *HyForth* 285
FPGA 156
frames 142
`FRE` *HyBASIC* 306
`free` command 115
`free` *HyForth* 286
`free` *hylang* 301
free map 146
`FREEFILE` *HyBASIC* 306
`from#` *hylang* 298
`from-bytes` *hylang* 296
`fsck` command 116
`fst` *hylang* 294
`FSTAT` 350
full format 145
`fun` *hylang* 293
function 89, 218
`FUNCTION` *HyBASIC* 306
functions, in HyBASIC 218
functions, in hylang 199
functions, in rc 89
`FWSTAT` 350

G

gateware 158
`GCD` *HyBASIC* 306
General MIDI 150
General MIDI drum number 151
`gensym` *hylang* 297
Gerber 40
`GET` *HyBASIC* 306
`get-base` *HyForth* 289
`get-current` *HyForth* 286
`get-dir` *HyForth* 288
`get-order` *HyForth* 286
`GETC` 351
`GETCWD` 351
`getenv` *HyForth* 287
`GETPID` 348
`GETPPID` 349
git 28
github.com/danstruthers/hydra-16 28
`glob` *hylang* 300
global 199
`GOSUB` *HyBASIC* 306
`GOTO` *HyBASIC* 306
`gpio` *HyForth* 291
`GPRINT` *HyBASIC* 306
graphic rendition 140
graphics, in HyBASIC 225
graphics, in HyForth 194
graphics, in hylang 211
`green` *HyForth* 284
`grep` command 116

H

`h` *hylang* 298
handler 219
handshake 163
hardware test 63, 80
hash 202
`hash-add-tag` *hylang* 298
`hash-call` *hylang* 298
`hash-clone` *hylang* 298
`hash-const?` *hylang* 298
`hash-create` *hylang* 298
`hash-get` *hylang* 298
`hash-key?` *hylang* 298
`hash-keys` *hylang* 298
`hash-lock` *hylang* 298
`hash-locked?` *hylang* 298
`hash-make-const` *hylang* 298
`hash-make-not-nil` *hylang* 298
`hash-make-private` *hylang* 298
`hash-private?` *hylang* 298
`hash-put` *hylang* 298
`hash-remove` *hylang* 298
`hash-tag?` *hylang* 298
`hash-values` *hylang* 298
`head` command 117
`head` *hylang* 294
header 105, 237, 244
hello 82
`hello` command 117
help 74
`here` *HyForth* 278
here string 198
hex 21
`hex` *HyForth* 281
`hex` *hylang* 296
`HEX$` *HyBASIC* 306
hexadecimal 21, 171
`hi` *hylang* 296
`hold` *HyForth* 279
`hold` *hylang* 293
`holds` *HyForth* 281
holes 146
hours to build 32
http://localhost:8016 29
http://localhost:8017 28
`hwtest` command 117
HyBASIC 19, 73, 74, 213
Hydra emulator 24
Hydra-16's files 28
hydra-16.html 24
`hydra?` *hylang* 301
HydraFS 95, 146
hydrafs.js 334
HydraOS 16, 19
hydrapc.js 334
HyForth 19, 73, 74, 84, 177
`+` *HyForth* 277
`-` *HyForth* 277
`*` *HyForth* 277
`/` *HyForth* 277
`*/` *HyForth* 277
`=` *HyForth* 278
`<` *HyForth* 278
`>` *HyForth* 278
`@` *HyForth* 278
`!` *HyForth* 278
`+!` *HyForth* 278
`,` *HyForth* 278
`.` *HyForth* 279, 281
`."` *HyForth* 279
`<#` *HyForth* 279
`#` *HyForth* 279
`#>` *HyForth* 279
`\` *HyForth* 279
`:` *HyForth* 279
`;` *HyForth* 279
`<>` *HyForth* 281
`?` *HyForth* 283
`--` *HyForth* 286
`:}` *HyForth* 286
`%` *HyForth* 287
hylang 19, 74, 197
`;` *hylang* 292
`:` *hylang* 292
`=` *hylang* 292
`?` *hylang* 292
`@` *hylang* 292
`#` *hylang* 292
`.` *hylang* 292
`~` *hylang* 292
`&_` *hylang* 292
`<=>` *hylang* 293
`+` *hylang* 295, 297
`-` *hylang* 295
`*` *hylang* 295
`/` *hylang* 295
`==` *hylang* 296
`<` *hylang* 296
`>` *hylang* 296
`<=` *hylang* 296
`>=` *hylang* 296
`<>` *hylang* 302
`!=` *hylang* 302
`><?` *hylang* 302
`??` *hylang* 302
hysong.js 335

I

`i` *HyForth* 280
I/O 16
I/O header 19
`i2c` *HyForth* 291
IC 34
IC1 to IC4 54
IC5 to IC8 54
idle task 81
`IF` *HyBASIC* 306
`if` *HyForth* 280
`if` *hylang* 293
`IMAG` *HyBASIC* 306
image file 30, 58
image file as `/pc` 30, 58
immediate 276
`immediate` *HyForth* 279
`IMP` *HyBASIC* 306
`>in` *HyForth* 279
`in?` *hylang* 294
include 185
`INCLUDE` *HyBASIC* 306
`include` *HyForth* 280
`include-file` *HyForth* 283
`included` *HyForth* 280

indenting 221
index-of *hylang* 297
indexing 238
indirect 238
init 78
init *hylang* 294
INIT_RUN 352
INKEY$ *HyBASIC* 306
INPUT 216
INPUT *HyBASIC* 306
INPUT$ *HyBASIC* 306
INSTR *HyBASIC* 306
INT *HyBASIC* 306
int? *HyForth* 289
integers 170
interrupt 78, 80, 241
interrupt entry 131
interrupt handler 241
interrupt line 131
inverse *hylang* 302
invert *HyForth* 278
ior 189
ior>text *HyForth* 288
IRQ_OWN 351
IRQ_RELEASE 351
IS *HyBASIC* 307
is *HyForth* 281
ISO-8859-15 318
item-at *hylang* 294
I²C 164
I²C bus 163

J

j *HyForth* 280
J11 37, 48, 55
J12 to J17 48
J26 37, 48
J3 37, 48, 53
J30 62
J4 54
J7 51
join *hylang* 294
joints, how many 32
jump table 252
jumper 37, 51, 338

K

k-alt-mask *HyForth* 284
k-ctrl-mask *HyForth* 284
k-delete *HyForth* 284
k-down *HyForth* 284
k-end *HyForth* 284
k-f1 *HyForth* 284
k-f12 *HyForth* 284
k-focus *HyForth* 284
k-home *HyForth* 284
k-insert *HyForth* 284
k-left *HyForth* 284
k-next *HyForth* 284
k-prior *HyForth* 284
k-resize *HyForth* 284
k-right *HyForth* 284
k-shift-mask *HyForth* 284
k-up *HyForth* 284
keeping your work 71
kernel 18
key 202, 318
key *HyForth* 279
key *hylang* 299
key? *HyForth* 284
key? *hylang* 299
kill command 118
KILL *HyBASIC* 307
kill *hylang* 301
KMESG 347

L

label 49, 118, 244, 247
label command 118
last *hylang* 294
last-index-of *hylang* 297
Latin-9 321
layers 156
layout 104
LBOUND *HyBASIC* 307
LCASE$ *HyBASIC* 307
lead in solder 33
lead-free 33
leave *HyForth* 280
leaving a program 67
LED 55, 163
LEFT$ *HyBASIC* 307
LEN *HyBASIC* 307
len *hylang* 294
LET *HyBASIC* 307
let *hylang* 293
lib *HyForth* 283
-lib *HyForth* 283
libraries 188
library *HyForth* 286
libs *HyForth* 283
line editor 112
LINE *HyBASIC* 307
line number 214
Lisp 197
list 86
LIST *HyBASIC* 307
list *HyForth* 286
list *hylang* 294
literal *HyForth* 279
little-endian 234
lo *hylang* 296
LOAD *HyBASIC* 307
load *HyForth* 286
load *hylang* 293
LOC *HyBASIC* 307
local *HyForth* 286
locals| *HyForth* 286
LOCATE *HyBASIC* 307
LOF *HyBASIC* 307
LOG *HyBASIC* 307
log *hylang* 296
login shell 92
LOOP *HyBASIC* 307
loop *HyForth* 280
+loop *HyForth* 280
loops, in HyBASIC 217
loops, in HyForth 181
loops, in hylang 201
loops, in rc 88
low byte first 234
lower? *hylang* 297
ls command 118
ls *hylang* 300
lshift *HyForth* 278
LTRIM$ *HyBASIC* 307

M

m* *HyForth* 277
machine code 234
macro 247, 250
magenta *HyForth* 284
map *hylang* 294
marker *HyForth* 281
math library 171
math.e *hylang* 302
math.gamma *hylang* 302
math.phi *hylang* 302
math.pi *hylang* 302
max *HyForth* 277
max *hylang* 295
maximum *hylang* 302
memory card 18, 52
method 203
MID$ *HyBASIC* 307
middle C 149
MIDI 149
min *HyForth* 277
min *hylang* 295
minimum *hylang* 302
minus stripe 36
mirror RAMs 19
mistakes, typing 67
mixer 148
mkdir command 118
MKDIR *HyBASIC* 307
=mkdir *HyForth* 288
mkdir *hylang* 300
mkfs command 119
MKN$ *HyBASIC* 307
MML 149, 153
mnemonic 234
MOD *HyBASIC* 307
mod *HyForth* 277
/mod *HyForth* 277
*/mod *HyForth* 277
mod *hylang* 302
mode 146
modifiers 174
MODINFO 348
mods command 119
mods *hylang* 301
modules 81, 119
monitor 29
more command 120
mount 96, 351
mount *HyForth* 287
mount *hylang* 301
move *HyForth* 278
moving a file 69
ms *HyForth* 284
multitasking 16, 73

music macro language 153
mv command 120

N

>n *HyForth* 289
n! *HyForth* 290
n* *HyForth* 289
n+ *HyForth* 289
n- *HyForth* 289
n. *HyForth* 289
n.base *HyForth* 289
n.s *HyForth* 289
n/ *HyForth* 289
n/mod *HyForth* 289
n0< *HyForth* 289
n0= *HyForth* 289
N: *HyForth* 276
n< *HyForth* 289
n= *HyForth* 289
n> *HyForth* 289
n>d *HyForth* 289
n>r *HyForth* 283
n>s *HyForth* 289
n>str *HyForth* 289
n@ *HyForth* 290
nabs *HyForth* 289
NAME *HyBASIC* 307
:name *hylang* 292
name *hylang* 292
$name *hylang* 292
name>compile *HyForth* 283
name>interpret *HyForth* 283
name>string *HyForth* 283
named bases 172
namespace 81, 93, 96
nand *HyForth* 289
natan *HyForth* 289
nbit? *HyForth* 289
nbytes *HyForth* 289
ncompare *HyForth* 289
nconstant *HyForth* 290
ncos *HyForth* 289
ndepth *HyForth* 289
ndrop *HyForth* 289
ndup *HyForth* 289
neg? *hylang* 296
negate *HyForth* 277
negative 173
negative number 235
negative? *hylang* 302
neq *hylang* 296
nested 197
NEW *HyBASIC* 307
new-window command 120
\newline *hylang* 292
newns *HyForth* 287
newns *hylang* 293
nexp *HyForth* 289
NEXT *HyBASIC* 307
nfib *HyForth* 289
nfloor *HyForth* 289
nformat *HyForth* 289
nfrom-bytes *HyForth* 289
ngcd *HyForth* 289
ninvert *HyForth* 289
nip *HyForth* 281
NJM4556A 148
nliteral *HyForth* 290
nlog *HyForth* 289
nlshift *HyForth* 289
nnegate *HyForth* 289
Node.js 28
nodejs.org 28
non-polarized 37
:noname *HyForth* 281
nor *HyForth* 289
NOT *HyBASIC* 307
not *hylang* 296
not-between? *hylang* 302
notch 37
note 241, 256, 348
note group 122, 344
note handler 256
note *HyForth* 288
note *hylang* 301
note-group *HyForth* 288
note-group *hylang* 301
NOTE_POST 352
NOTE_QUEUE 352
NOTIFY 348
nover *HyForth* 289
npi *HyForth* 289
npow *HyForth* 289
nr> *HyForth* 283
nrandom *HyForth* 289
nrot *HyForth* 289
nround *HyForth* 289
nrshift *HyForth* 289
ns command 121
ns *hylang* 301
nseed *HyForth* 289
nsin *HyForth* 289
NSINFO 351
nsqrt *HyForth* 289
nswap *HyForth* 289
ntan *HyForth* 289
nth *hylang* 294
null-modem 58
>number *HyForth* 279
number stack 195
number-bytes *hylang* 296
NUMERATOR *HyBASIC* 307
nvalue *HyForth* 290
nvariable *HyForth* 290
nxor *HyForth* 289

O

objects 203
OCT$ *HyBASIC* 307
octal 171
octave 149
odd? *hylang* 302
of *HyForth* 281
ohms 35
ON *HyBASIC* 307
on-note *HyForth* 288
on-note *hylang* 301
one? *hylang* 296
only *HyForth* 286
opcode 234
OPEN 350
OPEN *HyBASIC* 307
open *hylang* 300
open-blocks *HyForth* 286
open-dir *HyForth* 288
open-file *HyForth* 282
operand 234
operating system 19
operators 148, 304
OPM2151 (a YM2151 replacement) 40
OPTION *HyBASIC* 307
options 29, 85
OR *HyBASIC* 307
or *HyForth* 278
or *hylang* 293
order *HyForth* 286
OUTPUT *HyBASIC* 307
output-of *HyForth* 287
output-of *hylang* 293
over *HyForth* 277

P

pack *hylang* 302
pad *HyForth* 281
page 237, 327
page 0 237
page 1 237
page *HyForth* 284
PAGES_ALLOC 349
PAGES_FREE 349
PAINT *HyBASIC* 307
palette 156
PALETTE *HyBASIC* 307
parameters 218, 250
parse *HyForth* 281
parse-name *HyForth* 281
partition table 146
passes 258
pasting into the Hydra 24, 71
pasting text into the Hydra 24, 134, 137
patch 148
path 68, 93
pattern 88, 116
pause *HyForth* 288
/pc 332
pc *HyForth* 291
PC tool 58
PCM 148, 156
PEEK *HyBASIC* 307
peek *hylang* 301
peek-word *hylang* 301
PI *HyBASIC* 308
pi *hylang* 296
pick *HyForth* 281
picofarads 36
pid *hylang* 301
pin 1 37
pin headers 37
pin sockets 37
pipe 87, 168, 351

piped *HyForth* 287
place 20
plain *HyForth* 284
platform *hylang* 301
play command 121
PLAY *HyBASIC* 308
PLCC 37
PLCC socket 46
PLCC-32 adapter 49
POINT *HyBASIC* 308
pointer 161, 238
POKE *HyBASIC* 307
poke *hylang* 301
poke-word *hylang* 301
port 62
port A 163
POS *HyBASIC* 308
pos? *hylang* 296
positive? *hylang* 302
POST 79
postfix 178
postpone *HyForth* 280
pow *hylang* 296
power light 55, 62
power switch 23, 55
power-on self test 79
precision 171
PREEMPT_OFF 348
PREEMPT_ON 348
prefix 199, 204
PRESERVE *HyBASIC* 308
PRESET *HyBASIC* 308
previous *HyForth* 286
PRINT 213
PRINT *HyBASIC* 308
print *hylang* 299
print-to *hylang* 300
printing, in HyForth 180
priority encoder 241
proc *HyForth* 291
processor 15
product *hylang* 295
profile 92
program counter 234
program's name 84
programs, running 72
prompt 65, 67
prompt *HyForth* 287
prompt2 *HyForth* 287
ps command 121
ps *hylang* 301
PS_ON 55
PSET *HyBASIC* 308
PSG 148, 156
PUT *HyBASIC* 308
PUTC 351
PUTHEX 351
PUTS 351
PuTTY 59
pwd command 122

Q

Q-expression 200
quick format 145
quit *HyForth* 280
quote 85

R

>r *HyForth* 277
.r *HyForth* 281
r/o *HyForth* 282
r/w *HyForth* 282
R22 41
R65C22 51
R65C51 51
R: *HyForth* 276
r> *HyForth* 277
r@ *HyForth* 277
radix 173
RAM 16
RAM bank register 78
RAM banks 255
RAM disks 81, 95
RAM module 18, 27, 52, 344
rand *HyForth* 290
rand32 *HyForth* 290
random *HyForth* 290
random *hylang* 296
RANDOMIZE *HyBASIC* 308
range *hylang* 294
RATIONAL *HyBASIC* 308
rational.d *HyForth* 289
rational.d *hylang* 296
rational.n *HyForth* 289
rational.n *hylang* 296
rational? *HyForth* 289
rationals 170
raw 257
raw mode 135
rc 19, 84
rc command 122
rc: dat: not found 67
rc: LS: not found 67
READ 350
READ *HyBASIC* 308
read *hylang* 297
read-all *hylang* 299
read-buffer *hylang* 301
read-byte *hylang* 299
read-bytes *hylang* 300
read-dir *HyForth* 288
read-file *HyForth* 282
read-file *hylang* 300
read-line *HyForth* 282
read-line *hylang* 299
read-lines *hylang* 300
REAL *HyBASIC* 308
REBOOT 347
recip *hylang* 302
record 224
recurse *HyForth* 279
recursion 201, 230
red *HyForth* 284
REDIM *HyBASIC* 308
redirection 87
reference 34
reference book (the docs folder) 28
refill *HyForth* 281
register 76, 152, 165, 235
regular expression 116
REM *HyBASIC* 308
remove *hylang* 300
remove-all *hylang* 302
remove-fst *hylang* 302
remove-max *hylang* 302
remove-min *hylang* 302
rename *hylang* 300
rename-file *HyForth* 283
renaming a file 69
repeat *HyForth* 280
repeat *hylang* 302
replaces *HyForth* 285
reposition-file *HyForth* 282
repr *hylang* 297
require *HyForth* 280
required *HyForth* 280
reserves 244
reset button 61, 62, 67
reset chip 76
reset vector 76
resistor 35, 336
resistor network 36, 43
resize *HyForth* 286
resize-file *HyForth* 282
RESTORE *HyBASIC* 308
restore-input *HyForth* 281
RESUME *HyBASIC* 308
RETURN *HyBASIC* 308
reverse *HyForth* 284
reverse *hylang* 294
reverse Polish 178
RIGHT$ *HyBASIC* 307
rm command 122
rmdir command 123
RMDIR *HyBASIC* 307
RN 34
RND *HyBASIC* 308
roll *HyForth* 281
ROM 16
ROM bank register 78
ROMREAD 352
root 68, 93
rot *HyForth* 277
round *hylang* 302
RREG *HyBASIC* 308
RS-232 57
rseed *HyForth* 290
rshift *HyForth* 278
RTC 351
RTRIM$ *HyBASIC* 307
RUN *HyBASIC* 308
run *HyForth* 288
run *hylang* 301

S

#s *HyForth* 279
.s *HyForth* 283
s" *HyForth* 279
S-expression 200
s>d *HyForth* 277
s>n *HyForth* 289

s\" *HyForth* 281
safety 33
sample programs 72
SAVE *HyBASIC* 308
save *hylang* 299
save-buffers *HyForth* 286
save-input *HyForth* 281
saving a file 69, 71
sbit *HyForth* 290
scheduler 78
scom command 123
scope 202
score 153
scr *HyForth* 286
screen editor 69, 113
SCREEN *HyBASIC* 308
script 89, 185
SD card 27, 143
SD card 0 27
SD card module 143
SD card reader module 23
SD card reader, connecting 143
SDHC 144
SDK 250
SDSC 144
sdwrite.js 334
SDXC 144
search *HyForth* 285
search-wordlist *HyForth* 286
seats 162
seconds-of *hylang* 301
see *HyForth* 283
SEEK 350
SEEK *HyBASIC* 308
seek *hylang* 300
seg-attach *hylang* 301
seg-bank! *HyForth* 288
seg-create *hylang* 301
seg-detach *hylang* 301
seg-read *hylang* 301
seg-write *hylang* 301
SEG_ATTACH 349
SEG_CREATE 349
SEG_CREATE_IN 350
SEG_DETACH 349
SEG_MAP 349
SEGINFO 350
segment 115, 167, 244, 249, 325, 349
SELECT *HyBASIC* 308
SEM_ACQUIRE 349
SEM_FREE 349
SEM_NEW 349
SEM_RELEASE 349
SEM_TRY 349
semaphore 349
semitone 149
send *HyForth* 287
serial cable 57
serial console 24, 26
serial port 57
set *hylang* 293
set! *hylang* 293
set-base *HyForth* 289
set-current *HyForth* 286
set-dir *HyForth* 288
set-order *HyForth* 286
setenv *HyForth* 287
setenv *hylang* 301
SGN *HyBASIC* 308
sgr *HyForth* 284
sh *HyForth* 288
sh *hylang* 301
sh-out *HyForth* 287
sh-out *hylang* 301
SHARED *HyBASIC* 308
shell 65, 84
SHELL *HyBASIC* 308
SHELL$ *HyBASIC* 308
shifts 176
SHL *HyBASIC* 308
shl *hylang* 296
SHR *HyBASIC* 308
shr *hylang* 296
sign *HyForth* 279
SIN *HyBASIC* 308
sin *hylang* 296
slay command 123
SLEEP 348
sleep command 123
SLEEP *HyBASIC* 308
sleep *hylang* 301
sleep-until *hylang* 301
SLEEP_UNTIL 348
sliteral *HyForth* 285
slot 0 19, 156
sm/rem *HyForth* 277
SMC 156
snarf buffer 105
snd *hylang* 294
sockets 37, 336
sort command 123
sort *hylang* 294
sort-desc *hylang* 302
SOUND *HyBASIC* 308
sound, in HyBASIC 224
sound, in HyForth 193
sound, in hylang 210
source file 243
source *HyForth* 279
source-id *HyForth* 281
space *HyForth* 279
\space *hylang* 292
SPACE$ *HyBASIC* 308
space? *hylang* 297
spaces *HyForth* 279
SPAWN 348
spawn *HyForth* 287
spawn *hylang* 301
SPC *HyBASIC* 308
spec 131
SPI bus 19
SPI device headers 19
spi *HyForth* 291
split *hylang* 302
SPRITE *HyBASIC* 308
sprites 156
SQR *HyBASIC* 309
sqrt *hylang* 296
square *hylang* 302
square pad 37, 43
SRV_NAP 352
SRV_REGISTER 352
SRV_REPLY 352
SRV_TAKE 352
stack 177, 237, 240, 276
stack diagram 179, 276
stack underflow 184
stacking headers 55
standard error 253
standard files 86
standard input 253
standard output 253
starts-with? *hylang* 302
STAT 350
stat *hylang* 300
stat record 350
state *HyForth* 280
statement 247
static 234
STATIC *HyBASIC* 309
status 88, 105, 108
STATUS *HyBASIC* 309
status *HyForth* 287
STEP *HyBASIC* 309
STOP *HyBASIC* 309
stopping a program 66, 67
STR$ *HyBASIC* 309
str-chars *hylang* 297
str-join *hylang* 297
str-lower *hylang* 297
str-pad-left *hylang* 297
str-pad-right *hylang* 297
str-repeat *hylang* 297
str-replace *hylang* 297
str-split *hylang* 297
str-trim *hylang* 297
str-upper *hylang* 297
stream 208
string 183, 198, 214
STRING *HyBASIC* 309
/string *HyForth* 285
STRING$ *HyBASIC* 309
strings, in HyBASIC 217
strings, in HyForth 182
strings, in hylang 202
SUB 218
SUB *HyBASIC* 309
subset *hylang* 294
substitute *HyForth* 285
substring *hylang* 297
sum *hylang* 295
superblock 146
surface-mount 37
surface-mount card 52
SWAP *HyBASIC* 309
swap *HyForth* 277
switching off 61, 75
synonym *HyForth* 283
SYS *HyBASIC* 309
sys *HyForth* 288
sys *hylang* 301

sys- *HyForth* 288
sys- *hylang* 301
SYSINFO 347
sysinfo *hylang* 301
system call 130, 196, 245, 252, 347
SYSTEM *HyBASIC* 309

T

TAB *HyBASIC* 309
\tab *hylang* 292
tail command 124
tail *hylang* 294
take *hylang* 294
TAN *HyBASIC* 309
tan *hylang* 296
task 78
task 0 78
task 1 78
task-info *hylang* 301
TASKINFO 348
TASKMEM 352
TASKREAD 349
TASKSTEP 352
TASKSTOP 352
tbit *HyForth* 290
tee command 124
tell *hylang* 300
tenth *hylang* 302
Tera Term 59
terminal program 57
terminal window 24
"text" *hylang* 292
"""text""" *hylang* 292
thd *hylang* 294
THEN *HyBASIC* 309
then *HyForth* 280
through-hole card 52
throw codes 291
throw *HyForth* 282
thru *HyForth* 286
tick 81, 165, 232, 348
tick-rate *hylang* 301
ticks *hylang* 301
tiled 104
TIME 351
time *hylang* 301
TIME$ *HyBASIC* 309
time&date *HyForth* 284
TIME_SET 351
TIMER *HyBASIC* 309
times *hylang* 302
TL074 148
TO *HyBASIC* 309
to *HyForth* 281
to# *hylang* 298
TO-92 37
to-atom *hylang* 297
to-fixed *HyForth* 289
to-fixed *hylang* 295
to-rational *HyForth* 289
to-rational *hylang* 295
to-str *hylang* 297
to-sym *hylang* 297
top command 124
touch command 125
-trailing *HyForth* 285
transaction 165
traverse-wordlist *HyForth* 283
tree 93
troubleshooting 62
true *HyForth* 281
trunc *hylang* 302
truncate *HyForth* 289
truncate *hylang* 295
try *hylang* 293
tuck *HyForth* 281
turtle graphics 194
two's complement 235
TYPE *HyBASIC* 309
type *HyForth* 279
typing a line 66

U

u. *HyForth* 279
u.r *HyForth* 281
U43 37, 46
U6 37
u< *HyForth* 278
u> *HyForth* 281
UBOUND *HyBASIC* 307
UCASE$ *HyBASIC* 307
um* *HyForth* 277
um/mod *HyForth* 277
uncurry *hylang* 302
underline *HyForth* 284
unescape *HyForth* 285
union directory 98
uniq command 125
unloop *HyForth* 280
UNMOUNT 351
unmount *HyForth* 287
unmount *hylang* 301
unpack *hylang* 302
unsetenv *HyForth* 287
unsetenv *hylang* 301
UNTIL *HyBASIC* 309
until *HyForth* 280
unused *HyForth* 281
update *HyForth* 286
upper? *hylang* 297
usage 74
USB to RS-232 adapter 57
USB-to-serial adapter 23
use *hylang* 302
USING *HyBASIC* 309

V

VAL *HyBASIC* 309
val *hylang* 296
value 182, 198, 202
value *HyForth* 281
variable 85, 182, 214
variable *HyForth* 280
variables, in HyBASIC 214
variables, in HyForth 182
variables, in hylang 198
variables, in rc 85
vector RAM 241
VERA 156
Vera X 19, 26, 156, 326
Vera X's PSG 148
VIA 325
video *HyForth* 291
video RAM 156
VIEW *HyBASIC* 309
VT100 138

W

W Register 19
w/o *HyForth* 282
W65C02S 15, 234
W65C22N 51
W65C51N 51
WAIT 348
wait *HyForth* 287
wait *hylang* 301
WAKE 352
wc command 126
WDC's W65C02S data sheet 242
web browser 24
web emulator 24
WEND *HyBASIC* 309
which language first 73
WHILE *HyBASIC* 309
while *HyForth* 280
while *hylang* 293
white *HyForth* 284
WIDTH *HyBASIC* 309
window 0 101
WINDOW *HyBASIC* 309
window key 101
within *HyForth* 281
word *HyForth* 279
word *hylang* 296
wordlist *HyForth* 286
words 177, 230
words *HyForth* 283
worst-of *hylang* 302
WRITE *HyBASIC* 309
write *hylang* 299
write-bytes *hylang* 300
write-file *HyForth* 282
write-file *hylang* 300
write-line *HyForth* 282
write-to *hylang* 300
WSTAT 350

X

x *hylang* 301
XCALL 347
xd command 126
#xFF *hylang* 292
XMODEM 126
xmodem command 126
XOFF 59, 137
XON 59, 137
xor 176
XOR *HyBASIC* 309
xor *HyForth* 278
xor *hylang* 302

Y

Y1 37, 46
Yamaha YM2151 148
`yellow` *HyForth* 284
`YIELD` 348
`yield` *hylang* 301
YM2151 19, 148, 326
YM3012 148

Z

`>z` *HyForth* 288
`zcount` *HyForth* 288
zero page 237
`zero?` *hylang* 296
`zip` *hylang* 302
zones 213
ZSM 154